

UNIVERSIDAD INTERCONTINENTAL DE LA EMPRESA



FACULTAD DE INGENIERÍA Y TECNOLOGÍA EMPRESARIAL

TRABAJO FIN DE GRADO EN INGENIERÍA EN SISTEMAS INTELIGENTES

F1 StratLab: Sistema Multi-Agente Inteligente para Análisis de Telemetría y Optimización de Estrategias en Carreras de Fórmula 1

Autor:

Vega Sobral, Víctor

Director(es):

Fontenla Seco, Yago



Esta obra se encuentra sujeta a la licencia *Creative Commons* (CC-BY-NC-ND)

Compartir-Reconocimiento-No Comercial-Sin Obra Derivada

RESUMEN

La estrategia de carrera en Fórmula 1 condiciona el resultado final del Gran Premio y exige decisiones críticas en ventanas de pocos segundos sobre información masiva, heterogénea e incierta. Pese a la inversión millonaria de los equipos en sistemas internos de apoyo a la decisión, el conocimiento académico público sobre estos sistemas es escaso. Este trabajo presenta F1 StratLab, un sistema multi-agente abierto y extensible que aplica aprendizaje automático, aprendizaje profundo y modelos de lenguaje a la producción de recomendaciones estratégicas de carrera sobre datos públicos.

El sistema combina cuatro bloques. Primero, una canalización de ingeniería de características sobre telemetría descargada con fuentes de datos públicas, con agrupación de circuitos en cuatro arquetipos. Segundo, una familia de modelos predictivos de distintas disciplinas para las variables clave de la estrategia de carrera, desde el tiempo de vuelta y la degradación de neumático hasta la duración de la parada, el undercut, el adelantamiento y la probabilidad de coche de seguridad, validados con partición temporal estricta, con las temporadas 2023 y 2024 dedicadas a entrenamiento y la temporada 2025 a prueba, para mitigar la deriva conceptual entre reglamentos. Tercero, un flujo de canalización de procesamiento del lenguaje natural sobre la radio de equipo y los mensajes de dirección de carrera. Cuarto, un subsistema de generación aumentada por recuperación sobre el reglamento deportivo de la FIA, integrado en una arquitectura multi-agente en la que cada sub-agente envuelve uno o varios de los modelos anteriores y razona localmente sobre sus salidas, supervisado por un orquestador de tres capas (enrutador por mezcla de expertos, simulación Monte Carlo de quinientas trayectorias y síntesis con modelo de lenguaje) que entrega una recomendación estratégica vuelta a vuelta.

El sistema se valida sobre tres carreras representativas de la temporada 2025, con recomendaciones que diferían de la decisión real del muro de boxes en dos de ellas y resultaron mejor justificadas a posteriori. La aportación principal es la integración reproducible de los cuatro bloques anteriores en un sistema multi-agente académico abierto sobre datos públicos de Fórmula 1, distribuido bajo licencia Apache 2.0 frente a las plataformas cerradas de los equipos profesionales. El motor de simulación opera vuelta a vuelta sobre cualquier carrera de la temporada 2025, y la arquitectura modular está preparada para la ingestión de telemetría en tiempo real y el reentrenamiento sobre el nuevo reglamento técnico de la temporada 2026 en adelante.

Palabras clave:

Aprendizaje automático, Deriva conceptual, Estrategia de carrera, Fórmula 1, Ingeniería de características, Modelo de lenguaje, Procesamiento de lenguaje natural, Generación aumentada por recuperación, Sistema multi-agente, Telemetría.

ABSTRACT

Race strategy in Formula 1 largely determines the final outcome of a Grand Prix and requires critical decisions to be made within a few seconds, on the basis of massive, heterogeneous and uncertain information. Despite the millions invested by teams in internal decision support systems, public academic knowledge about such systems remains limited. This thesis presents F1 StratLab, an open and extensible multi-agent system that applies machine learning, deep learning and large language models to the generation of race strategy recommendations from public data.

The system combines four components. First, a feature engineering pipeline is applied to telemetry obtained from public data sources, with circuits grouped into four archetypes. Second, a family of predictive models from different disciplines is developed for the key variables of race strategy, from lap time and tire degradation to pit stop duration, undercut, overtake and safety car probability, and is validated with a strict temporal split, with the 2023 and 2024 seasons assigned to training and the 2025 season to testing, in order to mitigate concept drift across regulations. Third, a natural language processing pipeline is applied to team radio and race control messages. Fourth, a retrieval-augmented generation subsystem is built on the FIA sporting regulations and integrated into a multi-agent architecture in which each sub-agent wraps one or more of the previous models and reasons locally over their outputs, under the supervision of a three-layer orchestrator, comprising a mixture-of-experts router, a Monte Carlo simulation of five hundred trajectories and language model synthesis, which delivers a lap by lap race strategy recommendation.

The system is validated on three representative races from the 2025 season, with recommendations that differed from the real pit wall decision in two of them and were judged afterwards to be better justified. The main contribution is the reproducible integration of the four preceding components into an open academic multi-agent system over public Formula 1 data, distributed under the Apache 2.0 license in contrast to the closed platforms used by professional teams. The replay engine operates lap by lap over any race of the 2025 season, and the modular architecture is prepared for real-time telemetry ingestion and retraining on the technical regulations from the 2026 season onwards.

Keywords:

Concept drift, Feature engineering, Formula 1, Large language model, Machine learning, Multi-agent system, Natural language processing, Race strategy, Retrieval-augmented generation, Telemetry.

DEDICATORIA

A mis padres.

No podemos hacer grandes cosas, pero sí cosas pequeñas con un gran amor.

— Madre Teresa de Calcuta¹

¹Cita ampliamente recogida de los escritos y entrevistas de Madre Teresa de Calcuta, en la forma canónica con la que circula en castellano. Original en inglés: *We cannot do great things, only small things with great love.*

ÍNDICE DE CONTENIDOS

RESUMEN	II
ABSTRACT	III
DEDICATORIA	V
ÍNDICE DE FIGURAS	XII
ÍNDICE DE TABLAS	XIII
1 INTRODUCCIÓN	1
1.1 Motivación del trabajo	1
1.2 Planteamiento del problema	2
1.3 Objetivos	3
1.3.1 Objetivo general	3
1.3.2 Objetivos específicos	4
1.4 Antecedentes	4
1.5 Metodología	5
1.6 Organización del documento	7
2 MARCO TEÓRICO	8
2.1 Contextualización de la Fórmula 1 y decisiones estratégicas	8
2.1.1 Estructura de un fin de semana de carrera	8
2.1.2 Neumáticos y degradación	9
2.1.3 Eventos de pista	9
2.1.4 Conceptos estratégicos	10
2.1.5 La carrera como problema de decisión bajo incertidumbre	11
2.2 Deriva conceptual en series temporales deportivas	12
2.3 Gradient Boosting: XGBoost, LightGBM e HistGBT	15
2.4 Redes convolucionales temporales	16
2.5 Procesamiento de lenguaje natural para radio de Fórmula 1	18
2.5.1 Reconocimiento automático del habla: Whisper	18
2.5.2 Clasificación de sentimiento: RoBERTa	20
2.5.3 Clasificación de intención: SetFit y ModernBERT	20
2.5.4 Reconocimiento de entidades nombradas: BERT-BIO	20
2.5.5 Procesamiento de mensajes de dirección de carrera	21
2.6 Generación aumentada por recuperación	21
2.7 Agentes de modelos de lenguaje y orquestación multi-agente	22
2.8 Agrupamiento de circuitos	24
3 ANÁLISIS DE LA SITUACIÓN	26

3.1	Panorama actual de la inteligencia artificial en la Fórmula 1	26
3.1.1	Equipos punteros y sus partners tecnológicos	26
3.1.2	Software comercial y producción académica	28
3.2	Oportunidades del panorama actual	29
3.3	Requisitos del sistema	30
3.3.1	Requisitos funcionales	30
3.3.2	Requisitos no funcionales	32
3.4	Restricciones	33
4	DESARROLLO	35
4.1	Diseño	35
4.1.1	Visión de sistema en tres capas	35
4.1.2	Patrones y contratos de orquestación	37
4.1.3	Stack tecnológico	38
4.2	Implementación de la solución	40
4.2.1	Ingeniería de datos	40
4.2.1.1	Extracción y consolidación	40
4.2.1.2	Agrupamiento de circuitos en cuatro arquetipos	41
4.2.1.3	Construcción del dataset featurizado	43
4.2.2	Pace Agent	44
4.2.2.1	Datos y análisis exploratorio	44
4.2.2.2	Modelo entrenado e integración como sub-agente	45
4.2.3	Tire Agent	47
4.2.3.1	Datos y análisis exploratorio	48
4.2.3.2	Modelos entrenados e integración como sub-agente	49
4.2.4	Race Situation Agent	51
4.2.4.1	Datos y análisis exploratorio	51
4.2.4.2	Modelos entrenados e integración como sub-agente	53
4.2.5	Pit Strategy Agent	55
4.2.5.1	Datos y análisis exploratorio	55
4.2.5.2	Modelos entrenados e integración como sub-agente	57
4.2.6	Radio Agent	59
4.2.6.1	Datos y análisis exploratorio	59
4.2.6.2	Modelos entrenados e integración como sub-agente	60
4.2.7	RAG Agent	61
4.2.7.1	Datos y análisis exploratorio	61
4.2.7.2	Modelo de recuperación e integración como sub-agente	62
4.2.8	Strategy Orchestrator	63
4.2.8.1	Enrutamiento MoE entre sub-agentes	63
4.2.8.2	Simulación Monte Carlo	64
4.2.8.3	Síntesis con modelo de lenguaje	65
4.2.9	Interfaces de uso	67
4.2.9.1	Línea de comandos	68

4.2.9.2	Panel Streamlit y backend FastAPI con FastMCP	69
4.2.9.3	Réplica en Arcade de tres ventanas	71
5	RESULTADOS	76
5.1	Resultados de los modelos predictivos	76
5.1.1	Predicción de tiempo de vuelta	77
5.1.2	Agrupamiento de circuitos	78
5.1.3	Degradación de neumático	79
5.1.4	Probabilidad de adelantamiento	80
5.1.5	Probabilidad de coche de seguridad	81
5.1.6	Duración de parada en boxes	81
5.1.7	Probabilidad de undercut	82
5.1.8	Transcripción automática de radio	83
5.1.9	Análisis de sentimiento de radio	83
5.1.10	Clasificación de intención de radio	84
5.1.11	Reconocimiento de entidades de radio	85
5.1.12	Procesamiento de mensajes de dirección de carrera	86
5.1.13	Recuperación reglamentaria	86
5.2	Resultados por sub-agente	88
5.3	Validación end-to-end del orquestador	89
5.4	Comparación con la literatura previa	90
5.5	Casos de uso	92
5.5.1	Análisis post-carrera con Streamlit	92
5.5.2	Replay del GP de Hungría 2025 desde la CLI	93
5.5.3	Sesión en directo con Arcade sobre el GP de Qatar 2025	96
6	CONCLUSIONES y TRABAJOS FUTUROS	98
6.1	Cumplimiento de objetivos	98
6.2	Aportaciones principales	98
6.3	Refinamiento durante la validación: módulo RCMContextResolver	100
6.4	Limitaciones	101
6.5	Trabajos futuros	102
6.6	Valoración personal	103
7	BIBLIOGRAFÍA	104
8	ANEXOS	113
8.1	Anexo 1 Declaración de uso de IAG	113
8.2	Anexo 2 Manual de instalación, despliegue y uso	117
8.2.1	II.1 Requisitos del sistema	117
8.2.2	II.2 Instalación con uv, flujo recomendado	117
8.2.3	II.3 Instalación como herramienta global	118
8.2.4	II.4 Alternativa con pip y entorno virtual	118
8.2.5	II.5 Despliegue con Docker Compose	118
8.2.6	II.6 Variables de entorno	119

8.2.7	II.7 Descarga del conjunto de datos y de los modelos	119
8.2.8	II.8 Primera ejecución, tres caminos	120
8.2.8.1	Camino 1, CLI con Rich	120
8.2.8.2	Camino 2, modo Arcade de tres ventanas	120
8.2.8.3	Camino 3, Streamlit más FastAPI	121
8.2.9	II.9 Inventario de scripts en scripts	121
8.2.10	II.10 Resolución de incidencias frecuentes	121
8.3	Anexo 3 Material complementario	123
8.3.1	III.1 Mapa de notebooks del proyecto	123
8.3.2	III.2 Diagramas completos del sistema	123

ÍNDICE DE FIGURAS

2.1	Vista conceptual del bucle de decisión	12
2.2	Protocolo de validación temporal por bloques	14
2.3	Componentes arquitectónicos de una TCN	17
2.4	Arquitectura general de Whisper	19
2.5	Flujo de generación aumentada por recuperación	22
4.1	Arquitectura del sistema en tres capas	36
4.2	Matriz de correlación tras la poda	42
4.3	Selección del número de clusters	43
4.4	Pipeline de datos del sistema	44
4.5	Deriva inter-temporada del tiempo por vuelta	45
4.6	Predicción del modelo de ritmo frente al tiempo real por arquetipo de circuito	46
4.7	Flujo del agente de ritmo	47
4.8	Tasa de degradación por compuesto frente a la edad del neumático	48
4.9	Compromiso entre truncación y <i>padding waste</i> por tamaño de ventana	49
4.10	Flujo del agente de neumáticos	51
4.11	Distribución de pares de coches en el espacio brecha–diferencia de ritmo	52
4.12	Análisis exploratorio del corpus N13 para el clasificador de coche de seguridad	53
4.13	Importancias SHAP del clasificador de adelantamiento N12	54
4.14	Arquitectura del Race Situation Agent (N27)	55
4.15	Validación de la descomposición de duración de parada frente a OpenF1	56
4.16	Análisis exploratorio del corpus de undercuts del cuaderno N16	57
4.17	Arquitectura del Pit Strategy Agent (N28)	58
4.18	Distribución de entidades del esquema BIO del corpus de radio	60
4.19	Análisis exploratorio del corpus de mensajes de dirección de carrera	60
4.20	Arquitectura del Radio Agent (N29)	61
4.21	Arquitectura del RAG Agent (N30)	63
4.22	Distribuciones de la simulación Monte Carlo del orquestador	65
4.23	Esquema de salida <i>StrategyRecommendation</i> del orquestador	67
4.24	Panel multi-agente del simulador <i>f1-sim</i> en una vuelta	69
4.25	Página <i>Strategy</i> del panel Streamlit ejecutando el orquestador.	71
4.26	Menú inicial y ventana de replay del subsistema Arcade.	73
4.27	Panel estratégico del subsistema Arcade.	74
4.28	Ventana de telemetría en directo del subsistema Arcade.	75
5.1	Chat conversacional de Streamlit con llamada a la herramienta <i>get_lap_times</i>	93
5.2	Fragmento del informe generado por el botón <i>Download Report</i>	94
5.3	Panel Rich de <i>f1-sim</i> sobre el GP de Hungría 2025 en V19	95
5.4	Réplica visual con Arcade sobre el GP de Catar 2025 en la vuelta 7 tras la corrección	97

8.1	Vista global de F1 StratLab. Las tres superficies de uso (CLI, Arcade y Streamlit con FastAPI) comparten el núcleo de seis sub-agentes y el orquestador supervisor N31, el proveedor de modelos de lenguaje y el árbol de datos. El bloque inferior representa los modelos en producción y los artefactos descargados durante el primer arranque desde el conjunto abierto publicado en Hugging Face.	124
8.2	Flujo del orquestador supervisor N31. La capa de invocación dispara los sub-agentes especializados, la capa de enrutamiento por mezcla de expertos basada en reglas decide qué salidas pasan a la siguiente etapa, la capa de simulación Monte Carlo evalúa las cuatro acciones discretas STAY_OUT, PIT_NOW, UNDERCUT y OVERCUT sobre quinientas trayectorias por candidata y la capa de síntesis con modelo de lenguaje produce la recomendación final con salida estructurada de catorce campos Pydantic.	127
8.3	Arquitectura de tres ventanas del modo Arcade. El proceso <i>pyglet</i> del replay 2D actúa como dueño del bucle de simulación y publica el estado por TCP al puerto 127.0.0.1:9998 a diez hercios. Un único subproceso lanzado con <code>subprocess.Popen</code> sobre <code>src.arcade.dashboard</code> abre las dos ventanas Qt (cuadro de mando estratégico y telemetría) dentro de la misma <code>QApplication</code> , suscritas al mismo flujo TCP.	128
8.4	Flujo del chat conversacional con tool calling MCP. La consulta del usuario en la interfaz de Streamlit alcanza el endpoint <code>/api/v1/chat/stream</code> del <i>backend</i> FastAPI, que invoca <code>chat_engine.stream_chat</code> . El modelo <code>gpt-5.4-mini</code> decide la herramienta MCP a llamar mediante el mecanismo de <i>function calling</i> de OpenAI, el resultado de la herramienta se parte en el <i>payload</i> completo destinado a la interfaz y el <i>payload</i> reducido destinado a la segunda pasada del modelo, y el cliente renderiza las figuras Plotly de forma <i>inline</i> junto a la respuesta sintetizada.	129

ÍNDICE DE TABLAS

1.1	Fases metodológicas del desarrollo	6
2.1	Comparativa de variantes de <i>gradient boosting</i>	16
2.2	Modelos disponibles de Whisper	19
2.3	Agrupamiento de circuitos en cuatro arquetipos	25
3.1	Colaboraciones entre equipos de F1 y proveedores de IA	27
4.1	Stack tecnológico del sistema	39
4.2	Métricas del fine-tuning por compuesto y decisión de enrutado	50
5.1	Variantes evaluadas para la predicción de tiempo de vuelta	77
5.2	Selección del número de clusters de circuitos	78
5.3	Ablación de la TCN global de degradación	79
5.4	Modelo de degradación por compuesto sobre holdout 2025	79
5.5	Variantes evaluadas para la probabilidad de adelantamiento	80
5.6	Variantes evaluadas para la probabilidad de coche de seguridad	81
5.7	Variantes evaluadas para la duración de parada en boxes	82
5.8	Variantes evaluadas para la probabilidad de undercut	83
5.9	Métricas del modelo de sentimiento de radio	84
5.10	Métricas del modelo de intención de radio	84
5.11	Variantes evaluadas para el reconocimiento de entidades de radio	85
5.12	Variantes evaluadas para la recuperación reglamentaria	87
5.13	Latencia por sub-agente	88
5.14	Verificación de extremo a extremo del orquestador	90
5.15	Comparación con la literatura previa de aprendizaje automático en Fórmula 1	91
6.1	Cumplimiento de los cuatro objetivos específicos enunciados en el Capítulo 1. La columna de evidencia recoge una muestra representativa de las métricas obtenidas en el Capítulo 5.	99
8.1	Variables de entorno reconocidas por el sistema	119
8.2	Scripts disponibles en <code>scripts/</code>	122
8.3	Incidencias frecuentes y su resolución	122
8.4	Inventario de cuadernos Jupyter del proyecto F1 Strat Manager (parte 1 de 3: ingeniería de datos y aprendizaje supervisado clásico).	125
8.5	Inventario de cuadernos Jupyter del proyecto F1 Strat Manager (parte 2 de 3: procesamiento de lenguaje natural de radio y sub-agentes). Continuación de la Tabla 8.4.	126
8.6	Inventario de cuadernos Jupyter del proyecto F1 Strat Manager (parte 3 de 3: integración y validación). Continuación de la Tabla 8.5.	127

1 INTRODUCCIÓN

La suerte es preparación a la espera de una oportunidad.

— Ross Brawn, director técnico de Ferrari¹

1.1 Motivación del trabajo

La Fórmula 1 es la máxima categoría del automovilismo internacional: diez equipos compiten con dos pilotos cada uno a lo largo de un calendario de entre veinte y veinticuatro carreras (denominadas Grandes Premios) repartidas por todo el mundo. Se trata de un deporte gobernado por decisiones tomadas en cuestión de segundos y bajo un caudal constante de información: telemetría de cientos de sensores, comunicaciones por radio y eventos de pista que alteran el orden competitivo a cada vuelta.

La pericia del piloto y la sofisticación técnica del monoplaza son condición necesaria, pero no suficiente. Una decisión estratégica sobre cuándo parar, qué compuesto de neumático (la mezcla de goma utilizada, con distinto equilibrio entre velocidad y duración) montar al hacerlo, cuándo forzar un adelantamiento o cuándo gestionar el desgaste de los neumáticos puede invertir el resultado de una carrera incluso cuando el rival cuenta con un coche más rápido o una mejor posición en pista.

La importancia de esta estrategia puede ejemplificarse con el Gran Premio de Mónaco de 2023. En un año en el que Red Bull (el equipo dominante en el período 2022-2024) se alzó con la victoria en veintiuna de las veintidós carreras del campeonato, el equipo Aston Martin había arrancado la temporada con un coche sorprendentemente competitivo, alcanzando el podio en seis de las ocho primeras carreras con Fernando Alonso.

No obstante, las condiciones cambiantes del tramo final, con lluvia dispersa solo en algunas curvas, abrieron una ventana de incertidumbre suficiente para arrebatarse la victoria a Max Verstappen. Al terminar la vuelta 54, Aston Martin llamó a Fernando Alonso a boxes y le montó neumáticos medios, apostando a que el chubasco sería breve. Sin embargo, la lluvia se intensificó y obligó a una segunda parada inmediata para cambiar a intermedios. Red Bull, en cambio, acertó con una única parada directa a intermedios en la vuelta 55 y se llevó la victoria con casi treinta segundos de margen.

Lo que pudo parecer una cuestión de fortuna climática fue, en realidad, el resultado de un criterio estratégico y una experiencia superiores en la toma de decisiones críticas bajo incertidumbre, que evidenciaron el margen entre el equipo estratégico veterano y consolidado de Red Bull y un Aston Martin aún en construcción.

Precisamente por el peso que la toma de decisiones tiene en el resultado final, y por la imposibilidad de procesar humanamente, en tiempo real, todas las variables que confluyen en cada vuelta, los equipos han respondido volcando recursos en sistemas de Inteligencia Artificial (IA), ya sea con desarrollos propios o a través de colaboraciones con grandes empresas tecnológicas, como es el caso de Ferrari con Amazon Web Services (AWS), Red Bull con Oracle, McLaren con Dell, Cisco y Google, o Aston Martin con Cognizant (Carter, 2025). Esta tendencia se ha intensificado con acuerdos orientados específicamente a la IA generativa y los modelos de lenguaje (LLMs, por sus siglas en inglés), como la alianza firmada en 2024 entre Cohere y Aston Martin como socio oficial de IA generativa (Aston Martin F1 Team, 2026), o el acuerdo multianual

¹Fragmento de *Total Competition: Lessons in Strategy from Formula One*, de Ross Brawn y Adam Parr (2016). Traducción propia del original en inglés: *Luck is preparation waiting for an opportunity*.

que Anthropic cerró en 2026 con Williams, por el que su modelo Claude pasa a integrarse en la toma de decisiones estratégicas, el desarrollo del coche y las operaciones del equipo (Williams Racing, 2026). No obstante, ni la comunidad académica ni los aficionados disponen de acceso a cómo razonan² estos sistemas, qué modelos emplean o cómo integran fuentes heterogéneas de telemetría, radio y regulaciones, debido a la naturaleza competitiva de la categoría, que convierte a estos sistemas en un activo estratégico propio que no se comparte.

Frente a este enfoque cerrado, el presente *Trabajo de Fin de Grado* propone un sistema multi-agente *abierto y reproducible*, debidamente documentado, que asiste en la toma de decisiones estratégicas durante carreras de Fórmula 1. El sistema se articula en cinco capas:

- Extracción y preparación de los datos de telemetría.
- Ingesta e inferencia de modelos predictivos de ritmo por vuelta, degradación de neumáticos, probabilidad de adelantamiento, coche de seguridad y duración de paradas en boxes.
- Procesamiento del lenguaje natural sobre las comunicaciones de radio entre pilotos y muros, junto a los mensajes del director de carrera.
- Razonamiento a través de sub-agentes especializados que consumen las salidas de las capas anteriores.
- Un orquestador basado en un modelo de lenguaje que sintetiza las recomendaciones de los sub-agentes en una recomendación estratégica final.

Cada recomendación del orquestador es trazable hasta las inferencias de los modelos y el razonamiento de los sub-agentes que la sustentan, lo que permite auditar el proceso completo de decisión. La propuesta no aspira a competir en escala ni en presupuesto con las soluciones descritas, sino a ofrecer una alternativa accesible cuya lógica interna pueda inspeccionarse, reproducirse y adaptarse por la comunidad académica y aficionada, apoyándose además en fuentes de datos públicas.

La motivación de abordar un trabajo así combina el interés personal del autor por la Fórmula 1 con la oportunidad académica de aplicar una pila técnica actual, que incluye aprendizaje automático, procesamiento del lenguaje natural y agentes basados en modelos de lenguaje, sobre un dominio real caracterizado por su heterogeneidad de datos y la escasez de literatura científica abierta que lo aborde de forma integrada.

1.2 Planteamiento del problema

Desde una perspectiva técnica, el problema se formula como la integración coherente de fuentes de datos heterogéneas, modelos basados en distintas aproximaciones y paradigmas de la inteligencia artificial y una capa de orquestación en una arquitectura unificada capaz de producir recomendaciones estratégicas trazables. Esta integración debe satisfacer simultáneamente restricciones de latencia, condiciones de apertura y reproducibilidad para terceros, y cumplimiento del reglamento deportivo de la Federación Internacional del Automóvil (FIA).

La principal dificultad reside en que cada componente exige una solución propia, dada la distinta naturaleza de los datos de origen y del problema a resolver. Las fuentes de datos abarcan desde series numéricas

²En este trabajo el término *razonar* se emplea para referirse al proceso por el que un modelo de lenguaje llega desde una premisa inicial recogida en su *prompt* hasta un resultado final mediante una generación iterativa de *tokens*; se trata de un proceso no determinista y no debe interpretarse como razonamiento simbólico en el sentido clásico.

estructuradas, como los tiempos por vuelta, hasta lenguaje natural especializado, como el empleado en las comunicaciones por radio, pasando por documentos reglamentarios de la FIA en formato PDF; su consolidación en un conjunto unificado, limpio y fiable requiere un trabajo de ingeniería de datos previo a cualquier modelado. Las tareas de modelado son igualmente plurales: ninguna arquitectura única cubre todas las subtarefas, ya que cada variable estratégica impone su propio compromiso entre tipo de modelo, granularidad temporal y métrica de evaluación, combinando regresión, modelos de series temporales, clasificación y estimación de cuantiles. Reunir todas estas salidas (valores numéricos, etiquetas textuales, referencias al reglamento) en una única recomendación estratégica exige una capa final de integración que opera sobre representaciones dispares y justifica recurrir a un modelo de lenguaje como orquestador. A ello se añade la exigencia de que toda recomendación sea auditable hasta sus fuentes y se produzca en márgenes temporales compatibles con la necesidad de toma de decisiones en tiempo real propia de las carreras, dos objetivos en tensión dado que añadir trazabilidad tiende a incrementar la latencia. Por último, incluso dentro de un mismo ciclo regulatorio (periodo de varias temporadas durante el que se mantienen estables las reglas técnicas que rigen los monoplazas, típicamente de tres a cinco años) los equipos introducen avances anuales en el rendimiento de sus monoplazas, de modo que los patrones de tiempos, degradación y ventanas estratégicas cambian cada temporada; esta situación de deriva conceptual (Hovakimyan & Bravo, 2024) invalida la validación cruzada aleatoria y obliga a particiones temporales estrictas.

Ahora bien, ninguno de los trabajos publicados sobre modelado cuantitativo en Fórmula 1 aborda estos retos de forma conjunta. Heilmeyer et al. proponen un conjunto de herramientas de simulación Monte Carlo que modela con precisión la selección de compuestos y el momento óptimo de las paradas en boxes a partir de datos públicos de la interfaz de programación de aplicaciones (API, por sus siglas en inglés) Ergast, aunque opera exclusivamente sobre variables numéricas de tiempo en pista y no aborda la deriva conceptual presente en sus datos de las temporadas 2014 a 2019, periodo que incluye la revisión aerodinámica de 2017 (Heilmeyer et al., 2020a). Por su parte, el grupo de aprendizaje automático de la Universidad de Cambridge trabaja en colaboración con un equipo de Fórmula 1 de primer nivel para optimizar la estrategia de paradas; sin embargo, los términos de confidencialidad comercial asociados a dicha colaboración impiden el acceso público a sus resultados (Machine Learning at Cambridge Laboratory, 2019). El panorama de los proyectos de código abierto disponibles refleja el mismo sesgo: la práctica habitual son simulaciones Monte Carlo orientadas a estimar la ventana óptima de parada y la posición esperada al final de carrera, entrenadas con frecuencia sobre datos de distintos periodos reglamentarios sin distinguir entre ellos. Ninguno aborda de forma conjunta la predicción vuelta a vuelta de la degradación del neumático, el tiempo esperado por vuelta, la probabilidad de aparición del coche de seguridad y la probabilidad de éxito de un adelantamiento, ni integra dichas predicciones en una arquitectura de razonamiento multi-agente que produzca recomendaciones trazables. La integración de estos tres planos (predicción numérica multidimensional vuelta a vuelta, procesamiento de lenguaje natural y validación contra el reglamento deportivo) en un sistema construido sobre fuentes abiertas de un único ciclo regulatorio no está cubierta por la literatura académica ni por dichos proyectos ni, de forma accesible, por los sistemas propietarios ya descritos. El presente trabajo nace precisamente de esa ausencia.

1.3 Objetivos

1.3.1 Objetivo general

Diseñar, implementar y validar un sistema multi-agente que combine modelos predictivos de aprendizaje automático y aprendizaje profundo, un flujo de procesamiento de lenguaje natural (PLN) y un subsistema de generación aumentada por recuperación (RAG, por sus siglas en inglés) sobre el reglamento deportivo

de la FIA. Estos tres bloques (predicción numérica, análisis de radio y consulta del reglamento) son consumidos por sub-agentes especializados, cuyas conclusiones sintetiza un orquestador supervisor en una recomendación estratégica interpretable y trazable. Todo ello se apoya en datos públicos de las temporadas 2023 y 2024 para el entrenamiento, reservando la temporada 2025 como conjunto de prueba.

1.3.2 Objetivos específicos

1. Construir un conjunto de datos reproducible a partir de FastF1 (Oehrly, 2024) y OpenF1 (OpenF1 Project, 2024), las dos principales bibliotecas de Python que dan acceso público a la telemetría oficial y a los tiempos por vuelta de la Fórmula 1, para las temporadas 2023-2024 (entrenamiento) y 2025 (prueba), e implementar una familia de siete modelos predictivos para las variables clave: tiempo de vuelta, degradación de neumáticos, probabilidad de adelantamiento, probabilidad de coche de seguridad, duración de la parada en boxes y éxito del *undercut* (maniobra estratégica consistente en parar a boxes antes que un rival cercano para aprovechar el ritmo superior de unos neumáticos nuevos y rebasarlo cuando este realiza su propia parada), con agrupación de circuitos en cuatro arquetipos, ingeniería de características y partición temporal estricta para mitigar la deriva conceptual entre temporadas.
2. Desarrollar un flujo de PLN que, a partir de un corpus de audio sincronizado con los datos de telemetría vuelta a vuelta, transcriba las comunicaciones de radio del equipo, clasifique su contenido por sentimiento e intención, extraiga entidades propias del dominio F1 y procese los mensajes de dirección de carrera emitidos por la FIA.
3. Implementar un subsistema RAG sobre el reglamento deportivo de la FIA e integrarlo en una arquitectura de seis sub-agentes especializados coordinados por un orquestador supervisor, compatible con modelos de lenguaje locales y en la nube, de forma que las predicciones de los modelos, el análisis de radio y las referencias reglamentarias confluyan en una recomendación estratégica que exponga no solo la decisión final sino las inferencias de cada modelo, el razonamiento de cada sub-agente y los artículos del reglamento que la sustentan.
4. Implementar un motor de simulación vuelta a vuelta que integre los componentes anteriores en un flujo completo de reproducción de carrera, distribuido a través de tres superficies de uso (interfaz de línea de comandos, réplica de carrera con Arcade (Craven & Arcade contributors, 2025) y panel de análisis post-carrera con Streamlit (Snowflake Inc. & Streamlit contributors, 2025)), y validar el sistema de extremo a extremo sobre carreras de la temporada 2025, reportando las métricas de cada modelo predictivo por grupo de circuito y comparando las recomendaciones del orquestador con las decisiones reales de los equipos.

1.4 Antecedentes

Este Trabajo de Fin de Grado tiene su origen en un proyecto previo desarrollado en solitario por el autor al cierre del segundo cuatrimestre del tercer curso del Grado en Ingeniería en Sistemas Inteligentes. Aquel proyecto se planteó como trabajo final conjunto de las asignaturas de *Procesamiento del Habla y el Lenguaje Natural*, *Aprendizaje Automático Avanzado*, *Sistemas Inteligentes* y *Visión por Computador*, con libertad para elegir el dominio sobre el que articular las cuatro disciplinas. La Fórmula 1 se seleccionó tanto por el interés personal como por ofrecer un corpus de datos públicos lo suficientemente rico para sostener un trabajo común, en un dominio cuyo tratamiento académico abierto e integrado era aún escaso. El sistema resultante (Vega Sobral, 2025) se limitaba a los datos del Gran Premio de Barcelona de 2023,

circuito tradicionalmente empleado por los equipos como banco de pruebas polivalente por la variedad de su trazado, que combina rectas largas, curvas abiertas y cerradas y secciones de radio corto y largo, y cuyo comportamiento se considera razonablemente extrapolable al resto del calendario. Aquel sistema integraba un modelo primitivo de tiempos por vuelta y de degradación de neumáticos, un flujo de análisis de radio restringido al reconocimiento de intención, entidades y sentimiento, y una red neuronal capaz de identificar en vídeo el equipo al que pertenecía cada monoplace, trazando sobre ellos una línea que estimaba el intervalo de distancia y tiempo que los separaba. Aquellos primeros modelos sentaron las bases conceptuales de los empleados actualmente, particularmente en la predicción de tiempos por vuelta, la degradación de neumáticos y el análisis de radio. Las predicciones numéricas de tiempo y degradación confluían en un sistema experto de lógica condicional que, combinado con la telemetría recibida, emitía recomendaciones rudimentarias; los módulos de radio y visión, en cambio, operaban de forma desacoplada de ese bucle y no inyectaban sus inferencias en las decisiones tomadas durante la propia carrera.

Sobre esas bases, el sistema actual crece en datos y en modelos. El corpus se extiende a las temporadas completas de 2023 y 2024, y los circuitos se agrupan en cuatro arquetipos por *clustering* no supervisado, corroborado con fuentes externas fiables, para mitigar la variabilidad entre trazados y evitar el entrenamiento de un modelo por circuito. La familia predictiva, reducida en el informe *legacy* a dos modelos agregados, crece hasta los siete recogidos en los objetivos específicos, con novedades como el tratamiento de la degradación por compuesto con mapeo explícito a la nomenclatura Pirelli C1–C5 que sustituye al modelo primitivo. El sistema experto de lógica condicional, por su parte, cede el lugar a la arquitectura multi-agente descrita en el objetivo tercero, cuyo orquestador supervisor integra, esta vez sí, las inferencias de radio y del resto de módulos dentro del propio bucle de decisión vuelta a vuelta, cerrando así el hueco que el sistema experto dejaba sin cubrir. El módulo de visión por computador queda fuera de esta nueva versión, al no disponer de vídeo en directo ni de grabaciones completas de cada carrera que permitan emparejar sus detecciones con los datos reales de telemetría en ese instante, lo que limita su utilidad práctica dentro del alcance abordado.

El sistema actual hereda además de un segundo proyecto previo. Durante el primer cuatrimestre del cuarto curso, en el marco de la asignatura *Sistemas Interactivos Inteligentes*, el autor desarrolló junto a su compañero de grado Santiago Souto Ortega el proyecto F1 Telemetry Manager (Vega Sobral & Souto Ortega, 2025), que introdujo un panel de análisis de telemetría con chat multimodal de voz y texto sobre Streamlit. La superficie Streamlit del sistema actual nace de la unión entre los módulos heredados del proyecto *legacy* y aquel panel conversacional, ampliado con un flujo de voz renovado, la generación del informe de análisis post-carrera y la posibilidad de alternar entre modelos de lenguaje en la nube y locales. El repositorio de F1 Telemetry Manager se integra como submódulo Git bajo `src/telemetry/`, heredando únicamente los componentes elaborados por el autor; las capas de almacenamiento en la nube y de gestión de usuarios, responsabilidad del coautor, quedan fuera del alcance abordado.

1.5 Metodología

La metodología seguida responde a un enfoque de ingeniería aplicada, no de investigación teórica pura. El trabajo arranca con una fase común de ingeniería de datos, recogida en los cuadernos N01–N04, dedicada a descargar el corpus de las temporadas 2023–2025 desde las *APIs* públicas FastF1 y OpenF1, realizar un análisis exploratorio general sobre el conjunto consolidado, agrupar los circuitos por *clustering* no supervisado y derivar las características compartidas entre los modelos posteriores. A partir de esa base, para cada modelo desarrollado se recorre el mismo ciclo completo antes de pasar al siguiente, en lugar de acometer por separado todas las etapas de cada disciplina. Dicho proceso comprende, en primer lugar, el análisis exploratorio específico del modelo, la limpieza y preparación de los datos, la ingeniería

Tabla 1.1: Fases metodológicas del desarrollo del sistema, con entregables principales y cuadernos asociados.

Fase	Periodo	Entregable principal	Cuadernos
1	Feb 2026	Corpus 2023–2025 publicado en HuggingFace Hub, análisis exploratorio general, <i>clustering</i> de circuitos e ingeniería de características compartida	N01–N04
2	Feb–Mar 2026	Tiempos por vuelta y degradación por compuesto de neumático (TCN)	N05–N10
3	Mar 2026	Adelantamiento, coche de seguridad, duración de parada y éxito del undercut	N11–N16
4	Mar 2026	Procesamiento de radio (transcripción, sentimiento, intención, NER) y <i>parser</i> de mensajes de dirección de carrera	N17–N24
5	Abr 2026	Sistema multi-agente con orquestador y RAG sobre reglamento FIA	N25–N31
6	Abr 2026	Migración del código de los cuadernos al paquete <code>src/</code> y CLI distribuible	–
7	Abr 2026	Panel Streamlit ampliado y réplica de carrera en Arcade	–
8	Abr–May 2026	Pruebas de extremo a extremo comparando con casos reales y entrega de la memoria	N32, N34

de características específica, el análisis de correlaciones y la selección de variables. Posteriormente, se procede al entrenamiento iterativo y a la búsqueda de hiperparámetros, mediante *grid search* u optimización bayesiana según el caso, con incursiones puntuales de investigación cuando una decisión metodológica lo requiere, como la elección de estrategias de normalización o de ventanas temporales para mitigar la deriva conceptual entre temporadas. Todo este trabajo se desarrolla en cuadernos Jupyter numerados (N01–N34) con títulos descriptivos, cuyo orden reconstruye la cronología del proyecto. Contienen además figuras y explicaciones en Markdown reutilizadas en el Capítulo 4 y en el Capítulo 5. Una vez un modelo alcanza su versión estable en el cuaderno, su código se extrae a módulos Python del paquete `src/`, junto con los pesos serializados y los artefactos de preprocesado necesarios para su inferencia.

A partir de estos módulos, la entrega al usuario se articula en tres superficies desarrolladas en orden incremental. En primer lugar, se construyó una interfaz de línea de comandos (CLI) como producto mínimo viable, para validar el flujo del sistema extremo a extremo. Posteriormente, se amplió el panel heredado de Streamlit, orientado al análisis post-carrera con chat multimodal. Finalmente, se desarrolló la réplica de carrera sobre Arcade como superficie principal, en la que converge la riqueza completa del sistema. Cada superficie se apoya en la anterior una vez estable, prolongando al plano de entrega el mismo enfoque incremental que ya guiaba el trabajo en los cuadernos. Todo el código se distribuye como proyecto abierto bajo licencia Apache 2.0, repartido entre los repositorios `F1_Strat_Manager` y `F1_Telemetry_Manager`, este último integrado como submódulo. El avance del proyecto se articula mediante *releases* sucesivas etiquetadas en GitHub, que dejan una traza cronológica pública de cada hito del desarrollo. Los datos del corpus y los pesos entrenados, por su parte, se publican en HuggingFace Hub ([VforVitorio/f1-strategy-dataset](https://huggingface.co/VforVitorio/f1-strategy-dataset)) y se descargan en la primera ejecución, lo que evita versionar archivos pesados en Git y permite reejecutar el sistema sin reentrenar los modelos. La Tabla 1.1 recoge la cronología completa del desarrollo, con la correspondencia entre fases, cuadernos y versiones publicadas.

La partición del corpus por temporadas es la decisión metodológica más delicada del trabajo, pues condiciona directamente la capacidad de generalización de los modelos. Por esa razón, los datos de la temporada 2022 se excluyen de forma deliberada. Al tratarse del primer año del ciclo reglamentario de efecto suelo, los monoplazas todavía no habían consolidado su rendimiento, de modo que el salto de tiempos entre 2022 y 2023 resulta exponencial frente a la evolución incremental entre las temporadas siguientes e introduciría en el entrenamiento una distribución ajena al régimen estable que se busca modelar. En cambio, las temporadas 2023 y 2024, ya asentadas dentro del mismo ciclo, se emplean como conjunto de entrenamiento. La temporada 2025, por su parte, se reserva como conjunto de prueba por ser el último año del ciclo y, por

tanto, aquel en el que la pendiente de mejora respecto a 2024, tanto interanual como a lo largo de la propia temporada, es la más reducida, lo que minimiza el desfase distribucional entre entrenamiento y prueba. El detalle cuantitativo se retoma en el Capítulo 5.

1.6 Organización del documento

El resto de la memoria se organiza en cinco capítulos y tres anexos. El Capítulo 2 abre con una contextualización del dominio de la Fórmula 1 (estructura de una carrera, familia de neumáticos, eventos de pista y conceptos estratégicos como el undercut) antes de fundamentar los bloques teóricos del sistema: aprendizaje automático sobre series temporales, deriva conceptual, procesamiento del lenguaje natural, generación aumentada por recuperación y arquitecturas multi-agente basadas en modelos de lenguaje. El Capítulo 3 revisa el estado actual de la inteligencia artificial aplicada a la estrategia de Fórmula 1, en equipos profesionales y en la literatura académica, y deriva de él los requisitos funcionales y no funcionales del sistema. El Capítulo 4 detalla la arquitectura y la implementación: ingeniería de datos, familia de modelos predictivos, flujo de procesamiento de radio, subsistema RAG sobre el reglamento deportivo y coordinación multi-agente bajo un orquestador supervisor. El Capítulo 5 reporta las métricas obtenidas por modelo y arquetipo de circuito, evalúa el sistema de extremo a extremo sobre carreras de 2025 y contrasta sus recomendaciones con los *baselines* de referencia. El Capítulo 6 recoge el grado de cumplimiento de los objetivos, las aportaciones principales y las líneas de trabajo futuro. Cierran la memoria tres anexos: una declaración del uso de inteligencia artificial generativa durante la elaboración del trabajo, el manual de instalación y uso del sistema, y un anexo con material complementario.

2 MARCO TEÓRICO

Debes desaprender lo aprendido.

— Maestro Yoda, *El Imperio Contraataca* (1980)¹

Este capítulo recoge los fundamentos teóricos en los que se apoya el sistema, comenzando por una introducción al dominio de la Fórmula 1, aspecto clave para comprender la terminología empleada en el resto de la memoria. A continuación se desarrollan las bases teóricas de cada componente y se introducen los conceptos más relevantes asociados a cada uno de ellos: la deriva conceptual, el *gradient boosting*, las redes convolucionales temporales, diferentes áreas del procesamiento del lenguaje natural, la generación aumentada por recuperación y la orquestación multi-agente, entre otras. Para cada bloque se presentan además las distintas técnicas analizadas y evaluadas durante el desarrollo del proyecto, con énfasis en las finalmente seleccionadas y utilizadas.

2.1 Contextualización de la Fórmula 1 y decisiones estratégicas

Esta sección se organiza en cinco bloques, ordenados de lo más concreto a lo más general. Primero, se introduce el concepto de "fin de semana de carrera" se describe su estructura, junto con la familia de neumáticos Pirelli, continuando con los eventos de pista. Posteriormente, se definen algunos de los conceptos estratégicos claves que emplean los equipos. Finalmente se cierra con el marco de la carrera como problema de decisión bajo incertidumbre, con el objetivo de servir de puente hacia el resto del capítulo.

2.1.1 Estructura de un fin de semana de carrera

Un Gran Premio de Fórmula 1 se disputa a lo largo de un fin de semana de tres días. El viernes se celebran dos sesiones de entrenamientos libres (referidos como "libres.^{en} el resto del texto), el sábado la tercera y la sesión de clasificación, y el domingo la carrera (Fédération Internationale de l'Automobile, 2024). Este formato se altera en seis de las veinticuatro citas del calendario, donde la segunda sesión libre se sustituye por una clasificación *sprint* (formato corto que reordena el orden de salida de una carrera reducida disputada el sábado), se añade una carrera *sprint* de unos cien kilómetros el sábado con puntuación propia y la clasificación principal se adelanta al viernes. La carrera cubre una distancia objetivo de 305 km, salvo en Mónaco donde ronda los 260 km, y puntúa a los diez primeros clasificados con la secuencia 25-18-15-12-10-8-6-4-2-1, una vez eliminado a partir de 2025 el punto adicional por vuelta rápida que estuvo vigente entre 2019 y 2024.

A esta estructura se superpone el régimen de *parc fermé*, que entra en vigor al iniciarse la clasificación y prohíbe a los equipos modificar la configuración del coche de cara a la carrera, con lo que las decisiones tomadas durante los libres quedan bloqueadas hasta el final del domingo. Desde 2025, este régimen admite una excepción en los fines de semana *sprint*: tras la carrera *sprint* del sábado los equipos pueden reajustar la configuración antes de la clasificación principal, a partir de la cual el *parc fermé* vuelve a aplicarse hasta el final de la carrera del domingo. A ello se suma la gestión del cupo de componentes: cada pieza crítica del monoplaza, como el motor o la caja de cambios, dispone de un número limitado de usos durante la

¹Diálogo entre el maestro Yoda y Luke Skywalker durante el entrenamiento Jedi en Dagobah. Traducción propia del original en inglés: *You must unlearn what you have learned.*

temporada, y superar esa cuota implica sanciones automáticas en la parrilla de salida, lo que convierte la elección de configuraciones del viernes en decisiones de alto coste.

2.1.2 Neumáticos y degradación

Pirelli (proveedor oficial de neumáticos de la Fórmula 1) suministra los neumáticos de toda la parrilla y los clasifica en tres compuestos secos, etiquetados como blando, medio y duro, y dos para condiciones de lluvia, intermedio y extremo (Pirelli Motorsport, 2025b). Las etiquetas de los compuestos secos son relativas y cambian en cada Gran Premio: el catálogo real consta de cinco compuestos identificados como C1 a C5, del más duro al más blando, de los cuales Pirelli selecciona tres para cada fin de semana y los presenta a los equipos bajo las etiquetas blando, medio y duro. Un neumático etiquetado como blando en un Gran Premio puede ser el C3 y en otro el C5, de modo que su respuesta de degradación no es directamente comparable entre circuitos. En 2025, Pirelli amplió este catálogo con un sexto compuesto, el C6, aún más blando que el C5 y reservado a circuitos de baja exigencia para el neumático como Mónaco, Imola, Montreal y Bakú. Por su introducción reciente y la escasez de vueltas disponibles, el sistema desarrollado en este trabajo se entrena únicamente con el catálogo histórico C1-C5 y deja el C6 como extensión futura. La selección de compuestos para cada Gran Premio se realiza en función de la abrasión del asfalto, la carga lateral del trazado, la tipología de curvas y las temperaturas esperadas durante el fin de semana, y se publica oficialmente con antelación en el portal de prensa del fabricante. Los compuestos de lluvia, en cambio, no son elegidos por el equipo: entran en pista únicamente cuando las condiciones meteorológicas así lo exigen.

Dentro de una carrera, el tramo rodado con el mismo juego de neumáticos se denomina *stint*. La degradación de los neumáticos no sigue una curva lineal: se mantiene contenida durante la mayor parte del stint y se acelera de forma abrupta al rebasar cierto umbral, punto conocido como *cliff*, a partir del cual el tiempo por vuelta cae varios segundos y la adherencia lateral se deteriora bruscamente. El combustible, a diferencia de la degradación, evoluciona de manera predecible: los monoplazas arrancan la carrera con una masa máxima de 110 kg establecida por el reglamento técnico de la FIA (Fédération Internationale de l'Automobile, 2025), y su consumo se descuenta vuelta a vuelta a un ritmo aproximadamente lineal, lo que permite al sistema integrarlo implícitamente en la referencia de ritmo de la vuelta anterior sin necesidad de modelarlo como variable independiente. Por último, el reglamento deportivo obliga a utilizar al menos dos compuestos secos distintos durante la carrera, salvo si la sesión se declara oficialmente mojada (Fédération Internationale de l'Automobile, 2024), con la excepción del Gran Premio de Mónaco, donde la FIA introdujo en 2025 una regla de dos paradas obligatorias que fuerza al menos tres stints con compuestos distintos para recuperar variedad estratégica en un circuito especialmente propenso a carreras monótonas (Formula 1, 2025b).

2.1.3 Eventos de pista

La carrera se desarrolla bajo la supervisión continua de la dirección de carrera, que dispone de un conjunto de dispositivos para regular el ritmo, neutralizar incidentes y sancionar conductas contrarias al reglamento (Fédération Internationale de l'Automobile, 2024). El mecanismo más visible es el coche de seguridad (SC), que sale a pista tras un accidente o una caída grave de adherencia para encabezar el pelotón a baja velocidad y restablecer unas condiciones de pista seguras, con la consiguiente neutralización de los intervalos entre pilotos. Cuando la zona comprometida es reducida y basta con ralentizar el conjunto del pelotón sin coche físico en pista, la FIA puede recurrir en su lugar a un coche de seguridad virtual (VSC), que obliga a todos los participantes a rodar sobre un delta de tiempo fijado. Ambos mecanismos tienen un impacto estratégico muy alto, debido a que la bajada de ritmo del pelotón provoca que una parada ejecutada durante su despliegue solo pierda una fracción del tiempo habitual. Complementando a los coches de seguridad se encuentra el sistema de banderas, que señalan condiciones puntuales de pista mediante un

código cromático: verde para pista limpia y segura, amarilla para precaución local y roja para neutralización total de la sesión. También existen banderas relacionadas con conductas de los pilotos, como la bandera azul para ordenar a un piloto que va a ser doblado que ceda el paso a los coches más rápidos, blanca para advertir de un vehículo lento y negra con franja blanca para avisar al piloto de una conducta antideportiva antes de aplicar sanción formal.

En contraste con estos instrumentos reguladores existe un mecanismo dedicado a favorecer los adelantamientos: el sistema de reducción de resistencia aerodinámica (DRS, por sus siglas en inglés), que permite al piloto que rueda a menos de un segundo del coche precedente abrir el alerón trasero en las zonas señalizadas del circuito, reduciendo así su carga aerodinámica y aumentando su velocidad punta en las rectas. No obstante, su activación está condicionada a varios requisitos: solo puede utilizarse a partir de la segunda vuelta, tanto al comienzo de la carrera como tras el final de un período de coche de seguridad, y permanece totalmente suspendida mientras haya pista mojada o se muestren banderas amarillas dobles.

Al margen de los dispositivos reguladores, la propia pista puede imponer condiciones que reconfiguran la carrera. El cambio meteorológico es el más disruptivo: el inicio o el final de la lluvia obliga a sustituir el compuesto seco por el intermedio o el extremo, y el retorno a pista seca tras un chubasco activa el llamado *crossover*, el punto en el que los neumáticos de mojado pasan a ser más lentos que los de seco y se hace rentable volver a parar. Variables secundarias como la temperatura del asfalto y el viento dominante modulan además la evolución de la degradación a lo largo de la carrera. La dirección de carrera también sanciona las infracciones cometidas durante la sesión mediante distintos tipos de penalización: el *drive-through* obliga a atravesar el *pit lane* (calle de boxes) sin detenerse, el *stop-and-go* añade diez segundos de parada sin asistencia técnica, el recargo de cinco o diez segundos se aplica en la siguiente parada o al cronómetro final, y la sanción en parrilla se traslada a la carrera siguiente, habitualmente asociada al consumo excesivo del cupo de componentes ya mencionado. Estas sanciones tienen un impacto estratégico desproporcionado respecto a su tiempo nominal: un castigo de cinco segundos puede reabrir o cerrar la ventana de adelantamiento tras la siguiente parada, y una *drive-through* obliga a anticipar un paso por boxes que reordena toda la estrategia prevista de dos compuestos. Por último, los límites de pista, marcados por líneas blancas sobre el asfalto, se vigilan de forma automática: rebasarlos con las cuatro ruedas durante la clasificación invalida la vuelta registrada, y en carrera acumular tres avisos activa una penalización de tiempo, con castigos crecientes ante reincidencias.

2.1.4 Conceptos estratégicos

La estrategia de carrera se articula en torno a un repertorio reducido de maniobras y fenómenos que permiten alterar el orden entre pilotos, en numerosas ocasiones incluso sin necesidad de adelantamientos directos en pista. La maniobra canónica se conoce como *undercut*, donde el piloto se adelanta a la parada del rival para montar un neumático fresco y aprovechar las vueltas siguientes (idealmente en aire limpio) en las que el rival sigue degradando el compuesto antiguo, para tener un mayor ritmo y conseguir superarle cuando este salga de boxes (Brawn & Parr, 2016). En sentido opuesto, el *overcut* alarga deliberadamente el *stint*, ya sea para aprovechar el tiempo que el rival necesita para poner a temperatura el neumático nuevo tras su parada, ya sea para explotar el ritmo que pierde si queda atrapado en tráfico. El rango de vueltas en el que alguna de estas dos maniobras es viable se denomina *pit window* y queda determinado por la degradación prevista del compuesto actual, la distancia restante a meta y el comportamiento esperado de los rivales inmediatos. Esta ventana no es estática: un coche de seguridad desplegado la colapsa bruscamente al reducir el coste de parar, mientras que una penalización aplicada a mitad de carrera puede obligar a anticiparla o a prescindir de ella por completo.

A estas maniobras se superponen varios fenómenos y decisiones que condicionan cualquier estrategia. Los dos fenómenos físicos principales son la evolución progresiva del agarre del asfalto (*track evolution*) a lo largo del fin de semana por acumulación de restos de goma en la trayectoria óptima, y la distinción entre aire limpio y aire sucio: este último, producido por las turbulencias del coche precedente, reduce la carga aerodinámica disponible, eleva la temperatura del neumático y puede hacer estratégicamente infructuosa una parada si el piloto emerge de boxes encerrado en tráfico. Por ello, tanto el undercut como el overcut exigen una ventana de varias vueltas en aire limpio tras la parada y un delta de ritmo suficiente para compensar la pérdida de tiempo en el pit lane. Su peso estratégico depende fuertemente del trazado: en circuitos difíciles de adelantar como Mónaco, Singapur o Hungría se convierten en la herramienta principal para ganar posición, mientras que en trazados rápidos con zonas DRS largas como Monza o Bakú pierden protagonismo frente al adelantamiento directo en pista. La elección entre una y otra maniobra responde además al perfil de degradación del circuito: los trazados con alta degradación, como Singapur, favorecen al undercut porque el neumático fresco ofrece una ventaja inmediata, mientras que los de baja degradación y superficie poco agresiva, como Mónaco, tienden a favorecer al overcut, ya que el neumático nuevo tarda más en alcanzar su ventana térmica y conviene prolongar el stint cuando el rival ha parado. A ello se añaden las decisiones previas a la salida, como el compuesto de partida, el número de paradas previstas (habitualmente una o dos en seco, con excepciones reglamentarias como Mónaco) y los supuestos sobre el comportamiento esperado de cada compuesto, que a veces se ven contradichos durante la sesión por rendimientos inesperadamente buenos o malos y obligan a replantear la estrategia, situación agravada cuando se suma un cambio meteorológico con su correspondiente crossover.

2.1.5 La carrera como problema de decisión bajo incertidumbre

Desde una perspectiva teórica, el planteamiento anterior puede formalizarse como un problema de decisión secuencial bajo incertidumbre parcialmente observable: el estratega toma decisiones acotadas en el tiempo sobre un sistema del que solo ve una parte (su propio coche, parte del pelotón y algunos mensajes) y cuya evolución depende tanto de factores deterministas, como el reglamento y el régimen de parc fermé, como estocásticos, tales como la aparición de un coche de seguridad o la degradación inesperada de un compuesto. La heterogeneidad de variables expuesta en las secciones anteriores obliga a combinar modelos con dinámicas y granularidades distintas, de modo que la integración multi-modelo no es una elección estilística sino una exigencia estructural del escenario.

La literatura académica sobre este tipo de retos se apoya sobre todo en simulaciones Monte Carlo aplicadas a series temporales de tiempos por vuelta (Heilmeyer et al., 2020a), un enfoque que captura con precisión la mecánica de compuestos y paradas pero deja fuera las señales textuales y el contexto normativo que en la práctica condicionan las decisiones reales. En el plano industrial la diferencia es notable: los equipos de primer nivel operan con varios cientos de sensores por monoplace que generan del orden de gigabytes de telemetría propia por Gran Premio (McLaren Racing, 2024), y algunos han escalado la simulación estratégica hasta el orden de cien millones de combinaciones Monte Carlo por carrera, como se documentó para Red Bull en Hungría 2022 (GrandPrix247, 2024), si bien la lógica interna y la infraestructura de estos sistemas permanecen cerradas al análisis externo. Entre ese enfoque académico acotado a lo numérico y este entorno industrial opaco por confidencialidad queda un espacio intermedio, resumido en la Figura 2.1, que este capítulo articula con los fundamentos teóricos necesarios para coordinar modelos heterogéneos y adaptarlos a las variaciones entre temporadas, empezando por la deriva conceptual.

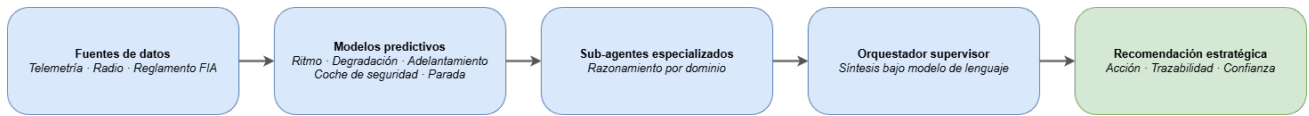


Figura 2.1: Vista conceptual simplificada del bucle de decisión del sistema. Las fuentes de datos (telemetría, radio y reglamento) alimentan modelos predictivos especializados cuyas salidas son consumidas por sub-agentes coordinados por un orquestador que emite la recomendación estratégica final. La arquitectura completa, con los modelos y agentes concretos, se detalla en el capítulo 4.

2.2 Deriva conceptual en series temporales deportivas

En el ámbito del aprendizaje automático supervisado, se denomina deriva conceptual (*concept drift*) al cambio que experimenta la distribución conjunta $P(X, y)$ de las variables de entrada y la variable objetivo entre el período empleado para entrenar el modelo y aquel en el que se despliega (Hovakimyan & Bravo, 2024). Es decir, la relación que el modelo aprendió entre lo que observa y lo que debe predecir deja de ser válida en producción, de modo que su rendimiento se degrada incluso cuando los datos de entrada parecen formalmente similares a los del entrenamiento. Conviene diferenciarla de dos fenómenos vecinos: la deriva de covariables (*covariate shift*), donde solo cambia la distribución marginal de las entradas $P(X)$ mientras la distribución condicional $P(y|X)$ permanece estable, situación en la que el modelo recibe ejemplos con un perfil distinto al habitual pero la regla que los conecta con la salida sigue siendo correcta. Y la deriva de etiquetas (*label shift*), donde se altera la frecuencia relativa de los valores objetivo sin modificar su relación funcional con las entradas, lo que se traduce en clases o magnitudes que aparecen en proporciones distintas a las del conjunto de entrenamiento aunque su correspondencia con las entradas no haya cambiado (Evidently AI, 2025).

La literatura distingue cuatro modalidades temporales de presentación (Hovakimyan & Bravo, 2024). La modalidad gradual avanza con continuidad a lo largo de un horizonte prolongado, como ilustra el desplazamiento del léxico habitual de un idioma a lo largo de las décadas: ningún día concreto marca el cambio, pero un modelo entrenado con textos de los años setenta clasifica peor el habla actual de lo que lo haría un modelo entrenado con textos contemporáneos (Hamilton et al., 2016). La modalidad incremental se manifiesta por escalones cuya separación es del orden de la propia escala temporal del problema, como ocurre en los detectores de fraude bancario, donde las técnicas de los atacantes se renuevan en oleadas sucesivas y el modelo debe acomodar cada peldaño antes de afrontar el siguiente (Dal Pozzolo et al., 2018). La modalidad recurrente describe regímenes que alternan periódicamente entre estados conocidos, como la estacionalidad del consumo eléctrico residencial entre verano e invierno o los patrones diarios de tráfico urbano, donde el sistema vuelve una y otra vez a los mismos estados en un orden previsible (Gama et al., 2014). La modalidad súbita, por último, completa la transición entre dos regímenes en una ventana corta, como sucedió con los hábitos de consumo en marzo de 2020, cuando los confinamientos sanitarios reconfiguraron en pocos días las distribuciones aprendidas por los modelos de demanda comercial y obligaron a recalibrar flujos de trabajo completos en cuestión de semanas (Vela et al., 2022). Cualquier serie temporal observada bajo un marco normativo o tecnológico en evolución sufre alguna combinación de estas modalidades, y el reto metodológico no consiste en detectarlas a posteriori sino en anticiparlas desde el diseño del protocolo de entrenamiento y validación.

En la Fórmula 1 conviven dos escalas de deriva con orígenes muy distintos. La primera opera entre ciclos reglamentarios y se manifiesta de modo súbito: cada gran reforma del reglamento técnico, como el retorno al efecto suelo introducido en 2022, redefine las restricciones de diseño aerodinámico y rompe la estacionariedad respecto a temporadas anteriores (Fédération Internationale de l'Automobile, 2024, 2025). La segunda opera dentro de un mismo ciclo reglamentario y avanza de modo gradual: los equipos invierten su cupo asignado de túnel de viento y simulación mediante dinámica de fluidos computacional

(CFD) en sucesivas mejoras aerodinámicas a lo largo de la temporada, de modo que el ritmo en un mismo circuito tiende a aumentar de un año al siguiente. Dicho de otro modo, aunque el reglamento permanezca formalmente inalterado, el monoplaza que lo cumple deja de ser el mismo, y una distribución de tiempos por vuelta válida en una temporada ya no captura con fidelidad la población de la siguiente.

Este avance no es uniforme entre escuderías. Al contrario, su magnitud depende de la efectividad del proceso de desarrollo de cada equipo, lo que añade una segunda capa de heterogeneidad sobre el efecto reglamentario. El caso de McLaren ilustra el extremo de esta variabilidad, comenzando la temporada 2023 con un monoplaza MCL60 situado entre los menos competitivos de la parrilla, hasta el punto de provocar un cambio total de dirección técnica del equipo en marzo de ese año, pero la actualización aerodinámica introducida a partir del Gran Premio de Austria transformó al coche en uno de los dos más rápidos del campeonato y permitió a la escudería cerrar la temporada en cuarta posición de constructores (Noble & Somerfield, 2023). Sobre esa base, se proclamó campeona del mundo de constructores en 2024 tras veintiséis años sin lograrlo (Formula 1, 2024) y revalidó el título en 2025 con seis carreras de antelación (Formula 1, 2025a), un recorrido difícilmente compatible con un patrón monótono y homogéneo de deriva entre temporadas. El problema operacional que esta heterogeneidad le plantea al modelo es estructural y no aleatorio: un regresor entrenado sobre temporadas pasadas y expuesto a las venideras no se equivoca por ruido sino por desplazamiento sistemático en una dirección específica, porque la variable objetivo se mueve sin que las variables de entrada lo hagan en la misma dirección, lo que rompe el supuesto de generalización por muestreo independiente e idénticamente distribuido sobre el que se construyen los modelos tabulares clásicos (Gama et al., 2014).

El reconocimiento de la deriva conceptual exige descartar de partida la validación cruzada aleatoria, que distribuye observaciones de todos los períodos disponibles entre los conjuntos de entrenamiento y validación. En un dominio temporalmente no estacionario, esta práctica permite que el modelo vea durante el entrenamiento ejemplos posteriores en el tiempo a aquellos que después se le exigirá predecir, lo que filtra información del futuro hacia el pasado y subestima de modo sistemático el error que el modelo cometerá una vez desplegado. La alternativa metodológicamente legítima es la validación cruzada por bloques temporales, donde los conjuntos de entrenamiento, validación y prueba respetan estrictamente el orden cronológico y no se solapan, de manera que la evaluación reproduce la separación temporal real entre la fase de aprendizaje y la fase de inferencia (Bergmeir & Benítez, 2012). Esto es, el modelo solo se mide sobre temporadas que en el momento del entrenamiento todavía no existían, lo que devuelve una estimación honesta de su rendimiento ante la deriva.

Sobre este principio, el sistema sigue un protocolo en dos fases, resumido en la Figura 2.2. En la primera, los datos de la temporada 2023 se emplean para el entrenamiento y los de 2024 para la validación, lo que permite seleccionar los hiperparámetros de cada modelo bajo un escenario que reproduce a pequeña escala la situación real de despliegue. Una vez fijados, en la segunda fase el modelo se reentrena sobre el conjunto combinado 2023 y 2024, y la temporada 2025 queda íntegramente reservada como conjunto de prueba para la evaluación final. Este protocolo se aplica de forma homogénea en todos los modelos del sistema, cuya implementación concreta se detalla en el Capítulo 4. Su objetivo es controlar el efecto de la partición temporal en la estimación de rendimiento, pero no neutraliza por sí mismo el sesgo estructural que la deriva induce sobre las predicciones cuando llega el momento de operar sobre la nueva temporada. Para ello, hace falta complementarlo con una segunda capa de mitigaciones aplicadas dentro del propio modelo, que se describen a continuación.

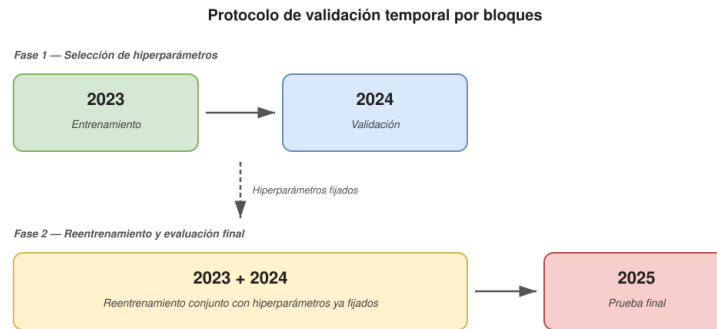


Figura 2.2: Protocolo de validación temporal por bloques en dos fases. Fase 1 (2023 y 2024) para selección de hiperparámetros, fase 2 con reentrenamiento sobre 2023-2024 y prueba final sobre 2025.

Las mitigaciones aplicadas en este trabajo abordan cada uno de los mecanismos de fallo identificados en el párrafo anterior, sin recurrir a detectores de deriva en línea. La primera, basada en anclas temporales explícitas, consiste en incorporar el año de la sesión como variable propia del vector de entrada, lo que permite al modelo aprender el desplazamiento entre temporadas como una dimensión más en lugar de tener que inferirlo a partir de las correlaciones residuales entre las demás variables. La segunda es la normalización relativa intra-sesión, donde determinadas variables vinculadas al ritmo se reformulan como cociente respecto a un referente de la propia sesión, típicamente el mejor tiempo limpio de la carrera, de modo que el descriptor resultante es invariante por construcción al nivel absoluto de *pace* y se mantiene comparable entre temporadas. Por ejemplo, si en 2025 todo el pelotón rueda un segundo más rápido que en 2023, el cociente de un piloto que va un 3 % por encima del ritmo de referencia se mantiene en 1,03 en ambas temporadas, mientras que su tiempo absoluto sí cambia. La tercera es la ponderación temporal exponencial del entrenamiento, que asigna un peso creciente a las observaciones más recientes mediante un esquema suave, sin descartar las temporadas pasadas, de modo que la frontera de decisión se calibra hacia la distribución actual sin perder la regularización que aportan los datos históricos. La normalización relativa se apoya en la idea de invariancia intra-serie popularizada por T. Kim et al. (2022), mientras que la ponderación temporal exponencial es un esquema clásico de *forgetting factors* adaptado al contexto de deriva gradual; ambas se inscriben en la línea de mitigación empírica que Y. Liu et al. (2023) discute para series temporales globales.

Durante el desarrollo del sistema se consideraron además dos alternativas más recientes y específicamente diseñadas para combatir la no estacionariedad en modelos secuenciales, ShiftS (Zhao et al., 2025) y TAFAS (H. Kim et al., 2025), junto con la familia RevIN (T. Kim et al., 2022). Las tres se descartaron por la complejidad adicional que introducen frente a la ganancia marginal esperada en un corpus del tamaño del aquí disponible, aunque su incorporación queda señalada como línea natural de extensión futura. Por último, el sistema renuncia deliberadamente a incorporar detectores de deriva en línea como ADWIN o DDM (Kraus & van der Aa, 2025), que están concebidos para flujos continuos de producción con cambios potencialmente súbitos. En la Fórmula 1, en cambio, la deriva relevante es gradual dentro de un ciclo reglamentario, el horizonte de despliegue es una temporada con veinticuatro eventos discretos y bien delimitados, y la combinación de la partición temporal por bloques con las tres mitigaciones descritas agota razonablemente el margen de mejora alcanzable sin acceso a las fuentes propietarias de los equipos. Sobre estos cimientos metodológicos, las siguientes secciones del capítulo describen las familias de modelos concretas que materializan estas decisiones, comenzando por el *gradient boosting*.

2.3 Gradient Boosting: XGBoost, LightGBM e HistGBT

El *gradient boosting* es una familia de algoritmos supervisados que construye un modelo predictivo como una suma ponderada de un número creciente de aprendices débiles, típicamente árboles de decisión poco profundos y entrenados secuencialmente para corregir los errores acumulados del conjunto anterior (Friedman, 2001). En cada iteración, se ajusta un árbol al negativo del gradiente de la función de pérdida sobre las predicciones del modelo hasta ese paso, lo que equivale a aplicar un descenso por gradiente directamente en el espacio de funciones. En otras palabras, el modelo final no aprende en un solo trazo, sino por correcciones acumuladas. Por tanto, el primer árbol captura la tendencia gruesa de los datos, el segundo refina los casos donde el primero se equivoca, el tercero los que aún resisten, y así sucesivamente hasta agotar el presupuesto de iteraciones definido. Esta dinámica explica por qué esta técnica suele dominar a las redes densas sobre datos tabulares heterogéneos con decenas o centenares de variables, escenario habitual de los conjuntos de entrenamiento empleados en este sistema. Conviene además situar esta familia dentro del paradigma del aprendizaje automático (AA) clásico basado en árboles, frente al aprendizaje profundo con redes neuronales al que el sistema recurre en las secciones siguientes para tareas que requieren modelar dependencias temporales largas o lenguaje natural.

Dentro de esta familia, el sistema emplea tres variantes con propiedades algorítmicas y roles diferenciados, sintetizadas en la Tabla 2.1. El modelo XGBoost (T. Chen & Guestrin, 2016) se utiliza para la regresión del tiempo por vuelta, donde resultan especialmente útiles las restricciones monotónicas sobre las variables anti-deriva descritas en el apartado anterior, además de técnicas de regularización explícitas como la L_1 y L_2 , que limitan el sobreajuste penalizando los pesos asignados a las hojas de los árboles. LightGBM (Ke et al., 2017) cubre las tres tareas de clasificación binaria del sistema, a saber, probabilidad de adelantamiento, probabilidad de despliegue de coche de seguridad en una ventana corta y éxito de un undercut. Fue el modelo elegido gracias a su tratamiento nativo de variables categóricas y su crecimiento por hojas, que encajan bien con la presencia de interacciones no lineales que estas tareas suelen exhibir. Finalmente, el HistGradientBoosting (Friedman, 2001) se reserva para los casos en los que importa caracterizar la dispersión de la salida y no solo su valor central, es decir, no se predice un único valor sino el rango plausible que la variable de salida puede tomar bajo las condiciones observadas. Tal es el caso de las paradas en boxes, donde se estima en forma de cuantiles $\alpha = 0,05/0,50/0,95$ mediante la pérdida *pinball* nativa de la implementación, una función asimétrica que penaliza de modo distinto los errores por encima y por debajo del cuantil objetivo.

La elección del *gradient boosting* como familia principal para los componentes tabulares del sistema responde a tres ventajas operativas que las redes densas no ofrecen con la misma facilidad, siendo la primera de ellas la interpretabilidad mediante SHAP (Lundberg & Lee, 2017). Esta técnica, propia de la teoría de juegos, descompone cada predicción del modelo en contribuciones aditivas atribuibles a cada variable de entrada, permitiendo auditar el razonamiento detrás de la salida. Se trata de un requisito imprescindible para que la recomendación final del orquestador sea trazable hasta el componente que la genera. La segunda es la calibración probabilística mediante una regresión logística de un parámetro (Platt, 1999), ajustada sobre el conjunto de validación de 2024 que transforma las puntuaciones o *scores* crudos de los clasificadores LightGBM en probabilidades comparables a la frecuencia empírica observada, condición necesaria para que las salidas de los clasificadores de adelantamiento (N12), coche de seguridad (N14) y undercut (N16) puedan agregarse coherentemente para la simulación Monte Carlo del orquestador. La tercera es la regresión por cuantiles nativa de HistGradientBoosting, ya descrita en el párrafo anterior, que devuelve la dispersión de la predicción sin necesidad de modelos auxiliares ni supuestos paramétricos sobre la distribución de la salida.

Tabla 2.1: Comparativa de las tres variantes de *gradient boosting* empleadas en el sistema.

Propiedad	XGBoost (T. Chen & Guestrin, 2016)	LightGBM (Ke et al., 2017)	HistGradientBoosting (Friedman, 2001)
Crecimiento del árbol	Por niveles (<i>level-wise</i>) con <code>tree_method=hist</code> ; expansión simétrica controlada por <code>max_depth</code>	Por hojas (<i>leaf-wise</i>) sobre la hoja con mayor reducción de pérdida; controlado por <code>num_leaves</code>	Por niveles sobre histogramas precomputados; controlado por <code>max_leaf_nodes</code> y <code>max_depth</code>
Discretización de variables	Histograma aproximado por percentiles ponderados; un <i>bin</i> por característica	Histograma con <i>Gradient-based One-Side Sampling</i> (GOSS) y <i>Exclusive Feature Bundling</i> (EFB) sobre variables dispersas	Binning determinista en como máximo 255 <i>bins</i> por característica, calculado una sola vez en preprocesado
Regresión por cuantiles nativa	No de forma directa; requiere objetivos personalizados o modelos auxiliares	Sí, mediante <code>objective="quantile"</code> con parámetro <code>alpha</code>	Sí, mediante <code>loss="quantile"</code> con parámetro <code>quantile</code> ; pérdida <i>pinball</i> integrada
Regularización	L1 (<code>reg_alpha</code>) y L2 (<code>reg_lambda</code>); poda por ganancia mínima (<code>gamma</code>); restricciones monotónicas y de interacción	L1, L2, <code>min_child_samples</code> , <code>min_split_gain</code> ; soporte categórico nativo y restricciones monotónicas	L2 (<code>l2_regularization</code>); <code>min_samples_leaf</code> , <code>max_leaf_nodes</code> ; restricciones monotónicas
Interpretabilidad	SHAP (<i>TreeExplainer</i>); importancia por ganancia, cobertura y frecuencia	SHAP (<i>TreeExplainer</i>); importancia por ganancia y <i>split</i>	SHAP (<i>TreeExplainer</i>); <code>permutation_importance</code> de scikit-learn
Calibración probabilística	Aplicable mediante Platt o isotónica sobre la salida <code>predict_proba</code> ; no usada en este trabajo	Platt (regresión logística sobre validación 2024) en N12, N14 y N16	No procede (regresor); cobertura empírica controlada vía cuantiles P05/P50/P95
Rol en el sistema	Regresión del tiempo por vuelta	Clasificación binaria (adelantamiento, coche de seguridad, undercut)	Regresión por cuantiles de la duración de parada

Estas tres propiedades, sumadas al rendimiento general del boosting sobre datos tabulares con interacciones no lineales, justifican que la primera capa predictiva del sistema descansa íntegramente en esta familia, con dos excepciones. La primera es el modelo de degradación de neumáticos, cuya estructura temporal continua y prolongada cae fuera del régimen para el que el boosting fue diseñado y exige la arquitectura que se introduce a continuación. La segunda es el procesamiento de las comunicaciones de radio, que opera sobre lenguaje natural y se aborda más adelante en el capítulo.

2.4 Redes convolucionales temporales

El horizonte temporal sobre el que opera el modelo de degradación de neumáticos abarca habitualmente entre quince y cuarenta vueltas por stint, según la duración de carrera y el compuesto montado. Para esta clase de horizontes, la familia canónica en aprendizaje profundo es la de las redes neuronales recurrentes (RNN, por sus siglas en inglés), que mantienen un estado oculto actualizado en cada paso de tiempo y propagan información a lo largo de la secuencia. Sus dos variantes más empleadas son la memoria a largo y corto plazo (*long short-term memory*, LSTM) (Hochreiter & Schmidhuber, 1997), que introduce un sistema de puertas de entrada, salida y olvido para controlar qué información retener o descartar, y la unidad recurrente con puerta (*gated recurrent unit*, GRU) (Cho et al., 2014), que simplifica el diseño anterior reduciendo el número de puertas a dos con rendimiento comparable y coste computacional menor. Ambas mitigan el problema clásico del *vanishing gradient*, que en las redes recurrentes simples impide aprender dependencias separadas por muchos pasos en el tiempo, pero no lo eliminan. También comparten dos limitaciones operativas relevantes para este sistema, siendo la primera la ejecución durante su entrenamiento, secuencial por construcción y no paralelizable a lo largo del eje temporal; y la segunda el campo efectivo del modelo, que depende de un equilibrio frágil entre el tamaño del estado oculto y la longitud de la ventana

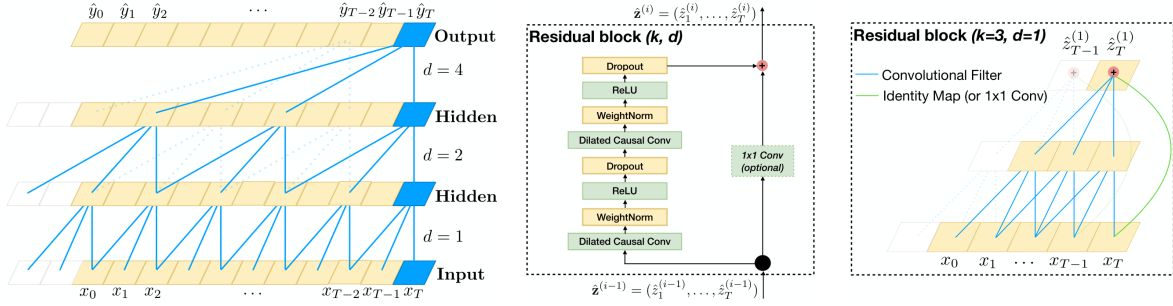


Figura 2.3: Componentes arquitectónicos de una TCN. (a) Convolución causal dilatada con factores de dilatación $d = 1, 2, 4$ y tamaño de filtro $k = 3$, donde el campo receptivo cubre la totalidad de la secuencia de entrada. (b) Bloque residual TCN. (c) Ejemplo de conexión residual con convolución 1×1 cuando la entrada y la salida tienen dimensiones distintas. Reproducido de Bai et al. (2018, Fig. 1). Los tres paneles se leen de izquierda a derecha: el panel izquierdo corresponde a (a), el central a (b) y el derecho a (c).

de entrenamiento. En el proyecto previo del autor (Vega Sobral, 2025) se evaluaron LSTM y GRU sobre el problema de degradación con un rendimiento modesto, y esa observación experimental es la que motiva en este trabajo el cambio a la familia que se introduce a continuación.

Las redes convolucionales temporales (*temporal convolutional networks*, TCN) (Bai et al., 2018) ofrecen una alternativa que conserva el sesgo inductivo temporal de las arquitecturas recurrentes pero esquiva las dos limitaciones operativas señaladas en el párrafo anterior. Su funcionamiento se apoya en tres bloques internos, ilustrados en la Figura 2.3. El primero son las convoluciones causales, en las que la salida en el instante t depende exclusivamente de entradas en instantes $\leq t$, lo que garantiza por construcción que el modelo no incorpore información futura al producir una predicción y lo hace apto para la inferencia vuelta a vuelta en vivo. El segundo son las convoluciones dilatadas, donde el filtro se aplica sobre la secuencia con saltos exponencialmente crecientes $(1, 2, 4, 8, \dots)$, de modo que el campo receptivo crece de forma exponencial con la profundidad mientras el número de capas se mantiene lineal; el campo receptivo deja entonces de depender de un equilibrio frágil entre estado oculto y ventana, como en las recurrentes, y pasa a ser un parámetro de diseño explícito y predecible. El tercero son los bloques residuales, que estabilizan el entrenamiento de redes profundas mediante conexiones identidad y permiten apilar capas suficientes para cubrir un stint completo sin degradación del gradiente. En el proyecto previo del autor (Vega Sobral, 2025) la familia recurrente sobre estos mismos datos arrojó un rendimiento modesto, y esa observación empírica es la que motiva el salto a la familia convolucional en este trabajo; evaluaciones académicas recientes sobre datos públicos de Fórmula 1 confirman además que la elección entre familias arquitectónicas es sensible al planteamiento concreto del problema (Sasikumar et al., 2025). La variante concreta empleada para el modelo de degradación es la TCN con convoluciones causales y dilatadas, sobre la que en el párrafo siguiente se construye una capa de cuantificación de incertidumbre.

No obstante, una predicción puntual de la TCN no basta para razonar sobre una parada en boxes: la decisión exige conocer tanto el valor central previsto como su rango plausible. Para estimar ese rango el sistema recurre a MC Dropout (Gal & Ghahramani, 2016), que no obliga a reentrenar la red ni a modificar su arquitectura. La idea consiste en mantener activo durante la inferencia el mismo *dropout* aplicado en el entrenamiento y ejecutar varias veces el cálculo sobre la entrada de interés, cada vez con una máscara aleatoria distinta. La dispersión obtenida en esas repeticiones constituye una aproximación a la distribución a posteriori del modelo, en otras palabras, a la incertidumbre que reflejaría un colectivo formado por todas las versiones de la red compatibles con los datos de entrenamiento, sin necesidad de construirlo de manera explícita.

Sobre esta TCN se ejecutan cincuenta repeticiones por inferencia y sus salidas se resumen en los percentiles P10, P50 y P90 de la degradación prevista. Estos tres valores se incorporan al grafo de agentes como medida de incertidumbre, y a partir de ellos el agente de neumáticos estima cuántas vueltas quedan hasta el cliff (jerga de *pit-wall* para la caída brusca de rendimiento que precede al pinchazo o al cruce térmico crítico) y emite un nivel de alerta asociado. En lugar de un valor único, la predicción se entrega como un intervalo, a partir del cual el resto del sistema decide el momento de la siguiente parada.

Esta TCN causal se ensayó también sobre el problema de la probabilidad de adelantamiento, pero se archivó al obtener un rendimiento sustancialmente inferior al del clasificador basado en LightGBM, finalmente seleccionado para esta tarea, como se detalla en el Capítulo 5. La lección que se extrae es directa: cuando el conjunto de entrenamiento es limitado y los patrones temporales relevantes ya están codificados como variables explícitas, el modelado de las secuencias crudas no compensa la complejidad añadida frente a un *boosting* tabular bien calibrado.

Se cierra así el tratamiento de los flujos numéricos vuelta a vuelta del sistema por parte de la familia convolucional. En las dos secciones siguientes se aborda la otra mitad de las entradas, donde el dato es el lenguaje natural y la familia pertinente cambia.

2.5 Procesamiento de lenguaje natural para radio de Fórmula 1

El sistema procesa dos flujos textuales con propiedades muy distintas. El primero son las comunicaciones por radio entre piloto y muro (los ingenieros del equipo apostados en el *pit wall*, frente a su garaje), caracterizadas por voces con ruido de motor de fondo, jerga propia del deporte y nombres propios, características que exigen una cadena de modelos de procesamiento de lenguaje natural (PLN) sobre la transcripción. El segundo son los mensajes de dirección de carrera, emitidos por la FIA con plantillas fijas y vocabulario controlado, donde un analizador determinista basado en reglas resulta más robusto, sencillo y mantenible que un modelo entrenado.

Las subsecciones siguientes recorren los componentes de esa cadena (transcripción, sentimiento, intención y reconocimiento de entidades) antes de cerrar con el analizador determinista que aborda el segundo flujo.

2.5.1 Reconocimiento automático del habla: Whisper

La cadena arranca con el reconocimiento automático del habla (*automatic speech recognition*, ASR), tarea que se encarga de transcribir la señal de audio captada por la radio del equipo en una secuencia de texto procesable por los modelos posteriores. Para este paso el sistema emplea Whisper (Radford et al., 2022), un *transformer* con arquitectura codificador-decodificador (Figura 2.4). Fue entrenado por OpenAI sobre aproximadamente 680 000 horas de audio multilingüe etiquetado de forma débil, y esa escala de preentrenamiento le confiere robustez frente al ruido de motor, las transmisiones a bordo y las grabaciones de baja fidelidad típicas de este escenario. De los seis tamaños disponibles, recogidos en la Tabla 2.2, se selecciona la variante *turbo* por su equilibrio entre coste de inferencia y calidad sobre los audios cortos del corpus de radio. Sus dos limitaciones reconocidas, la jerga específica del *paddock* (la zona de garajes y equipos donde se concentra la actividad técnica del fin de semana), con expresiones como *box box* o *lift and coast* y los nombres propios de pilotos y equipos, especialmente vulnerables a la transcripción fonética en un modelo entrenado mayoritariamente sobre habla genérica, se mitigan más tarde en la cadena por el módulo de reconocimiento de entidades.

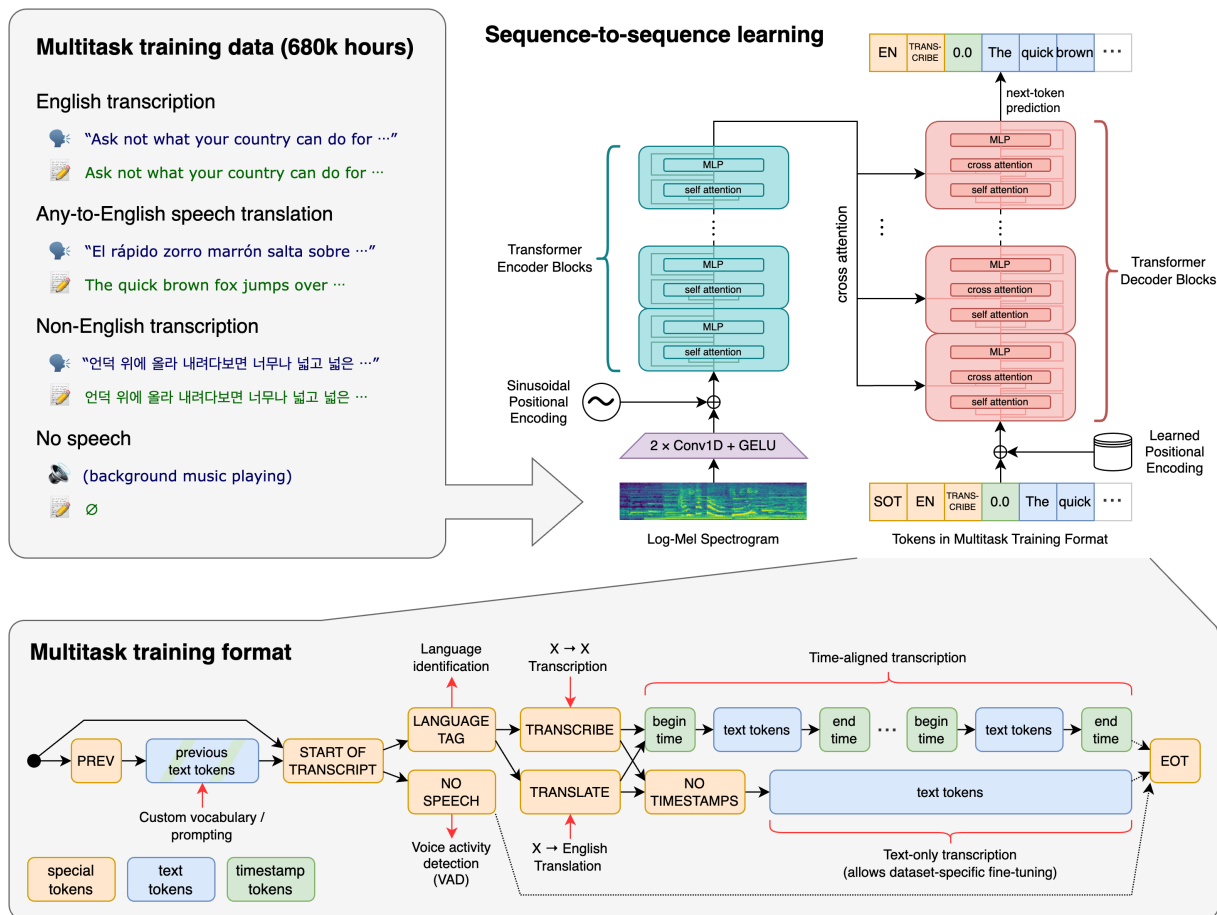


Figura 2.4: Arquitectura general de Whisper. La señal de audio se convierte en un espectrograma logarítmico de Mel que alimenta el codificador. El decodificador genera los *tokens* de texto de forma autorregresiva, condicionado por una secuencia de *tokens* de tarea iniciales que permiten al mismo modelo cubrir transcripción, traducción al inglés, identificación de idioma y detección de actividad de voz. Reproducido de Radford et al. (2022, Fig. 1).

Tabla 2.2: Modelos disponibles de Whisper. Parámetros por modelo según (Radford et al., 2022), con requisitos de VRAM, velocidad relativa y disponibilidad de la variante *turbo* según (OpenAI, 2024).

Tamaño	Parámetros	Variante monolingüe	Variante multilingüe	VRAM mínima	Velocidad relativa
<i>tiny</i>	39 M	<i>tiny.en</i>	<i>tiny</i>	~1 GB	~10×
<i>base</i>	74 M	<i>base.en</i>	<i>base</i>	~1 GB	~7×
<i>small</i>	244 M	<i>small.en</i>	<i>small</i>	~2 GB	~4×
<i>medium</i>	769 M	<i>medium.en</i>	<i>medium</i>	~5 GB	~2×
<i>large</i>	1 550 M	—	<i>large</i>	~10 GB	1×
<i>turbo</i>	809 M	—	<i>turbo</i>	~6 GB	~8×

2.5.2 Clasificación de sentimiento: RoBERTa

Sobre la transcripción producida por Whisper, la cadena aplica un clasificador de sentimiento que asigna cada mensaje a una de tres categorías (negativo, neutro o positivo). El estado emocional del piloto es una señal estratégica útil al orquestador, complementaria a las métricas del coche o a otras partes del mensaje de radio. El sistema emplea RoBERTa (Y. Liu et al., 2019), una optimización del modelo BERT (Devlin et al., 2019). Su diferencia clave radica en que RoBERTa descarta la tarea de predicción de la siguiente frase, entrena durante más tiempo sobre un corpus mayor y conserva únicamente el modelado de lenguaje enmascarado, lo que aporta una base más estable para el ajuste fino (*fine-tuning*) en dominios específicos. Dicho ajuste se aplica sobre un corpus etiquetado de radios por parte del autor con las tres clases mencionadas. La alternativa más sencilla, un clasificador basado en léxico como VADER (Hutto & Gilbert, 2014), se descarta porque su léxico procede de texto general en redes sociales y no recoge ni la jerga del paddock ni el registro operativo e irónico de las comunicaciones entre piloto y muro.

2.5.3 Clasificación de intención: SetFit y ModernBERT

La radio se etiqueta manualmente también según la intención del mensaje, distinguiendo entre cinco categorías operativas: información, problema, orden, advertencia y pregunta. Estas etiquetas permiten al agente de radios encaminar de forma diferenciada un aviso de fallo o una orden a boxes en lugar de tratar todo el flujo de radio como texto plano. El conjunto de datos disponible para el entrenamiento es deliberadamente pequeño, del orden de trescientas setenta frases anotadas manualmente, lo que descarta de partida el ajuste fino clásico de transformers grandes por el evidente desajuste entre el número de parámetros del modelo y el de ejemplos disponibles. Una primera tentativa con DeBERTa-v3-large (He et al., 2023) confirmó este límite empíricamente: con trescientos cuatro millones de parámetros, el modelo no logró converger sobre el corpus de entrenamiento y colapsó prediciendo siempre la clase mayoritaria, con un F1 macro inferior al de un clasificador trivial. La alternativa adoptada es SetFit (Tunstall et al., 2022), un procedimiento de *few-shot* en dos etapas en el que primero se entrena un codificador de oraciones mediante *contrastive learning* sobre pares dentro y fuera de cada clase, y a continuación se ajusta un clasificador lineal sobre los *embeddings* resultantes. Esta partición permite obtener clasificadores robustos con cientos de ejemplos en lugar de miles, exactamente el régimen en el que opera el corpus disponible. Como base se emplea ModernBERT-base (Warner et al., 2024), una evolución reciente de BERT con ventana de contexto extendida y eficiencia mejorada, mejor adaptada a la longitud de los mensajes radiados completos sin necesidad de truncamiento agresivo.

2.5.4 Reconocimiento de entidades nombradas: BERT-BIO

El último componente del flujo aplica un reconocimiento de entidades nombradas (*named entity recognition*, NER) sobre la transcripción de radios para extraer nueve etiquetas semánticas del dominio: acción, situación, incidente, instrucción estratégica, cambio de posición, llamada a boxes, condición de pista, problema técnico y meteorología. A diferencia de las tareas NER clásicas, no se trata de extraer nombres propios sino de etiquetar tokens con roles semánticos a nivel de cláusula, lo que condiciona la elección del paradigma. Una primera tentativa con modelos de la familia GLiNER (Zaratiana et al., 2024) y NuNER (Bogdanov et al., 2024) en régimen zero-shot, basados en codificación textual de las etiquetas y extracción de spans, devolvió un rendimiento prácticamente nulo. Ambos modelos fueron entrenados para extraer entidades nominales en corpus generalistas y no logran proyectar sus espacios de representación sobre roles clausulares específicos del dominio. El intento de adaptación mediante ajuste fino sobre el corpus anotado solo aporta una mejora marginal, lo que confirma que el problema no es de hiperparámetros sino de adecuación entre el paradigma

del modelo y la naturaleza de la tarea. La solución finalmente adoptada parte de un BERT-large (Devlin et al., 2019) preentrenado sobre el corpus CoNLL-03 (Tjong Kim Sang & De Meulder, 2003), referencia estándar de NER en prensa inglesa, con esquema de etiquetado BIO (*Begin-Inside-Outside*) que asigna a cada token una etiqueta para indicar si abre, continúa o queda fuera de una entidad. Sobre esa base de etiquetado BIO ya conocida por el modelo, el ajuste fino al dominio F1 converge y alcanza un rendimiento equivalente al del modelo desarrollado en el proyecto legacy (Vega Sobral, 2025) para la misma tarea. La lección metodológica que se desprende es que la elección del paradigma correcto (clasificación token-level con BIO) sobre uno diseñado para otra tarea (extracción de spans en GLiNER) es lo que determina el rendimiento, no el tamaño del modelo subyacente.

2.5.5 Procesamiento de mensajes de dirección de carrera

A diferencia del flujo de radio entre piloto y muro, los mensajes de dirección de carrera (*Race Control Messages*, RCM) son mensajes oficiales emitidos por la dirección de carrera a través del sistema de cronometraje de la FIA, no transcripciones de audio humano. Se obtienen ya estructurados desde la API pública FastF1, con cinco categorías predefinidas (Flag, SafetyCar, Drs, CarEvent y Other) y un vocabulario cerrado por el reglamento deportivo (Fédération Internationale de l'Automobile, 2024). Sobre este corpus, el sistema aplica un procesador determinista basado en patrones léxicos sobre las plantillas conocidas de cada categoría, apoyado por expresiones regulares puntuales para extracciones complementarias como el número de coche implicado, en lugar de un modelo de aprendizaje. La justificación es metodológica antes que pragmática. En un dominio donde «SAFETY CAR DEPLOYED» siempre significa lo mismo y «DRS ENABLED» también, ningún modelo estadístico puede mejorar a un conjunto de reglas bien escritas, y a cambio se ganan tres propiedades operativas relevantes para el sistema, siendo la primera la robustez frente a variantes léxicas inexistentes, la segunda la trazabilidad regla a regla de cada decisión y la tercera la mantenibilidad por extensión sin reentrenamiento. La salida del procesador es un objeto estructurado con tipo de evento (uno de veinticinco reconocidos, entre ellos las distintas banderas, despliegues de coche de seguridad real y virtual, transiciones del DRS, sanciones, investigaciones e infracciones de límites de pista), alcance (pista, sector o coche) y número de coche cuando procede. La elección refleja un principio que recorre toda esta sección, según el cual el grado de complejidad del modelo debe ajustarse a la naturaleza del lenguaje a procesar, y el lenguaje formal y cerrado de los RCM no exige más que reglas explícitas. Con este último componente queda completo el procesamiento de los flujos textuales generados durante la carrera. En la siguiente sección se aborda una fuente textual de naturaleza distinta, las regulaciones deportivas y técnicas de la FIA, cuya complejidad léxica devuelve al sistema al terreno del aprendizaje.

2.6 Generación aumentada por recuperación

El reglamento deportivo de la FIA constituye la fuente textual más voluminosa del sistema, con varios cientos de páginas por temporada de lenguaje jurídico-técnico revisado anualmente. Ni un procesador determinista (el vocabulario está abierto y depende del contexto) ni la inyección del reglamento entero en el prompt del modelo de lenguaje (límite de tokens, coste por consulta e irrelevancia masiva en cada respuesta) son viables sobre un texto de este volumen. La solución adoptada es la generación aumentada por recuperación (*retrieval-augmented generation*, RAG) (Lewis et al., 2020), paradigma en el que el modelo genera respuestas condicionadas en pasajes recuperados de una base documental externa, en lugar de depender solo del conocimiento codificado durante el preentrenamiento. La ganancia es doble. Por un lado, el modelo se ancla a citas verificables del articulado sin necesidad de memorizar el reglamento en sus propios pesos, con lo que mitiga las alucinaciones, dicho de otro modo, la tendencia a producir afirmaciones plausibles pero no respaldadas. Por otro lado, cada recomendación que apele al reglamento queda trazable

Flujo de recuperación aumentada por generación

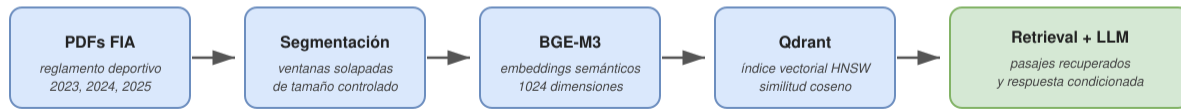


Figura 2.5: Flujo de generación aumentada por recuperación sobre el reglamento deportivo. Construcción del índice mediante segmentación, codificación con BGE-M3 y almacenamiento en Qdrant con HNSW, e inferencia mediante recuperación por similitud coseno y concatenación al *prompt*.

hasta el artículo concreto que la sustenta, una propiedad ya identificada como requisito metodológico en el Capítulo 1.

El índice de búsqueda se construye en cuatro etapas, resumidas en la Figura 2.5. La primera es la segmentación del texto en ventanas solapadas de tamaño controlado, suficiente para que las frases que cruzan los bordes aparezcan completas en al menos una de ellas. La segunda es la generación de embeddings semánticos con BGE-M3 (J. Chen et al., 2024), modelo multilingüe que proyecta cada extracto en un vector de mil veinticuatro dimensiones donde la proximidad geométrica corresponde a similitud de contenido, idea introducida en su forma actual por Sentence-BERT (Reimers & Gurevych, 2019). La tercera es el almacenamiento de esos vectores en una instancia local de Qdrant, indexada con la estructura *Hierarchical Navigable Small World* (HNSW) (Malkov & Yashunin, 2020) para permitir búsquedas aproximadas por similitud coseno con coste sublineal sobre el conjunto indexado. La cuarta es el retrieval condicionado, donde cada consulta se embebe con el mismo modelo, se recuperan los pasajes más próximos en el espacio vectorial y se concatenan al prompt para condicionar la generación de la respuesta.

El corpus indexado lo forman las tres versiones del reglamento deportivo correspondientes a las temporadas 2023, 2024 y 2025 (Fédération Internationale de l'Automobile, 2024). Cada pasaje devuelto por la búsqueda lleva los metadatos necesarios para que el sub-agente consumidor filtre por temporada, de modo que la respuesta provenga de la versión vigente del reglamento, y por umbral de relevancia, descartando los extractos cuya similitud con la consulta queda por debajo de un valor mínimo. Este componente no se invoca permanentemente, sino solo cuando otros sub-agentes detectan situaciones que requieren consultar reglamento, como una posible sanción, un procedimiento de pit lane o una neutralización en curso. Su integración dentro de la arquitectura multi-agente, junto al resto de sub-agentes especializados y al orquestador que sintetiza sus salidas, se describe en la siguiente sección.

2.7 Agentes de modelos de lenguaje y orquestación multi-agente

La arquitectura multi-agente que articula los componentes anteriores responde a una restricción concreta del problema, que es que cada predictor del sistema opera con un sesgo inductivo diferente, una métrica de evaluación que le es propia y un régimen de incertidumbre que no se traduce trivialmente al de los demás. Forzar un único modelo a producir simultáneamente el ritmo por vuelta, la degradación del compuesto, la probabilidad de adelantamiento, la salida del flujo de radio y la consulta al reglamento implicaría perder esa especialización y, lo que pesa más en este trabajo, dejar la decisión final sin una cadena clara de trazabilidad hasta cada inferencia parcial. La alternativa adoptada distribuye estas competencias entre sub-agentes especializados que un supervisor consulta y agrega. Se trata de un paradigma cuya raíz conceptual está en la mezcla de expertos (*mixture of experts*, MoE) (Jacobs et al., 1991), donde una red de *gating* aprende a enrutar cada entrada hacia el subconjunto de expertos que mejor la sirve. La versión

moderna y dispersa de esa idea, escalada a modelos de lenguaje (Shazeer et al., 2017), demostró que el enrutamiento condicional permite ampliar la capacidad de un sistema sin asumir un coste lineal en cómputo, principio que se reaprovecha aquí aunque a otro nivel, ya que la unidad enrutada no es una subred del modelo sino un sub-agente entero, y la decisión la toma un orquestador explícito gobernado por reglas en lugar de una compuerta aprendida.

Sobre ese principio, la implementación concreta de los sub-agentes especializados se apoya en LangGraph, un marco de orquestación de agentes basado en grafos de estado donde cada nodo es una función *Python* que recibe y devuelve una porción del estado compartido y cada arista una transición condicional entre nodos. Cinco de los seis sub-agentes se materializan como agentes ReAct (Yao et al., 2023), patrón en el que un modelo de lenguaje alterna ciclos de razonamiento explícito y de invocación a herramientas, implementadas aquí como llamadas a los modelos predictivos descritos en las secciones anteriores. El sexto, el sub-agente de radio, sigue un patrón *NLP-first* en el que la canalización determinista de procesamiento del lenguaje natural se ejecuta primero y solo al final un modelo de lenguaje sintetiza el resultado en una salida estructurada, de modo que las alertas críticas extraídas por los modelos no puedan ser modificadas ni omitidas por el agente. Los seis sub-agentes se reparten en dos categorías según su lógica de activación. La primera, de activación permanente, reúne los sub-agentes de ritmo (N25), neumáticos (N26), situación de carrera (N27) y radio (N29), que se consultan en cada vuelta porque su salida es relevante con independencia del contexto. La segunda, condicional, agrupa al sub-agente de estrategia de paradas (N28) y al sub-agente de regulaciones (N30), que se invocan solo cuando reglas explícitas detectan que su consulta aportaría valor, recuperando a escala de sistema la misma lógica de enrutamiento de la mezcla de expertos. Este nivel de control fino sobre qué sub-agente se activa y qué porción del estado comparten justifica la elección de LangGraph frente a alternativas como AutoGen (Wu et al., 2023), orientado también a aplicaciones multi-agente con modelos de lenguaje pero que prioriza la conversación libre entre agentes en detrimento de la determinación explícita del flujo de estado.

Una vez consultados los sub-agentes pertinentes, el supervisor no remite directamente sus salidas al modelo de lenguaje final, sino que las hace converger primero en una capa de simulación Monte Carlo que pondera escenarios alternativos a partir de las distribuciones de incertidumbre que cada sub-agente expone como salida. Esta capa se inspira en una referencia canónica de simulación Monte Carlo aplicada a estrategia de carrera (Heilmeyer et al., 2020a), donde se demuestra que el muestreo sobre tiempos por vuelta y duración de paradas reproduce el comportamiento agregado de una carrera bajo distintas políticas de boxes. Sobre ese principio, el orquestador genera quinientas trayectorias por cada uno de cuatro candidatos discretos de acción (mantenerse en pista, parar de inmediato, ejecutar un undercut o forzar un overcut) y resume cada conjunto en su valor esperado $E[S]$, sus percentiles $P_{10}[S]$ y $P_{90}[S]$, y una puntuación agregada $\alpha \cdot E[S] + (1 - \alpha) \cdot P_{10}[S]$ que pondera explícitamente el peor caso mediante el décimo percentil. El parámetro α es ajustable según el perfil de riesgo del usuario, lo que permite mover el sistema de un extremo agresivo a uno conservador sin alterar el resto de la cadena. La incorporación deliberada del término de peor caso recoge la idea central de DeLLMa (O. Liu et al., 2024), propuesta para combinar razonamiento probabilístico explícito con modelos de lenguaje, según la cual la incertidumbre debe entrar como variable formal del proceso de decisión y no quedar absorbida en el lenguaje natural posterior. El conjunto de cuatro vectores de puntuación así obtenido, con su esperanza, sus dos percentiles y su agregado robusto, es la evidencia cuantificada que la siguiente capa de síntesis recibe como entrada.

Esa capa de síntesis es un modelo de lenguaje al que el orquestador presenta simultáneamente las puntuaciones de Monte Carlo, las observaciones cualitativas que el muestreo no absorbe (las etiquetas de intención y sentimiento producidas por el sub-agente de radio y los pasajes recuperados por el sub-agente de regulaciones cuando ha sido invocado) y un resumen del estado de carrera vigente. La salida del modelo no se produce como texto libre, sino que se canaliza a través del mecanismo *structured output* de la integración

con LangChain, que delega la validación al esquema Pydantic StrategyRecommendation. Ese esquema fija los campos obligatorios de la respuesta, entre ellos la acción recomendada (uno de los cuatro candidatos puntuados por Monte Carlo o un quinto valor de aviso), el razonamiento textual que justifica esa decisión a partir de las salidas previas, la vuelta planificada para la parada en boxes cuando proceda, el compuesto que se montaría a continuación, el código del rival apuntado por undercut u overcut y un nivel de confianza explícito. La presencia de esa capa de síntesis encima de los sub-agentes y de la simulación cuantitativa es coherente con la arquitectura de mezcla de agentes (*Mixture of Agents*, MoA) (Wang et al., 2024), que formaliza la idea de que las salidas de varios agentes especializados pueden refinarse iterativamente en una capa adicional de modelo de lenguaje sin perder la traza de cada contribución parcial. Toda la cadena descrita hasta aquí, desde los predictores tabulares hasta la recomendación estructurada que devuelve el orquestador al usuario, opera sobre datos cuyo régimen depende fuertemente del circuito concreto en el que se compite, lo que justifica el último elemento metodológico que el capítulo introduce a continuación.

2.8 Agrupamiento de circuitos

La dependencia del circuito previamente mencionada se observa con claridad al comparar trazados extremos del calendario como Mónaco frente a Monza o Marina Bay frente a Silverstone, donde el ritmo, patrón de degradación, frecuencia de las paradas y el impacto del coche de seguridad varían lo suficiente como para invalidar cualquier supuesto de homogeneidad. Frente a esta variabilidad caben dos extremos metodológicos, ninguno satisfactorio. Entrenar a un único modelo global ciego al circuito introduce un ruido sistemático en las predicciones, ya que el modelo aprende un comportamiento promedio que no se ajusta a casi ningún Gran Premio en particular. En el otro extremo, entrenar a un modelo independiente por circuito tampoco es viable, porque con un corpus de veinticuatro Grandes Premios y dos a tres temporadas el conjunto de datos por circuito se queda demasiado pequeño para sostener cualquier ajuste razonable. La alternativa adoptada es intermedia y consiste en agrupar los circuitos en un conjunto reducido de clusters de comportamiento estratégico similar e introducir el cluster resultante como *feature* adicional en todos los modelos predictivos descritos en las secciones anteriores.

Sobre esos circuitos, el sistema aplica K-Means clásico, algoritmo de particionamiento iterativo que asigna cada punto al centroide más cercano y recalcula los centroides hasta convergencia, con métrica euclídea sobre *features* previamente estandarizadas. La decisión metodológicamente importante no está en el algoritmo sino en la elección de *features*. En lugar de partir de descriptores geométricos preestablecidos como la carga aerodinámica, la longitud del trazado o el porcentaje de recta, las *features* que alimentan el clustering se extraen del comportamiento observado en carrera, entre ellas el ritmo medio por circuito, la variabilidad de los tiempos por vuelta, la tasa de degradación efectiva, el número medio de paradas, la temperatura media de pista y la velocidad media por sector. La pregunta a la que responde el agrupamiento no es entonces «cómo se diseñó este circuito» sino «cómo se comporta estratégicamente este circuito en carrera», una distinción cuyo encaje con las clasificaciones tradicionales de la prensa motorsport y con la categorización oficial de compuestos por circuito se contrasta en el párrafo siguiente. El número final de clusters se fija en cuatro tras validar la elección con el método del codo y el índice de silueta (Qureshi, 2025).

Aplicado el procedimiento se obtienen cuatro grupos de circuitos cuya composición y rasgo distintivo recoge la Tabla 2.3. La validez del agrupamiento se contrasta con dos referencias externas. La primera es la nominación oficial de compuestos por circuito que Pirelli publica como suministrador único (Pirelli Motorsport, 2025a), donde la escala interna con la que el fabricante identifica los trazados de máxima exigencia para el neumático coincide en buena parte con el núcleo del cluster de circuitos rápidos de carga sostenida (*Long High-Speed Circuits*) en casos como Suzuka, Silverstone, Spa o Sakhir, lo que aporta una convergencia parcial entre dos criterios independientes, uno físico-mecánico y otro estadístico-

Tabla 2.3: Agrupamiento de los circuitos del calendario en cuatro arquetipos según su comportamiento estratégico observado en carrera.

Cluster	Circuitos	Rasgo distintivo
Caos estratégico (<i>Strategic Chaos</i>)	Mónaco, Melbourne, Montréal, Zandvoort	Alta varianza estratégica y mayor dificultad de predicción
Circuitos rápidos de carga sostenida (<i>Long High-Speed Circuits</i>)	Spa, Silverstone, Suzuka, Shanghái, Sakhir, Las Vegas	Degradación elevada por carga sostenida en curvas rápidas
Estrategia conservadora en caliente (<i>Warm Conservative Strategy</i>)	Monza, Budapest, Bakú, Marina Bay, Miami, Imola, Austin, Barcelona	Perfiles térmicos y de degradación que favorecen estrategias de una a dos paradas
Atípicos de alto estrés (<i>High-Stress Outliers</i>)	Lusail, Mexico City, Spielberg, São Paulo	Estrés combinado de pianos, altitud y cargas estructurales extremas

comportamental. La inclusión de Las Vegas en ese mismo cluster matiza la convergencia, pues responde a las velocidades medias altas que generan sus rectas largas en condiciones nocturnas frescas y no a una exigencia térmica sostenida sobre el neumático. La segunda referencia es la dicotomía clásica con la que la prensa motorsport organiza el calendario entre circuitos de alta carga aerodinámica y circuitos de potencia (Holding, 2023), criterio del que el agrupamiento guiado por los datos diverge de manera informativa. El cluster de caos estratégico (*Strategic Chaos*) reúne a Mónaco, Melbourne, Montréal y Zandvoort, casos en los que la dicotomía geométrica los colocaría en categorías distintas pero que comparten muros próximos a la pista, mínimas zonas de escape y abundancia de coches de seguridad, factores que dominan la varianza estratégica observada con independencia de que el trazado sea callejero, semipermanente o permanente sobre dunas costeras. El cluster de estrategia conservadora en caliente (*Warm Conservative Strategy*) integra a Monza junto a Budapest, casos opuestos en carga aerodinámica pero equiparables a efectos de carrera, ya que sus perfiles térmicos y de degradación efectiva favorecen sobre ambos estrategias de una a dos paradas. El cluster de atípicos de alto estrés (*High-Stress Outliers*) aísla a Lusail, Mexico City, Spielberg y São Paulo por una combinación de cargas estructurales extremas que ninguna taxonomía geométrica clásica recoge como categoría diferenciada, ya sea por pianos agresivos sobre cargas laterales prolongadas, por la altitud que reduce la densidad del aire disponible para la generación de carga aerodinámica o por la conjunción de ambos factores. La convergencia parcial con la categorización oficial de compuestos y la divergencia razonada con la clasificación de la prensa motorsport sostienen así la validez de un agrupamiento construido íntegramente sobre el comportamiento observado en carrera.

El cluster resultante se incorpora a varios de los modelos predictivos numéricos del sistema, en concreto a los que dependen del ritmo y la dinámica de pista del trazado, en particular, el tiempo por vuelta, la degradación del neumático, la probabilidad de adelantamiento y la probabilidad de aparición del coche de seguridad, con un papel doble que conviene matizar, ya que en los modelos basados en árboles entra como variable categórica directa al lado del resto de variables predictoras, mientras que en la red convolucional temporal de degradación actúa como referencia de normalización al sustraer del tiempo por vuelta de cada observación el promedio de ritmo de su cluster antes de modelar el patrón temporal restante. Cerrado este recorrido teórico por los componentes del sistema, el capítulo siguiente revisa el panorama actual de la inteligencia artificial aplicada a la estrategia de Fórmula 1 en equipos profesionales, software comercial y producción académica, identifica las carencias del estado del arte y deriva de ellas los requisitos que el sistema desarrollado debe cubrir.

3 ANÁLISIS DE LA SITUACIÓN

Para quien no sabe a qué puerto se dirige, ningún viento le es favorable.

— Lucio Anneo Séneca, *Cartas a Lucilio*¹

El Capítulo 2 presenta los fundamentos teóricos sobre los que se construye el sistema de asistencia estratégica a la decisión en Fórmula 1 y cierra situando el agrupamiento de circuitos como variable transversal disponible para los modelos predictivos del sistema que dependen del ritmo y la dinámica del trazado. El presente capítulo cambia de plano y aborda el contexto aplicado en el que ese sistema se inserta. La primera sección recorre el panorama actual del uso de la inteligencia artificial en el campeonato, distinguiendo entre las soluciones desplegadas por los equipos profesionales y la producción académica reciente. La segunda identifica las limitaciones que persisten en ese panorama y que justifican el alcance del trabajo. La tercera deriva de ahí los requisitos funcionales y no funcionales del sistema propuesto, anclados a los componentes que los implementan. La última recoge las restricciones operativas reales bajo las cuales se ha desarrollado.

3.1 Panorama actual de la inteligencia artificial en la Fórmula 1

El uso de la inteligencia artificial en la Fórmula 1 se ha consolidado en los últimos años como una capa de soporte permanente del trabajo de los equipos (Carter, 2025). Las colaboraciones tecnológicas mencionadas en el Capítulo 1 reflejan, más que un fenómeno comercial puntual, una infraestructura computacional que abarca desde la dinámica de fluidos y la simulación estratégica hasta el procesamiento del lenguaje natural y la integración multimodal de la telemetría. Esta sección desarrolla ese panorama en dos planos complementarios. El primero describe los equipos punteros y los proveedores tecnológicos con los que operan, identificando las tecnologías concretas documentadas y los hitos públicos disponibles. El segundo recoge el software comercial empleado por la categoría y la producción académica más reciente sobre los mismos problemas, con el objetivo de contrastar lo que se publica de forma abierta frente a lo que permanece como activo propietario.

3.1.1 Equipos punteros y sus partners tecnológicos

El mapa de colaboraciones entre los principales equipos de Fórmula 1 y proveedores externos de inteligencia artificial concentra las manifestaciones industriales más visibles de la disciplina dentro del campeonato. La Tabla 3.1 sintetiza los acuerdos documentados públicamente, indicando en cada caso la familia de inteligencia artificial aportada (analítica predictiva, simulación estocástica, sensorización inteligente o modelos de lenguaje generativos) y la fuente que la respalda. El texto que sigue desarrolla esa información agrupándola por familia, en lugar de hacerlo equipo por equipo, para evidenciar la cobertura que cada uno de estos planos alcanza en el paddock.

La primera de estas familias, la analítica predictiva sobre datos de carrera, agrupa las soluciones que aplican aprendizaje automático clásico al ritmo, la degradación de los neumáticos y las decisiones de parada en boxes. El caso de Mercedes en el Gran Premio de México de 2019 ha sido recogido por la

¹Fragmento de *Epistulae morales ad Lucilium* (*Cartas a Lucilio*), Ep. 71.3, de Lucio Anneo Séneca (s. I d. C.). Traducción propia del original en latín: *Ignoranti, quem portum petat, nullus suus ventus est.*

Tabla 3.1: Colaboraciones documentadas públicamente entre equipos de Fórmula 1 y proveedores externos de inteligencia artificial, indicando la familia principal aportada y la fuente. La cobertura no es exhaustiva, ya que excluye las herramientas internas no publicadas por los equipos.

Equipo	Proveedor	Familia de inteligencia artificial	Fuente
Ferrari	AWS	Analítica estratégica desplegada en la nube sobre datos de carrera y de telemetría	(Carter, 2025)
Mercedes	G42	Algoritmos predictivos de degradación de neumáticos y de ventanas de parada (caso del Gran Premio de México 2019)	(Carter, 2025; GrandPrix247, 2024)
Red Bull	Oracle	Simulación Monte Carlo de estrategia, con alrededor de cien millones de trayectorias en la preparación del Gran Premio de Hungría 2022	(GrandPrix247, 2024)
McLaren	Google / Cisco / Dell	Gemelos digitales, sensorización inteligente con entre trescientos y seiscientos sensores por monoplace y analítica en tiempo real desde centros de datos móviles	(Regan, 2024)
McLaren	Alteryx	Analítica predictiva y exploración de datos históricos de carrera	(Marr, 2023)
Aston Martin	Cohere	Modelos de lenguaje generativos para asistencia al razonamiento estratégico y consulta documental interna	(Aston Martin F1 Team, 2026)
Williams	Anthropic	Acuerdo multianual para la integración de modelos de lenguaje generativos en los flujos de trabajo del equipo	(Williams Racing, 2026)

prensa especializada como hito de la integración de analítica avanzada en el muro, dado que una parada en boxes apoyada por modelos predictivos de degradación contribuyó a la victoria de Lewis Hamilton en aquella carrera (GrandPrix247, 2024); la atribución específica al partner industrial G42 procede de fuentes adicionales recogidas en (Carter, 2025). En la misma familia operan la analítica histórica de carrera entre McLaren y Alteryx (Marr, 2023) y la analítica estratégica desplegada por Ferrari sobre la infraestructura en la nube de AWS (Carter, 2025), ambas orientadas a la exploración cuantitativa del comportamiento de pilotos y neumáticos en sesiones reales del campeonato.

La segunda familia comprende la simulación estocástica y la inferencia bajo incertidumbre aplicadas a la decisión estratégica vuelta a vuelta, donde el caso de referencia documentado es la colaboración entre Red Bull y Oracle. Para la preparación del Gran Premio de Hungría 2022 se publicó una estimación cercana a cien millones de trayectorias Monte Carlo evaluadas, con el objetivo de cubrir el espacio combinado de compuestos, ventanas de parada, irrupciones del coche de seguridad y comportamientos plausibles de los rivales (GrandPrix247, 2024). El nivel de cómputo aplicado responde a una doble exigencia, ya que es preciso muestrear con suficiente densidad un espacio de incertidumbre que cruza variables continuas como el ritmo por vuelta con variables discretas como la elección de compuesto y la vuelta de parada, y al mismo tiempo mantener un tiempo de respuesta compatible con la ventana real de toma de decisión durante el fin de semana de carrera.

La tercera comprende la sensorización inteligente y la analítica en tiempo real, es decir, la infraestructura encargada de capturar, transmitir y procesar la telemetría del coche desde la pista hasta el muro y la fábrica durante la propia carrera. El caso documentado con mayor detalle es la colaboración de McLaren con Google, Cisco y Dell en torno al Centro Tecnológico de Woking, donde cada monoplace incorpora entre trescientos y seiscientos sensores cuyas señales alimentan gemelos digitales del coche y se canalizan a través de centros de datos móviles desplegados en cada circuito (Regan, 2024). La densidad y la velocidad de este flujo condicionan directamente la calidad de los modelos predictivos y de la simulación estratégica descritos en los planos anteriores, ya que la latencia entre la observación física en pista y la inferencia disponible en el muro determina cuánta información puede incorporarse a la decisión durante la propia

ventana de oportunidad. La característica común a estos tres planos es que los detalles de implementación permanecen como activo competitivo de cada equipo y no se publican en forma reutilizable.

Cierra el cuadro la inteligencia artificial generativa y los modelos de lenguaje aplicados al flujo interno del equipo, plano de aparición más reciente en el campeonato. Como ya se anticipó en el Capítulo 1, los acuerdos de Aston Martin con Cohere en 2024 (Aston Martin F1 Team, 2026) y de Williams con Anthropic en 2026 (Williams Racing, 2026) ilustran la integración de modelos generativos en la decisión estratégica, la consulta documental, la lectura del reglamento y los flujos cotidianos del equipo. Aunque la corta vida pública de estos acuerdos impide aún medir su impacto deportivo concreto, su sola existencia confirma que la frontera entre los datos numéricos clásicos del campeonato y el conocimiento textual asociado al mismo, a saber, reglamento, comunicaciones de radio y documentación interna, se ha desplazado al primer plano del trabajo de los equipos. La siguiente subsección contrasta este conjunto de soluciones desplegadas dentro de los equipos con la oferta de software comercial accesible al ecosistema y con la producción académica abierta sobre los mismos problemas.

3.1.2 Software comercial y producción académica

El primero de estos planos, el del software comercial empleado por la categoría como columna vertebral de su trabajo de ingeniería, opera fundamentalmente sobre dinámica computacional de fluidos (CFD, por sus siglas en inglés) y simulación numérica clásica, ámbitos que no se inscriben en la inteligencia artificial pero condicionan de cerca su explotación. Alpine, denominada Renault hasta 2021, publicó junto con Siemens un caso de estudio según el cual su uso de Siemens STAR-CCM+ genera del orden de sesenta terabytes de datos CFD por semana (Siemens Digital Industries, 2021), mientras que Red Bull articula sus desarrollos aerodinámicos y los ensayos virtuales de impacto sobre ANSYS Fluent y LS-DYNA (Digital Engineering 24/7, 2024). La intensidad de estos cómputos no es libre, ya que la Federación Internacional del Automóvil limita por reglamento técnico el número de horas de procesador que cada equipo puede dedicar a CFD durante una temporada, y la maqueta a escala del sesenta por ciento en el túnel de viento, único entorno físico autorizado, está prevista para su eliminación progresiva hacia 2030 (Partridge, 2025; Raceteq, 2024). Estas herramientas, aunque ajenas al núcleo de inteligencia artificial que articula este capítulo, conviven en el flujo cotidiano de los equipos con los componentes descritos en la subsección anterior y delimitan el ecosistema computacional en el que la inteligencia artificial se inserta.

En el segundo plano, la producción académica reciente ha publicado piezas individuales sobre varios de los subproblemas que la industria aborda en privado. El referente canónico de simulación Monte Carlo aplicada a estrategia de motorsport (Heilmeier et al., 2020a) ya se emplea en el Capítulo 2 como base de la capa estocástica del orquestador. En el plano aerodinámico, las redes neuronales informadas por la física se han aplicado a la optimización del alerón delantero de un monoplaza, con un R^2 de 0,968 (Cheng et al., 2025). Sobre la decisión de parada en boxes, las arquitecturas Bi-LSTM y TCN-GRU alcanzan una métrica F_1 cercana al cincuenta y ocho por ciento (Sasikumar et al., 2025). Una red TabNet entrenada sobre datos históricos de las temporadas 2010 a 2023 predice la posición final del piloto y los puntos del constructor con R^2 de 0,75 y 0,71 respectivamente (Urdhwareshe, 2025). Cierra el panorama académico el grupo de aprendizaje automático de la Universidad de Cambridge, que mantiene un proyecto de modelado completo de carreras de Fórmula 1 cuya componente aplicada permanece sometida a confidencialidad comercial (Machine Learning at Cambridge Laboratory, 2019). La caracterización común de estas referencias es la cobertura de subproblemas concretos sin integración entre planos, ya que ninguna combina predicción numérica vuelta a vuelta con procesamiento de lenguaje natural sobre la radio o consulta a la regulación, lo que constituye precisamente el hueco que la siguiente sección desarrolla.

3.2 Oportunidades del panorama actual

El panorama anterior dibuja un dominio donde la inteligencia artificial está consolidada pero fragmentada, próxima al usuario industrial pero ajena a quien quiera reproducir el trabajo desde fuera. Ese desequilibrio entre intensidad y apertura genera un espacio concreto para una propuesta integrada, abierta y trazable, articulado en seis vertientes que cubren la apertura y reproducibilidad de los modelos publicados, la integración de extremo a extremo de los componentes hoy disponibles por separado, el procesamiento abierto de las comunicaciones por radio, la incorporación del reglamento deportivo en el bucle de decisión, la reproducibilidad de un punto de referencia sobre datos públicos y la explicabilidad y trazabilidad de la recomendación final. Cada vertiente se argumenta en su propio bloque y, leídas en conjunto, definen el alcance que el sistema desarrollado asume como propio.

Apertura y reproducibilidad. El cierre tecnológico de los equipos profesionales bloquea cualquier intento de replicar sus modelos, auditar sus decisiones o construir variantes académicas comparables (Carter, 2025), y deja al margen del progreso colectivo una parte sustancial del conocimiento acumulado en torno a la disciplina. Un sistema cuyos pesos, código y trazas de decisión queden expuestos a inspección externa abre la puerta a esa progresión, asumiendo a cambio que opera con datos de menor resolución que la telemetría interna de los equipos.

Integración de extremo a extremo. Los trabajos académicos disponibles resuelven con solvencia subproblemas individuales, ya sea la optimización aerodinámica mediante redes informadas por la física (Cheng et al., 2025), la decisión de parada con redes recurrentes y convolucionales temporales (Sasikumar et al., 2025) o la predicción de resultados de carrera con TabNet (Urdhwareshe, 2025), pero ninguno articula simultáneamente la predicción numérica vuelta a vuelta, el procesamiento del lenguaje natural sobre la radio, la consulta documental sobre el reglamento y la coordinación multi-agente entre todos esos componentes. La integración de esos cuatro planos en una arquitectura común aún no se ha publicado.

Procesado abierto de las comunicaciones por radio. El canal de radio que une al piloto con su muro transporta intención táctica, estado emocional, alertas y referencias a entidades del dominio (compuesto montado, vuelta de parada, rivales, banderas), información que ningún canal público transcribe ni estructura para uso analítico. Recuperarla en línea exige una cadena completa de reconocimiento automático del habla, clasificación de sentimiento e intención y extracción de entidades nombradas, sin precedente público integrado para la Fórmula 1.

Reglamento deportivo dentro del bucle de decisión. El reglamento deportivo de la Federación Internacional del Automóvil cambia cada temporada y se distribuye únicamente como PDF jurídico-técnico de varios cientos de páginas, formato hostil tanto a la consulta humana en tiempo real como al análisis programático (Fédération Internationale de l'Automobile, 2024). Los acuerdos firmados por los equipos con proveedores de inteligencia artificial generativa empiezan a abordar este flujo en privado, sin contraparte equivalente en el dominio público que lo conecte con la cadena de razonamiento estratégico vuelta a vuelta.

Reproducibilidad sobre datos públicos. Las APIs públicas que sirven datos de Fórmula 1 ofrecen telemetría multifrecuencia, simultánea para veinte coches y atravesada por cambios reglamentarios periódicos que introducen deriva conceptual a escala estacional (Hovakimyan & Bravo, 2024). Un trabajo que documente con rigor el particionamiento temporal, fije las semillas aleatorias y publique pesos junto a los cuadernos que los generan, todo ello sobre estas mismas APIs, es un esfuerzo que la literatura abierta no ha consolidado todavía como punto de referencia compartido.

Explicabilidad y trazabilidad. Los modelos publicados sobre Fórmula 1 entregan valores numéricos o probabilidades crudas sin un mecanismo nativo que conecte la salida con las inferencias intermedias ni con las fuentes documentales consultadas. Un ingeniero de estrategia que confíe en una recomendación necesita entender qué predicción de cada subsistema y qué fragmento de regulación la sustentan, capacidad ausente en lo publicado y accesible en cambio cuando la explicabilidad se construye desde el propio diseño.

Tomadas en conjunto, las seis vertientes anteriores describen un sistema concreto, multi-agente, abierto y reproducible, que integra predicción numérica vuelta a vuelta, procesamiento del lenguaje natural sobre la radio y recuperación documental sobre el reglamento bajo una capa de razonamiento explicable. La sección siguiente enuncia los requisitos funcionales y no funcionales que esa arquitectura debe satisfacer y los ancla a los componentes concretos que los implementan. La aspiración expresada por esos requisitos, sin embargo, debe entenderse a la luz de las restricciones operativas con las que el trabajo realmente se ha desarrollado, declaradas al cierre del capítulo.

3.3 Requisitos del sistema

Los requisitos del sistema se derivan del análisis del panorama realizado en las secciones anteriores, y se enuncian en términos verificables, anclados a los componentes concretos del proyecto que los implementan. Los requisitos funcionales se agrupan en cuatro bloques temáticos, en concreto, predicción mediante aprendizaje automático, procesamiento de las comunicaciones por radio y de los eventos de pista, razonamiento y orquestación multi-agente, y entrega al usuario a través de tres superficies de uso. Los requisitos no funcionales recogen, por su parte, las propiedades transversales que el sistema debe satisfacer con independencia del componente concreto, entre ellas la latencia, la reproducibilidad, la modularidad, la apertura del *stack*, la explicabilidad de las recomendaciones y la integridad temporal de las características de entrenamiento.

3.3.1 Requisitos funcionales

Los requisitos funcionales (en adelante, RF) se enumeran a continuación con la nomenclatura RF-XX y se anclan al cuaderno o módulo del proyecto que los implementa.

RF-01: Predicción del tiempo por vuelta. El sistema debe predecir el delta de tiempo por vuelta de cada piloto a partir de las variables de telemetría disponibles, con un error medio absoluto inferior a medio segundo en el conjunto de prueba de la temporada 2025. Anclado al cuaderno N06, basado en *XGBoost*.

RF-02: Predicción de la degradación del neumático con incertidumbre. El sistema debe predecir la degradación vuelta a vuelta del compuesto montado y devolver, además del valor central, los percentiles P10, P50 y P90 de la distribución predictiva, con anclaje explícito a la nomenclatura oficial Pirelli C1 – C5. Anclado a los cuadernos N09 y N10, basado en una red convolucional temporal (TCN) con *Monte Carlo Dropout*.

RF-03: Estimación de la probabilidad de adelantamiento. El sistema debe estimar la probabilidad de que un piloto adelante a otro dentro de una ventana de tres vueltas. Anclado al cuaderno N12, basado en *LightGBM* con calibración de Platt.

- RF-04: Estimación de la probabilidad de coche de seguridad.** El sistema debe estimar la probabilidad de despliegue del coche de seguridad en una ventana de cinco vueltas como prior contextual del orquestador. Anclado al cuaderno N14, basado en *LightGBM*.
- RF-05: Predicción de la duración de la parada en boxes.** El sistema debe predecir la duración física de la parada en boxes en forma de cuantiles P05, P50 y P95 mediante regresión por cuantiles nativa, sin necesidad de modelos auxiliares ni supuestos paramétricos. Anclado al cuaderno N15, basado en *HistGradientBoosting* con pérdida *pinball*.
- RF-06: Predicción del éxito de un undercut.** El sistema debe predecir la probabilidad de éxito de una maniobra de undercut sobre un rival concreto. Anclado al cuaderno N16, basado en *LightGBM* con calibración de Platt.
- RF-07: Reconocimiento automático del habla en radio.** El sistema debe transcribir las comunicaciones por radio entre piloto y muro a partir de la señal de audio, con tolerancia al ruido de motor y a la jerga propia de la categoría. Anclado al cuaderno N18, basado en la variante *turbo* de Whisper.
- RF-08: Análisis textual de la radio.** El sistema debe procesar la transcripción de cada mensaje de radio para extraer simultáneamente sentimiento (negativo, neutro, positivo), intención (información, problema, orden, advertencia, pregunta) y entidades del dominio de la Fórmula 1, devolviendo una salida estructurada en formato JSON (*JavaScript Object Notation*). Anclado a los cuadernos N20 (RoBERTa), N21 (*SetFit* sobre *ModernBERT-base*), N22 (BERT-BIO) y N24 (canalización unificada).
- RF-09: Procesamiento de mensajes de dirección de carrera.** El sistema debe convertir los mensajes textuales emitidos por la dirección de carrera de la FIA en eventos estructurados, distinguiendo banderas, despliegues de coche de seguridad y de coche de seguridad virtual, autorizaciones y restricciones de DRS y penalizaciones. Anclado al cuaderno N23, mediante un analizador determinista basado en reglas.
- RF-10: Recuperación documental sobre el reglamento deportivo.** El sistema debe permitir consultas en lenguaje natural al reglamento deportivo de la FIA mediante generación aumentada por recuperación, con índice vectorial local sobre los reglamentos de las temporadas 2023, 2024 y 2025. Anclado al cuaderno N30, basado en BGE-M3 (1024 dimensiones), Qdrant local y top-5.
- RF-11: Orquestación multi-agente con patrón supervisor.** El sistema debe coordinar a seis sub-agentes especializados, en concreto los agentes de ritmo (*PaceAgent*), neumáticos (*TireAgent*), situación de carrera (*RaceSituationAgent*), estrategia de paradas (*PitStrategyAgent*), radio (*RadioAgent*) y reglamento (*RagAgent*), implementados como agentes *ReAct* sobre LangGraph (salvo el *RadioAgent*, que sigue un patrón *NLP-first*), mediante un orquestador supervisor que agrega sus salidas, ejecuta una capa de simulación Monte Carlo y emite una recomendación estratégica estructurada. Anclado a los cuadernos N25–N31, con 500 trayectorias Monte Carlo por candidato resumidas en la puntuación $\alpha E[S] + (1 - \alpha) P_{10}[S]$.
- RF-12: Recomendación estratégica estructurada.** La recomendación final debe escogerse entre cinco acciones discretas (STAY_OUT, PIT_NOW, UNDERCUT, OVERCUT, ALERT), acompañarse de un modo de ritmo recomendado (PUSH, NEUTRAL, MANAGE, LIFT_AND_COAST) y de un nivel de riesgo (AGGRESSIVE, BALANCED, DEFENSIVE), e incluir el razonamiento textual que la sustenta y un nivel de confianza explícito.
- RF-13: Interfaz en vivo de réplica de carrera.** El sistema debe ofrecer una interfaz visual en tiempo real con tres ventanas sincronizadas (réplica de carrera sobre Arcade, panel estratégico Qt y panel de telemetría Qt), con difusión local de eventos.

RF-14: Interfaz post-carrera de análisis. El sistema debe ofrecer una interfaz web post-carrera, sobre Streamlit y FastAPI, para análisis exhaustivo y exploración multimodal de los modelos predictivos.

RF-15: Interfaz de línea de comandos sin entorno gráfico. El sistema debe poder ejecutarse sin entorno gráfico mediante una interfaz de línea de comandos, expuesta a través de los puntos de entrada `f1-strat`, `f1-sim`, `f1-arcade` y `f1-streamlit` declarados en `pyproject.toml`.

3.3.2 Requisitos no funcionales

Los requisitos no funcionales (en adelante, RNF) se enumeran a continuación con la nomenclatura RNF-XX y recogen las propiedades transversales del sistema.

RNF-01: Latencia. La canalización unificada de procesamiento de lenguaje natural debe mantener un percentil 95 de latencia por mensaje inferior a cien milisegundos sobre una unidad de procesamiento gráfico (GPU, por sus siglas en inglés). La inferencia de los sub-agentes predictivos debe ser compatible con operación vuelta a vuelta sin acumulación de retraso. La verificación cuantitativa de este requisito sobre el cuaderno N24 se reporta en el Capítulo 5.

RNF-02: Reproducibilidad. Las semillas aleatorias deben quedar fijadas en todos los cuadernos de entrenamiento; el árbol de dependencias se versiona en `pyproject.toml` y `uv.lock`; el corpus consolidado y los pesos entrenados se publican en HuggingFace Hub bajo el repositorio `VforVitorio/f1-strategy-dataset`.

RNF-03: Modularidad. Cada sub-agente debe ser intercambiable y exponer una interfaz uniforme `run_*_agent(lap_state) -> *Output` validada por esquema tipado, con baterías de pruebas aisladas por agente para evitar acoplamientos no documentados.

RNF-04: Apertura del *stack* y portabilidad del modelo de lenguaje. El sistema debe construirse íntegramente sobre dependencias de licencia libre, sin componentes propietarios en su pila técnica. La capa de orquestación final debe poder consumir un modelo de lenguaje en la nube o un modelo local servido sobre una instancia compatible, configurable de manera transparente para el usuario. Las dependencias concretas y la mecánica de configuración del proveedor de modelo se detallan en el Capítulo 4.

RNF-05: Explicabilidad. Cada recomendación del orquestador debe acompañarse de un razonamiento en lenguaje natural y de los vectores de puntuación Monte Carlo de los cuatro candidatos discretos, de manera que la decisión final sea trazable hasta las inferencias de los modelos y el razonamiento de los sub-agentes que la sustentan.

RNF-06: Despliegue. La instalación debe completarse con un único comando, `uv sync o uv tool install`, sobre cualquier máquina con Python compatible. Adicionalmente, el sistema debe poderse desplegar mediante `docker-compose` para los componentes de *backend* (FastAPI) y *frontend* (Streamlit).

RNF-07: Mantenibilidad. El proyecto debe disponer de pruebas automatizadas en el directorio `tests/` (`test_smoke.py`, `test_agents.py`, `test_simulation.py`, `test_arcade_dashboard_imports.py` y pruebas de canalización de radio) y formato de código uniforme verificado mediante `black` y `ruff`.

RNF-08: Integridad temporal. Ningún modelo de producción puede emplear características que requieran información futura de la misma sesión, es decir, no se permiten medianas a nivel de sesión, mejores tiempos absolutos ni cualquier estadístico que dependa de vueltas posteriores a la vuelta predicha. Esta regla deriva del análisis de deriva conceptual y particionamiento temporal estricto descrito en el Capítulo 2.

3.4 Restricciones

El alcance del sistema queda acotado por seis restricciones operativas que conviene declarar antes de pasar al desarrollo, ya que han condicionado decisiones de diseño que se retomarán en el Capítulo 4. Las dos primeras son de orden material y se refieren a las fuentes de datos numéricos y al corpus de radio etiquetado manualmente por el autor. Las dos siguientes acotan los recursos físicos disponibles y el calendario del proyecto. Las dos últimas documentan, respectivamente, la dependencia operativa del modelo de lenguaje empleado por la capa de razonamiento y una funcionalidad inicialmente prevista que se desestimó al constatarse la ausencia de una fuente real que la sustentara.

Datos. El sistema opera exclusivamente sobre las fuentes públicas FastF1 y OpenF1, sin acceso a la telemetría propietaria de los equipos, lo que limita la resolución disponible y excluye el bus CAN completo y los canales crudos de suspensión y motor. Estas APIs reportan el compuesto montado con la nomenclatura genérica del fin de semana (*soft*, *medium*, *hard*), por lo que su mapeo a la nomenclatura oficial Pirelli C1 – C5, necesaria para el modelado entre carreras, se ha realizado manualmente a partir de las notas de prensa del fabricante. El reparto temporal del corpus es estricto, con las temporadas 2023 y 2024 reservadas para entrenamiento y la temporada 2025 para la prueba final, lo que evita el salto distribucional introducido por el cambio reglamentario aerodinámico de 2022.

Corpus de radio anotado manualmente. El conjunto de entrenamiento de los modelos de procesamiento de radio se compone de aproximadamente quinientos treinta mensajes transcritos y etiquetados manualmente por el autor sobre las grabaciones públicas de OpenF1. Esta anotación condiciona dos aspectos relevantes del rendimiento del subsistema de procesamiento de lenguaje natural. Por un lado, la clase advertencia del clasificador de intención queda representada por unos setenta y cinco ejemplos, lo que reduce su capacidad de generalización frente al resto de categorías. Por otro, el reconocedor de entidades nombradas alcanza un valor de F_1 a nivel de *token* de 0,42, cifra que el corpus disponible no permite mejorar sin un esfuerzo de etiquetado superior al alcance del trabajo.

Hardware. El entrenamiento y la inferencia local del sistema se realizan sobre un equipo portátil dotado de una tarjeta gráfica NVIDIA GeForce RTX 5070 Laptop con 8 GB de memoria GDDR7 dedicada, un procesador AMD Ryzen AI 9 365 y 32 GB de memoria RAM. Esta configuración acota la huella admisible para los modelos del sistema y obliga a emplear arquitecturas ligeras, en concreto XGBoost, LightGBM, HistGradientBoosting, BERT-base y BERT-large y una red convolucional temporal compacta, descartando el entrenamiento de modelos de lenguaje de gran tamaño. La capa de razonamiento del orquestador delega, en consecuencia, en modelos de lenguaje servidos por API en la nube o en una instancia local de LM Studio compatible con el presupuesto de memoria de vídeo disponible.

Tiempo. La planificación inicial contemplaba dieciocho semanas de desarrollo. El alcance real del proyecto se ha extendido hasta abril de 2026 para incorporar los cuadernos N15 y N16 sobre paradas y undercut, así como la arquitectura multi-ventana de la réplica de carrera sobre Arcade.

Dependencia del modelo de lenguaje. La capa de razonamiento del orquestador y los sub-agentes delegan en un modelo de lenguaje servido por la API en la nube de OpenAI o por una instancia local de LM Studio, intercambiables a través de una variable de configuración. Esta dependencia es, con diferencia, la más crítica del sistema completo, ya que el rendimiento global de la cadena de decisión queda condicionado por la capacidad de razonamiento, la latencia y la disponibilidad del modelo escogido, ninguno de cuyos pesos ni datos de entrenamiento se han controlado dentro del marco del presente trabajo. Sobre la configuración actual, los resultados del orquestador son satisfactorios en condiciones nominales gracias al uso intensivo de prompts de sistema cuidadosamente diseñados,

salida estructurada validada por esquemas y aporte explícito de las inferencias parciales como contexto de cada consulta, técnicas que se desarrollan en el Capítulo 4. La solución óptima a fecha de mayo de 2026 sería, no obstante, disponer del hardware y de un corpus específicamente curado del dominio, con terminología técnica, jerga del paddock, articulado del reglamento deportivo, comentario estratégico de carrera y casos históricos relevantes, para realizar un ajuste fino de un modelo de lenguaje de pesos abiertos, por ejemplo de la familia Gemma de Google DeepMind (Gemma Team, 2025), con el objetivo de dotar al razonador final de un conocimiento profundo y especializado en Fórmula 1 que ningún modelo generalista alcanza por defecto. Ese ejercicio excede holgadamente el alcance del trabajo actual y queda apuntado como línea natural de extensión en el Capítulo 6.

Alcance descartado. La ingesta en tiempo real mediante Kafka y la difusión por WebSocket entre subsistemas se desestimaron en abril de 2026 al constatar la ausencia de una fuente de telemetría en directo accesible al desarrollo. La reincorporación de este componente queda señalada como línea de trabajo futuro en el Capítulo 6.

Establecidos el panorama, las oportunidades del campo, los requisitos del sistema y las restricciones operativas que lo enmarcan, el Capítulo 4 recorre el proceso completo de construcción del sistema, desde los modelos predictivos individuales y la canalización de procesamiento de lenguaje natural hasta el orquestador multi-agente y las tres superficies de uso finales, justificando en cada paso las decisiones de diseño tomadas a la luz de las alternativas inicialmente plausibles y de las restricciones operativas que el trabajo asume.

4 DESARROLLO

La evolución es a menudo la clave, una vez que ha saltado la chispa de una dirección adecuada.

— Adrian Newey, diseñador de Fórmula 1¹

El capítulo anterior enuncia quince requisitos funcionales, ocho requisitos no funcionales y seis restricciones operativas que delimitan el alcance del sistema. Por lo tanto, resta ahora mostrar cómo cada uno de ellos se traduce en una decisión concreta de arquitectura, en un modelo entrenado o en una superficie de uso.

La exposición se ordena en tres planos. El primero, el diseño, justifica la separación del sistema en tres capas, junto a los patrones de orquestación que las conectan y la pila tecnológica que las soporta. El segundo es la implementación, donde se recorren uno por uno los sub-agentes del grafo y se aplica el mismo esquema a cada bloque, primero la ingeniería de datos y el análisis exploratorio que lo alimenta y después el modelo entrenado y su integración como agente. El tercero, los casos de uso, encadena las tres superficies (línea de comandos, panel de análisis post-carrera y réplica visual síncrona) sobre un núcleo común a todas ellas.

La construcción del sistema responde a una evolución, no a un punto de partida. Esta versión hereda los cimientos de F1Strat-Legacy y F1 Telemetry Manager (Vega Sobral, 2025; Vega Sobral & Souto Ortega, 2025) y consolida la transición desde un motor experto basado en reglas hacia un sistema multiagente con razonamiento probabilístico y de lenguaje. La validación cuantitativa de cada decisión queda reservada para Capítulo 5.

4.1 Diseño

El diseño se aborda en tres niveles sucesivos de concreción que descienden desde la arquitectura abstracta hacia su materialización tecnológica. La descomposición del sistema en componentes con responsabilidades diferenciadas se motiva en el Subsección 4.1.1, donde se justifica frente a las alternativas más planas inicialmente plausibles. Una vez fijada esa partición, los contratos que conectan los componentes entre sí, tanto el mensaje que entra a la capa de agentes como el formato estructurado de salida que cada uno emite hacia el orquestador, se establecen en el Subsección 4.1.2. Sobre ambas elecciones se asienta finalmente la pila tecnológica que las materializa, agrupada por capa y por función en el Subsección 4.1.3. Cada subsección clausura, en suma, cuestiones que las siguientes presuponen como ya respondidas, de manera que la implementación posterior arranca con las decisiones globales ya tomadas.

4.1.1 Visión de sistema en tres capas

El sistema se organiza en tres capas con responsabilidades exclusivas y comunicación unidireccional, tal como se representa en Figura 4.1. La capa inferior es la encargada de consolidar los datos crudos en una representación canónica de la sesión vuelta a vuelta. La capa intermedia ejecuta la inferencia distribuida de los seis sub-agentes especializados que abordan cada dimensión estratégica de la carrera.

¹Fragmento de *How to Build a Car: The Autobiography of the World's Greatest Formula 1 Designer*, de Adrian Newey (2017). Traducción propia del original en inglés: *Evolution is often the key once the spark of a good direction has been set.*

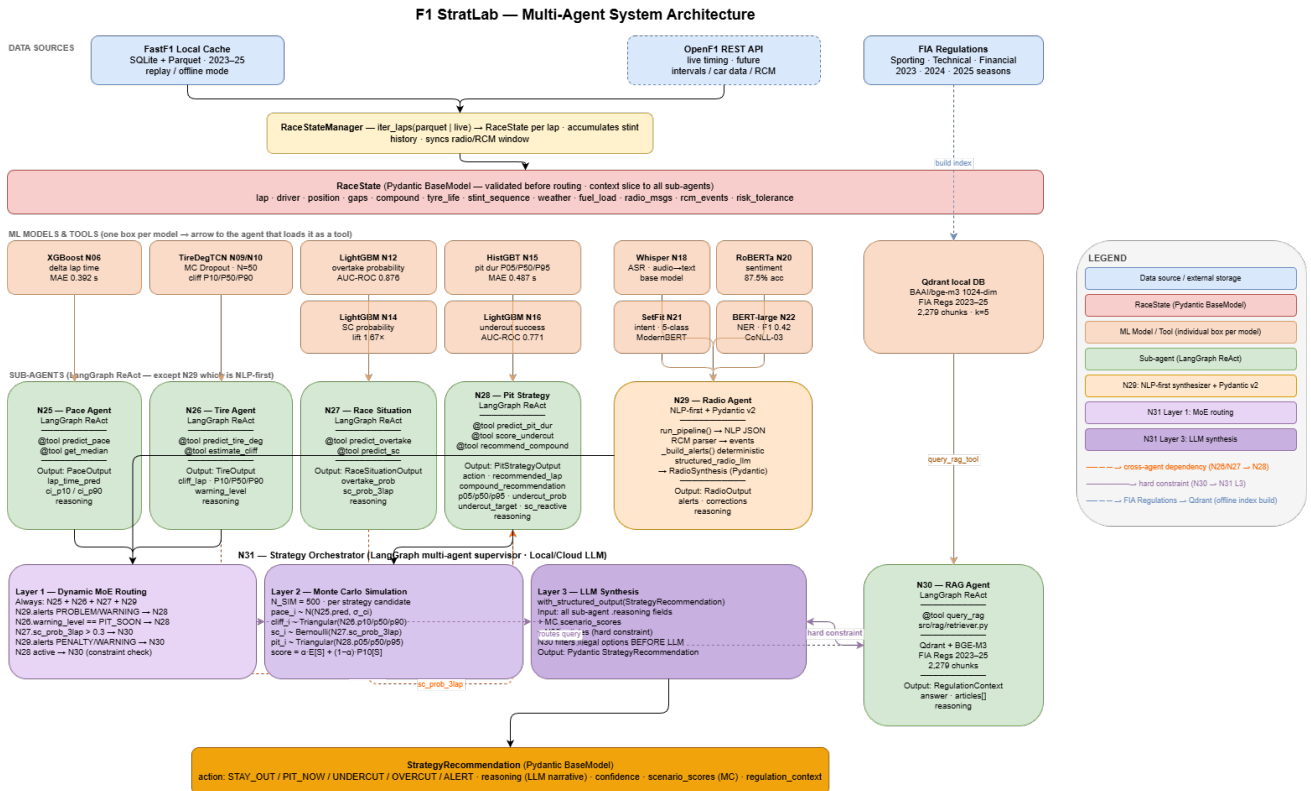


Figura 4.1: Arquitectura del sistema en tres capas. La capa inferior consolida los datos de FastF1, OpenF1 y los reglamentos FIA en el contrato canónico que recorre todo el sistema, instanciado a través del componente RaceStateManager. La capa intermedia ejecuta los seis sub-agentes especializados con sus modelos entrenados asociados (XGBoost N06, TCN con MC Dropout N09/N10, LightGBM N12 y N14, HistGBT N15, LightGBM N16, Whisper N18, RoBERTa N20, SetFit N21, BERT-large N22, Qdrant + BGE-M3). La capa superior corresponde al orquestador N31, que enruta selectivamente los sub-agentes condicionales, ejecuta la simulación Monte Carlo y delega la síntesis final al modelo de lenguaje, devolviendo un StrategyRecommendation validado por esquema. Diagrama propio del proyecto.

Finalmente, la capa superior los coordina, pondera sus salidas mediante la simulación estocástica y emite la recomendación final acompañada del razonamiento que la sustenta.

La capa inferior descansa sobre dos componentes complementarios. El primero, RaceReplayEngine, reproduce de forma determinista la cronología completa de una sesión a partir del corpus de carreras consolidado en el Subsección 4.2.1. El segundo, RaceStateManager, agrega esa cronología en un objeto canónico denominado lap_state, que consolida para cada vuelta y piloto la telemetría, la posición, el compuesto montado y las condiciones meteorológicas en un único punto de entrada a la capa de inferencia. La definición formal del contrato y las razones de diseño detrás de esta elección se desarrollan en el Subsección 4.1.2.

Sobre ese contrato actúa la capa intermedia, que agrupa los seis modelos del sistema en sub-agentes funcionales dedicados respectivamente al ritmo (*PaceAgent*), a la degradación del neumático (*TireAgent*), a la situación de carrera (*RaceSituationAgent*), a la estrategia de paradas (*PitStrategyAgent*), al procesamiento de la radio (*RadioAgent*) y a la consulta del reglamento (*RagAgent*). Cada uno consume el lap_state y devuelve una salida estructurada conforme a un esquema tipado propio, lo que permite tratar al conjunto como una colección homogénea de servicios cuya implementación interna queda encapsulada. Por encima de todos ellos, la capa superior encadena sus salidas, ejecuta una simulación Monte Carlo sobre cuatro acciones discretas y delega la síntesis final a un modelo de lenguaje guiado por el patrón supervisor, descrito en detalle en el Subsección 4.2.8.

La opción de mantener una arquitectura más plana se descartó por motivos de verificabilidad y trazabilidad. Una arquitectura monolítica que mezclase la extracción de datos, la inferencia numérica y el razonamiento del modelo de lenguaje forzaría a unificar tipos de respuesta, escalas temporales y modos de validación incompatibles entre sí. Una canalización estrictamente procedimental, sin sub-agentes intermedios, impediría someter las predicciones a esquemas de salida verificables y obligaría al modelo de lenguaje final a interpretar variables crudas sin un protocolo común. Ambos riesgos se neutralizan al fijar dos contratos explícitos, uno de entrada y uno de salida, cuya formalización se aborda en el Subsección 4.1.2.

4.1.2 Patrones y contratos de orquestación

Los componentes de la arquitectura quedan vinculados entre sí mediante dos contratos formales. La entrada a la capa de inferencia es un único objeto canónico, denominado `lap_state`, construido por el método `get_lap_state()` de `RaceStateManager` y compuesto siempre por cinco bloques estables, a saber, el número de vuelta, el estado del piloto consultado, la lista de rivales, las condiciones meteorológicas y los metadatos de sesión. Mantener un solo punto de entrada para los seis sub-agentes evita que la lógica de extracción se duplique y permite que la sustitución de la fuente de datos, ya sea sesión FastF1 viva o reconstrucción desde *parquet*, no afecte a los modelos. Por la parte de salida, los seis sub-agentes especializados entregan un `dataclass` con campos derivados (`PaceOutput`, `TireOutput`, `RaceSituationOutput`, `PitStrategyOutput`, `RadioOutput` y `RegulationContext`), lo que permite calcular automáticamente el nivel de alerta o de amenaza durante su construcción. El orquestador, en cambio, devuelve un modelo Pydantic, `StrategyRecommendation`, formato exigido por la API de salida estructurada del modelo de lenguaje para garantizar que la recomendación final se valida por esquema antes de emitirse.

Cinco de los seis sub-agentes (*PaceAgent*, *TireAgent*, *RaceSituationAgent*, *PitStrategyAgent* y *RagAgent*) se construyen como agentes ReAct en el sentido de Yao et al. (2023), es decir, un agente basado en un modelo de lenguaje que alterna razonamiento textual y llamadas a un conjunto reducido de *tools* deterministas que encapsulan la inferencia numérica de los modelos entrenados correspondientes. El sexto, el *RadioAgent*, es también un sub-agente del sistema pero sigue un patrón distinto, denominado *NLP-first* (canalización determinista de PLN previa al modelo de lenguaje), en el que la canalización de procesamiento del lenguaje natural ejecuta primero la transcripción, el sentimiento, la intención y la extracción de entidades de forma determinista, y solo después un modelo de lenguaje sintetiza el resultado en una salida estructurada validada por esquema. La razón del corte es que las alertas críticas de radio (banderas, penalizaciones, problemas mecánicos) deben extraerse con garantías deterministas que el patrón ReAct no asegura, ya que en ReAct es el propio agente quien decide cuándo invocar cada tool. En todos los casos, sea ReAct o *NLP-first*, el sub-agente devuelve la salida tipada al orquestador y la separación entre razonamiento del agente y cálculo numérico en sus componentes internos se mantiene, lo que conserva la inferencia trazable.

Sobre estos seis sub-agentes se encuentra el orquestador, que actúa como un supervisor implementado manualmente en tres capas. La primera es la encargada de ejecutar los cuatro sub-agentes que siempre se encuentran activos, es decir, los dedicados al ritmo de carrera, al neumático, a la situación y a la radio. A continuación se aplica un enrutado determinista de tipo mezcla de expertos (*mixture-of-experts*, MoE) sobre los dos condicionales, dedicados a la estrategia de paradas y a la consulta del reglamento. Esa activación selectiva se inscribe en la noción de cómputo selectivo formalizada por Bengio et al. (2013) y aplicada a sistemas multi-agente como activación dirigida de herramientas (Wu et al., 2023). Posteriormente, la segunda capa ejecuta una simulación Monte Carlo sobre cuatro estrategias candidatas y agrega los resultados en puntuaciones probabilísticas. La tercera delega la síntesis al modelo de lenguaje con salida estructurada, que recibe las salidas de los sub-agentes activados junto con las puntuaciones Monte Carlo y devuelve el `StrategyRecommendation` ya validado, descrito en detalle en el Subsección 4.2.8.

La elección del supervisor manual se valoró desde el inicio frente a la alternativa más natural a este nivel sistémico, una compuerta aprendida (*gating network*) en el sentido clásico de la mezcla de expertos (Jacobs et al., 1991), donde una red entrenada decide qué expertos activar mediante una distribución *softmax* sobre el conjunto, refinada en su versión moderna y dispersa por Shazeer et al. (2017). Esa vía se descartó por dos motivos, la ausencia de un corpus etiquetado en el que cada estado de carrera vaya emparejado con su subconjunto óptimo de sub-agentes activados y la imposibilidad de auditar regla a regla un vector de pesos *softmax*, requisito recogido en el RNF-05 de explicabilidad. La alternativa de un enrutador conversacional sustentado en un modelo de lenguaje también se desestimó por la latencia añadida que supondría una llamada extra al modelo antes de cualquier inferencia, contraria al RNF-01 de operación vuelta a vuelta.

Por debajo de esos patrones de orquestación, cada sub-agente expone dos puntos de entrada con la misma lógica interna pero distinta fuente de datos: `run_*_agent()` opera sobre una sesión FastF1 viva, y `run_*_agent_from_state()` reconstruye los metadatos desde el `lap_state` y un `DataFrame` histórico. En el alcance de este trabajo se emplea exclusivamente la segunda vía, ya que el sistema procesa carreras ya disputadas sin canal de *live timing* accesible; la primera queda incorporada para un eventual despliegue en directo. La equivalencia entre ambas vías es la condición que permite cambiar la fuente de datos sin tocar los modelos entrenados ni las herramientas que los envuelven. La pila tecnológica concreta sobre la que descansa todo lo anterior se enumera en la siguiente subsección.

4.1.3 Stack tecnológico

La pila tecnológica del sistema responde a tres principios. El primero es el uso íntegro de software de código abierto, en cumplimiento del RNF-04 declarado en Capítulo 3. El segundo obedece a la separación entre el motor de inferencia local y el modelo de lenguaje, que puede consumir tanto un proveedor en la nube como una instancia local servida por LM Studio sin alterar la pila. El tercero es la reproducibilidad estricta del entorno mediante `uv` como gestor de paquetes y `uv.lock` como manifiesto inmutable de versiones, lo que permite reconstruir el sistema con un solo comando sobre cualquier máquina compatible con Python 3.10 a 3.12. Los pesos entrenados, los índices Qdrant precomputados y el corpus consolidado se publican en *HuggingFace Hub* bajo el repositorio `VforVitorio/f1-strategy-dataset` y se descargan de forma diferida en la primera ejecución, lo que evita versionar binarios pesados en Git y mantiene la instalación en una sola orden.

Ahora bien, el marco de aprendizaje profundo merece mención aparte. PyTorch 2.11 se instala desde la *wheel* compilada para CUDA 12.8 (`cu128`), única que soporta la arquitectura *Blackwell* de la familia GeForce RTX 50 declarada en el Capítulo 3 sobre la que se entrenan y despliegan los modelos. La selección por plataforma se gestiona desde la sección `[tool.uv.sources]` del `pyproject.toml`, que dirige a Linux y Windows hacia el índice `pytorch-cu128` y a macOS hacia el índice de CPU, sin que el usuario tenga que intervenir manualmente al ejecutar `uv sync`.

La Tabla 4.1 resume los componentes elegidos en cada capa. La distribución del sistema se completa con cuatro puntos de entrada declarados en `pyproject.toml`, en concreto `f1-strat` como router general, `f1-sim` para la simulación sin entorno gráfico, `f1-arcade` para la réplica visual y `f1-streamlit` para el panel post-carrera, lo que permite invocar las tres superficies desde la línea de comandos sin scripts auxiliares.

Más allá de las dependencias enumeradas, el desarrollo se organiza sobre un flujo Git de tres ramas (`test`, `dev` y `main`), con `main` protegida frente a empujes forzados y borrados accidentales. Toda fusión a la rama principal exige superar tres comprobaciones automáticas que GitHub Actions (GitHub, 2024b), plataforma nativa de integración continua basada en flujos de trabajo declarados en YAML, ejecuta en paralelo sobre cada *pull request*: pruebas unitarias con `pytest`, verificación de estilo y formato con `ruff` y comprobación

Tabla 4.1: Resumen del stack tecnológico agrupado por capa funcional. Las versiones se toman literalmente del `pyproject.toml` y del `uv.lock` en su estado de abril de 2026. Las dependencias auxiliares de tokenización, validación menor y herramientas de formato quedan fuera del resumen.

Capa	Componente	Tecnología y versión	Propósito
Runtime	Intérprete y entorno	Python 3.10–3.12, uv	Base de ejecución y resolución de <i>wheels</i>
Datos	Adquisición	FastF1 3.7 (Oehrly, 2024), OpenF1 (OpenF1 Project, 2024)	Telemetría, radio y mensajes de dirección
Datos	Tabular y serialización	Pandas 2.3, NumPy 1.26, PyArrow 23	Manipulación y escritura <i>parquet</i>
Aprendizaje profundo	Marco GPU	PyTorch 2.11 + torchvision 0.26 (cu128)	TCN y <i>transformers</i> de PLN en GPU
Aprendizaje tabular	<i>Gradient boosting</i>	XGBoost 3.2, LightGBM 4.6, HistGB (sklearn)	Tiempo de vuelta, adelantamiento, SC, parada y <i>undercut</i>
Aprendizaje tabular	<i>Tuning</i>	Optuna 4.7	Búsqueda bayesiana de hiperparámetros
PLN	ASR	Whisper <i>turbo</i>	Transcripción de radio piloto-muro
PLN	Clasificación y NER	Transformers 5.1, SetFit 1.1, sentence-transformers 5.2	Sentimiento, intención y entidades F1
Recuperación	Almacén vectorial	Qdrant local 1.16	Índice del reglamento FIA
Recuperación	<i>Embeddings</i>	BGE-M3 (<i>HuggingFace Hub</i>)	1024 dimensiones, similitud coseno
Agentes	Orquestación	LangGraph 1.0, LangChain 1.2, langchain-openai 1.1	Sub-agentes ReAct y conexión con el modelo de lenguaje
Agentes	Validación	Pydantic 2.12	Esquemas de salida estructurada
<i>Backend</i>	API REST y servidor	FastAPI 0.128, Uvicorn 0.40	Submódulo <code>src/telemetry</code>
<i>Backend</i>	Difusión local	WebSockets 15, socket TCP en puerto 9998	<i>Replay</i> hacia paneles Qt en directo
Frontend web	Panel post-carrera	Streamlit 1.54, Plotly 6.5	Análisis multimodal con FastMCP
Réplica 2D	Visualización	Arcade 3.3 (pyglet)	Réplica visual sincronizada
Paneles Qt	Estrategia y telemetría	PySide6 6.11, pyqtgraph 0.14	Panel estratégico y curvas en directo
Despliegue	Contenedores	Docker Compose, python:3.10-slim	Servicios backend: 8000 y frontend: 8501
Calidad	Pruebas y formato	pytest 9.0, ruff 0.15, black 26.1	RNF-07 (mantenibilidad)

estática de tipos con `mypy` sobre el subsistema de recuperación. La automatización se extiende al ciclo de publicación mediante `release-please` (Google APIs, 2024), herramienta que infiere la siguiente versión semántica a partir de los mensajes de `commit` acumulados desde la última etiqueta siguiendo la convención abierta *Conventional Commits* (Conventional Commits, 2019), abre el *pull request* con las notas de versión y, al fusionarse, dispara la construcción de la *wheel* y la distribución de fuentes y su publicación como artefactos del *release* de GitHub. Sobre esta base, Dependabot (GitHub, 2024a) vigila las dependencias declaradas y el grafo de seguridad de forma continuada, abriendo automáticamente *pull requests* con la actualización cuando detecta vulnerabilidades, y el revisor automático de Copilot complementa la inspección humana de cada uno. El conjunto materializa el RNF-07 de mantenibilidad declarado en el Capítulo 3 y deja la rama `main` en un estado siempre desplegable, condición técnica para que el sistema pueda salir del aula y operar en un entorno de producción real.

4.2 Implementación de la solución

La implementación se despliega en nueve subsecciones que siguen tres tipos de exposición. La primera (Subsección 4.2.1) describe la ingeniería de datos transversal que precede a cualquier inferencia y comprende la consolidación del corpus, el agrupamiento de circuitos en arquetipos y la construcción del dataset featurizado del que beben todos los modelos. Las seis siguientes (Subsección 4.2.2 a Subsección 4.2.7) recorren los seis sub-agentes especializados bajo un patrón uniforme dividido en dos bloques, primero los datos y el análisis exploratorio que sustentan cada modelo, y después el modelo entrenado y su integración como agente sobre el contrato `lap_state`. Las dos últimas (Subsección 4.2.8 y Subsección 4.2.9) rompen ese patrón por su naturaleza, ya que el orquestador no posee un modelo propio entrenado sino una mecánica de coordinación entre los anteriores, y las interfaces de uso no son agentes sino superficies de salida del sistema.

4.2.1 Ingeniería de datos

La capa de datos del sistema persigue un objetivo práctico: disponer de un corpus consolidado de carreras reales, organizado por año y por gran premio, que los modelos puedan leer como una sola tabla sin tener que invocar las APIs originales ni combinarlas en tiempo de inferencia. Construirlo obliga a integrar dos canales abiertos de telemetría y cronometraje complementarios y a resolver los desajustes entre ambos. A ese trabajo de integración se suma el RNF-08 de integridad temporal (Capítulo 3), que condiciona directamente la construcción de las variables. La subsección desarrolla los tres bloques que aterrizan ambas ideas. Comienza con la consolidación del corpus y la resolución del mapeo Pirelli ausente en los dos canales, prosigue con el agrupamiento de los circuitos en cuatro arquetipos que actúa como variable transversal de pista para los modelos sensibles al ritmo y a la dinámica del trazado, y cierra con la construcción del dataset featurizado de cincuenta y tres características del que parten todos los modelos numéricos del sistema.

4.2.1.1 Extracción y consolidación

El corpus que alimenta toda la capa de inferencia se consolida a partir de dos canales abiertos y complementarios. FastF1 (Oehrly, 2024) es una biblioteca de Python que envuelve el flujo de cronometraje en directo del campeonato y expone una gran variedad de datos como vueltas, telemetrías, paradas y datos meteorológicos por sesión. Para complementar esta información, se usa OpenF1 (OpenF1 Project, 2024), una interfaz REST comunitaria que consume ese mismo flujo y reexpone dos magnitudes que la librería anterior no cubre con detalle suficiente, como son los intervalos vuelta a vuelta entre coches y los audios de radio piloto-muro junto con los mensajes de dirección de carrera (*Race Control Messages*, RCM) emitidos durante el evento. El reparto entre ambos canales es estrictamente operativo, donde FastF1 aporta vueltas, sectores y telemetría base, mientras que OpenF1 se encarga de cubrir los intervalos finos necesarios para detectar ventanas de undercut, además de proporcionar las radios y los mensajes de control de carrera ya mapeados al piloto y vuelta correspondiente por coincidencia de intervalos.

Sobre ambos canales se construye un corpus consolidado de sesenta y nueve grandes premios, cuarenta y siete entre las temporadas 2023 y 2024 y veintidós de la temporada 2025, dando un total de 68.122 vueltas con datos (22.106 en 2023, 23.256 en 2024 y 22.760 en 2025). El resultado se serializa en tres parquet anuales bajo `data/processed/laps_featured_YYYY.parquet`, lo que evita cualquier llamada a las APIs originales en tiempo de inferencia.

No obstante, sobre dicha consolidación pesa un desajuste que es recurrente en ambos canales de datos. Ninguno de ellos expone los compuestos absolutos C1 a C5 con los que Pirelli identifica sus mezclas reales, sino solo las etiquetas relativas SOFT, MEDIUM y HARD, cuyo significado cambia de un gran premio a otro

según la dotación nominal de la cita y lo que provocaría una mala praxis a la hora de entrenar y producir buenas inferencias en el modelo de degradación de neumáticos. Como este modelo requiere del código absoluto, se ha realizado un etiquetado manual a través de las fuentes de Pirelli Preview oficial publicadas antes de cada uno de los Grandes Premios, y se han serializado en `data/tire_compounds_by_race.json`, cuya estructura por año, circuito y nominal queda fijada como contrato de lectura para toda la capa de inferencia posterior.

Conviene además precisar que el corpus arranca en 2023 y no excluye temporadas previas mediante un corte rígido asociado al cambio reglamentario de 2022, sino que asume la deriva inter-temporadas (estimada entre 0,5 y 1,5 segundos por temporada como consecuencia del desarrollo aerodinámico) y la mitiga mediante los tres mecanismos descritos en el Sección 2.2: una normalización relativa intra-sesión, la inclusión de `Year` como variable de entrada (cuya importancia agregada en familias XGBoost sobre F1 se ha cuantificado en el 23,8 % (Bansal et al., 2025)) y una ponderación muestral exponencial que prioriza las temporadas más recientes. Resueltos así el corpus y los compuestos, queda por incorporar la dimensión de pista, dado que el comportamiento del coche, la degradación del neumático y la frecuencia de incidentes varían más entre circuitos que entre temporadas, lo que se aborda en el siguiente apartado agrupando los trazados del calendario en cuatro arquetipos derivados de su propia telemetría.

4.2.1.2 Agrupamiento de circuitos en cuatro arquetipos

La motivación para agrupar los circuitos en arquetipos de comportamiento estratégico, los cuatro grupos resultantes y su papel diferenciado en cada familia de modelos predictivos quedan razonados en Sección 2.8, donde la Tabla 2.3 sintetiza la composición y el rasgo distintivo de cada grupo. En este apartado se documenta cómo se materializa dicho agrupamiento.

El cuaderno Jupyter notebooks/data_engineering/NO3_circuit_clustering.ipynb construye paso a paso el conjunto de variables, ajusta el modelo y serializa tanto los pesos del clasificador como el escalador asociado en `data/models/k_means_circuit_clustering/`, junto con las asignaciones de cluster por circuito en `data/processed/circuit_clustering/`, lo que permite reutilizar la inferencia desde cualquier punto del sistema sin necesidad de reajustar el modelo.

Las quince variables se distribuyen en las cuatro dimensiones razonadas en el Sección 2.8, con la siguiente repartición concreta. La primera, velocidad y complejidad, recoge el ritmo medio del trazado, la dispersión de los tiempos de vuelta y el mejor tiempo absoluto, magnitudes que sirven para separar circuitos rápidos y cortos como Spielberg de circuitos largos como Spa-Francorchamps. La segunda, patrones de degradación del neumático, captura la pendiente de una regresión lineal del tiempo de vuelta frente a la edad del neumático, la longitud máxima de los stints observados y la varianza dentro de cada uno de ellos. La tercera, estrategia de paradas, agrega el número medio de paradas por piloto y la dispersión de los momentos de detención durante la carrera. La cuarta, condiciones ambientales, promedia las temperaturas de pista y aire junto con la presión atmosférica como aproximación indirecta a la altitud del circuito. El conjunto resultante se compone de cuatro variables de complejidad y velocidad, tres de degradación, cuatro de estrategia y cuatro ambientales, calculadas sobre las vueltas con duración inferior a ciento ochenta segundos para descartar efectos distorsionantes como coches de seguridad, banderas rojas o errores ocasionales en la captura de las telemetrías de los sensores de algunos monoplazas.

El análisis exploratorio sobre estas dimensiones aporta dos hallazgos que condicionan los pasos siguientes. El más llamativo afecta a la dimensión de la degradación, donde la regresión lineal devuelve pendientes mayoritariamente negativas en lugar de positivas, es decir, los coches resultan más rápidos conforme la edad de sus neumáticos aumenta. Este comportamiento contraintuitivo, ya constatado en Vega Sobral (2025), se explica por el efecto del consumo de combustible. Como cada kilogramo de combustible aligera el

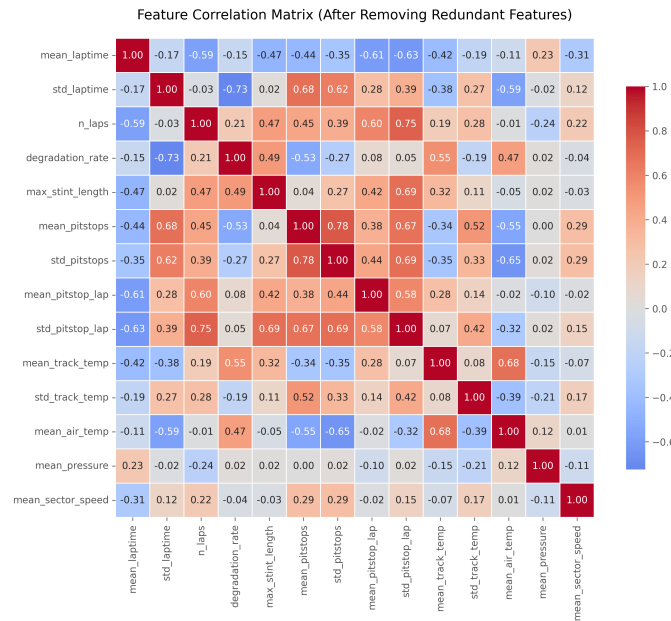


Figura 4.2: Matriz de correlación de las catorce variables del agrupamiento tras descartar `min_laptime` y `stint_variance` por su redundancia con `mean_laptime` y `std_laptime`, respectivamente. Ningún par conserva un coeficiente absoluto por encima de 0,78. Generada en el cuaderno N03.

coche en aproximadamente tres centésimas de segundo por vuelta y el depósito se vacía progresivamente a lo largo de la carrera, ese aligeramiento compensa o invierte el desgaste real del neumático en la mayoría de los circuitos, fenómeno bien documentado por la prensa especializada en estrategia de Fórmula 1. Por tanto, la pendiente medida actúa como firma estratégica conjunta del circuito (combustible más degradación) y se considera aceptable a efectos del agrupamiento, ya que diferencia bien casos extremos como Lusail (pendiente cercana a las nueve décimas negativas, es decir, el coche se acelera casi un segundo con el aligeramiento porque el combustible domina al desgaste elevado) frente a Imola (pendiente prácticamente nula, donde ambos efectos se equilibran). Sin embargo, esta lectura conjunta motiva que el conjunto de datos posterior incorpore una corrección explícita de carga de combustible cuando el objetivo sea aislar la degradación pura, decisión que se aborda en el apartado siguiente.

El segundo hallazgo se sitúa en la dimensión ambiental: la presión atmosférica revela un *outlier* pronunciado en Ciudad de México (en torno a 782 hectopascales, frente al rango habitual de 900 a 1000) por los más de dos mil doscientos metros de altitud del Autódromo Hermanos Rodríguez, lo que distorsiona también la velocidad punta absoluta por la menor densidad del aire. Ambos efectos justifican el primer refinamiento, que sustituye la velocidad punta por la media de las velocidades registradas en los puntos de detección de los tres sectores, huella del régimen del circuito más fiel que la velocidad punta absoluta. Tras esta incorporación el conjunto pasa a dieciséis variables; un análisis de correlación detecta dos pares por encima de 0,90 (ritmo medio con mejor tiempo absoluto, 0,974; varianza intra-stint con dispersión global, 0,919), y el segundo refinamiento poda el miembro redundante de cada par para evitar que la métrica euclídea sobrepondere dimensiones duplicadas, dejando catorce variables finales. La estructura resultante se recoge en Figura 4.2. Como paso previo al ajuste, el conjunto se estandariza con `StandardScaler` para neutralizar las diferencias de escala.

La elección final de cuatro grupos sobre estas catorce variables se sustenta en una validación numérica concreta. K-Means se ajusta con veinte reinicios aleatorios y semilla fija para garantizar reproducibilidad entre ejecuciones. El barrido del coeficiente de silueta para valores de k entre dos y ocho devuelve un máximo absoluto en $k = 2$ con valor 0,222, un ligero descenso a 0,198 en $k = 3$ y un repunte a 0,201 en

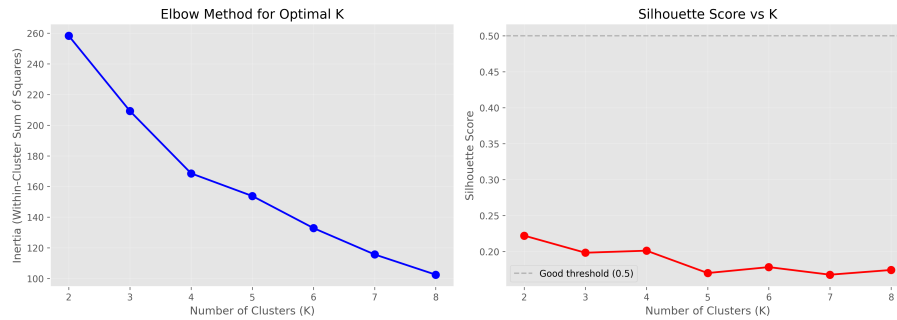


Figura 4.3: Validación conjunta del número de clusters mediante el método del codo (izquierda) y el coeficiente de silueta (derecha) para valores de k entre dos y ocho. La inercia presenta un punto de inflexión visible en $k = 4$; el coeficiente de silueta alcanza su máximo en $k = 2$ (0,222) y oscila entre 0,17 y 0,20 a partir de $k = 3$, siempre por debajo del umbral de buena separación 0,5. Generada en el cuaderno N03.

$k = 4$, todos ellos muy por debajo del umbral de buena separación 0,5, mientras que el codo de inercia se sitúa de manera nítida en $k = 4$ (inercia 168,55 frente a 258,27 en $k = 2$). Pese a no maximizar la silueta, la elección de cuatro grupos ofrece una resolución estratégica que una dicotomía de $k = 2$ no proporcionaría, y se alinea con la enumeración tradicional de arquetipos de circuito reconocida en la literatura (Gomathi L., 2025); la validación conjunta del codo y la silueta se reproduce en Figura 4.3.

Una vez completado el ajuste, el estimador y el escalador se guardan como `kmeans_circuit_classifier_k4.pkl` y `circuit_feature_scaler.pkl` bajo `data/models/k_means_circuit_clustering/`, y la tabla de asignaciones como `circuit_clusters_k4.parquet` en `data/processed/circuit_clustering/`. La temporada 2025 se procesa cargando los artefactos congelados y aplicando `predict`, lo que garantiza que las definiciones de cluster son idénticas entre entrenamiento y prueba, cumple el RNF-08 y produce `circuit_clusters_k4_2025.parquet` como variable categórica que el sistema consume vuelta a vuelta. Con la dinámica de pista codificada como atributo discreto y consistente entre temporadas, el siguiente apartado integra esta variable en el dataset featurizado del que parten todos los modelos numéricos.

4.2.1.3 Construcción del dataset featurizado

El cuaderno N04 parte de las 37 columnas heredadas de N01 y añade 34 variables derivadas en seis bloques hasta producir las 53 columnas de `laps_featured.parquet` que alimentan todos los modelos del sistema. El primero desconfunde los dos efectos superpuestos en el tiempo de vuelta crudo, la degradación del neumático y el aligeramiento por quema de combustible ya analizado en el Subsubsección 4.2.1.2, obteniendo `FuelAdjustedLapTime` como referencia libre del sesgo de carga. El segundo captura el momentum intra-stint con doce variables secuenciales (tiempo previo, delta y tendencia de segundo orden), donde la primera vuelta de cada stint se deja deliberadamente como valor nulo para preservar la semántica de ausencia de antecesor. El tercero estima la tasa local de degradación mediante una regresión lineal sobre ventanas deslizantes de tres vueltas, diseño más sensible al *graining* (desgaste por delaminación superficial de la goma) temprano y al cliff tardío que un promedio global del stint. El cuarto añade el contexto térmico de la sesión porque el comportamiento de un mismo compuesto varía radicalmente entre circuitos cálidos y fríos; el quinto incorpora la fase de carrera, las vueltas restantes y el estado de pista. El sexto integra la variable de clúster construida en el Subsubsección 4.2.1.2 junto con dos referencias relativas al arquetipo, cerrando el *pipeline* cuyo flujo se recoge en Figura 4.4.

La restricción de integridad temporal del RNF-08 condiciona la construcción de varias variables. El caso más ilustrativo es `lap_time_pct_of_race_fastest`: dado que F1 mejora entre 0,5 y 1,5 segundos por vuelta de un año al siguiente en el mismo circuito por desarrollo aerodinámico, una vuelta un 5 % más lenta

Data Pipeline — From Raw Telemetry to Strategy Recommendations

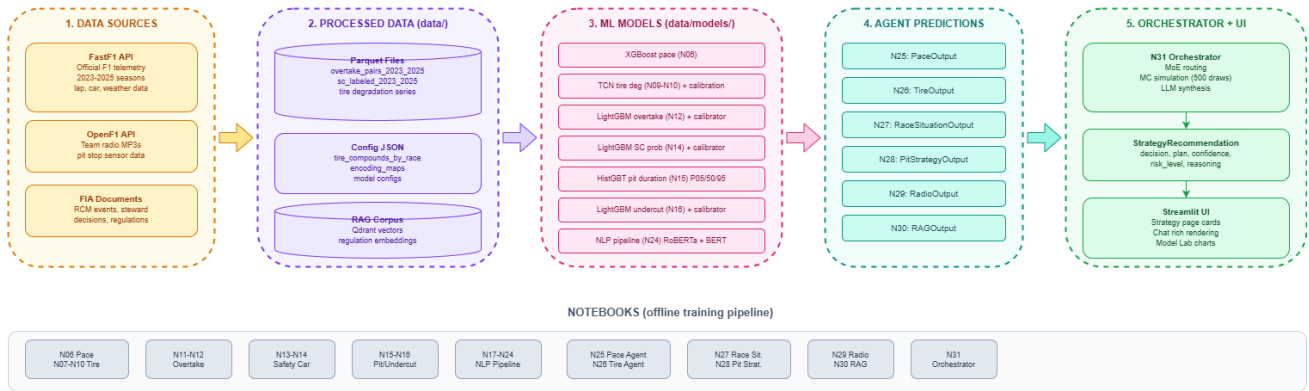


Figura 4.4: Pipeline de datos del sistema, desde las fuentes de adquisición hasta el dataset featurizado listo para inferencia. La capa izquierda agrupa las tres fuentes de entrada: FastF1 para telemetría y cronometraje, OpenF1 para intervalos y radios, y los documentos FIA para el reglamento. La capa central consolida esas fuentes en los artefactos procesados, entre los que destacan `laps_featured.parquet` (53 columnas, 68.122 vueltas) producido por N04, las tablas de asignación de clúster de N03 y el corpus vectorial de Qdrant para el agente RAG. La capa derecha muestra los modelos entrenados y los sub-agentes que los envuelven, cuya implementación se desarrolla en las subsecciones siguientes. Diagrama propio del proyecto.

que la mejor de esa misma carrera y temporada es genuinamente comparable con otra vuelta igualmente un 5 % más lenta en años distintos, mientras que los tiempos absolutos no lo son; por eso la variable se obtiene dividiendo cada tiempo de vuelta por el mínimo de ese gran premio y año, de modo que el cociente queda agnóstico al nivel de *pace* de la temporada y no incorpora estadísticos de otras carreras. Este criterio descarta alternativas como medianas anuales o mínimos globales, que filtrarían información futura de forma implícita. La aplicación de los tres mecanismos anti-deriva y la partición 2023 / 2024 / 2025 como entrenamiento, validación y prueba siguen el protocolo del Sección 2.2.

4.2.2 Pace Agent

El agente de ritmo (*Pace Agent* en el código) constituye el primer eslabón de la capa de inferencia y se ocupa de estimar el tiempo de cada vuelta y la dispersión esperada de esa estimación a partir del estado de la carrera al final de la vuelta anterior. Su salida no la consume otro sub-agente, sino directamente el orquestador, que la incorpora al prompt de síntesis del modelo de lenguaje como referencia cuantitativa del ritmo previsto y de su intervalo de confianza, y la utiliza también para calibrar la dispersión empleada en la simulación Monte Carlo de las cuatro estrategias candidatas. Por debajo, el sub-agente apoya su inferencia en un único regresor de *gradient boosting* entrenado con XGBoost (Sección 2.3), que predice el incremento del tiempo respecto a la vuelta anterior y al que se suma después `PrevLapTime` para reconstruir el tiempo absoluto. La cuantificación de incertidumbre se resuelve con un *bootstrap* que perturba ligeramente las variables más sensibles y devuelve los percentiles inferior y superior de la distribución resultante. La construcción se reparte en tres cuadernos: `N05_laptime_eda.ipynb` consolida y filtra las variables, `N06_laptime_model.ipynb` entrena el modelo y selecciona sus hiperparámetros, y `N25_pace_agent.ipynb` envuelve la inferencia como agente ReAct sobre el contrato `lap_state`.

4.2.2.1 Datos y análisis exploratorio

El cuaderno `N05_laptime_eda.ipynb` parte del conjunto de datos introducido en el Subsubsección 4.2.1.3 y aplica una segunda capa de filtrado específica del problema, hasta acotar el conjunto a 39 variables

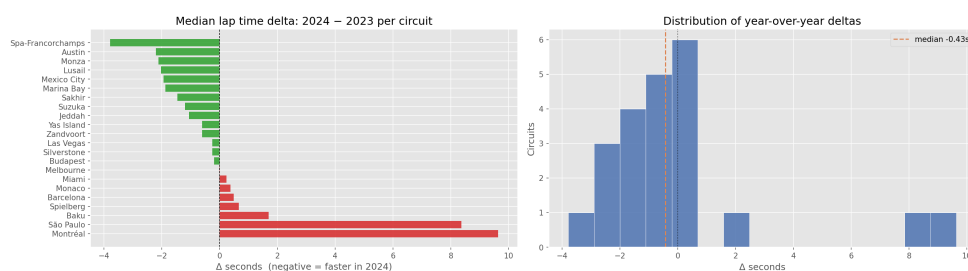


Figura 4.5: Diferencia mediana del tiempo por vuelta entre las temporadas 2024 y 2023 para los veintidós circuitos comunes. Valores negativos indican que el circuito fue más rápido en 2024. La interpretación de los casos extremos se discute en el texto. Generada en el cuaderno N05.

predictivas. El objetivo, `LapTime_s`, está dominado por una señal secuencial muy fuerte: el tiempo de la vuelta inmediatamente anterior alcanza una correlación de 0,989 con el de la vuelta actual, lo que convierte a `Prev_LapTime` en el ancla natural del ritmo del piloto. A esta señal se suma la variable categórica de clúster de circuito construida en el Subsubsección 4.2.1.2, cuya correlación de $-0,72$ refleja la pertenencia del trazado a uno de los cuatro arquetipos. El resto de variables presenta correlaciones lineales por debajo de 0,1, pero el modelo las aprovecha a través de las interacciones no lineales que captura un *ensemble* de árboles. Conviene señalar por último la exclusión deliberada de `FuelAdjustedLapTime`, cuya correlación de 0,999 no refleja una capacidad predictiva real sino una fuga de información (*data leakage*): la variable se construye sumando al propio `LapTime_s` una corrección por la carga de combustible, por lo que entrenar con ella equivaldría a darle al modelo el resultado que debe predecir. Este criterio se aplicó al conjunto completo y descartó también los tiempos por sector, que suman por definición `LapTime_s`, junto con otras variables derivadas del propio objetivo.

El análisis exploratorio aporta además una constatación cuantitativa de la deriva entre temporadas introducida en el Sección 2.2. De los veintidós circuitos que aparecen tanto en 2023 como en 2024, catorce resultaron más rápidos en 2024 con una mediana de mejora de $-0,43$ segundos por vuelta, valor que cae en el extremo inferior del rango esperado de 0,5 a 1,5 segundos anuales y que se atribuye a la evolución aerodinámica de los monoplazas. La distribución completa se recoge en Figura 4.5. La interpretación del extremo izquierdo del gráfico exige distinguir las reducciones explicables por esa evolución natural de aquellas atribuibles a circunstancias puntuales: Spa-Francorchamps marca $-3,79$ s, pero la carrera de 2023 se disputó sobre pista mojada mientras que la de 2024 fue seca, por lo que esa diferencia no informa sobre el desarrollo del coche sino sobre el cambio de condiciones. En contraste, el Gran Premio de Yas Marina presenta $-0,60$ s con ambas ediciones disputadas en seco, a temperaturas comparables y sin cambios en el trazado, lo que sí refleja la mejora real del paquete aerodinámico entre temporadas. La consecuencia metodológica es la misma: en lugar de predecir `LapTime_s` en valor absoluto, donde el desplazamiento entre temporadas sesgaría sistemáticamente las predicciones de 2025, se predice el delta respecto a la vuelta anterior, magnitud invariante por construcción al nivel absoluto de *pace* de la temporada y reconstruye en inferencia sumando `Prev_LapTime`.

4.2.2.2 Modelo entrenado e integración como sub-agente

El modelo se construye en `N06_laptime_model.ipynb` sobre el conjunto de 39 variables descrito en el apartado anterior. La elección recae en XGBoost (T. Chen & Guestrin, 2016) por las propiedades razonadas en el Sección 2.3, con un único matiz operativo añadido en este problema: el soporte nativo para valores ausentes resulta crítico aquí porque la primera vuelta de cada stint carece de antecesor y `Prev_LapTime` entra como nulo legítimo en la inferencia. Una decisión metodológica clave es la formulación del problema en términos de delta: el modelo predice el incremento del tiempo respecto a la vuelta anterior y el tiempo

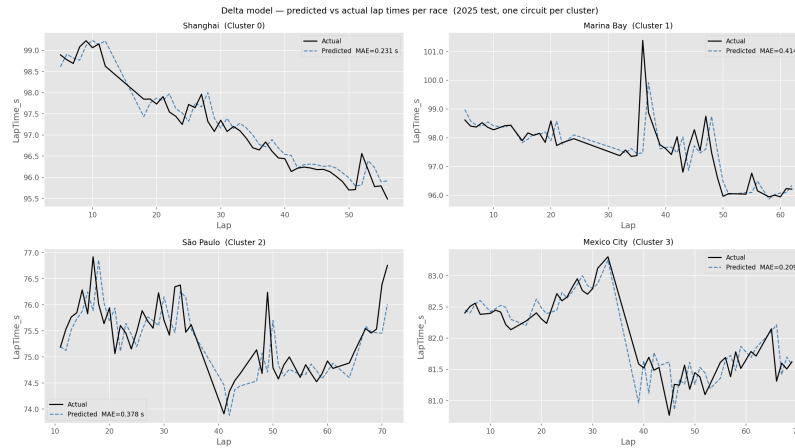


Figura 4.6: Comparación entre el tiempo por vuelta real (línea continua negra) y el reconstruido a partir de la predicción del modelo (línea discontinua azul) sobre cuatro carreras de la temporada 2025, una representativa de cada arquetipo de clúster: Shanghái (Cluster 0), Marina Bay (Cluster 1), São Paulo (Cluster 2) y Ciudad de México (Cluster 3). El error medio absoluto reportado en cada panel ilustra la consistencia del modelo entre arquetipos, con valores entre 0,21 y 0,41 segundos sobre circuitos no utilizados durante el entrenamiento. Generada en el cuaderno N06.

absoluto se reconstruye a posteriori sumando `Prev_LapTime`, lo que aísla al regresor de la deriva intertemporada del nivel absoluto de *pace*. La búsqueda de hiperparámetros se realiza con `RandomizedSearchCV` sobre 150 combinaciones de siete parámetros siguiendo el protocolo de bloques temporales del Sección 2.2, con 2023 como entrenamiento y 2024 como validación; una vez fijados, el modelo se reentrena sobre la combinación 2023+2024 y se evalúa sobre el *holdout* íntegro de 2025. La configuración retenida del modelo delta fija `max_depth=5`, `learning_rate=0,08`, `min_child_weight=30`, `reg_lambda=10` y `subsample=0,7`, sin restricciones monótonicas sobre las variables porque la formulación delta admite incrementos negativos de tiempo (evolución de pista, recuperación térmica, rebufo) y la regularización por profundidad y peso mínimo de hoja contiene por sí sola la varianza vuelta a vuelta. Sobre el *holdout* de 2025, no expuesto durante el entrenamiento ni la validación, el modelo alcanza un error medio absoluto (MAE) de 0,410 segundos y un coeficiente de determinación R^2 de 0,995, con veinte de los veinticuatro circuitos por debajo de los 0,5 segundos de error. La motivación inicial de la reformulación delta fueron precisamente Shanghái y Austin, que con el modelo absoluto presentaban MAE de 5,81 y 4,15 segundos por cambios estructurales del trazado entre temporadas; sobre el target delta ambos descienden a 0,35 y 0,45 segundos respectivamente, lo que confirma que el problema original residía en el nivel absoluto y no en la dinámica vuelta a vuelta. El modelo final se serializa en `data/models/lap_time/xgb_laptime_delta_final.json` y se carga sin reentrenamiento desde el sub-agente. La Figura 4.6 ilustra esta consistencia sobre cuatro carreras del conjunto de prueba 2025, una por cada arquetipo de circuito.

La envoltura como sub-agente se realiza en `N25_pace_agent.ipynb` y se traslada al módulo `src/agents/pace_agent.py`, siguiendo el patrón ReAct ya formalizado en el Subsección 4.1.2. Sobre la inferencia base se añade una capa de cuantificación de incertidumbre por bootstrap paramétrico: se realizan doscientas ejecuciones del modelo introduciendo en cada una un ruido gaussiano del 2 % sobre el subconjunto de variables sensibles a la perturbación local de la vuelta (`Prev_LapTime`, `Prev_SpeedST`, `mean_sector_speed`, `AirTemp`, `TrackTemp` y `TyreLife`), y se devuelven los percentiles 10 y 90 de la distribución resultante como envoltura de la predicción puntual. La salida tipada `PaceOutput` reúne seis campos: el tiempo absoluto reconstruido, el delta respecto a la vuelta anterior, el delta respecto a la mediana de la sesión para ese gran premio, año y compuesto, los dos percentiles de la incertidumbre y el razonamiento textual emitido por el modelo de lenguaje. El razonamiento se construye a través de dos herramientas deterministas que el agente invoca vía `LangGraph` (LangChain Inc., 2024): `predict_pace_tool`, que devuelve la predicción y

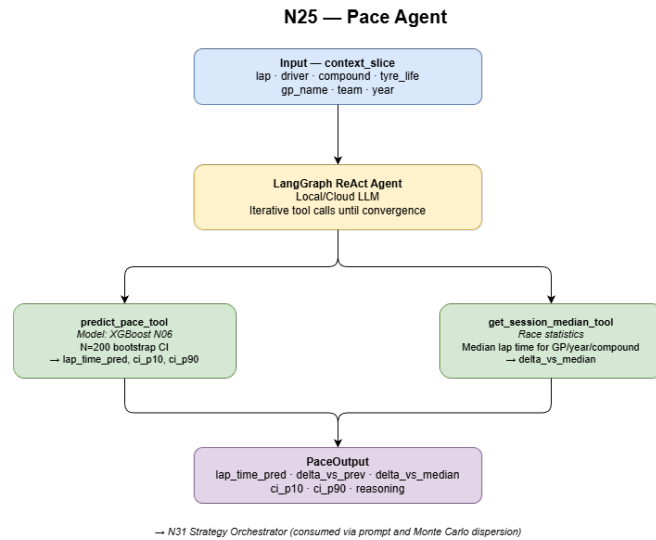


Figura 4.7: Flujo del sub-agente N25. La entrada `context_slice` pasa al agente ReAct sobre LangGraph, que orquesta dos llamadas deterministas (`predict_pace_tool` y `get_session_median_tool`) y devuelve `PaceOutput` al orquestador. Diagrama propio.

los percentiles del bootstrap, y `get_session_median_tool`, que recupera la mediana histórica del tiempo por vuelta para la combinación de gran premio, año y compuesto. La separación entre el cálculo numérico, encapsulado en las herramientas, y el razonamiento, asumido por el modelo de lenguaje, mantiene trazable cada predicción hasta su origen. El flujo completo del sub-agente se ilustra en la Figura 4.7.

4.2.3 Tire Agent

El agente de neumáticos (*Tire Agent* en el código) es el segundo eslabón de la capa de inferencia y se ocupa de cuantificar la degradación acumulada del neumático en uso y de estimar cuántas vueltas restan hasta su cliff, ya introducido en el Sección 2.4. Su salida tiene dos consumidores claramente diferenciados: el orquestador, que la incorpora como ancla de la simulación Monte Carlo al ser el único componente del sistema que aporta una distribución temporal hasta el momento de la parada; y el agente de estrategia de paradas, cuya activación condicional desde el orquestador se dispara cuando la alerta de este sub-agente alcanza el nivel `PIT_S00N`. Por debajo, el sub-agente apoya su inferencia en una red convolucional temporal causal (TCN, Sección 2.4) que recibe la secuencia del stint en curso codificada con cuarenta y dos variables, y la incertidumbre se cuantifica mediante MC Dropout (Sección 2.4), técnica que mantiene activo el *dropout* durante la inferencia y promedia varias pasadas para obtener percentiles calibrados. El entrenamiento sigue dos fases sucesivas: una primera versión global que mezcla los seis compuestos disponibles, y una segunda fase de fine-tuning especializado por compuesto, con un esquema de enrutado que asigna a cada compuesto su modelo dedicado salvo cuando los datos disponibles son insuficientes y obligan a recurrir al global. La construcción se reparte en cinco cuadernos: `N07_tireddeg_eda.ipynb` (análisis exploratorio de las curvas de degradación), `N08_tireddeg_sequence_config.ipynb` (calibración de las ventanas de secuencia por compuesto), `N09_tireddeg_tcn.ipynb` (entrenamiento global), `N10_tireddeg_compound_finetuning.ipynb` (fine-tuning por compuesto) y `N26_tire_agent.ipynb` (envoltura como agente ReAct sobre el contrato `lap_state`).

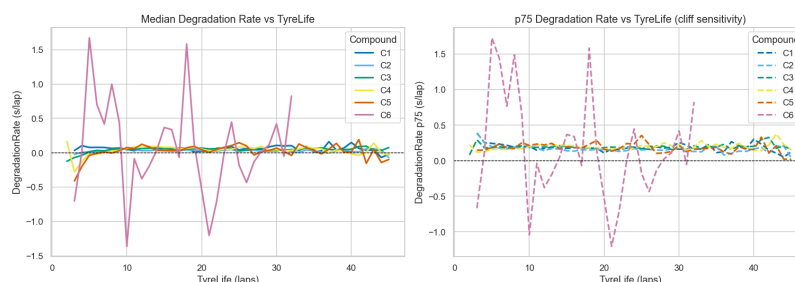


Figura 4.8: Tasa de degradación frente a la edad del neumático, expresada como mediana (izquierda) y percentil 75 (derecha) por compuesto. La traza rosa de C6 corresponde a las 46 vueltas exclusivas de 2025 referidas en el texto. Generada en el cuaderno N07.

4.2.3.1 Datos y análisis exploratorio

El análisis exploratorio se construye en `N07_tireddeg_eda.ipynb` y fija como objetivo de predicción la variable `FuelAdjustedDegAbsolute`, definida en el Subsección 4.2.1.3 como la degradación acumulada del neumático desde el inicio del stint corregida por el efecto del consumo de combustible. Sobre el corpus consolidado en el Subsección 4.2.1, el filtrado a vueltas secas y stints completos deja un subconjunto de entrenamiento de 64.271 vueltas distribuidas en 3.222 stints para las temporadas 2023 y 2024, cifras inferiores a las 68.122 vueltas del dataset general por la exclusión deliberada de los stints interrumpidos por bandera roja, coche de seguridad o lluvia. La distribución del objetivo presenta una asimetría marcada (mediana de 0,12 segundos pero con un 41 % de valores negativos en las primeras vueltas) provocada por la fase de calentamiento del neumático, durante la cual el ritmo todavía mejora. El análisis por compuesto, partiendo del mapeo Pirelli C1-C6 ya introducido en el Subsección 4.2.1, confirma que el sistema dispone de suficientes datos sobre los compuestos C1 a C5 (entre 2.623 vueltas para C5 y 27.754 para el dominante C3), pero apenas 46 vueltas registradas de C6, todas ellas pertenecientes a la temporada 2025, ya que Pirelli reintrodujo este compuesto solo a partir de ese año en unos pocos grandes premios seleccionados. Como 2025 queda reservada como conjunto de prueba, el corpus de entrenamiento no contiene ni una sola vuelta sobre C6, lo que descarta este compuesto del proceso y obliga a tratarlo en inferencia como caso particular. El hallazgo metodológicamente más relevante del EDA, derivado directamente del corpus 2023-2024 analizado en este trabajo, es que las curvas medias por compuesto no presentan un cliff universal observable en el agregado: incluso el percentil 75 de la tasa permanece por debajo de 0,25 segundos por vuelta para C1-C5 a lo largo de todo el rango de edad del neumático, según se aprecia en Figura 4.8. Esto indica que el momento del cliff es un evento específico de la combinación de circuito, temperatura e historia del propio stint y no un patrón medio que la red pueda absorber con un único modelo global; esta evidencia motiva directamente la estrategia de especialización por compuesto que se detalla en el siguiente apartado.

Una red convolucional temporal causal exige entradas de longitud fija, lo que obliga a fijar una ventana temporal por encima de la cual los stints largos se truncan y por debajo de la cual los cortos se rellenan. La distribución empírica de longitud de stint es bimodal, con dos picos en torno a 17-20 vueltas (estrategia agresiva de dos paradas) y 25-30 vueltas (estrategia estándar de una parada), y los percentiles globales se sitúan en $p_{50} = 21$, $p_{75} = 29$ y $p_{90} = 36$ vueltas. La calibración de la ventana se desarrolla en `N08_tireddeg_sequence_config.ipynb` y descarta la opción inicial basada en el percentil 90 al detectar, durante la prueba de cordura sobre el modelo global, que producía un 50 % de relleno medio y diluía la señal aprovechable durante el entrenamiento. La decisión final fija las ventanas en el percentil 75 por compuesto, identificado como el codo de la curva de compromiso entre truncación y *padding waste*: en ese punto ambas magnitudes se equilibran en torno al 25 %, lo que minimiza la pérdida combinada de información útil. Las ventanas resultantes son de 25 vueltas para C1, 31 para C2, 30 para C3, 26 para C4 y 22 para C5, con una ventana global de 28 vueltas como referencia. La estrategia de relleno aplica *left zero-padding* a los

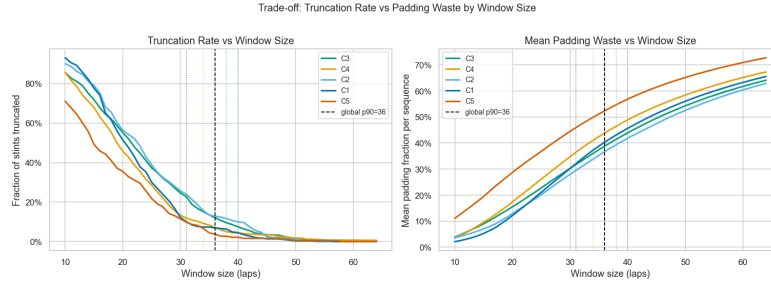


Figura 4.9: Compromiso empírico entre truncación de *stints* largos (izquierda) y fracción media de relleno por secuencia (derecha) en función del tamaño de ventana, desglosado por compuesto. La línea discontinua vertical marca el percentil 90 global ($p_{90} = 36$) inicialmente probado, descartado al producir aproximadamente un 50 % de relleno medio. El codo de las curvas se sitúa en torno al percentil 75 (28-31 vueltas según el compuesto), donde ambas magnitudes caen en el entorno del 20-30 % y minimizan la pérdida combinada de información útil. Generada en el cuaderno N08.

stints cortos y truncación desde el inicio para los largos, con enmascaramiento de las posiciones rellenadas durante el entrenamiento. El enrutado en inferencia, formalizado en `routing_config.json`, asigna a cada uno de los compuestos C1 a C5 su modelo dedicado y dirige a C6 al modelo global como *fallback*, dada la ausencia de datos sobre este compuesto identificada en el párrafo anterior. Esta especialización por compuesto se alinea con la práctica establecida en el modelado de estrategia en Fórmula 1, donde la sensibilidad de la degradación al compuesto montado obliga a tratar cada mezcla como un régimen distinto (Heilmeyer et al., 2020b). La Figura 4.9 sintetiza el compromiso que sustenta la elección del percentil 75.

4.2.3.2 Modelos entrenados e integración como sub-agente

El cuaderno `N09_tiredreg_tcn.ipynb` entrena la versión global del modelo sobre los cinco compuestos con datos suficientes (C1-C5), cargando las cuarenta y dos variables de entrada del apartado anterior y reproduciendo la arquitectura presentada en el Sección 2.4 con la siguiente parametrización concreta: proyección lineal inicial de cuarenta y dos a sesenta y cuatro canales, aproximadamente ciento dos mil parámetros entrenables y un campo receptivo de sesenta y una vueltas que cubre con margen la ventana global de veintiocho. La implementación es propia, escrita en PyTorch y modularizada en tres clases reutilizables (`CausalConv1dBlock`, `TCNResidualBlock` y `TireDegTCN`), y el bucle de entrenamiento se construye sobre PyTorch Lightning para abstraer la gestión de épocas, validación, *checkpoints* y métricas. La función de pérdida es la de Huber con $\delta = 1,0$ segundos, escogida por su robustez frente a la asimetría del objetivo, y el optimizador es AdamW con tasa de aprendizaje 10^{-3} y *weight decay* 10^{-4} . El entrenamiento sigue el protocolo de bloques temporales del Sección 2.2. En la primera fase, con 2023 como entrenamiento y 2024 como validación, se emplea un planificador con *warm restarts* (Loshchilov & Hutter, 2017) ($T_0 = 10$, $T_{\text{mult}} = 2$) hasta que la parada temprana fija el presupuesto de épocas; en la segunda, con 2023 y 2024 combinados, se aplica *cosine decay* monótono ajustado a ese presupuesto, sin reinicios. El modelo global obtiene un MAE de 0,866 segundos en validación 2024 y de 0,708 segundos sobre el holdout de 2025.

El cuaderno `N10_tiredreg_compound_finetuning.ipynb` parte de los pesos del modelo global y los ajusta de forma especializada sobre cada compuesto, con la misma arquitectura pero la ventana de secuencia específica fijada en el apartado anterior, una tasa de aprendizaje diez veces menor (10^{-4}) para evitar el olvido catastrófico, y el mismo esquema de dos fases con *warm restarts* y *cosine decay*. La comparación contra el modelo global sobre el holdout de 2025, recogida en la Tabla 4.2, muestra un patrón asimétrico: el fine-tuning mejora claramente C2 con un descenso de 0,158 segundos en MAE, permanece neutro en C1 y C3 con variaciones por debajo de los 0,01 segundos, y resulta contraproducente en C4 y C5, donde el modelo especializado se degrada respectivamente en 0,094 y 0,194 segundos respecto al global.

Tabla 4.2: MAE en holdout 2025 del modelo *fine-tuned* por compuesto frente al global, diferencia entre ambos y modelo seleccionado en producción.

Compuesto	MAE <i>fine-tuned</i> (s)	Δ MAE vs global (s)	Modelo en producción
C1	0,709	+0,001	global
C2	0,550	−0,158	<i>fine-tuned</i>
C3	0,714	+0,006	global
C4	0,802	+0,094	global
C5	0,901	+0,194	global
C6	—	—	global (<i>fallback</i>)

Este resultado motiva una decisión de enrutado *a posteriori*, no basada en una regla rígida de número de stints mínimos sino en la evidencia empírica del propio holdout: la inferencia en producción asigna a C2 su modelo dedicado y devuelve a C1, C3, C4 y C5 al global, mientras que C6, sin datos de entrenamiento, también queda enrutado al global como *fallback*. La lectura metodológica es que el fine-tuning aporta valor cuando el compuesto reúne volumen suficiente y un sesgo predictivo específico que el global diluye, pero penaliza cuando los datos son escasos y el regularizador implícito del entrenamiento conjunto sobre cinco compuestos prevalece sobre la especialización. Conviene además situar estas magnitudes en su contexto: el objetivo es una variable muy asimétrica, con curtosis 63 y picos coincidentes con los cliffs específicos de cada combinación de circuito y condiciones, irreducibles a un patrón medio aprendible. La justificación de una arquitectura secuencial profunda no recae por tanto en superar en MAE absoluto a un modelo tabular más simple, sino en habilitar el razonamiento causal sobre la posición dentro del stint y la cuantificación de incertidumbre mediante MC Dropout, cualidades que el agente aprovecha aguas abajo al traducir la predicción en un nivel de alerta discreto, donde el error en segundos pierde relevancia frente a la posición relativa al umbral de cliff. Esta lectura debe situarse además en el panorama del propio dominio: la predicción de la degradación del neumático es uno de los problemas más complejos de la estrategia en Fórmula 1 y un quebradero de cabeza permanente para los equipos (Sasikumar et al., 2025), donde incluso los modelos académicos abiertos sobre telemetría pública obtienen errores comparables a los que aquí se reportan. El sistema descrito se apoya únicamente en los datos públicos de FastF1 y OpenF1, sin acceso a sensores embarcados de presión y temperatura ni a la telemetría privada de los equipos, lo que sitúa los errores observados en un orden de magnitud compatible con el estado del arte académico accesible y razonable bajo las restricciones del entorno abierto.

La envoltura como sub-agente se construye en `N26_tire_agent.ipynb` y se traslada al módulo `src/agents/tire_agent.py`, siguiendo el patrón ReAct ya formalizado en el Subsección 4.1.2. El agente expone dos herramientas deterministas que comparten el modelo seleccionado por el enrutado: `predict_tire_deg_tool` ejecuta un único *forward pass* y devuelve la degradación acumulada y la tasa instantánea, mientras que `estimate_laps_to_cliff_tool` mantiene activo el *dropout* durante la inferencia y promedia $N = 50$ pasadas según el método descrito en Sección 2.4, obteniendo los percentiles 10, 50 y 90 de las vueltas restantes hasta el cliff. La incertidumbre total combina la dispersión empírica de las pasadas MC con un sigma calibrado por compuesto en `mc_dropout_calibration.json`, cargado al inicializar el agente. El umbral de cliff se calibra por compuesto a partir del percentil 75 de la degradación observada en la última vuelta de los stints del entrenamiento, situándose en tres segundos para C1 y dos segundos para C2 a C6. El nivel de alerta se deriva del percentil P10 de las vueltas restantes contra dos umbrales globales: PIT_SOON si es inferior a tres, MONITOR si es inferior a siete y OK en caso contrario, con *overrides* por gran premio o clúster cargados desde `tire_agent_config_v1.json`. La salida tipada `TireOutput` reúne nueve campos (compuesto, edad actual del neumático, tasa de degradación, los tres percentiles de las vueltas restantes, nombre del gran premio, nivel de alerta y razonamiento textual) y se enruta hacia el orquestador y,

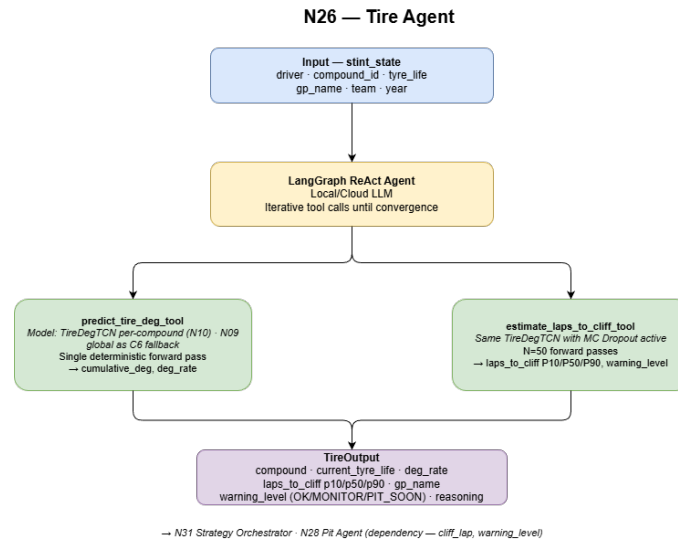


Figura 4.10: Flujo del sub-agente N26. El agente ReAct sobre LangGraph invoca dos herramientas con la TireDegTCN seleccionada por enrutado. `predict_tire_deg_tool` devuelve degradación y tasa instantánea. `estimate_laps_to_cliff_tool` aplica MC Dropout para los percentiles P10, P50 y P90. La salida `TireOutput` se enruta al orquestador y al sub-agente de paradas cuando la alerta alcanza `PIT_SOON`. Diagrama propio.

cuando el nivel alcanza `PIT_SOON`, hacia el agente de estrategia de paradas. El flujo completo se ilustra en la Figura 4.10.

4.2.4 Race Situation Agent

El agente de situación en carrera evalúa en cada vuelta dos amenazas de naturaleza opuesta, la probabilidad de que el piloto propio sea adelantado en los compases siguientes y la probabilidad de que se despliegue un coche de seguridad en el horizonte inmediato. Ambas señales condicionan la estrategia de manera asimétrica, ya que una amenaza de adelantamiento eleva la urgencia de una parada defensiva mientras que un coche de seguridad previsible abre una ventana de parada gratuita capaz de recomponer el orden en carrera. El módulo N27, construido sobre el patrón ReAct ya descrito en el Subsección 4.1.2, integra dos clasificadores LightGBM binarios calibrados mediante una regresión logística de un parámetro (Platt, 1999) y produce una salida unificada `RaceSituationOutput` con un nivel de amenaza discreto que el orquestador N31 emplea para activar o inhibir los sub-agentes de parada y de radio.

4.2.4.1 Datos y análisis exploratorio

El análisis exploratorio del modelo de probabilidad de adelantamiento parte del cuaderno `N11_overtake_eda.ipynb`, que construye el corpus de entrenamiento sobre las setenta carreras de las temporadas 2023 a 2025. El etiquetado de cada par de coches consecutivos en pista combina condiciones de seguridad y de relevancia táctica, de modo que solo se etiquetan los pares en los que la vuelta transcurre con bandera verde, ninguno de los dos pilotos entra en boxes ese giro, ambos ruedan con compuestos secos, la separación entre ellos no supera los 2,5 segundos y la vuelta es posterior a la tercera para descartar el desorden de la salida. El corpus resultante reúne 28.494 pares etiquetados, de los cuales 2.404 (8,44 %) son adelantamientos efectivos. El análisis exploratorio confirma la centralidad del DRS (*Drag Reduction System*, alerón trasero móvil que reduce la resistencia aerodinámica cuando el perseguidor se sitúa a menos de un segundo del coche precedente), ya que el 80,5 % de los adelantamientos registrados ocurre con una separación inferior a ese umbral, lo que convierte a la brecha al coche de delante en la palanca táctica dominante del modelo, como ilustra la Figura 4.11. La tasa varía además de forma marcada entre circuitos, desde los

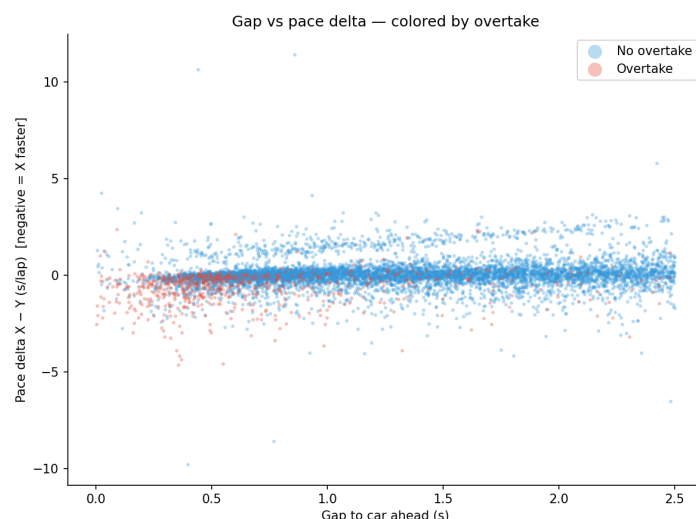


Figura 4.11: Diagrama de dispersión de los 28.494 pares etiquetados del corpus de adelantamiento, situados en el espacio formado por la brecha al coche precedente (eje horizontal) y la diferencia de ritmo entre atacante y defensor (eje vertical; valores negativos indican que el atacante es más rápido). Los puntos rojos señalan adelantamientos efectivos y los azules no adelantamientos. La concentración de puntos rojos en la esquina de baja brecha y ritmo favorable confirma que la ventana DRS es la condición necesaria dominante: a partir de 1,0 segundo de separación los adelantamientos desaparecen prácticamente en cualquier escenario de diferencia de ritmo. Generada en el cuaderno N11.

grandes premios de alta velocidad y trazados amplios hasta circuitos urbanos como Mónaco, diferencia que reproduce las distintas condiciones de sobrepaso propias de cada arquetipo del Subsección 4.2.1. A las once variables base, brecha, diferencia de ritmo, edades del neumático de atacante y defensor, compuestos de ambos y clúster del circuito, se añaden cuatro derivadas pensadas para capturar el contexto temporal de la persecución, entre ellas la media móvil de tres vueltas del ritmo del atacante (`pace_delta_rolling3`), la tendencia reciente de la brecha (`gap_trend`) y dos interacciones multiplicativas entre brecha, ritmo y estado DRS.

El corpus del clasificador de coche de seguridad se construye en `N13_sc_edu.ipynb`, que etiqueta cada vuelta en carrera con la variable `sc_within_3_laps`, indicadora de si en los tres giros siguientes se despliega un coche de seguridad o un coche de seguridad virtual. Tras descartar las vueltas que ya transcurren bajo neutralización, el corpus suma 3.275 vueltas con 114 positivos, lo que arroja una tasa base del 3,5 %. La temporada 2024 destaca por una tasa de incidencia notablemente inferior a la de 2023 y 2025, una anomalía que la validación cruzada en dos particiones temporales neutraliza promediando los resultados de ambos pliegos. El hallazgo más relevante del análisis es que los mensajes de dirección de carrera, pese a aparecer en el 18,8 % de las vueltas del corpus, se limpian de las notificaciones de despliegue del propio coche de seguridad para evitar una fuga de información directa, y las señales reactivas resultantes quedan entre las variables de menor correlación con el objetivo; por el contrario, el estado estructural del pelotón, principalmente la dispersión de los tiempos de vuelta del conjunto del campo y la edad máxima del neumático en pista, anticipa los escenarios de riesgo con mayor fiabilidad. La consecuencia operativa es que `sc_prob_3lap` se emplea en el sistema como un prior contextual blando que eleva la vigilancia del orquestador ante condiciones de carrera propensas al incidente, no como una predicción determinista del siguiente despliegue. La Figura 4.12 sintetiza los cuatro ángulos del análisis exploratorio.

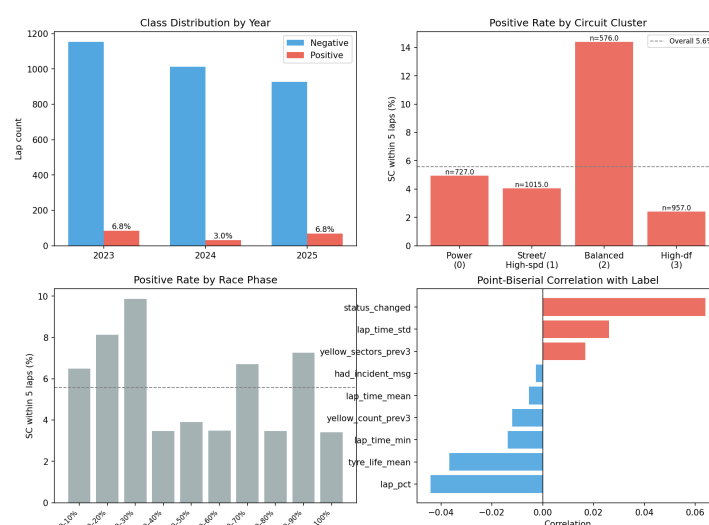


Figura 4.12: Resumen exploratorio del corpus construido en `N13_sc_eda.ipynb`, visualizado con ventana de cinco vueltas. *Superior izquierda*, distribución de clases por temporada. *Superior derecha*, tasa por clúster de circuito. *Inferior izquierda*, tasa por fase de carrera. *Inferior derecha*, correlación punto-biserial de las variables con el objetivo. La discusión de cada panel se desarrolla en el texto. Generada en el cuaderno N13.

4.2.4.2 Modelos entrenados e integración como sub-agente

El cuaderno `N12_overtake_model.ipynb` entrena el clasificador de adelantamiento siguiendo el protocolo de bloques temporales del Sección 2.2. La elección de LightGBM (Ke et al., 2017) responde a su eficiencia con conjuntos tabulares de tamaño medio y a su soporte nativo para la ponderación de clases, de modo que el desequilibrio del 8,44 % se corrige con un peso de 12,05 asignado a los positivos mientras la búsqueda de hiperparámetros fija `n_estimators = 717`. La calibración de Platt (Sección 2.3) reduce la desviación media entre probabilidad estimada y tasa observada de 0,58 a 0,26 puntos. Sobre el holdout de 2025 el modelo obtiene un área bajo la curva precisión-recall (AUC-PR, por sus siglas en inglés) de 0,549, un factor de mejora de 7,2 veces respecto a la línea base aleatoria de 0,076, y un área bajo la curva ROC (AUC-ROC, *receiver operating characteristic*) de 0,876, con un umbral operativo de 0,798 optimizado sobre la validación. El análisis SHAP (*SHapley Additive exPlanations*, método de interpretabilidad basado en valores de Shapley que atribuye a cada variable la contribución marginal a la predicción), recogido en la Figura 4.13, confirma la jerarquía que el análisis exploratorio anticipaba. La variable de mayor importancia es `gap_ahead_s`, con un valor medio de 0,582, seguida de `pace_delta_rolling3` con 0,461, lo que refuerza que la tendencia sostenida de la persecución supera en capacidad predictiva a la diferencia de ritmo instantánea de la vuelta en curso (0,201). La posición de `LapNumber` en tercer lugar, con 0,389, responde a una relación no lineal con la predicción que refleja la mayor concentración de adelantamientos en las fases donde atacante y defensor ruedan con compuestos distintos.

El ensayo TCN paralelo (N12B) anunciado en el Sección 2.4, implementado con una ventana de ocho vueltas y aproximadamente siete mil parámetros, obtuvo un AUC-PR de 0,10 sobre el mismo holdout y se archivó en favor del enfoque tabular con ingeniería de variables explícita.

El clasificador de coche de seguridad, entrenado en `N14_sc_model.ipynb`, afronta un desequilibrio más severo, con una tasa base del 3,5 %. La selección de la ventana de predicción responde a un análisis comparativo entre horizontes, en el que la ventana de tres vueltas obtiene un *lift* de 1,67 veces sobre la línea base, frente a 1,44 para cinco vueltas y 1,29 para siete, de modo que el horizonte más corto concentra

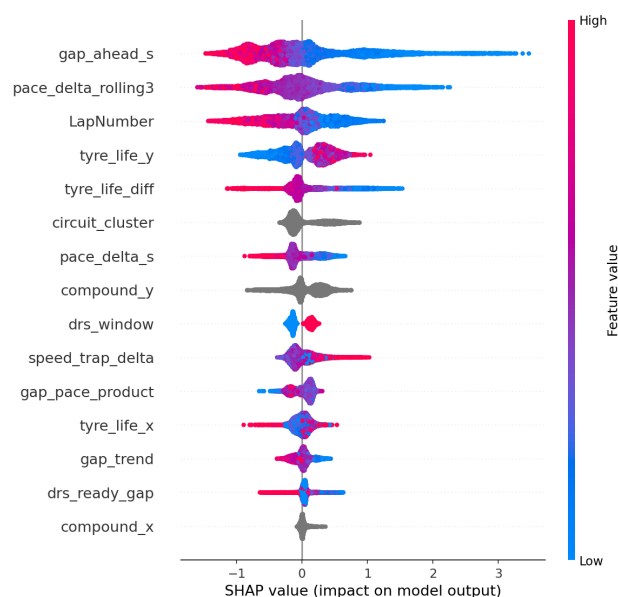


Figura 4.13: Importancias SHAP del modelo LightGBM de probabilidad de adelantamiento (N12), calculadas sobre el conjunto completo. La jerarquía y los valores numéricos se discuten en el texto. Generada en el cuaderno N12.

mejor la señal estructural de riesgo. El entrenamiento sigue el protocolo del Sección 2.2, y la anomalía de 2024 (3,0 % frente al 6,8 % de 2023 y 2025) se neutraliza promediando los resultados de los dos pliegos de validación cruzada temporal. El modelo final, con `n_estimators = 341`, alcanza un AUC-PR de 0,072 y un AUC-ROC de 0,641 sobre el holdout de 2025. El umbral de decisión se fija en 0,234 para maximizar la métrica F_2 , que pondera el *recall* el doble que la precisión, dado que un falso negativo (no detectar un coche de seguridad inminente) tiene mayor coste estratégico que elevar innecesariamente la vigilancia. El análisis SHAP confirma el patrón estructural frente al reactivo que el análisis exploratorio anticipó, con `lap_time_std_z` y `tyre_life_max` concentrando las importancias más elevadas, mientras que `rcm_incident_count_prev3`, el recuento de mensajes de incidente en los tres giros previos, no supera los 0,02 puntos de importancia media, lo que confirma que las señales reactivas no aportan capacidad predictiva adicional una vez que el estado estructural del pelotón está representado.

El cuaderno `N27_race_situation_agent.ipynb` integra los dos clasificadores en el agente de situación en carrera, cuya arquitectura se recoge en la Figura 4.14. El agente expone al LLM del patrón ReAct dos herramientas, `predict_overtake_tool` y `predict_sc_tool`, que invocan respectivamente el modelo N12 y el modelo N14 con sus calibraciones de Platt. A partir de sus salidas, el agente construye el nivel de amenaza discreto `threat_level` mediante una lógica de umbrales en la que HIGH se activa cuando la probabilidad de coche de seguridad supera 0,30 o la probabilidad de adelantamiento supera 0,80, y MEDIUM se activa con umbrales de 0,15 y 0,40 respectivamente. La salida consolidada `RaceSituationOutput` reúne seis campos (`overtake_prob`, `sc_prob_3lap`, `threat_level`, `gap_ahead_s`, `pace_delta_s` y `reasoning`) que el orquestador N31 consume para decidir si activa el sub-agente de parada en boxes cuando `warning_level` alcanza PIT_SOON, o bien el sub-agente de neutralización cuando `sc_prob_3lap` supera 0,30, cerrando el ciclo de análisis entre los módulos de diagnóstico y los de decisión.

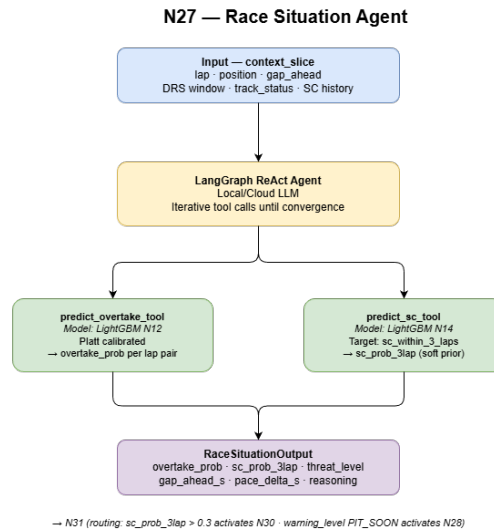


Figura 4.14: Arquitectura del agente de situación en carrera (N27). El agente LangGraph ReAct invoca `predict_overtake_tool` (LightGBM N12 con Platt) y `predict_sc_tool` (LightGBM N14). La salida `RaceSituationOutput` con `threat_level` dirige el enrutado del orquestador hacia N28 y N30 según las reglas detalladas en el texto. Exportado del diagrama N27.

4.2.5 Pit Strategy Agent

El agente de estrategia de paradas resuelve la decisión crítica y central de toda la carrera, cuándo entrar en boxes y con qué compuesto salir en la siguiente etapa. Para ello combina las salidas de dos modelos predictivos cuyas naturalezas son complementarias. El primero es un regresor cuantílico que estima la distribución del tiempo físico de parada en los percentiles P05, P50 y P95, y el segundo un clasificador binario que estima la probabilidad de que un undercut sobre un rival concreto se convierta en una ganancia de posición. El módulo N28, construido sobre el patrón ReAct ya descrito en el Subsección 4.1.2, expone tres herramientas al LLM e integra como contexto las salidas que han emitido previamente los agentes de neumáticos (`laps_to_cliff` de N26) y de situación (`sc_prob_3lap` de N27) para producir una decisión estructurada `PitStrategyOutput` con una de cinco acciones discretas, todo ello sujeto a un conjunto de salvaguardas tácticas que filtran recomendaciones inviables (vueltas finales, ventanas mínimas de stint, compatibilidad con la regla FIA de dos compuestos).

4.2.5.1 Datos y análisis exploratorio

El cuaderno `N15_pit_duration.ipynb` construye el corpus a partir de las paradas registradas por FastF1 sobre las temporadas descritas en el Subsección 4.2.1, identificando cada parada por la pareja `PitInTime/PitOutTime` y filtrando los *drive-through* con un umbral de `TyreLife_out` ≤ 5 sobre la vuelta de salida. El corpus inicial reúne 2.373 paradas, que tras descartar artefactos de duración fuera del rango de 15 a 60 segundos quedan en 2.205 paradas válidas con una mediana de 23,4 segundos y una desviación típica de 4,6 segundos. El objetivo del modelo es la duración del cambio físico de neumático, no la duración total que el coche pasa en el carril de boxes, que incluye además el recorrido fijo del propio pit lane. Una primera tentativa de modelar esa duración total se descartó al obtener un MAE de 2,3 segundos sobre el holdout, idéntico al de la mediana por equipo y circuito, porque la longitud del trazado de boxes domina por completo la señal y la captura ya el propio factor de circuito sin que el modelo aporte información adicional, lo que motiva extraer y aislar esos segundos antes de aprender el componente realmente predictivo. La aproximación final descompone la duración en dos componentes, `pit_duration_s = circuit_traversal + physical_stop_est`, donde el *traversal* se estima como el percentil 5 de la duración por

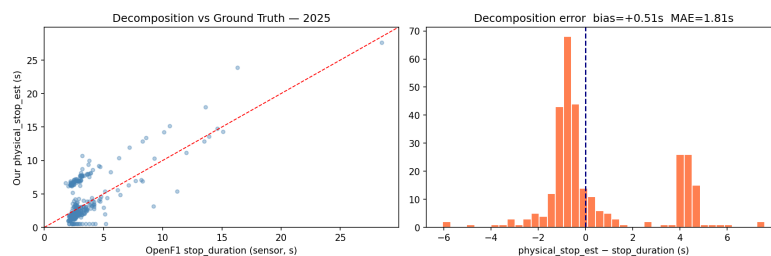


Figura 4.15: Validación de la descomposición $\text{pit_duration_s} = \text{circuit_traversal} + \text{physical_stop_est}$ frente a la telemetría sensorial de OpenF1 sobre 305 paradas emparejadas de 2025. *Izquierda:* dispersión entre la estimación propia y la duración medida por los sensores de plataforma del *pit lane*, donde la masa de puntos se concentra en torno a la diagonal salvo en la cola superior, asociada a paradas con artefactos de cronometraje en FastF1. *Derecha:* distribución del error de descomposición con sesgo medio de +0,51 segundos y MAE de 1,81 segundos, valores compatibles con que la descomposición captura razonablemente el componente físico de la parada al margen del recorrido fijo del circuito. Generada en el cuaderno N15.

circuito menos un suelo de 1,5 segundos correspondiente al cambio de neumático más rápido posible. Esta descomposición aísla el componente físico variable, en el rango aproximado de 0,5 a 15 segundos, que sí depende de la calidad del equipo y de las condiciones puntuales, y se valida frente a la telemetría sensorial de OpenF1 con un sesgo medio de 0,513 segundos sobre 305 paradas de 2025, según ilustra la Figura 4.15. El corpus se acota además al subrango de paradas normales entre 2,0 y 4,5 segundos, excluyendo deliberadamente penalizaciones de cinco y diez segundos, fallos de gato y *unsafe release* (liberación insegura del coche durante la parada cuando alguno de los mecánicos no ha completado su tarea), eventos no predecibles desde variables previas a la parada que el agente trata por una vía separada a partir de los mensajes de dirección de carrera.

El cuaderno `N16_undercut.ipynb` etiqueta como *undercut* aquellos pares de coches en los que un piloto X entra en boxes hasta cinco vueltas antes que un rival Y al que precedía en pista, y verifica el resultado tres vueltas después de la parada de Y, activando la etiqueta binaria `undercut_success` cuando X termina por delante de Y. El corpus inicial reúne 7.149 pares, que se reducen a 2.041 al exigir una proximidad máxima de tres posiciones que descarte coincidencias entre cabeza y cola del pelotón. Tres filtros adicionales eliminan los pares que no responden a una decisión táctica deliberada. El primero descarta los pares entre compañeros de equipo, dado que los *pit stacks* responden a la coordinación interna del equipo y un *undercut* entre compañeros es excepcional como movimiento estratégico planificado. El segundo elimina los pares con un coche de seguridad o un coche de seguridad virtual desplegado entre las dos paradas, un 34,1 % del corpus inicial, porque pese a que un *undercut* o un *overcut* puede convertir durante una neutralización, el resultado depende en la mayoría de los casos de un factor externo de fortuna (un accidente ajeno o un incidente que provoca el despliegue) y no del mérito de la maniobra propia, lo que introduciría ruido en la señal que el modelo debe aprender. El tercero descarta las combinaciones con compuestos de lluvia, escenario estructuralmente distinto al de neumáticos secos. El corpus final consta de 1.032 pares con una tasa de éxito del 37,2 %. El análisis exploratorio identifica tres factores dominantes que la Figura 4.16 sintetiza, la diferencia de posiciones `pos_gap` (cuanto menor sea, mayor probabilidad), las vueltas de adelanto `Lap_gap` (relación monótona, del 27 % al 45 % de éxito a medida que el *undercut* se anticipa) y la identidad del circuito (variabilidad entre el 13 % del Gran Premio de Australia y el 62 % del de Bélgica). Un hallazgo contraintuitivo es que los *undercuts* exitosos se producen entre coches con neumáticos de edad similar, mientras que los fallidos predominan cuando X arrastra ya un desgaste significativo respecto a Y, lo que sugiere que el *undercut* funciona como maniobra anticipatoria y no como recurso desesperado.

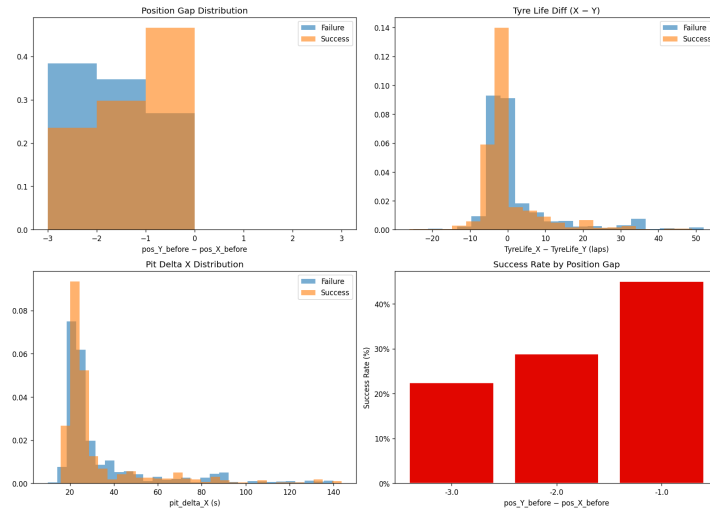


Figura 4.16: Análisis exploratorio del corpus de *undercuts* construido en `N16_undercut.ipynb`. *Superior izquierda*, distribución de `pos_gap` segregada por resultado. *Superior derecha*, distribución de la diferencia de edad de neumático entre atacante y defensor. *Inferior izquierda*, distribución de `pit_delta_X`. *Inferior derecha*, tasa de éxito por `pos_gap`. Los hallazgos y cifras se discuten en el texto. Generada en el cuaderno N16.

4.2.5.2 Modelos entrenados e integración como sub-agente

El modelo del cuaderno N15 entrena tres regresores cuantílicos independientes con `HistGradientBoostingRegressor` y pérdida *pinball* en los percentiles 0,05, 0,50 y 0,95, siguiendo el protocolo de bloques temporales del Sección 2.2 con 2023 y 2024 como entrenamiento y 2025 como holdout. La elección de la regresión cuantílica frente a una predicción puntual responde a la naturaleza de la decisión aguas abajo, ya que el orquestador necesita conocer la probabilidad de que la parada quepa en una ventana temporal concreta y no la duración esperada, especialmente cuando la cola derecha de la distribución refleja eventos de alto coste estratégico. La configuración fija `max_iter = 400`, tasa de aprendizaje 0,05 y veinte hojas máximas, con menor regularización en los modelos de cola (`min_samples_leaf = 20`, $l_2 = 0,5$) para no colapsarlos al centro de la distribución. El conjunto final de nueve variables excluye `circuit`, cuya importancia por permutación resultó negativa ($-0,848$ segundos: barajarla mejoraba el MAE de prueba) por sobreajuste a las tendencias de equipo de 2023 y 2024, y la sustituye por el indicador binario `tight_pit_box` para los circuitos de pit lane confinado (Mónaco, Singapur y Hungría). El modelo P50 alcanza un MAE de 0,487 segundos sobre el holdout de 2025, una mejora del 12 % respecto a la línea base de mediana por equipo y circuito (0,555 segundos), y la cobertura empírica del intervalo P05–P95 es del 70,5 %, por debajo del 90 % nominal debido al sesgo de cola que el corpus de paradas normales no alcanza a representar plenamente.

El modelo del cuaderno N16 entrena un clasificador binario `LightGBM` con la misma configuración base de los modelos del Subsección 4.2.4, calibrado con Platt sobre la temporada 2024. El conjunto de trece variables combina contexto de carrera (`pos_gap`, `Lap_gap`, `lap_race_pct`, `pos_X_before`), estado de neumático (`tyre_life_diff`, edades absolutas, identificadores de compuesto y su delta) y dos codificaciones del objetivo calculadas únicamente sobre el conjunto de entrenamiento, `circuit_undercut_rate` y `team_x_undercut_rate`, que recogen la heterogeneidad observada en el análisis exploratorio. La búsqueda de hiperparámetros se realiza con Optuna sobre 80 ensayos en un único pliegue temporal (entrenamiento 2023, validación 2024) maximizando AUC-PR, y el modelo final se reentrena sobre 2023 y 2024 combinados. Sobre el holdout de 2025 alcanza un AUC-PR de 0,674 frente a una línea base de 0,345, lo que supone un factor de mejora de 1,95, y un AUC-ROC de 0,771. El umbral operativo se fija en 0,522 mediante optimización de la métrica F_1 sobre la validación calibrada. El análisis SHAP confirma la jerarquía exploratoria, con `pos_gap` (0,690), `pos_X_before` (0,630) y `circuit_undercut_rate` (0,603) capitalizando el grueso de la

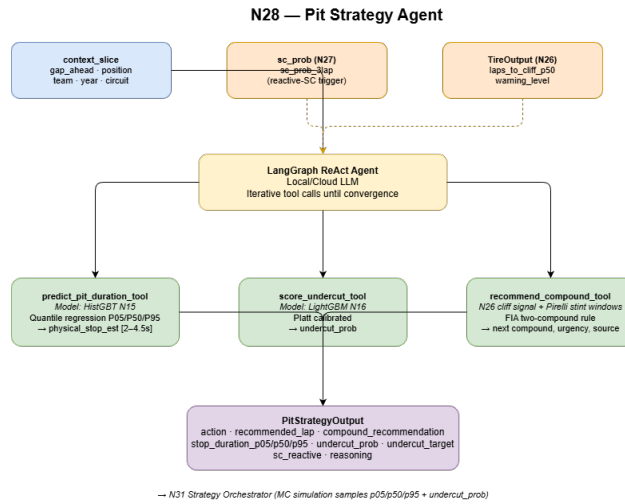


Figura 4.17: Arquitectura del agente de estrategia de paradas (N28). El agente LangGraph ReAct recibe context_slice, laps_to_cliff_p50 de N26 y sc_prob_3lap de N27, e invoca tres herramientas (predict_pit_duration_tool con los regresores cuantílicos HistGBT, score_undercut_tool con el clasificador LightGBM y recommend_compound_tool sobre la jerarquía descrita en el texto). La salida PitStrategyOutput se consume por el orquestador. Exportado del diagrama N28.

importancia, mientras que los identificadores de compuesto contribuyen apenas, dado que la edad de neumático ya recoge buena parte del estado táctico. La calibración traslada la media de probabilidades de 0,380 a 0,318 sobre una tasa real de 0,345.

El agente N28 integra los dos modelos en un patrón ReAct que expone tres herramientas al LLM, predict_pit_duration_tool, score_undercut_tool y recommend_compound_tool, e incorpora como contexto las salidas previas del agente de neumáticos (laps_to_cliff_p10 de N26) y del agente de situación (sc_prob_3lap de N27). La herramienta de duración devuelve los tres percentiles P05, P50 y P95 sumados al *traversal* del circuito. La de undercut aplica el clasificador calibrado y descarta automáticamente los pares con compuestos de lluvia, fuera del alcance del modelo. La de compuesto sigue una jerarquía de dos prioridades, primero el cliff de neumático del propio agente N26 (*cliff imminent* con tres o menos vueltas, *cliff soon* con diez o menos), y como respaldo las ventanas medias de stint reportadas por Pirelli y descritas en Heilmeyer et al. (2020b) (dieciocho vueltas para SOFT, treinta para MEDIUM y treinta y ocho para HARD), respetando siempre la regla FIA de dos compuestos. La salida estructurada PitStrategyOutput reúne diez campos, una de cinco acciones discretas internas del sub-agente (PIT_NOW, STAY_OUT, UNDERCUT, OVERCUT y REACTIVE_SC), la vuelta recomendada, el compuesto, los tres percentiles de duración, la probabilidad de undercut con su rival objetivo, una bandera sc_reactive y el razonamiento textual. La acción REACTIVE_SC se utiliza para señalar una parada disparada por un coche de seguridad activo o inminente; el orquestador la consume junto con la bandera homónima y, cuando construye la StrategyRecommendation final del RF-12, la traduce al valor ALERT del esquema externo, reservando los otros cuatro nombres para los casos cuyo significado es idéntico en ambas capas. Por encima de las reglas de decisión, el sistema impone salvaguardas tácticas explícitas que filtran recomendaciones inviables (sin paradas antes de la quinta vuelta salvo coche de seguridad o daño confirmado, sin paradas con tres vueltas o menos por delante salvo cliff inminente, vidas mínimas de stint por compuesto), de modo que el LLM nunca propone una decisión que la lógica táctica del deporte rechazaría a priori. La Figura 4.17 recoge la arquitectura completa del agente y su conexión con el orquestador N31, que muestrea los tres percentiles y la probabilidad de undercut en sus simulaciones Monte Carlo.

4.2.6 Radio Agent

El agente de radio (*Radio Agent* en el código) constituye el quinto sub-agente de la capa de inferencia y la única vía por la que el sistema convierte las comunicaciones habladas y los mensajes de dirección de carrera en señales estructuradas. Su salida no la consume otro sub-agente, sino directamente el orquestador, que incorpora al prompt de síntesis del modelo de lenguaje las observaciones del piloto y, sobre todo, lee de ella las alertas operativas que disparan condicionalmente al sub-agente de estrategia de paradas y al sub-agente de reglamento. A diferencia de los otros cinco, este sub-agente no sigue el patrón ReAct sino el patrón *NLP-first* ya descrito en el Subsección 4.1.2, que ejecuta primero, en orden fijo, una canalización determinista de procesamiento de lenguaje natural antes de ceder la síntesis al modelo de lenguaje. Esa canalización combina cuatro modelos de aprendizaje automático, cuyas bases teóricas se desarrollan en el Subsección 2.5.1, el Subsección 2.5.2 y el Subsección 2.5.3 junto con el subapartado de NER del mismo capítulo, con un procesador determinista basado en reglas para los *Race Control Messages*. La construcción se reparte entre el cuaderno `N17_radio_labeling.ipynb`, donde se fija el corpus etiquetado a mano, los cuadernos `N18–N23` que entrenan o configuran cada componente, el cuaderno `N24_radio_pipeline.ipynb` que los compone en un único *pipeline* unificado y `N29_radio_agent.ipynb` que envuelve la inferencia como sub-agente sobre el contrato `lap_state`.

4.2.6.1 Datos y análisis exploratorio

El cuaderno `N17_radio_labeling.ipynb` construye el corpus de entrenamiento de los componentes de aprendizaje automático etiquetando manualmente 659 mensajes de radio extraídos de la temporada cubierta por el sistema. La anotación se realiza en tres ejes simultáneos sobre la misma transcripción, una clase de sentimiento (negativo, neutro o positivo), una clase de intención dentro del repertorio operativo característico del paddock (información, problema, orden, advertencia o pregunta) y un esquema BIO (*beginning-inside-outside*, formato estándar de etiquetado por *token* en tareas de reconocimiento de entidades) con nueve entidades semánticas a nivel de *token*, recogidas en la Figura 4.18, que extienden la noción clásica de NER para capturar roles propios del dominio (acción, situación, incidente, instrucción estratégica, cambio de posición, llamada a boxes, condición de pista, problema técnico y meteorología). Tras la anotación se descartan un 5,2 % de mensajes posteriores al final de la carrera, irrelevantes para una decisión en tiempo real, y otros cuarenta y nueve mensajes son retirados en una segunda revisión por inconsistencia entre etiquetas. El corpus resultante se publica como *dataset* abierto en HuggingFace para favorecer la reproducibilidad, y en producción se descarga de forma diferida mediante el *script* `build_radio_dataset.py` cuando el sub-agente lo requiere por primera vez.

El segundo flujo textual del sistema, los *Race Control Messages* emitidos por la dirección de carrera de la FIA, no requiere etiquetado manual porque ya llega estructurado por la API pública FastF1 con cinco categorías de origen (`Flag`, `SafetyCar`, `Drs`, `CarEvent` y `Other`) y un vocabulario cerrado por el reglamento deportivo. El análisis del corpus completo, sintetizado en la Figura 4.19, identifica veinticinco tipos de evento concretos dentro de esas cinco categorías sobre los que se diseña el procesador determinista del cuaderno `N23`. La cobertura empírica es del 100 % en los eventos críticos (banderas, despliegues de coche de seguridad real y virtual, transiciones del DRS) y del 92 % en la categoría residual `Other`, lo que confirma que un conjunto de patrones léxicos sobre las plantillas conocidas resulta suficiente para este flujo y que un modelo entrenado no aportaría valor adicional sobre un dominio formal y cerrado.

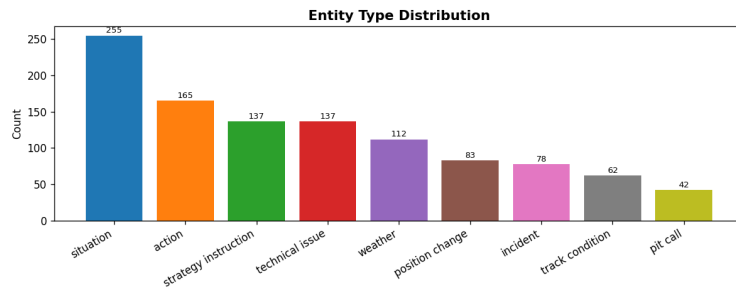


Figura 4.18: Distribución de las nueve entidades semánticas etiquetadas en el corpus de radio según el esquema BIO. La clasificación se aleja de la NER clásica de nombres propios para capturar roles operativos del dominio: las categorías mayoritarias son las acciones del piloto (ACTION) y las situaciones de carrera (SITUATION), seguidas de las llamadas a boxes (PIT_CALL) y los problemas técnicos (TECHNICAL_ISSUE), todas ellas con peso suficiente para entrenar el clasificador *token-level*. Las categorías más raras (cambio de posición, condición de pista, meteorología) se mantienen en el esquema porque su valor estratégico cuando aparecen compensa su frecuencia. Generada en el cuaderno N22 sobre el corpus etiquetado en N17.

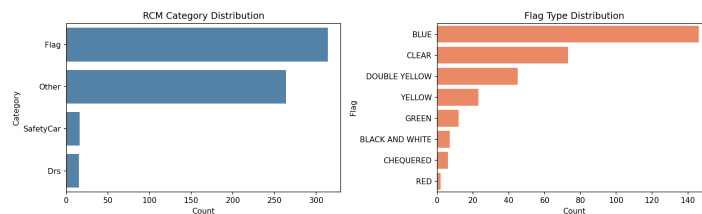


Figura 4.19: Análisis exploratorio del corpus de *Race Control Messages* sobre el que se diseña el procesador determinista N23. Los paneles recogen la distribución por categoría de origen, la frecuencia de los veinticinco tipos de evento concretos identificados dentro de ellas, la cobertura empírica del procesador frente a los mensajes anotados manualmente para validación y la proporción de mensajes residuales que requieren reglas adicionales. La concentración de los eventos críticos en patrones léxicos estables justifica la decisión metodológica de usar reglas explícitas en lugar de un clasificador entrenado. Generada en el cuaderno N23.

4.2.6.2 Modelos entrenados e integración como sub-agente

El pipeline `N24_radio_pipeline.ipynb` articula los cinco componentes en un único objeto invocable que recibe un fragmento de audio o un mensaje de dirección de carrera y devuelve un JSON estructurado con todos los campos derivados. La transcripción se delega a la variante *turbo* de Whisper descrita en el Subsección 2.5.1, el sentimiento al modelo RoBERTa ajustado sobre el corpus N17 (Subsección 2.5.2), la intención a un clasificador SetFit con codificador ModernBERT-base sobre el mismo corpus (Subsección 2.5.3), y el reconocimiento de entidades a un BERT-large preentrenado en CoNLL-03 con esquema BIO ajustado al dominio F1, cuya selección frente a alternativas *zero-shot* como GLiNER y NuNER se documenta en el subapartado correspondiente del marco teórico. El procesador determinista de *Race Control Messages* (N23) corre en paralelo a la rama de audio sobre los mensajes ya estructurados que entrega FastF1. La canalización completa alcanza un percentil 95 de latencia de 59,4 ms sobre GPU, holgadamente por debajo del techo de 100 ms exigido por el RNF-01 de operación vuelta a vuelta, y se valida con la prueba de humo del cuaderno N34, que procesa de extremo a extremo las veintiocho radios y setenta y seis mensajes de dirección de carrera del Gran Premio de Bahrein de 2025.

La envoltura como sub-agente se construye en `N29_radio_agent.ipynb` y se traslada al módulo `src/agents/radio_agent.py` siguiendo el patrón *NLP-first* formalizado en el Subsección 4.1.2. La inversión metodológica respecto al patrón ReAct mayoritario es deliberada, ya que las alertas críticas de carrera (banderas, despliegues de coche de seguridad, sanciones, problemas mecánicos confirmados) deben extraerse con garantías deterministas que un agente ReAct no asegura, dado que en ReAct es el propio

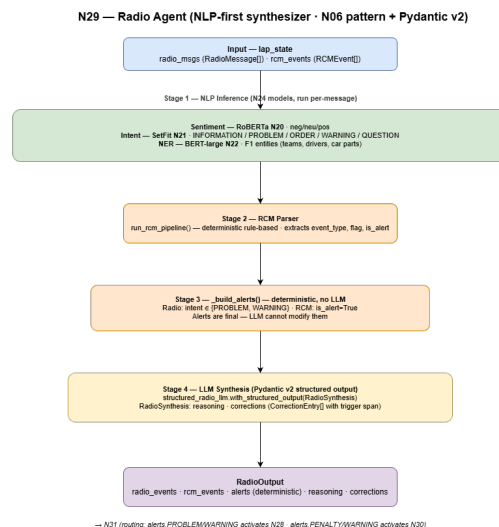


Figura 4.20: Arquitectura del agente de radio (N29). La canalización N24 ejecuta los cuatro componentes ML sobre el audio (Whisper, RoBERTa, SetFit-ModernBERT y BERT-large CoNLL-03) y el procesador determinista N23 estructura en paralelo los *Race Control Messages*. El modelo de lenguaje produce `RadioOutput` con alertas ya marcadas por banderas `is_alert` y el orquestador enruta cada alerta según su tipo. Exportado del diagrama N29.

modelo de lenguaje quien decide cuándo invocar cada tool y podría omitir una llamada o malinterpretar su resultado. La canalización determinista del N24 se ejecuta primero y el modelo de lenguaje recibe ya marcadas las banderas `is_alert` sobre las que debe sintetizar, sin posibilidad de omitirlas ni inventarlas. La salida tipada `RadioOutput` reúne cinco campos, la lista de eventos de radio con sus etiquetas y entidades, la lista de eventos de *Race Control*, la lista de alertas operativas derivadas de las banderas deterministas, una lista de correcciones cuando aparece discrepancia entre la transcripción cruda y las entidades extraídas, y el razonamiento textual del modelo de lenguaje. La Figura 4.20 recoge la arquitectura completa del agente y la conexión con el orquestador, que activa el sub-agente de estrategia de paradas cuando una alerta es de tipo `PROBLEM` o `WARNING` y el sub-agente de reglamento cuando es de tipo `PENALTY` o `WARNING`.

4.2.7 RAG Agent

El agente de reglamento (*RagAgent* en el código) constituye el sexto y último sub-agente de la capa de inferencia y la única vía por la que el sistema consulta el texto literal del reglamento deportivo de la FIA durante la carrera. A diferencia de los cinco sub-agentes anteriores, su activación es condicional y la decide el orquestador según las reglas que se detallan en el último párrafo de esta subsección. La construcción reutiliza la cadena del paradigma RAG presentada en el Sección 2.6 sobre el corpus de los reglamentos deportivos de las temporadas 2023, 2024 y 2025, y se materializa en el cuaderno `N30_rag_agent.ipynb`, que envuelve un *retriever* sobre el índice vectorial precomputado y expone al modelo de lenguaje una única tool, `query_rag_tool`, basada en recuperación multi-consulta, que devuelve los pasajes relevantes para que el LLM sintetice una salida tipada `RegulationContext`.

4.2.7.1 Datos y análisis exploratorio

El sub-agente opera sobre el corpus normativo identificado en el Sección 2.6, las tres ediciones del reglamento deportivo de la FIA correspondientes a las temporadas 2023, 2024 y 2025 (Fédération Internationale de l'Automobile, 2024). Los archivos se descargan en sus versiones *Issue 8* de diciembre de 2023, *Issue 7* de julio de 2024 e *Issue 5* de abril de 2025, con tamaños cercanos a 1,3, 1,7 y 1,3 megabytes respectivamente, mediante el script `scripts/download_fia_pdfs.py` a partir del manifiesto `data/rag/fia_known_urls.json`, que centraliza las URLs oficiales y permite reproducir la descarga sin intervención manual. La estructura interna de cada documento combina una tabla de contenidos al inicio,

con escaso contenido normativo propio, y un cuerpo articulado en epígrafes Art. XX.Y que se identifican mediante expresiones regulares y permiten anotar cada fragmento posterior con la referencia exacta del artículo del que procede.

Sobre los tres documentos opera la canalización presentada en el Sección 2.6, materializada en el script `scripts/build_rag_index.py` y con sus parámetros operativos fijados aquí. La ventana de segmentación se establece en 512 caracteres con un solape de 64 entre fragmentos consecutivos, valores que equilibran la cantidad de contexto disponible por fragmento con la fragmentación necesaria para que el recuperador localice secciones específicas del reglamento sin arrastrar contenido irrelevante. Los fragmentos resultantes se almacenan en una colección `fia_regulations` de Qdrant indexada por similitud coseno, y la canalización es idempotente por construcción, ya que cada fragmento se identifica por una huella SHA-256 que permite reejecutar el proceso sin duplicar entradas. El índice final reúne 2 279 fragmentos distribuidos de forma coherente entre los tres años, en torno a 760 por documento.

4.2.7.2 Modelo de recuperación e integración como sub-agente

La recuperación, descrita conceptualmente en el Sección 2.6, se concreta aquí con sus valores operativos. El modelo BGE-M3 emite simultáneamente representaciones densas, dispersas y multivector, y el sistema utiliza solo su componente denso de 1024 dimensiones, suficiente para la longitud y el tamaño del corpus normativo y compatible con la métrica coseno declarada en la Tabla 4.1. La cantidad de fragmentos recuperados por consulta se fija en cinco a través del parámetro `top_k` de la configuración `RagConfig`, valor que cubre los artículos relevantes a una pregunta concreta sin saturar al modelo de lenguaje con texto redundante. Esta capa de recuperación queda envuelta en una herramienta determinista que el sub-agente invoca durante su razonamiento, como se describe a continuación.

El sub-agente se construye en el cuaderno `N30_rag_agent.ipynb` y se traslada al módulo `src/agents/rag_agent.py`, siguiendo el patrón ReAct ya descrito en el Subsección 4.1.2 mediante la primitiva `create_agent` de LangChain, que monta por debajo el grafo de estado de LangGraph. El agente expone al modelo de lenguaje una única herramienta, `query_rag_tool`, que dada una pregunta libre invoca al recuperador BGE-M3 y devuelve los cinco fragmentos más relevantes con su artículo, su año y su puntuación de similitud. El prompt de sistema orienta al agente a producir respuestas concisas, de dos a tres oraciones, citando explícitamente los artículos consultados y, cuando la primera consulta no localiza el contenido relevante, el patrón ReAct le permite invocar nuevamente la herramienta sobre una reformulación de la pregunta. Esta capacidad de *multi-query retrieval* permite que una pregunta sobre la regla de los dos compuestos, por ejemplo, se descomponga internamente en consultas separadas sobre la regla principal, sus excepciones por meteorología y las penalizaciones aplicables, lo que amplía la cobertura semántica sobre la base de un único marco conceptual.

La activación del sub-agente la decide el orquestador N31 en lugar de ejecutarlo en cada vuelta como hacen los cuatro modelos siempre activos, regla codificada en `src/agents/strategy_orchestrator.py` que dispara la consulta al reglamento en tres escenarios complementarios. El primero es una probabilidad de coche de seguridad en las próximas tres vueltas superior a 0,30 según el agente de situación, condición ante la que el sistema verifica las restricciones reglamentarias de las paradas en régimen de neutralización. El segundo es la detección por parte del agente de radio de una intención de tipo PENALTY o WARNING, que disparan la consulta al reglamento para verificar la infracción que la dirección de carrera está señalando. El tercero es la activación previa del agente de estrategia de paradas, dado que toda decisión de pit cercana se contrasta contra el reglamento (compuestos obligatorios, ventanas de pit lane y procedimientos de salida) antes de elevarse al orquestador. La salida del sub-agente, encapsulada en el `dataclass` `RegulationContext`, reúne cinco campos: la pregunta original elevada al agente, la respuesta sintetizada por el modelo de lenguaje,

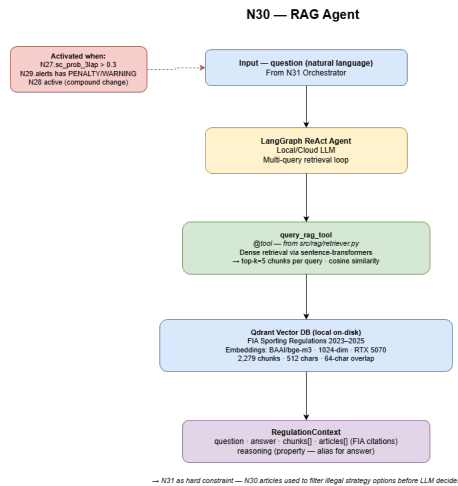


Figura 4.21: Arquitectura del agente de reglamento (N30). El agente LangGraph ReAct dispone de `query_rag_tool`, que invoca al recuperador BGE-M3 sobre el índice Qdrant precomputado y devuelve los cinco fragmentos más relevantes. La salida se consolida en `RegulationContext`. Las reglas de activación se detallan en el texto. Exportado del diagrama N30.

la lista completa de fragmentos recuperados con su texto, su artículo, su año y su puntuación de similitud, la lista deduplicada de identificadores de artículo (en el formato `Article XX.Y`) que el orquestador puede mostrar como referencias citables, y un alias `reasoning` sobre el campo de respuesta para mantener la interfaz uniforme con el resto de sub-agentes. La Figura 4.21 recoge la arquitectura completa del agente.

4.2.8 Strategy Orchestrator

El orquestador estratégico (*Strategy Orchestrator* en el código, N31) es el componente que cierra la cadena de inferencia y produce la recomendación final que recibe el usuario. Se encarga de recoger todas las salidas tipadas de los seis sub-agentes anteriores, decidir qué información combinar en cada vuelta y finalmente traducir esa combinación en una única acción discreta, acompañada de un plan de ejecución y de una serie de contingencias condicionales. Su diseño se materializa como un supervisor manual de tres capas, ya anticipado en el Subsección 4.1.2 y en la teoría general del cap. 2: una primera capa de enrutamiento determinista del tipo *mixture of experts* (Jacobs et al., 1991; Shazeer et al., 2017) que decide qué sub-agentes condicionales activar, una segunda de simulación Monte Carlo (Heilmeier et al., 2020a) que pondera cuatro estrategias candidatas a partir de las distribuciones de incertidumbre que cada sub-agente expone, y una tercera de síntesis mediante un modelo de lenguaje con *structured output* (O. Liu et al., 2024; Wang et al., 2024) que reconcilia los resultados numéricos con las observaciones cualitativas y los valida contra el esquema `StrategyRecommendation`. La construcción se reparte entre el cuaderno `N31_strategy_orchestrator.ipynb`, donde se prueba la cadena de extremo a extremo, y el módulo `src/agents/strategy_orchestrator.py` en producción, que expone dos puntos de entrada equivalentes (`run_strategy_orchestrator` sobre sesión FastF1 viva y `run_strategy_orchestrator_from_state` sobre el `lap_state` reconstruido) en línea con la convención de doble entrada presentada en el Subsección 4.1.2.

4.2.8.1 Enrutamiento MoE entre sub-agentes

La función `_decide_agents_to_call` concreta el enrutamiento *mixture of experts* anunciado en el párrafo anterior con cuatro reglas *if-else* independientes y una cascada de seguridad. El sub-agente de estrategia de paradas (N28) se activa cuando la salida `warning_level` del agente de neumáticos alcanza `PIT_SOON`,

lo que indica que el cliff es inminente, o cuando el agente de radio detecta una alerta de tipo `PROBLEM` o `WARNING`, lo que apunta a un fallo mecánico o a un aviso operativo que puede forzar una parada no planificada. El sub-agente de reglamento (N30) se activa de forma análoga cuando la probabilidad de coche de seguridad estimada por N27 supera el umbral de 0,30 o cuando la radio detecta una alerta de tipo `PENALTY` o `WARNING` que justifique consultar el artículo correspondiente. Una quinta regla cierra el conjunto y refleja la dependencia operativa entre ambos condicionales, ya que toda activación de N28 dispara automáticamente N30 para que cualquier decisión de parada se contraste contra el reglamento (compuestos obligatorios, ventana de pit lane, procedimientos de salida) antes de elevarse al modelo de lenguaje.

El conjunto que devuelve esta función puede ser vacío cuando ninguna condición se dispara y el orquestador prosigue solo con los cuatro sub-agentes de activación permanente (*always-on*), unitario cuando se activa exclusivamente N30 por una alerta de tipo `PENALTY` sin parada asociada, o binario cuando la cascada de N28 arrastra a N30, y se evalúa una sola vez por vuelta antes de invocar a los sub-agentes correspondientes. Las llamadas a los sub-agentes permanentes se distribuyen sobre dos líneas de ejecución coordinadas por un `ThreadPoolExecutor`, ya que los modelos cuya inferencia es *thread-safe* se paralelizan (N25 sobre XGBoost y N27 sobre LightGBM con sus llamadas HTTP al modelo de lenguaje) mientras que los sub-agentes que operan sobre PyTorch (N26) y MLX (N29) se serializan para evitar conflictos en sus respectivos *runtimes* de inferencia. Esta separación, codificada en `_run_always_on_agents_from_state`, mantiene la latencia por vuelta acotada al máximo entre el camino paralelo de N25 y N27 y la cadena secuencial N26-N29, sin que el enrutamiento condicional añada llamadas innecesarias a sub-agentes inactivos.

4.2.8.2 Simulación Monte Carlo

La función `_run_mc_simulation` extrae quinientas muestras independientes con semilla fija (`seed 42`) sobre cinco distribuciones que recogen, una a una, las salidas probabilísticas de los sub-agentes activados en esa vuelta. El ritmo predicho por N25 se modela como una variable normal (Casella & Berger, 2002) centrada en `lap_time_pred` y con desviación típica $\sigma = (CI_{90} - CI_{10}) / (2 \cdot 1,645)$ derivada del intervalo de confianza del *bootstrap*, mientras que las vueltas restantes hasta el cliff de N26 entran como una distribución triangular (Johnson et al., 1995) sobre los percentiles P10, P50 y P90, lo que captura tanto la asimetría del problema como la posición más probable del cliff. La presencia de coche de seguridad de N27 y el éxito del undercut de N28 se tratan ambos como variables Bernoulli (Casella & Berger, 2002) con probabilidades `sc_prob_3lap` y `undercut_prob` respectivamente, y la duración de la parada se muestrea desde otra triangular sobre los percentiles P05, P50 y P95 reportados por N28. Cuando alguno de los sub-agentes condicionales no ha sido activado en esa vuelta, el orquestador aplica priors débiles que evitan tanto la pérdida de la simulación como la inyección de información ficticia, en concreto una triangular fija (2,2; 2,8; 3,8) para la duración de la parada y un valor de 0,5 para la probabilidad de undercut. Una función auxiliar `_clamp_triangular` corrige finalmente el caso degenerado de la primera vuelta de carrera, donde los tres percentiles del cliff pueden coincidir por ausencia de historial reciente de degradación, ampliando el rango por un ε mínimo para evitar el fallo de `numpy.random.triangular` sin distorsionar el muestreo posterior.

Sobre cada muestra se evalúa la función `simulate_lap_window`, que estima la ganancia o pérdida de posiciones respecto a `STAY_OUT` sobre una ventana de cinco vueltas a partir de cinco constantes calibradas a mano sobre el corpus 2023-2024 y de inspiración general en el modelo Monte Carlo de Heilmeyer et al. (2020a), en concreto una ganancia de 0,25 segundos por vuelta del compuesto fresco, una pérdida de 0,80 segundos por vuelta tras pasar el cliff, una conversión de 1,5 segundos por posición ganada o perdida, un ahorro de 8,0 segundos al parar bajo coche de seguridad y la ventana de cinco vueltas mencionada. Los cuatro candidatos evaluados son `STAY_OUT`, `PIT_NOW`, `UNDERCUT` y `OVERCUT`, mientras que la quinta acción del esquema, `ALERT`, queda fuera del muestreo por ser una respuesta excepcional gobernada por banderas deterministas en lugar de por una distribución probabilística. Los quinientos resultados por

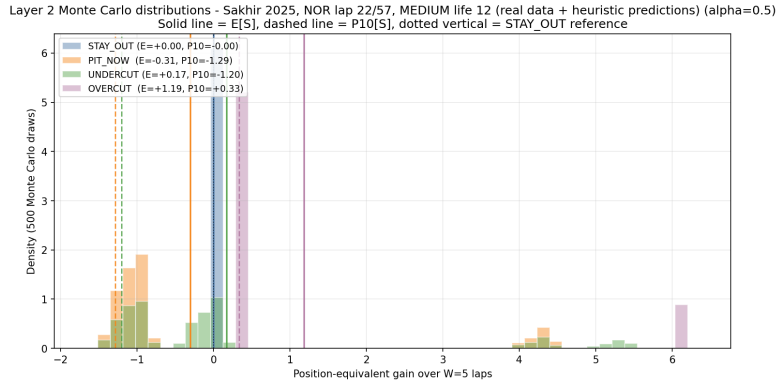


Figura 4.22: Distribuciones de la simulación Monte Carlo de la capa 2 del orquestador sobre el ejemplo cargado por defecto en `N31_mc_visualization.ipynb`, una vuelta real del Gran Premio de Baréin de 2025 (Lando Norris, vuelta 22 de 57, P3 con compuesto MEDIUM y vida 12 vueltas, $\alpha = 0,5$). Los valores observables (tiempo de vuelta, compuesto, edad de neumático, posición y meteorología) se leen del corpus `laps_featured_2025.parquet`, mientras que los valores que en producción aporta cada sub-agente predictivo (banda del *cliff*, probabilidad de coche de seguridad, duración de la parada y probabilidad de *undercut*) se derivan en el cuaderno mediante heurísticas transparentes apoyadas en el contexto de la propia vuelta, lo que permite reproducir la figura sin invocar al *pipeline* completo de inferencia. Cada estrategia se muestrea con quinientas trayectorias y se representa con su histograma normalizado, con la esperanza $E[S]$ marcada como línea sólida y el percentil $P_{10}[S]$ como línea discontinua del mismo color. La línea vertical punteada en cero corresponde a la referencia STAY_OUT. La bimodalidad observable en PIT_NOW, UNDERCUT y OVERCUT refleja la disyuntiva del coche de seguridad, ya que el grupo de muestras situadas a la derecha concentra los casos en los que el coche de seguridad efectivamente se despliega y la estrategia se beneficia del ahorro asociado.

estrategia se resumen en su esperanza $E[S]$, sus percentiles $P_{10}[S]$ y $P_{90}[S]$ y una puntuación agregada $\alpha \cdot E[S] + (1 - \alpha) \cdot P_{10}[S]$, ponderada por el parámetro de tolerancia al riesgo α presente en la `RaceState` (por defecto 0,5, neutra), siguiendo el principio del razonamiento bajo incertidumbre formalizado en O. Liu et al. (2024) de incorporar el peor caso como variable explícita y no implícita en el lenguaje natural posterior. La Figura 4.22 ilustra el funcionamiento sobre un escenario representativo del Gran Premio de Baréin de 2025 en la vuelta 22, donde la bimodalidad de cada distribución refleja la presencia o ausencia de coche de seguridad en la ventana y la puntuación final consolida el valor esperado y el peor caso en un único número que la siguiente capa toma como indicación cuantitativa del mejor candidato.

La transferencia de los resultados de Monte Carlo hacia la capa de síntesis no se reduce a una sola cifra agregada, ya que ese resumen colapsaría las distribuciones y privaría al modelo de lenguaje de la información que el muestreo ha producido con tanto detalle. En su lugar, la función `_build_orchestrator_prompt` inyecta en el prompt una tabla literal con los estadísticos resumen (E , P_{10} , P_{90} y *score*) de las cuatro estrategias candidatas, acompañada de una línea adicional con la estrategia mejor puntuada como sugerencia inicial pero no como decisión vinculante, mientras que el *system prompt* le indica al modelo de lenguaje que esa sugerencia es una entrada más de evidencia y que puede ser sobrescrita por la combinación de salidas de los sub-agentes, por las restricciones del reglamento o por la activación de una contingencia condicional. Independientemente de la decisión final, el diccionario `mc_results` completo se anexa después al campo `scenario_scores` de `StrategyRecommendation` dentro de la función `_assemble_recommendation`, lo que preserva la trazabilidad cuantitativa para la interfaz de usuario y para el análisis posterior de la carrera sin que el modelo de lenguaje tenga capacidad alguna de modificar esos números a posteriori.

4.2.8.3 Síntesis con modelo de lenguaje

La tercera capa delega la decisión final a un modelo de lenguaje configurado mediante el mecanismo structured output de la integración con LangChain, que liga la salida del modelo a un esquema Pydantic

auxiliar y lo valida contra él antes de devolver el resultado al sistema. La función `_get_orchestrator_llm` construye el cliente bajo demanda atendiendo a la variable de entorno `F1_LLM_PROVIDER`, que conmuta entre la API real de OpenAI cuando se ejecutan las pruebas finales y un servidor LM Studio local cuando el sistema opera sin conexión, siempre con temperatura nula para garantizar el determinismo del esquema. La distribución de modelos asigna `gpt-5.4-mini` al orquestador, que sostiene la mayor carga de razonamiento integrador, y `gpt-4.1-mini` a los seis sub-agentes especializados, cuya inferencia está más acotada y se beneficia de una latencia menor, mientras que el cliente de LM Studio admite además cualquier modelo de pesos abiertos servido localmente con la misma interfaz, lo que permite sustituir cualquiera de los dos por un modelo alternativo sin tocar el resto de la pila. La elección óptima a medio plazo, ya razonada entre las restricciones del Capítulo 3, sería el ajuste fino de un modelo abierto de la familia Gemma de Google DeepMind (Gemma Team, 2025) sobre un corpus específicamente curado de Fórmula 1, con el objetivo de dotar al razonador final del conocimiento del dominio que ningún modelo generalista alcanza por defecto, ejercicio que excede el alcance del trabajo actual y queda recogido como línea natural de extensión en el Capítulo 6. La elección de un esquema auxiliar `_LLMSynthesis` en lugar del esquema final `StrategyRecommendation` responde a un requisito técnico de la API de OpenAI, que prohíbe campos con tipo diccionario abierto en estructuras con `additionalProperties: false`, restricción que el campo `scenario_scores` de la salida final habría incumplido y que el orquestador resuelve dejando al modelo solo los campos de razonamiento y reincorporando los diccionarios cuantitativos durante la fase de ensamblado posterior.

Antes de cada llamada al modelo, la función `_build_orchestrator_prompt` consolida todas las evidencias disponibles en una composición que sigue el patrón de mezcla de agentes descrito por Wang et al. (2024). El bloque inicial del prompt recoge los valores numéricos verbatim de cada sub-agente activado en la vuelta (predicción de ritmo y banda de confianza de N25, percentiles del cliff y nivel de alerta de N26, probabilidad de adelantamiento y de coche de seguridad de N27, duración esperada de la parada y probabilidad de undercut de N28 cuando aplica, y conteo de eventos de radio y mensajes de dirección de carrera de N29), acompañados del razonamiento textual que cada uno ha emitido. A continuación se inyecta la tabla de Monte Carlo descrita en el apartado anterior y, cuando N30 se ha activado, un bloque de regulación marcado explícitamente como restricción dura que impide al modelo devolver una decisión que vulnere el reglamento. Por encima de las reglas de decisión locales de cada sub-agente, el prompt define seis salvaguardas estratégicas que vetan recomendaciones inviables, en concreto el bloqueo de paradas antes de la quinta vuelta salvo coche de seguridad o daño confirmado por radio, el bloqueo de paradas con tres o menos vueltas restantes salvo cliff inminente, la restricción de `REACTIVE_SC` a coches de seguridad realmente desplegados, las vidas mínimas de stint por compuesto (8, 12 y 15 vueltas para `SOFT`, `MEDIUM` y `HARD`), la compatibilidad entre compuesto y vueltas restantes, y un descuento de un nivel en las amenazas de N27 durante las tres primeras vueltas para neutralizar el caos de la salida.

El esquema `StrategyRecommendation` reúne catorce campos agrupados en cuatro bloques temáticos, recogidos en la Figura 4.23, que estructuran la respuesta del orquestador como un plan ejecutable en lugar de como una recomendación reducida a un solo número. El primer bloque contiene la decisión primaria (acción discreta, razonamiento textual y nivel de confianza), el segundo los detalles de ejecución de la parada (vuelta objetivo, compuesto siguiente y rival objetivo del undercut), el tercero las instrucciones para el piloto (modo de ritmo, tiempo objetivo y postura de riesgo) y el cuarto la planificación multi-vuelta (lista de contingencias condicionales con sus disparadores, lista de riesgos clave y vuelta esperada de fin de stint), junto con dos campos adicionales que el modelo de lenguaje no rellena directamente sino que el orquestador anexa en código tras la llamada (las puntuaciones de Monte Carlo por estrategia y el contexto de regulación de N30). La función `_assemble_recommendation` compone la `StrategyRecommendation` final mezclando la salida estructurada del modelo con tres campos de N28 (`recommended_lap`, `compound_recommendation` y `undercut_target`) que se utilizan como respaldo determinista cuando el modelo deja en `None` sus

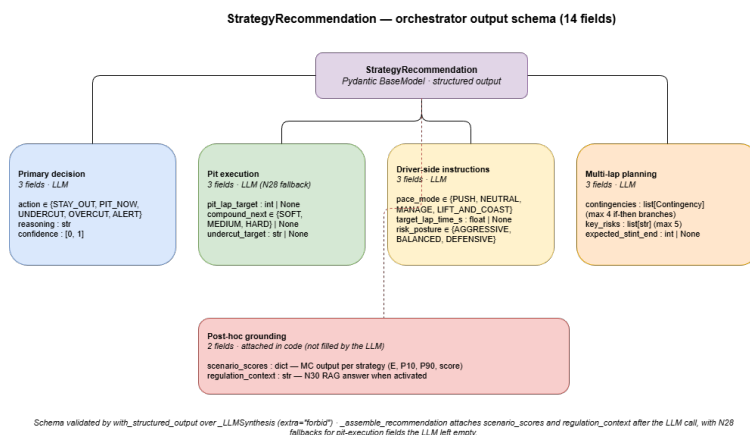


Figura 4.23: Esquema de la salida `StrategyRecommendation` del orquestador (N31) con los catorce campos organizados en cuatro bloques temáticos. El bloque azul recoge la decisión primaria (acción, razonamiento, confianza), el verde los detalles de ejecución de la parada (vuelta objetivo, compuesto siguiente, rival objetivo del *undercut*), el amarillo las instrucciones para el piloto (modo de ritmo, tiempo objetivo, postura de riesgo) y el naranja la planificación multi-vuelta (contingencias condicionales, riesgos clave, vuelta esperada de fin de *stint*). El bloque rosa, conectado por arista discontinua, contiene los dos campos que el modelo de lenguaje no rellena directamente y que el orquestador anexa en código tras la llamada, las puntuaciones de Monte Carlo y el contexto de regulación de N30. El esquema se valida mediante `with_structured_output` sobre el modelo Pydantic auxiliar `_LLMSynthesis` con `extra="forbid"`, lo que impide que el modelo invente campos no declarados.

equivalentes, lo que garantiza que ningún detalle de ejecución producido por el agente de paradas se pierda silenciosamente entre capas y que la decisión del modelo prevalezca cuando este la fija de manera explícita.

4.2.9 Interfaces de uso

El sistema expone tres superficies de uso diferenciadas, cada una orientada a una situación operativa distinta y todas ellas apoyadas sobre el mismo motor de inferencia descrito en las secciones anteriores. La primera es una interfaz de línea de comandos (CLI) desarrollada también con Python, concebida como producto mínimo viable y como herramienta de validación fácilmente reproducible, que ejecuta de extremo a extremo toda la cadena sin necesidad de capa gráfica y presenta sus resultados en forma de tablas Rich en terminal. La segunda combina un panel desarrollado con Streamlit, un servidor FastAPI y un servidor MCP (*Model Context Protocol*, protocolo abierto que estandariza la exposición de herramientas a los modelos de lenguaje) implementado mediante FastMCP, cuyo objetivo es el análisis post-carrera con datos congelados y que añade la posibilidad de consultar los resultados de otras temporadas, comparar telemetrías de pilotos y conversar con un asistente capaz de invocar dinámicamente las herramientas del backend a través del protocolo MCP. La tercera es una réplica de carrera desarrollada con Arcade, que sincroniza en tiempo real tres ventanas (un mapa bidimensional del circuito, un panel estratégico y un panel de telemetría) mediante un servidor TCP local de difusión de eventos, pensada como interfaz demostrativa que muestra la cadena de inferencia funcionando vuelta a vuelta junto a la propia visualización de los vehículos en pista. Las cuatro entradas correspondientes (`f1-strat`, `f1-sim`, `f1-streamlit` y `f1-arcade`) se declaran como *console scripts* en el `pyproject.toml` y quedan disponibles tras la instalación del paquete con `uv tool install`, lo que permite invocar cualquiera de las superficies mediante un único comando, con independencia del directorio de trabajo.

4.2.9.1 Línea de comandos

La interfaz de línea de comandos articula su uso en torno a dos puntos de entrada complementarios y un pequeño complemento de depuración. El primero, `f1-strat` (`scripts/f1_cli.py`), abre un menú interactivo basado en la librería `Rich` que guía al usuario en la elección entre una simulación de un solo piloto y un duelo cabeza a cabeza, y construye sobre la marcha los argumentos del simulador real mediante asistentes específicos para la selección de carrera, piloto y rango de vueltas implementados en `scripts/cli/pickers.py`. El segundo, `f1-sim` (`scripts/run_simulation_cli.py`), es el simulador propiamente dicho, sin capa gráfica alguna, que ejecuta la cadena completa de inferencia vuelta a vuelta sobre el Gran Premio escogido y vuelca el estado vigente del piloto seguido junto con las salidas tipadas de los seis sub-agentes y la recomendación final del orquestador en una tabla `Rich` actualizada en cada giro, como ilustra la Figura 4.24. La tercera entrada, `scripts/debug_agent.py`, es una utilidad orientada al desarrollador que permite invocar a un sub-agente concreto sobre un `lap_state` mínimo para inspeccionar su salida sin levantar el resto del sistema, especialmente útil al instrumentar nuevas reglas o al diagnosticar discrepancias entre la salida de un modelo y la capa que la consume.

Esta superficie funciona además como producto mínimo viable del sistema, ya que valida la cadena de inferencia completa sin depender de capa gráfica alguna y produce los rastros textuales reproducibles que sustentan los casos de uso de la Sección 5.5, lo que permite ejecutarla en cualquier entorno con Python instalado y comparar salidas frente a referencias guardadas. El paquete completo se distribuye como un *wheel* mediante `uv build` y los lanzamientos publicados en GitHub Releases, de modo que la instalación con `uv tool install` deja disponibles las cuatro entradas (`f1-strat`, `f1-sim`, `f1-streamlit` y `f1-arcade`) sin necesidad de clonar el repositorio ni de configurar manualmente las dependencias. La documentación operativa de la simulación, con los argumentos soportados y las trazas esperadas, vive en `documents/dev_docs/SIMULATION_GUIDE.md`, y la cobertura de pruebas automatizadas del módulo se cierra con `tests/test_simulation.py` en cumplimiento del RNF-07.

Lap	Tyre	Age	Pos	Lap (s)	Gap Fwd	Rival	Decision	Plan	Conf	Stay	Pit	Ucut	Ocut	Reasoning	
20	MED/C2	20	1	93.864	-	P2 MED/20 +1.5s	STAY_OUT	MNGR+DEF	-	0.88	0.000	-1.197	-1.181	0.392	STAY_OUT on lap 20 cliff P50 is still the pace delta is on current medium set stop. The situation support pati
21	MED/C2	21	1	95.220	-	P2 MED/21 +1.3s	STAY_OUT	MNGR+DEF	-	0.71	0.000	-1.254	-0.934	0.368	STAY_OUT on lap 21 cliff P50 is still a now while the car is slower than the pri pace loss is real bu pit-now thre
Lap 21															
● Pace	▲next +0.471s						pred 95.69s vs median +1.68s ±2.66s								
● Tire	▲cliff in ~11 laps						Lap 21: predicted 95.691s (+0.471s, slower than prev). CI [92.9-98.2s], +1.678s vs median. range 10-11 laps deg 2.594s/lap 0%								
● Situation	▲threat LOW						Warning level: W0M700, laps to cliff (P50): 11.1. Although the P10 estimate suggests a comfortable margin before the degradation cliff, the tyre life of 21 laps exceeds the typ...								
● Radio	▲no alerts						overtake 0% safety car 2% Threat level: LOW. P(overtake) = 0.0 (gap > 2.5s, so no overtake chance), P(SC 3-lap) = 0.023 (low SC risk). The gap ahead is too large for overtaking and the Safety Car probabi...								
○ Pit	idle						0 radios - 1 rcm								
○ RAG	idle						The Race Control Message indicates that the incident involving car 55 (Sainz) at Turn 17 was reviewed and no further investigation or penalties will be applied. This means there...								
● Pace mode	MANAGE						target 95.691s								
● Risk	DEFENSIVE														
○ Stint end	lap 31														
● Branch	IF SC deployed within 1... +2 more + PIT_NOW, OVERTAKE														
● Risks	Tyre cliff is approach... +3 more														
	Rival P2 MED/21 +1.3s														
Lap 22 / 25 - Summary screen															

Figura 4.24: Captura de la salida vuelta a vuelta de f1-sim en modo cabeza a cabeza sobre el Gran Premio de Suzuka de 2025, con Max Verstappen como piloto principal en primera posición y Lando Norris como rival inmediato en segunda posición a aproximadamente 1,3 segundos por detrás, sobre el rango de vueltas 20 a 26 de la carrera. La parte superior es una tabla histórica que registra las decisiones de las vueltas anteriores con la elección del orquestador (STAY_OUT en ambas), el nivel de confianza, las puntuaciones de Monte Carlo de las cuatro estrategias y un razonamiento abreviado. El bloque central, encabezado por la vuelta en curso (vuelta 21 en la captura), muestra en seis filas el estado de cada sub-agente con su predicción cuantitativa, su nivel de alerta (• activo, ○ inactivo) y el razonamiento textual emitido en esa vuelta. El bloque inferior consolida la planificación multi-vuelta del orquestador (modo de ritmo, postura de riesgo, vuelta esperada de fin de *stint*, contingencias condicionales, riesgos identificados y rival más cercano). El código de colores (verde para señales sanas, amarillo y naranja para alertas crecientes, rojo para riesgos confirmados) permite identificar de un vistazo el estado de la cadena completa en cada vuelta sin necesidad de capa gráfica.

4.2.9.2 Panel Streamlit y backend FastAPI con FastMCP

El segundo entregable se compone de dos mitades complementarias, un servidor FastAPI que centraliza el acceso al motor de inferencia y un panel Streamlit que consume sus servicios desde el navegador. La pieza servidora, alojada en `src/telemetry/backend/`, organiza su API en seis *routers* bajo el prefijo `/api/v1/` que cubren servicio de telemetría desde los *parquets* de FastF1, mapas de dominación de circuito (mapa de calor de velocidad, acelerador y freno por sector), comparación entre las vueltas más rápidas de dos pilotos, asistente conversacional con LM Studio y enrutamiento de consultas, voz (transcripción con Whisper en `/voice/transcribe` y síntesis con Edge-TTS en `/voice/synthesize` para chat por voz de extremo a extremo) y estrategia. El *router* de estrategia es el más voluminoso y expone tanto los seis sub-agentes individuales (pace, tire, situation, pit, radio y regulations) como el orquestador completo a través del *endpoint* `/strategy/orchestrator`, además de un canal en *server-sent events* sobre `/strategy/simulate` para simulaciones vuelta a vuelta consumibles desde el navegador conforme se generan. Por encima de la API REST tradicional, el backend monta un servidor FastMCP en la ruta `/mcp` mediante *Streamable HTTP* (FastMCP 3.x) que expone diez herramientas manuales, siete que envuelven cada sub-agente con parámetros simples de gran premio, piloto y vuelta y tres utilidades auxiliares para enumerar carreras, pilotos y rangos de vueltas disponibles, junto con un conjunto adicional generado automáticamente a partir de la propia especificación OpenAPI del backend mediante `FastMCP.from_openapi()`, que convierte los *endpoints* de telemetría y comparación en herramientas MCP sin código manual. Esta doble vía REST y MCP garantiza que cualquier capacidad del sistema sea invocable tanto programáticamente como en lenguaje natural por clientes conversacionales externos como Claude Desktop o Cursor sin duplicar la lógica subyacente, mientras que el propio chat interno del frontend dispatcha las mismas herramientas mediante un cliente MCP en proceso, lo que evita el sobre coste de las llamadas HTTP locales sin renunciar al contrato MCP compartido.

El frontend Streamlit, lanzado mediante el *console script* `f1-streamlit` (`scripts/run_streamlit.py`) que envuelve a `python -m streamlit run` sobre `src/telemetry/frontend/app/main.py`, presenta seis páginas con responsabilidades disjuntas que cubren los distintos modos de análisis post-carrera. La navegación entre ellas se gestiona con un enrutador propio sobre `st.session_state` en lugar del sistema multi-página por defecto de Streamlit, lo que permite mantener la barra de navegación superior visible y conservar el contexto de selección de carrera, sesión y piloto al cambiar de vista. Las páginas que solo dependen de telemetría externa (*Dashboard*, *Comparison* y *Chat*) cubren las tres temporadas indexadas (2023, 2024 y 2025), mientras que las que invocan al motor de inferencia con sus modelos predictivos entrenados (*Strategy*, *Race Analysis* y *Model Lab*) se restringen a 2025 porque es la única temporada para la que el sistema dispone de los pesos entrenados de los seis sub-agentes y de los datos enriquecidos en el *parquet* consolidado, una restricción que las propias páginas avisan con un banner amarillo al cargarse.

La página *Dashboard* actúa como punto de entrada y presenta, sobre los selectores comunes de año, gran premio, sesión y pilotos (selección múltiple con paleta de color por equipo), una gráfica de tiempos de vuelta con controles de carga junto con siete gráficas de telemetría sincronizadas (velocidad, acelerador, freno, revoluciones, marcha, DRS y delta) y una vista de dominación de circuito por sector que asigna cada zona del trazado al piloto que mejor la transita. La página *Strategy*, ilustrada en la Figura 4.25, es la única que ejecuta de extremo a extremo el orquestador N31 sobre una vuelta concreta, y para ello presenta selectores de gran premio, piloto, un control deslizante del rango de vueltas a analizar y un control deslizante de tolerancia al riesgo entre 0 y 1 que se transmite al cálculo Monte Carlo, tras lo cual el botón *Run strategy* dispara la cadena y muestra cinco bloques en orden, una instantánea de vuelta de estilo CLI con las cinco métricas críticas (compuesto, edad del neumático, hueco al rival anterior, tiempo de vuelta y probabilidad de coche de seguridad), la tarjeta de *StrategyRecommendation* con la decisión y su razonamiento, un botón de descarga del resultado completo como JSON, una gráfica de barras con las puntuaciones de las cuatro estrategias candidatas y un bloque de pestañas con el detalle por sub-agente. La página *Race Analysis* sustituye la vista por vuelta del *Strategy* por una vista de carrera completa sobre hasta tres pilotos, con cinco pestañas dedicadas a la degradación de los neumáticos a lo largo de la carrera, el análisis del hueco a rivales por vuelta, la inteligencia de radio etiquetada con la salida del N29, los artículos del reglamento FIA citados a lo largo del Gran Premio y un resumen agregado del corpus consultado.

La página *Comparison* ofrece la comparación delta entre las vueltas rápidas de dos pilotos cualesquiera con una animación sincronizada en la que las trayectorias se centran y se rotan para superponerse sobre el trazado del circuito, lo que ilustra las diferencias de línea, frenada y trazado curva a curva. La página *Chat* habilita la interfaz conversacional con el sistema mediante un asistente que invoca dinámicamente las herramientas del servidor MCP del backend descritas en el párrafo anterior, con un panel lateral que persiste el histórico de conversaciones y permite crear nuevas, dos modos de interacción seleccionables (texto y voz) que se conmutan desde la propia barra lateral, la generación de informes resumen del intercambio acumulado descargables como fichero, y un canal de voz de extremo a extremo (entrada por micrófono, transcripción con Whisper, síntesis de la respuesta con Edge-TTS) que se activa al pasar al modo de voz y permite al usuario formular consultas habladas sin teclado. La página *Model Lab* cierra el conjunto como banco de pruebas para los seis sub-agentes ejecutados de forma individual sobre vueltas concretas sin pasar por el orquestador, con seis pestañas (Pace, Tyres, Overtake, Safety car, Pit strategy y Radio) que permiten ejecutar cada modelo en un rango de vueltas, comparar la predicción frente al valor real cuando aplica (caso del agente de ritmo, que dibuja la curva predicha contra la observada) o invocar el clasificador de radio en dos modos, una búsqueda sobre transcripciones reales del corpus o un análisis libre sobre un texto escrito a mano por el usuario. La aplicación al completo se despliega con un único comando `docker-compose up`, que levanta los dos servicios (backend en el puerto 8000 y frontend en el 8501) en contenedores aislados con sus dependencias congeladas.

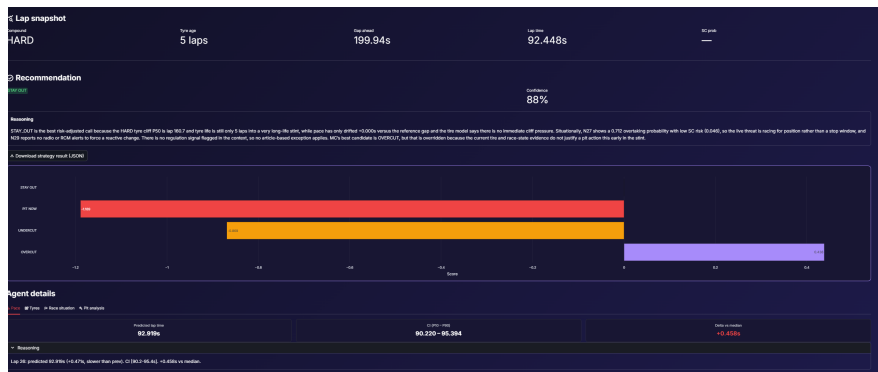


Figura 4.25: Página *Strategy* del panel Streamlit ejecutando el orquestador N31 sobre el Gran Premio de Suzuka de 2025 con Max Verstappen como piloto, configurada en el rango de vueltas 20 a 26 y con la vuelta 26 como vuelta analizada. La vista superpone los cinco bloques descritos en el texto, instantánea de vuelta tipo CLI con las métricas críticas (compuesto HARD, cinco vueltas de edad del neumático, tiempo de vuelta de 92,448 s y probabilidad de coche de seguridad), tarjeta de *StrategyRecommendation* con decisión *STAY_OUT* al 88 % de confianza acompañada del razonamiento del modelo, botón de descarga del resultado en JSON, gráfica de barras con las puntuaciones Monte Carlo de las cuatro estrategias candidatas y bloque de pestañas con el detalle por sub-agente (en la captura, el agente de ritmo predice 92,919 s con intervalo [90,220, 95,394] y un *delta* de +0,458 s frente a la mediana del pelotón). El valor anómalamente alto del hueco al rival anterior (199,94 s) refleja que Verstappen lidera la carrera en esa vuelta y el sistema no encuentra un coche por delante al que medirse, motivo por el que el campo se cubre con una constante centinela en lugar de un hueco real.

4.2.9.3 Réplica en Arcade de tres ventanas

El tercer entregable de la subsección replica las funcionalidades del panel Streamlit en una experiencia de visualización síncrona en escritorio orientada a la inmersión, donde el usuario sigue el desarrollo de la carrera con la posición del coche en pista, la recomendación estratégica y la telemetría instantánea repartidas entre tres ventanas coordinadas. La aplicación se distribuye como *console script* *f1-arcade* (definido en *pyproject.toml* y enlazado a *src/arcade/main.py:main*), de modo que tras un *uv tool install* desde el repositorio el binario queda disponible en el PATH del sistema y se invoca con un único *f1-arcade* sin argumentos. Esta forma canónica abre primero un menú interactivo por teclado, *MenuView*, mostrado en el panel izquierdo de la Figura 4.26, con los campos año (entre 2023 y 2025), ronda (entre 1 y 23, mostrada junto al nombre del Gran Premio leído de *data/tire_compounds_by_race.json*), modo de uno o dos pilotos, código del piloto principal, código del rival (oculto cuando se elige el modo de un solo piloto), equipo (autorrellenado a partir del piloto cuando este aparece en el mapeo *DRIVER_TO_TEAM_2025*) y un selector de estrategia activado o desactivado, navegable con las flechas (arriba y abajo cambian el campo enfocado, izquierda y derecha modifican los valores discretos, las teclas alfabéticas editan los *strings* de piloto y equipo) y confirmable con *ENTER*, mientras que *ESC* cierra la aplicación. Activar el campo de estrategia fuerza la temporada a 2025 por la misma razón discutida en el panel Streamlit, los pesos de los seis sub-agentes solo están disponibles para esa temporada. Para usuarios con preferencias ya fijadas existe el atajo de línea de comandos *f1-arcade -viewer -year 2025 -round 3 -driver VER -team 'Red Bull Racing' -driver2 LEC -strategy*, que salta el menú y carga directamente el replay con los parámetros indicados, equivalente a recorrer el menú campo por campo y pulsar *ENTER* en la última pantalla.

El subsistema arranca dos procesos del sistema operativo que operan en colaboración pero con bucles de eventos independientes, una decisión de diseño motivada por la incompatibilidad práctica entre el bucle de Pyglet, basado en OpenGL, y el bucle de Qt, basado en su propio sistema de señales y *slots*, cuando ambos se ejecutan en el mismo intérprete Python con el guardián global del intérprete (GIL) en juego. El proceso principal, lanzado por *f1-arcade*, aloja la ventana de reproducción construida con la biblioteca *arcade* (un envoltorio de alto nivel sobre Pyglet), un servidor TCP (*Transmission Control*

Protocol, protocolo de transporte orientado a conexión sobre el que se transmiten datos fiables entre los dos procesos del subsistema) local que se levanta en 127.0.0.1:9998 y un hilo en segundo plano (SimConnector) que invoca al motor de simulación vuelta a vuelta y al pipeline de estrategia. Cuando el modo de estrategia está activado, este proceso lanza con `subprocess.Popen([sys.executable, m, "src.arcade.dashboard"])` un segundo proceso completamente independiente que arranca un único `QApplication` de PySide6 dentro del cual conviven simultáneamente las dos ventanas Qt restantes, el panel estratégico y la malla de telemetría, compartiendo el mismo bucle de eventos y el mismo intérprete pero quedando aisladas del proceso de Pyglet. Ambos procesos se comunican exclusivamente a través del canal TCP, sobre el que el proceso principal emite mensajes en formato *JSON Lines* (un objeto JSON terminado en salto de línea por mensaje) varias veces por segundo con la última instantánea de sesión, telemetría del piloto principal y del rival, recomendación del orquestador y razonamiento por cada uno de los seis sub-agentes, mientras que las dos ventanas Qt actúan como consumidores pasivos, cada una con su propio cliente `TelemetryStreamClient` que lee el flujo y reparte el contenido a sus *widgets*. Esta separación en dos procesos es la pieza clave que permite que la animación del coche en pista se mantenga fluida cuando el orquestador invoca al modelo de lenguaje en una nueva vuelta, dado que la inferencia ocurre en el hilo en segundo plano del proceso principal y nunca bloquea ni el bucle de renderizado de Pyglet ni el bucle de eventos de Qt en el otro proceso. El subsistema cuenta además con su propia copia adaptada del orquestador N31 en `src/arcade/strategy_pipeline.py` que reutiliza la lógica de capas y los seis sub-agentes descritos en Subsección 4.2.8, lo que le permite operar con autonomía completa frente al backend FastAPI y reducir la latencia al evitar el sobre coste del protocolo HTTP (*HyperText Transfer Protocol*), ejecutándose una vez por vuelta y delegando la inferencia del LLM en el proveedor configurado por la variable `F1_LLM_PROVIDER`.

La primera de las tres ventanas, etiquetada como replay y construida sobre la biblioteca *arcade* (un envoltorio de alto nivel sobre Pyglet) a veinticinco fotogramas por segundo, recrea el desarrollo de la carrera vuelta a vuelta a partir del *parquet* consolidado de FastF1 cargado al inicio mediante `SessionLoader().load(year, round, gp)`, tal como ilustra el panel derecho de la Figura 4.26. La vista superpone el trazado del circuito con sus zonas de DRS resaltadas en verde, los iconos de los dos coches pintados con el color del equipo correspondiente al piloto principal y al rival cuando este existe, un panel meteorológico con temperatura del aire, temperatura de pista, humedad, dirección y velocidad del viento e indicador de lluvia, dos paneles de información de piloto con compuesto de neumático, edad del juego, marcha actual, estado de DRS, hueco al rival anterior y posterior y velocidad instantánea, una tabla de clasificación con las veinte posiciones de la carrera, cada una resaltada con el compuesto montado por el piloto y su brecha al líder, una barra de progreso que indica vuelta actual sobre vuelta total junto con eventos especiales como banderas amarillas, intervenciones del coche de seguridad o abandonos, y una leyenda de controles inferior. La reproducción se gobierna íntegramente por teclado, con la barra espaciadora para pausar y reanudar, las flechas izquierda y derecha para retroceder y avanzar vuelta a vuelta, las teclas + y - para regular la velocidad de la animación entre la mitad y cuatro veces el tiempo real y ESC para volver al menú o cerrar la aplicación. El bucle de actualización de Pyglet se mantiene independiente del bucle de Qt en el otro proceso para evitar que el guardián global del intérprete introduzca pausas perceptibles en la animación cuando el orquestador invoca al modelo de lenguaje en una nueva vuelta, una decisión de diseño ya motivada en el párrafo anterior y que se materializa visualmente en la fluidez con la que se renderizan los iconos de los coches al avanzar la simulación entre vueltas, incluso mientras la inferencia del orquestador se ejecuta en segundo plano.

La segunda ventana, el panel estratégico construido con `PySide6.QMainWindow` y mostrado en la Figura 4.27, traduce a interfaz visual la cadena de inferencia descrita en el Subsección 4.2.8, organizándose en dos columnas con responsabilidades complementarias. La columna izquierda concentra la decisión final del orquestador con un botón cromáticamente codificado por acción (verde para `STAY_OUT`, rojo para `PIT_NOW`, naranja para `UNDERCUT`, púrpura para `OVERCUT` y amarillo para `ALERT`), una barra de confianza,



Figura 4.26: Menú inicial (*izquierda*) y ventana de replay (*derecha*) del subsistema Arcade. El menú MenuView muestra el formulario de configuración por teclado con los campos descritos en el texto, en este caso Suzuka 2025 con modo de dos pilotos, NOR como piloto principal, VER como rival, equipo McLaren autorrellenado a partir del piloto principal y selector de estrategia activado, junto con la línea de ayuda inferior que documenta los atajos de navegación. La ventana de replay corresponde al instante en pausa de la sexta vuelta sobre cincuenta y tres del Gran Premio de Suzuka de 2025 y reúne en una única vista el trazado del circuito con las zonas de DRS resaltadas en verde, la posición sobre la pista de los dos coches pintados con los colores del equipo correspondiente, el panel meteorológico (45 °C de temperatura de pista, 18 °C de aire, 55 % de humedad, viento del sur a 12 km/s y condiciones secas), los dos paneles de información del piloto principal (NOR a 235 km/h en sexta marcha, compuesto medio, hueco nulo al líder PIA y 2,05 s de margen sobre LEC) y del rival (VER a 245 km/h en sexta marcha, compuesto medio y 2,07 s por detrás del líder), la tabla de clasificación con los veinte pilotos resaltados por su compuesto montado, la barra de progreso de la vuelta sobre el total y la leyenda inferior de atajos de teclado.

etiquetas para el modo de ritmo y la postura de riesgo, una línea de plan multivuelta con la vuelta de parada recomendada y el compuesto siguiente, una gráfica de barras horizontales con las cuatro puntuaciones Monte Carlo de los escenarios candidatos y un grupo de pestañas que dan acceso al razonamiento textual del orquestador y al de cada sub-agente con resaltado de sintaxis. La columna derecha despliega una malla de tres filas por dos columnas con seis tarjetas, una por cada uno de los sub-agentes presentados desde el Subsección 4.2.2 hasta el Subsección 4.2.7, donde cada tarjeta replica visualmente las salidas tipadas de su agente correspondiente (predicción puntual, banda de incertidumbre, probabilidad estimada, percentiles de duración, etiqueta de alerta o fragmento textual recuperado, según el caso) sin volver a explicar la mecánica subyacente, ya cubierta en sus respectivas subsecciones.

Cada tarjeta se actualiza al recibirse un nuevo mensaje sobre el canal TCP descrito en el párrafo de arquitectura, a través de una señal `data_received` que dispara `MainWindow.on_data_received` y reparte el contenido a la tarjeta correspondiente, con las gráficas de ritmo y neumáticos acumulando histórico local entre vueltas para preservar la curva temporal completa. Las tarjetas que el orquestador no ha activado para una vuelta dada (típicamente *pit* cuando no toca parar todavía o reglamento cuando no se ha invocado al RAG) aparecen atenuadas con etiquetas neutras como “regulation loaded” o “no alerts”, distinguiendo visualmente la información presente de la latente sin generar ruido. La instantánea recogida en la Figura 4.27 corresponde a una vuelta concreta del Gran Premio de Suzuka de 2025 elegida como ilustración del comportamiento típico del panel, mientras que en el Capítulo 5 se discuten varios casos de uso completos sobre carreras reales para evaluar la coherencia del sistema en conjunto y el desempeño individual de cada sub-agente frente a las decisiones del muro de boxes en su momento.

La tercera y última ventana, recogida en la Figura 4.28, presenta la telemetría comparada del piloto principal y del rival sobre la vuelta en curso mediante una malla de cuatro paneles construida con `pyqtgraph` y alimentada por el mismo flujo TCP que abastece al panel estratégico, lo que garantiza que las tres ventanas reflejen el mismo instante de la simulación. Los paneles se distribuyen en una cuadrícula de dos por dos con escalas fijas y eje horizontal compartido en metros recorridos a lo largo del trazado, lo que permite

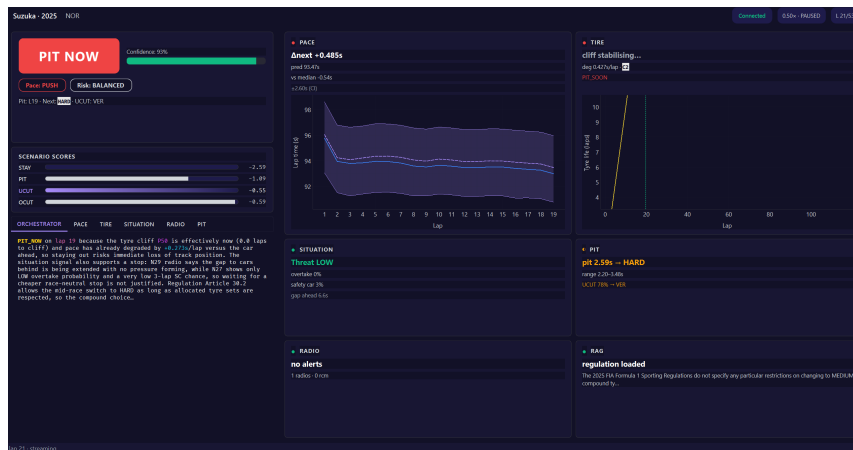


Figura 4.27: Panel estratégico del subsistema Arcade construido con PySide6.QMainWindow, capturado en la vuelta 21 de 53 del Gran Premio de Suzuka de 2025 con NOR como piloto seguido. La columna izquierda muestra la decisión del orquestador (PIT_NOW al 93 % de confianza, modo *push*, postura *balanced*, vuelta de parada L19, compuesto HARD, objetivo de *undercut* VER) junto con las puntuaciones Monte Carlo de los cuatro escenarios y el razonamiento textual. La columna derecha presenta las seis tarjetas de sub-agentes con sus salidas individuales. La recomendación coincide con la decisión real de McLaren en esa carrera, que convirtió a NOR de MEDIUM a HARD entre las vueltas 21 y 22.

alinean visualmente los puntos de frenada, las levantadas del acelerador y los picos de velocidad entre los dos coches sin necesidad de procesamiento adicional. El panel superior izquierdo dibuja el delta de tiempo entre el rival y el piloto principal, recortado a una banda de aproximadamente más-menos tres segundos para hacer perceptibles las diferencias subsegundo, donde la línea horizontal en cero corresponde al piloto principal seguido y la traza discontinua marca al rival, de modo que un valor negativo indica que el rival va por delante en ese punto del trazado. El panel superior derecho representa la velocidad instantánea entre cero y trescientos cincuenta kilómetros por hora, mientras que los dos paneles inferiores siguen el mismo esquema cromático y muestran la presión de freno y la apertura del acelerador en una escala porcentual entre cero y ciento cinco, con la línea sólida en el color asignado a cada métrica para el piloto principal y la línea discontinua naranja del color del equipo rival para el segundo coche. Esta vista permite contrastar de un vistazo si una pérdida de tiempo entre los dos coches proviene de una frenada más tardía, una salida de curva más agresiva o una combinación de ambas, cerrando así la réplica visual del subsistema Arcade con un grado de detalle que el panel de reproducción y el panel estratégico no podrían absorber sin saturar.



Figura 4.28: Ventana de telemetría en directo del subsistema Arcade construida con `pyqtgraph` sobre una malla de dos por dos con escalas fijas y eje horizontal en metros recorridos. La captura corresponde a la vuelta veinte del Gran Premio de Suzuka de 2025, con NOR como piloto principal y VER como rival, sobre los primeros dos kilómetros y medio del trazado. El panel superior izquierdo muestra el delta de tiempo entre rival y piloto principal, con VER situado aproximadamente un segundo y medio por delante de NOR a lo largo de la sección representada (valor negativo del delta indica rival por delante). El panel superior derecho recoge el perfil de velocidad de ambos coches con picos próximos a los 310 km/h en las rectas y trazas prácticamente superpuestas a la resolución de la gráfica, lo que confirma que en condiciones de aire sucio la diferencia de pace entre los dos coches es marginal. El panel inferior izquierdo dibuja la presión de freno en porcentaje, con cuatro zonas de frenada visibles y un solapamiento casi exacto de los puntos de aplicación entre los dos pilotos, mientras que el panel inferior derecho representa la apertura del acelerador, donde aparecen pequeñas divergencias en la modulación a media curva en las dos secciones técnicas del primer sector. La línea sólida en el color asignado a cada métrica corresponde al piloto principal y la línea discontinua naranja al rival.

5 RESULTADOS

Ni el más sabio conoce el fin de todos los caminos.

— Gandalf el Gris (La Comunidad del Anillo)¹

El Capítulo 4 justifica el diseño del sistema, desarrolla la implementación de cada uno de los sub-agentes y modelos predictivos del sistema y presenta las tres superficies que lo hacen accesible. Resta ahora mostrar las métricas y verificar empíricamente qué consigue el sistema una vez confrontado con datos reales del campeonato y, no menos importante, dónde se encuentran sus límites.

La exposición se ordena de menor a mayor escala de integración. El Sección 5.1 reporta las métricas de cada modelo predictivo en una tabla por componente. En ella figuran la versión final, las variantes intermedias, las alternativas descartadas y, donde procede, una línea base de referencia, cubriendo así tanto las elecciones de arquitectura como las distintas configuraciones de los conjuntos de entrenamiento y prueba. El Sección 5.2 eleva el plano y mide el comportamiento de cada sub-agente una vez integrado con su prompt, su esquema de salida y sus salvaguardas. Posteriormente, el Sección 5.3 cierra la cadena con las pruebas de extremo a extremo del orquestador sobre una carrera completa y sobre una muestra representativa de radios y mensajes de control. El Sección 5.4 sitúa después los resultados frente a los trabajos previos identificados en el Capítulo 2. El capítulo se cierra con el Sección 5.5, donde tres carreras del calendario 2025 sirven de banco de prueba para las tres superficies del sistema. Cada una recoge una decisión del muro de boxes que la carrera reveló a posteriori como acierto o como error, y la recomendación emitida por el sistema se contrasta con esa decisión real.

El recorrido no oculta los casos en los que el rendimiento se queda por debajo de lo esperado, una variante terminó archivada o una línea base sin entrenamiento acaba siendo competitiva. La decisión es deliberada: una evaluación que omitiera los resultados negativos contaría una historia parcial. Los casos de carrera que cierran el capítulo lo subrayan desde el lado contrario, mostrando situaciones en las que muros de boxes con décadas de experiencia tomaron decisiones legítimamente inciertas que solo el tiempo reveló como acierto o como error, sin que el sistema reclame ver con mayor nitidez caminos que entonces permanecían cerrados a todos.

5.1 Resultados de los modelos predictivos

El protocolo de validación temporal por bloques descrito en el Sección 2.2 se aplica de forma homogénea a todos los modelos de estrategia, sobre el corpus consolidado en el Subsección 4.2.1. Los modelos del pipeline de radio, condicionados por el tamaño limitado del conjunto etiquetado, emplean particiones estratificadas que se declaran en cada subsección. La teoría de cada arquitectura ya se cubre en el Capítulo 2 y su implementación en el Capítulo 4, por lo que cada subsección se limita a reportar las variantes ensayadas, la tabla comparativa con la decisión adoptada y una discusión breve de los hallazgos.

¹Fragmento de *La Comunidad del Anillo*, primera parte de *El Señor de los Anillos*, de J. R. R. Tolkien (1954), libro I, capítulo 2 «La sombra del pasado». Traducción al castellano de Matilde Horne y Luis Domènech para la edición Minotauro. Original en inglés: *For even the very wise cannot see all ends.*

Tabla 5.1: Variantes evaluadas durante el diseño del regresor de tiempo de vuelta (cuaderno N06). Las celdas – corresponden a métricas no reportadas en el cuaderno. El detalle del recorrido y la lectura del empate con la persistencia naïve se discuten en la prosa adyacente.

Variante	Train	Test	MAE (s)	RMSE (s)	R ²	Decisión
Baseline XGB (<i>default</i>)	2023	2024	0,719	1,303	–	Referencia inicial
XGB con year_circuit_median	2023	2024	0,615	0,926	–	<i>Leakage</i>
XGB absoluto sin leakage	2023	2024	0,653	1,747	0,970	Absoluto retenido
Per-cluster con leakage	2023	2024	2,149	–	–	Archivada
Per-cluster sin leakage	2023	2024	2,002	–	–	Archivada
XGB absoluto retrain	2023+2024	2025	0,817	2,435	0,946	<i>Outliers</i> Shanghái/Austin
XGB delta (val)	2023	2024	0,461	–	–	Mejor en validación
XGB delta retrain	2023+2024	2025	0,410	0,765	0,995	Modelo de producción
Persistencia	–	2025	0,408	–	–	<i>Training-free</i>
$\hat{y}_k = \text{Prev_LapTime}_k$	–	2025	3,062	–	–	<i>Training-free</i>
Mediana team $\times$ circuito	–	2025	3,062	–	–	<i>Training-free</i>

5.1.1 Predicción de tiempo de vuelta

Sobre el conjunto featurizado descrito en el Subsubsección 4.2.1.3 se evaluaron sucesivamente varias configuraciones del regresor XGBoost (*eXtreme Gradient Boosting*) del tiempo de vuelta, cuya familia se introduce en el Sección 2.3. La línea de partida fue un baseline sin búsqueda de hiperparámetros sobre el vector completo de variables. La búsqueda aleatoria con restricciones monotónicas mejoró esa referencia, pero apoyó parte de su señal en year_circuit_median, una variable que resume el ritmo absoluto de la sesión y resulta por tanto susceptible de filtrar información futura entre temporadas, motivo por el que se descartó para producción. La versión reentrenada sin esa variable se conservó como modelo absoluto y, en paralelo, se probó una variante per-cluster con un regresor independiente por arquetipo de circuito que terminó archivada al producir un MAE medio sustancialmente peor que el global. Sobre el holdout 2025, el modelo absoluto reveló un comportamiento bimodal: el grueso de circuitos quedaba con error contenido, pero Shanghái y Austin lo disparaban en varios segundos al haber movido su ritmo absoluto fuera del rango aprendido durante el entrenamiento, una manifestación directa de la deriva descrita en el Sección 2.2. La respuesta metodológica fue reformular el objetivo como diferencia entre vueltas consecutivas, eliminar las variables absolutas de velocidad y degradación incompatibles con esa formulación, eliminar las restricciones monotónicas (un delta puede ser negativo) y reentrenar sobre la unión de las dos temporadas de entrenamiento con una malla específica de hiperparámetros. El modelo resultante se contrasta además con dos líneas base *training-free* extraídas de data/eval/pace_baselines.csv, persistencia naïve (predecir el tiempo de la vuelta anterior) y mediana team $\times$ circuito de la temporada en curso, que sirven de techo natural del problema antes de invocar aprendizaje automático.

La Tabla 5.1 concentra tres hallazgos. En primer lugar, la reformulación del objetivo a diferencia entre vueltas consecutivas amortigua el efecto de los circuitos donde el ritmo absoluto se aleja del rango de entrenamiento y reduce a la mitad el MAE sobre holdout 2025 sin añadir variables ni cambiar la arquitectura, lo que confirma que el problema no estaba en la capacidad del modelo sino en la formulación del objetivo. En segundo lugar, la variante seleccionada empata con la persistencia naïve (0,410 frente a 0,408 segundos), un resultado honesto que conviene reconocer: la diferencia es despreciable y, sin embargo, la importancia relativa de las variables se redistribuye desde Prev_LapTime, responsable de más de la mitad de la ganancia en el modelo absoluto, hacia magnitudes físicas como TrackTemp, LapsSincePitStop, AirTemp y los lag-1 de degradación, lo que mantiene un estimador robusto en los escenarios donde el tiempo de la vuelta anterior deja de ser representativo (cambio de compuesto, coche de seguridad, lluvia, primera vuelta del

Tabla 5.2: Métricas internas de KMeans para $K \in \{2, \dots, 8\}$ sobre 24 circuitos y 14 características (cuaderno N03). El máximo de silueta se alcanza en $K = 2$, pero ningún valor supera el umbral 0,5 asociado a una separación nítida, indicio de que los circuitos forman un continuo de propiedades antes que clases disjuntas. La configuración $K = 4$ se selecciona por el codo en la inercia y por su correspondencia con cuatro arquetipos interpretables (*strategic chaos*, *long high-speed*, *warm conservative*, *high-stress outliers*) aprovechables aguas abajo como variable categórica de entrada. La decisión es interna a la familia KMeans; otras familias de *clustering* no se evaluaron.

K	Inercia	Silueta	Decisión
2	258,27	0,222	Máximo de silueta
3	209,21	0,198	Alternativa más simple
4	168,55	0,201	Codo + dominio F1
5	153,80	0,170	–
6	132,87	0,178	–
7	115,67	0,168	–
8	102,45	0,174	–

stint). En tercer lugar, las dos variantes per-cluster quedaron archivadas con y sin leakage, lo que indica que la información de cluster es más útil como variable de entrada de un único modelo global que como criterio de partición del conjunto de entrenamiento. El modelo de producción es, por tanto, el XGBoost delta reentrenado sobre 2023 y 2024 sin restricciones monotónicas, cuya integración dentro del agente de ritmo y su intervalo de confianza P10-P90 se describen en el Subsección 4.2.2, y cuya traducción en decisiones operativas se aborda en el Sección 5.2.

5.1.2 Agrupamiento de circuitos

El agrupamiento descrito en el Subsubsección 4.2.1.2 parte de los veinticuatro circuitos del calendario y un vector de catorce características de telemetría agregadas, tras descartar dos redundantes por correlación interna. Sobre ese vector estandarizado se aplicó la familia KMeans introducida en el Sección 2.8 con $K \in \{2, \dots, 8\}$ y `random_state=42`, y en cada configuración se midió la inercia y el coeficiente de silueta cuya validación gráfica conjunta se reproduce en la Figura 4.3. La comparación se restringió deliberadamente a esta familia algorítmica: el reducido tamaño del corpus desaconsejaba alternativas como DBSCAN (Ester et al., 1996), cuya estimación de los hiperparámetros ε y *minPts* resulta inestable con tan pocas muestras y produce además puntos clasificados como ruido que romperían el contrato de partición completa requerido por los modelos descendentes, así como los modelos de mezcla gaussiana (Reynolds, 2009), que exigirían estimar matrices de covarianza de orden 14×14 con apenas seis observaciones esperadas por componente. El índice de Davies-Bouldin no se calculó por aportar información redundante con la dupla inercia-silueta dentro de una misma familia de partición euclídea.

La Tabla 5.2 muestra que la silueta queda muy por debajo de 0,5 en todos los valores de K : la hipótesis de clases disjuntas no se sostiene y la decisión por $K = 4$ se basa en el codo de la inercia y en su correspondencia con la taxonomía de cuatro arquetipos que la prensa motorsport y los proveedores oficiales del campeonato manejan habitualmente (circuitos de potencia, de alta carga aerodinámica, callejeros, esto es, trazados urbanos sobre calles cerradas al tráfico, como Mónaco o Bakú, y de degradación elevada) (Holding, 2023), antes que en una métrica que premiase una separación que no existe. Al reasignar los circuitos de 2025 con el modelo entrenado sobre 2023 y 2024, once trazados migran al Cluster 1 y solo Montreal mantiene su clase original, un rebalanceo coherente con la deriva inter-temporadas descrita en el Sección 2.2 y con el comportamiento bimodal observado en el modelo absoluto de la Subsección 5.1.1. Ni DBSCAN ni los modelos de mezcla gaussiana quedan descartados de forma definitiva: con un corpus de circuitos más extenso, sea por incorporación de temporadas pre-2022 o por ampliación del calendario, su evaluación volvería a estar justificada como línea natural de trabajo futuro.

Tabla 5.3: Estudio de *ablación* de la TCN global de degradación sobre el holdout 2025 (cuaderno N09). La versión v4 se selecciona por la estabilidad de su programación cosenoidal monótonica del *learning rate*, no por mejora numérica frente a v1, ya que ambas convergen a métricas equivalentes hasta el tercer decimal. Las variantes v2 y v3 documentan ramas exploratorias archivadas (incremento de `d_model` y *warm restart* truncado, respectivamente).

Variante	MAE (s)	RMSE (s)	R ²	Decisión
TCN v1 (<i>baseline</i>)	0,708	1,123	0,605	Reemplazada por v4
TCN v2 (<code>d_model=128</code>)	0,737	1,139	0,593	Peor en todo, archivada
TCN v3 (<i>warm restart</i> truncado)	0,724	1,126	0,602	Inestable, archivada
TCN v4 (cosenoidal monótonica)	0,708	1,123	0,605	Modelo de producción

Tabla 5.4: Modelo de degradación por compuesto sobre el holdout 2025 (cuaderno N10). El enrutado se justifica en la Tabla 4.2 del cap 4. El *sigma* procede de cincuenta pasadas de MC Dropout y C1 y C3 heredan el del modelo global al no recalibrarse.

Compuesto	Modelo en producción	MAE (s)	σ MC (s)
C1	global v4	0,943	(hereda global)
C2	<i>fine-tuned</i>	0,550	0,124
C3	global v4	0,622	(hereda global)
C4	global v4	1,138	0,150
C5	global v4	0,974	0,155
C6	global v4 (<i>fallback</i>)	1,010	0,262

5.1.3 Degradación de neumático

El modelo de degradación se entrena con los datos de 2023 y 2024 y se evalúa sobre el holdout 2025 (cuaderno N09), con la canalización de N08, que aplica padding causal y ventana global de veintiocho vueltas sobre el corpus de stints en seco. Sobre esa preparación, la arquitectura es una TCN con cuatro bloques residuales, kernel tres y dilatación exponencial, cuyos fundamentos teóricos se desarrollan en el Sección 2.4 y cuya implementación se detalla en el Subsección 4.2.3. El espacio de variantes considera cuatro decisiones encadenadas. La primera de ellas compara el muestreo per-stint, descartado por un MAE de validación sustancialmente peor, con el per-lap, que multiplica por veinte el corpus de entrenamiento y se adopta como base. A continuación, la segunda toma la forma de un estudio de *ablación* entre cuatro versiones de la propia TCN (v1 a v4), procedimiento que modifica de manera controlada un único factor cada vez para aislar su contribución al rendimiento (Meyes et al., 2019), y selecciona v4 por la estabilidad de su programación cosenoidal monótonica del *learning rate*. Sobre la versión seleccionada, la tercera evalúa el fine-tuning por compuesto cuyo enrutado se recoge en la Tabla 4.2 del cap 4, donde solo C2 mejora sobre el global y queda como única especialización en producción, mientras C1, C3, C4 y C5 caen al global por resultados neutros o por empeoramiento, y C6 carece de muestras para reentrenar. Por último, la cuarta evalúa una variante incremental (Modelo B) sobre la derivada del objetivo, archivada como predictor único por su R^2 bajo y reincorporada como variable `tcn_wear_rate` dentro del modelo global. Al margen del recorrido sobre la familia TCN, la alternativa LSTM no se reentrena en este trabajo: el legacy del autor (Vega Sobral, 2025) ya recorrió la migración de LSTM a TCN sobre un objetivo distinto de predicción multipaso de tiempo por vuelta, donde la red recurrente cedió ante la TCN por su mayor campo receptivo y por la fuerte autocorrelación intra-stint, y este trabajo hereda esa decisión arquitectónica sin reabrirla.

El estudio de ablación de la Tabla 5.3 deja métricas equivalentes hasta el tercer decimal entre v1 y v4 en MAE, RMSE y R^2 ; el reentrenamiento sin truncar el ciclo cosenoidal (v4) estabiliza el modelo en un punto reproducible y, por esa razón, se adopta para producción. El desglose por compuesto de la Tabla 5.4 sobre el modelo definitivo deja ver dos asimetrías relevantes. La primera, que solo C2 obtiene una ganancia clara con fine-tuning, mientras que C4 y C5 empeoran tras la especialización, lo que explica que el enrutado de producción mantenga el resto en el global aunque la documentación narrativa del cuaderno los marcara inicialmente como neutrales. La segunda, el orden creciente del *sigma* estimado por MC Dropout (*Monte*

Tabla 5.5: Variantes evaluadas durante el diseño del clasificador de adelantamiento (cuadernos N11, N12 y N12B) sobre el holdout 2025. La línea base aleatoria toma como probabilidad la tasa positiva observada (0,084).

Variante	AUC-PR	AUC-ROC	Decisión
Línea base aleatoria (tasa 0,084)	0,076	0,500	Referencia
LightGBM por defecto	0,526	–	Punto de partida
LightGBM Optuna + Platt	0,549	0,876	Modelo de producción
CatBoost Optuna	0,552	–	Marginal, descartado
TCN causal (N12B)	0,099	–	Archivada (<i>negative result</i>)

Carlo Dropout, introducido en la Sección 2.4), coherente con el orden del MAE: la incertidumbre marginal sube precisamente en los compuestos donde el modelo predice con mayor error. La cobertura empírica P10–P90 sobre el holdout 2025 muestra que las sigmas crudas infracoberturan el intervalo (0,21 global frente al 0,80 nominal); tras aplicar el factor de escala derivado del conjunto de calibración, la cobertura sube a 0,88 global y al rango 0,79–0,92 entre compuestos, lo que valida el régimen MC Dropout como aproximación operativa de la incertidumbre marginal. Por último, el MAE crece un factor próximo a $\times 2,6$ entre los tramos medio (*TyreLife* 6–15) y muy avanzado (31 vueltas en adelante), una limitación atribuible a la escasez de stints largos en entrenamiento. La traducción de estas magnitudes en decisiones de pit y en niveles de alerta se aborda en el Sección 5.2.

5.1.4 Probabilidad de adelantamiento

La predicción de adelantamiento es un problema de clasificación binaria sobre pares piloto-rival, con tasa base global del 8,4 % en el holdout de 2025. Para esta tarea se emplea un LightGBM (*Light Gradient Boosting Machine*) calibrado con Platt sobre los *scores* crudos, ajustado mediante una búsqueda Optuna de cien ensayos; la familia teórica se introduce en el Sección 2.3 y la implementación dentro del sistema se describe en el Subsección 4.2.4. Las variantes evaluadas encadenan tres decisiones. La primera fija el LightGBM por defecto como referencia inicial. La segunda incorpora la búsqueda Optuna y la calibración de Platt, que se adoptan como configuración final. La tercera evalúa dos alternativas que no llegan a producción. CatBoost ofrece una ganancia marginal que no justifica el coste en infraestructura y estabilidad. Una TCN causal ensayada en el cuaderno N12B sobre la misma serie temporal, con la teoría introducida en el Sección 2.4, cede ante el LightGBM: las dieciocho mil secuencias disponibles quedan muy por debajo del volumen necesario para que un modelo puramente secuencial recupere la señal que la ingeniería de variables ya inyecta de forma explícita.

La Tabla 5.5 muestra que el LightGBM separa con holgura las dos clases sobre el holdout: su AUC-ROC de 0,876 queda muy por encima del clasificador aleatorio y su AUC-PR multiplica la tasa base por un factor próximo a 7,2. La señal dominante que el modelo captura no es la mera ventaja puntual del perseguidor sino el régimen DRS: el 80,5 % de los adelantamientos observados sobre el corpus completo se produce con el coche que persigue dentro de la ventana de un segundo en el punto de detección, condición que habilita la apertura del alerón móvil en las zonas DRS de la siguiente sección de pista. El 19,5 % restante corresponde a pases en zonas sin DRS, donde el adelantamiento depende de combinaciones de ritmo sostenido y desgaste relativo de neumático que el modelo también capta como variables de peso comparable. La calibración por Platt aporta el resto del valor: traduce los *scores* crudos en probabilidades fiables, condición necesaria para que el modelo se integre en la cadena de inferencia descrita en el Sección 5.2. Frente a esto, la TCN causal de N12B queda archivada como resultado negativo documentado, en línea con el patrón ya observado en la degradación de neumático del Subsección 5.1.3.

Tabla 5.6: Tres horizontes de predicción de coche de seguridad evaluados sobre el holdout 2025 (cuaderno N14). El *lift* se calcula como AUC-PR dividido por la tasa base propia de cada horizonte.

Variante	Tasa base	AUC-PR	Lift	AUC-ROC	Decisión
LightGBM 7 vueltas vista	0,091	0,117	1,29×	0,564	AUC-PR alto, <i>lift</i> bajo
LightGBM 5 vueltas vista	0,068	0,099	1,44×	0,596	Intermedio
LightGBM 3 vueltas vista (Optuna + Platt)	0,043	0,072	1,67×	0,641	Modelo de producción

5.1.5 Probabilidad de coche de seguridad

La aparición del coche de seguridad se modela como clasificación binaria sobre un evento poco frecuente, con tasas positivas entre el 4,3 % y el 9,1 % de las vueltas según el horizonte considerado. Se emplea de nuevo un LightGBM calibrado con Platt y ajustado mediante una búsqueda Optuna de cien ensayos, esta vez sobre el protocolo de validación temporal por bloques del Sección 2.2; la familia algorítmica se introduce en el Sección 2.3 y la implementación dentro del sistema se describe en el Subsección 4.2.4. Las variantes contrastan una línea base aleatoria con tres horizontes de predicción (dentro de tres, cinco y siete vueltas vista). El criterio de selección no es el AUC-PR absoluto sino el *lift* relativo sobre la tasa base, indicador más informativo cuando la tasa base cambia de un horizonte a otro, y bajo ese criterio el horizonte de tres vueltas se adopta como configuración final.

La Tabla 5.6 deja dos lecturas en paralelo. La primera es cuantitativa: el clasificador supera al azar en términos relativos (AUC-ROC 0,641 y *lift* 1,67× sobre la tasa base), pero su precisión absoluta es modesta: funciona como estimador de riesgo contextual antes que como predictor fino de incidentes concretos, y eleva la probabilidad asignada a las vueltas donde concurren condiciones propensas al incidente. La segunda procede del análisis SHAP, que deja al fondo a las variables precursoras explícitas, como el conteo reciente de mensajes de incidente o el cambio de signo del estado de pista, y privilegia en cambio variables ambientales: la varianza estandarizada de los tiempos por vuelta del pelotón, la edad máxima del neumático en pista, la temperatura del asfalto y la tasa histórica de coche de seguridad del circuito. El modelo no anticipa la causa específica del próximo despliegue: identifica qué combinaciones de ritmo, desgaste y entorno han precedido históricamente a los despliegues observados. La traducción de esta señal contextual en una regla de decisión operativa, junto con su interacción con las salidas del clasificador de adelantamiento, se aborda en el Sección 5.2.

5.1.6 Duración de parada en boxes

La duración de la parada en boxes se modela como problema de regresión cuantílica. El tiempo total se descompone en dos términos: el recorrido del pit lane, prácticamente determinista para cada circuito, y la parada física propiamente dicha, que es la componente que el modelo predice. La configuración de producción agrupa tres regresores HistGradientBoosting (en adelante, HistGBT) independientes ajustados con función de pérdida *quantile* sobre los percentiles 5, 50 y 95; la familia algorítmica corresponde a la misma del Sección 2.3 y la implementación dentro del sistema queda recogida en el Subsección 4.2.5. Las variantes contrastan una línea base de mediana *team*×circuito con dos refinamientos sucesivos. El primero introduce la descomposición entre recorrido del pit lane (*circuit traversal*) y parada física, sin la cual el regresor no superaba a la línea base. El segundo aplica un filtro de paradas normales (componente física en el rango [2,0; 4,5] s) y se adopta como configuración final. Como decisión accesoria, la variable *circuit* se evaluó como rasgo categórico directo y se descartó al detectar que su barajado mejoraba el MAE de prueba, evidencia de sobreajuste a la composición de equipos de boxes de 2023 y 2024 que en 2025 ya no operaba en su totalidad; el efecto residual del trazado se reincorpora mediante una variable binaria de *tight pit box* para los circuitos con calle estrecha.

Tabla 5.7: Variantes evaluadas para la estimación de la duración de parada en boxes (cuaderno N15) sobre 254 paradas del holdout 2025. La descomposición *circuit traversal* más parada física es la decisión central: el modelo entrenado sobre la duración total empata con la línea base, y solo separando la componente física se obtiene la mejora del doce por ciento sobre la mediana *team*×circuito. La banda de incertidumbre nominal del 90 % (cuantiles 5 y 95) cubre empíricamente el 70,5 % de las paradas reales, lectura honesta de que la cola superior del fenómeno (paradas largas por *jack failure* o *unsafe release*) queda infrarrepresentada en el corpus de entrenamiento. La validación externa contra la telemetría de OpenF1 sobre las paradas emparejadas confirma este sesgo, con un error medio de +0,51 s y un percentil 90 del error de aproximadamente +4,40 s.

Variante	MAE (s)	Pinball P05	Pinball P95	Cobertura emp.	Decisión
Baseline mediana <i>team</i> ×circuito	0,555	–	–	–	Referencia
HistGBT cuantílico, banda P10–P90	–	–	–	60,3 %	Banda estrecha, descartada
HistGBT cuantílico, banda P05–P95 (paradas normales)	0,487	0,038	0,110	70,5 %	Modelo de producción

La Tabla 5.7 contiene tres lecturas relevantes. La mejora del doce por ciento sobre la mediana *team*×circuito es modesta en magnitud absoluta pero consistente, y se explica casi en su totalidad por el efecto del propio identificador de equipo, única variable con importancia por permutación claramente positiva. La cobertura empírica del intervalo P05–P95 se queda en el 70,5 % frente al 90 % nominal: la banda subestima sistemáticamente el riesgo de la cola superior, donde las paradas largas anormales (fallos del gato, liberaciones inseguras) están presentes en el holdout pero infrarrepresentadas en el corpus de entrenamiento. El modelo restringe deliberadamente su alcance a paradas dentro del rango [2,0; 4,5] s de componente física. Esa restricción cubre la gran mayoría de los escenarios estratégicamente relevantes y deja fuera de propósito los eventos anómalos: una decisión de alcance que conviene declarar explícitamente para no diluirla en una métrica agregada. La incorporación de la distribución cuantílica completa en las decisiones aguas abajo y la elección del percentil de referencia para los escenarios pesimistas se cubren en el Sección 5.2.

5.1.7 Probabilidad de undercut

El undercut se modela como clasificación binaria sobre pares atacante-defensor: el evento positivo ocurre cuando el atacante para en boxes antes que el defensor, lo hace dentro de las cinco vueltas siguientes y termina la secuencia con posición ganada. El corpus se restringe a parejas con diferencia inicial de hasta tres posiciones, lo que reduce los 7.149 pares brutos a 2.041 candidatos estratégicamente plausibles y deja una tasa de éxito del 34,5 % sobre el holdout 2025 (conjunto de prueba reservado, año no visto durante el entrenamiento). Se emplea de nuevo un LightGBM calibrado con Platt, ajustado esta vez mediante una búsqueda Optuna de ochenta ensayos sobre el protocolo del Sección 2.2, con entrenamiento en 2023 y validación en 2024; la familia algorítmica está descrita en el Sección 2.3 y la implementación dentro del sistema en el Subsección 4.2.5. Las variantes contrastan la tasa base como predictor trivial frente al clasificador ajustado, sin *ablation* explícita (eliminación controlada de un subconjunto de variables para medir su aportación) de los *target encodings* por circuito y equipo; su contribución se infiere a posteriori desde el análisis SHAP del modelo final.

Dos lecturas conviven en la Tabla 5.8. La principal es la mejora sustantiva sobre la tasa base, con un factor próximo a dos sobre el holdout que sostiene al clasificador como discriminador efectivo de las parejas con recorrido real de undercut, ganancia que se mantiene incluso después del descenso esperable entre validación cruzada y prueba final. La segunda se refiere al perfil del modelo: tras la calibración Platt, las puntuaciones medias quedan ligeramente subconfiadas respecto a la tasa real, indicio de que el clasificador es conservador en su reclamo de eventos y privilegia la precisión sobre la cobertura. La elección concreta de la frontera de decisión sobre las probabilidades calibradas, y el equilibrio entre precisión y recall que esa elección implica, se trata en el Sección 5.2.

Tabla 5.8: Variantes evaluadas para la probabilidad de undercut (cuaderno N16). La columna de validación cruzada temporal (entrenamiento 2023, validación 2024) sirve de referencia interna para la búsqueda Optuna; el rendimiento sobre el holdout 2025 cae 0,10 en AUC-PR frente a esa validación (de 0,772 a 0,674), brecha esperable dado el cambio entre temporadas que confirma que la validación cruzada temporal funcionó como *proxy* fiable del rendimiento final. La calibración Platt es monótonica y por construcción no altera las métricas de discriminación; su efecto está en trasladar las puntuaciones crudas, ligeramente sobreconfiadas respecto a la tasa real, a probabilidades coherentes.

Variante	AUC-PR	AUC-ROC	Lift	Decisión
Línea base (tasa éxito 34,5 %)	0,345	0,500	–	Referencia
LightGBM Optuna sobre val. 2024	0,772	–	–	Mejor configuración en CV
LightGBM Optuna + Platt sobre 2025	0,674	0,771	1,95×	Modelo de producción

5.1.8 Transcripción automática de radio

La transcripción automática de la radio piloto-muro se delega a la variante *turbo* de Whisper en modo *zero-shot*, sin fine-tuning sobre el corpus de Fórmula 1; el modelo está descrito en el Subsección 2.5.1 y la implementación dentro del sistema en el Subsección 4.2.6. La elección del modelo *zero-shot* responde a dos condicionantes empíricos. El primero es de orden funcional: la calidad de la transcripción que necesita el sistema es la suficiente para alimentar las cadenas descendentes de sentimiento, intención y entidades, no una transcripción gold con precisión léxica fina. El segundo es de orden práctico: con 530 mensajes etiquetados manualmente, el corpus disponible es órdenes de magnitud menor que el conjunto sobre el que se preentrenó la variante *turbo*, y un fine-tuning supervisado difícilmente aportaría ganancia sobre la cobertura léxica del modelo base. El *benchmark* de latencia sobre cincuenta audios representativos en una RTX 5070 con CUDA 12.8 arroja una mediana de 183 ms y un percentil 95 de 326 ms por audio, intervalo holgadamente compatible con el ritmo natural de las comunicaciones de radio durante una carrera, donde los mensajes llegan distanciados varios segundos. La tasa de error por palabra (*Word Error Rate*, WER) no se mide formalmente: el corpus carece de transcripciones gold pareadas y la elaboración manual de esa referencia léxica excede el alcance de este trabajo, por lo que queda anotada como verificación pendiente.

Sin medición directa del WER, esta capa actúa como cota superior de la calidad alcanzable por los análisis textuales descendentes: cualquier error de transcripción introduce ruido en las cadenas de sentimiento, intención y reconocimiento de entidades, cuyos resultados se reportan en el Subsección 5.1.9 y siguientes, y cuya integración como agente se aborda en el Sección 5.2.

5.1.9 Análisis de sentimiento de radio

El análisis de sentimiento de los mensajes de radio se plantea como clasificación multiclase sobre tres etiquetas: positivo, neutro y negativo. El corpus está formado por los 530 mensajes etiquetados manualmente en el cuaderno N17, con división estratificada 371 / 79 / 80 entre entrenamiento, validación y prueba. Para esta tarea se emplea RoBERTa-base ajustado fino sobre el corpus de radio con función de pérdida ponderada por las frecuencias inversas de cada clase; la arquitectura está descrita en el Subsección 2.5.2 y la implementación dentro del sistema en el Subsección 4.2.6. Se contrastan dos enfoques radicalmente distintos. El primero es una línea base VADER (*Valence Aware Dictionary and sEntiment Reasoner*), clasificador léxico de polaridad sobre inglés general que se aplica sin entrenamiento; el cuaderno N19 documenta que su salida sobre el corpus no se alinea par a par con el etiquetado manual, por lo que la comparación se reduce a contraste de distribuciones agregadas, con una sobreestimación de 34 puntos porcentuales en la clase positiva y una subestimación de 38 en la neutral. El segundo enfoque es el propio RoBERTa-base, que se adopta como modelo de producción tras seis épocas con *early stopping* (detención anticipada por degradación de la pérdida de validación).

Tabla 5.9: Resultados del modelo de producción RoBERTa-base ajustado fino sobre el conjunto de prueba (cuaderno N20, 80 mensajes). VADER no figura en la tabla al admitir solo comparación de distribuciones agregadas, no par a par.

Clase	Precision	Recall	F1	Soporte
Positivo	0,75	0,75	0,75	8
Neutro	0,87	0,93	0,90	57
Negativo	0,73	0,53	0,62	15
Accuracy			0,84	80
Macro avg	0,78	0,74	0,75	80
Weighted avg	0,83	0,84	0,83	80

Tabla 5.10: Resultados del modelo de producción SetFit + ModernBERT sobre el conjunto de prueba (cuaderno N21, 80 mensajes). El cuaderno documenta como resultado negativo previo un ajuste fino de DeBERTa-v3-large que colapsó a la clase mayoritaria sobre el mismo conjunto de prueba (accuracy 0,40, macro F1 0,11, todas las predicciones iguales a INFORMATION), motivo por el que la arquitectura se cambió al enfoque *few-shot* contrastivo. La clase WARNING colapsa también en el modelo de producción (F1 0,00 sobre cinco ejemplos), límite intrínseco de la *back-translation* cuando la clase de partida tiene poca variabilidad léxica; la clase QUESTION, en cambio, alcanza F1 0,88 sobre ocho ejemplos gracias a los marcadores interrogativos característicos.

Clase	Precision	Recall	F1	Soporte
INFORMATION	0,58	0,69	0,63	32
PROBLEM	0,67	0,59	0,63	17
ORDER	0,53	0,56	0,54	18
WARNING	0,00	0,00	0,00	5
QUESTION	0,88	0,88	0,88	8
Accuracy			0,61	80
Macro avg	0,53	0,54	0,53	80
Weighted avg	0,58	0,61	0,59	80

Dos hechos relevantes se desprenden de la Tabla 5.9. La macro F1 de 0,75 está dominada por la clase neutral, cuyo recall del 93 % y soporte de 57 mensajes sobre 80 capturan más del setenta por ciento del problema; las clases extremas, con apenas 8 ejemplos positivos y 15 negativos, tienen recall del 75 % y del 53 % respectivamente, y arrastran la macro F1 a la baja. Además, el encabezado del cuaderno N20 anuncia un 87,5 % de accuracy mientras que el `classification_report` ejecutado en la misma celda devuelve 0,84; se reporta esta segunda cifra como autoritativa por ser la única reproducible al re-ejecutar el cuaderno. La extensión del corpus de etiquetado para reducir el desbalance queda como línea de mejora pendiente, y el efecto agregado de esta cota de calidad sobre las cadenas descendentes se aborda en el Sección 5.2.

5.1.10 Clasificación de intención de radio

La clasificación de intención del mensaje de radio es un problema multiclase de cinco etiquetas: INFORMATION, PROBLEM, ORDER, WARNING y QUESTION. Se aborda sobre los 529 mensajes etiquetados manualmente del cuaderno N17, con partición estratificada 370 / 79 / 80 entre entrenamiento, validación y prueba. Para esta tarea se adopta SetFit con codificador ModernBERT-base y cabezal de regresión logística; el paradigma está descrito en el Subsección 2.5.3 y la implementación dentro del sistema en el Subsección 4.2.6. Se contrastan dos enfoques radicalmente distintos. El primero es un ajuste fino clásico de DeBERTa-v3-large directamente sobre el corpus original, evaluado en el cuaderno N21 como referencia y descartado al observar el colapso de las predicciones a la clase mayoritaria. El segundo es el propio SetFit sobre ModernBERT, entrenado de manera contrastiva y reforzado con un aumento por *back-translation* que duplica el corpus de entrenamiento a 740 muestras (370 originales más 370 sintéticas generadas por traducción inglés-francés-inglés) y lleva todas las clases minoritarias a 148 ejemplos cada una; este segundo enfoque es el que se adopta como modelo de producción.

Tabla 5.11: Variantes evaluadas para el reconocimiento de entidades de radio (cuaderno N22) sobre el conjunto de prueba de 40 mensajes. Las cifras de precisión y recall son ponderadas a nivel de token según el esquema BIO. Las celdas – corresponden a métricas no reportadas en el cuaderno.

Variante	Precision	Recall	F1	Decisión
NuNER <i>zero-span</i> (<i>zero-shot</i>)	0,000	0,000	0,000	Descartada
GLiNER large-v2.1 (<i>zero-shot</i>)	0,029	0,011	0,016	Descartada
NuNER ajustada al dominio F1	–	–	0,068	Descartada
BERT-BIO del <i>legacy</i> (Vega Sobral, 2025)	–	–	0,430	Referencia
BERT-large CoNLL-03 BIO	0,421	0,426	0,415	Modelo de producción

El perfil multiclase de la Tabla 5.10 es muy desigual. La clase QUESTION alcanza una F1 de 0,88 sobre apenas ocho ejemplos, indicio de que los patrones léxicos interrogativos de la radio son lo bastante característicos como para que el modelo los identifique incluso con pocas muestras. En el extremo opuesto, la clase WARNING colapsa por completo (F1 0,00 sobre cinco ejemplos): el aumento por *back-translation* pasa su tamaño de entrenamiento de 23 a 148 muestras, pero el procedimiento traduce y retraduce variantes léxicas sobre los mismos patrones de fondo y no introduce diversidad semántica real, limitación intrínseca de la técnica cuando la clase de partida tiene baja variabilidad.

Las tres clases intermedias (INFORMATION, PROBLEM y ORDER) se confunden con frecuencia entre sí: comparten estructura imperativa y referencia a entidades del dominio, y el modelo separa peor las fronteras que cuando hay un marcador léxico tan claro como el signo de interrogación.

El descarte previo de DeBERTa-v3-large refuerza la lectura general: con 370 ejemplos de entrenamiento, un modelo con 435 millones de parámetros sobreparametriza el problema y converge a la trivialidad de predecir siempre la clase mayoritaria, mientras que SetFit, al apoyar la clasificación sobre un espacio de embeddings preentrenado y entrenar solo un cabezal de regresión logística, alcanza un rendimiento operativamente suficiente para el sistema (accuracy 0,61, macro F1 0,53). La integración de este componente en la cadena descendente y el tratamiento operativo de la clase WARNING como señal de baja confianza se abordan en el Sección 5.2.

5.1.11 Reconocimiento de entidades de radio

El reconocimiento de entidades sobre la radio es un problema de etiquetado de tokens con esquema BIO sobre nueve categorías semánticas del dominio (situación, acción, instrucción estratégica, problema técnico, meteorología, cambio de posición, incidente, condición de pista y llamada a boxes), aplicado a los 399 mensajes que contienen al menos un *span* dentro del corpus N17 y partidos en 319 mensajes de entrenamiento, 40 de validación y 40 de prueba. Para esta tarea se adopta un BERT-large preentrenado sobre CoNLL-03 y ajustado fino sobre el corpus F1; el paradigma está descrito en el Subsección 2.5.4 y la implementación dentro del sistema en el Subsección 4.2.6. Se contrastan dos enfoques estructuralmente distintos. El primero extrae *spans* guiados por etiquetas, representado por GLiNER en régimen *zero-shot* y por su ajuste fino sobre el corpus F1, y por NuNER en sus variantes *zero-shot* y ajustada; ambos modelos se entrenaron para identificar nombres propios en corpus generalistas y no consiguen proyectar su espacio de representación sobre etiquetas semánticas clausales que solapan entre sí. El segundo, finalmente adoptado, clasifica token a token con esquema BIO sobre un BERT-large preentrenado en CoNLL-03 (Subsección 2.5.4). Esa semejanza estructural con la tarea reduce el problema a un ajuste de cabezal, y el rendimiento alcanzado resulta equivalente al del modelo BIO previo del legacy del autor (Vega Sobral, 2025).

La Tabla 5.11 admite dos lecturas en paralelo. En términos absolutos, una F1 ponderada de 0,415 sobre el conjunto de prueba es baja: los modelos NER de referencia en CoNLL-03, evaluados sobre prensa inglesa con entidades nominales cortas, alcanzan habitualmente F1 por encima de 0,90. Esa distancia, sin embargo, refleja el techo intrínseco del problema tal y como aquí se ha planteado: los *spans* del corpus F1 son cláusulas semánticas de varias palabras que solapan entre sí (*strategy instruction* puede incluir literalmente la *action* y la *position change* que describen la misma orden táctica), y el esquema BIO penaliza cualquier desalineamiento parcial de frontera como error de igual peso que una clasificación errónea de tipo, lo que deprime la métrica frente a la utilidad operativa real del modelo. En la cadena descendente, la inferencia no exige fronteras exactas: tolera que un mismo *span* se etiquete con la categoría correcta aunque su rango difiera del oro en uno o dos tokens, y convierte así una fracción significativa de los falsos positivos a nivel de token en aciertos a nivel de mensaje, comportamiento que se observa cualitativamente al inspeccionar las salidas de la canalización completa. El empate con el modelo equivalente del legacy del autor (Vega Sobral, 2025) confirma además que la limitación es de dataset y no de arquitectura: la exploración de modelos más pequeños tipo *bert-base-NER* o de esquemas de etiquetado a nivel de *span* en lugar de token queda como línea de mejora futura. La traducción de estas entidades en disparadores de las cadenas estratégica y reglamentaria se aborda en el Sección 5.2.

5.1.12 Procesamiento de mensajes de dirección de carrera

El procesamiento de los mensajes de dirección de carrera prescinde por completo del aprendizaje automático: se aplica un analizador determinista basado en expresiones regulares sobre las tripletas (Categoría, Bandera, Mensaje) emitidas por la FIA, cuya formulación y justificación de diseño se desarrollan en el Subsección 2.5.5 y cuya implementación dentro del sistema se describe en el Subsección 4.2.6. La razón de la elección, repasada brevemente, es que el vocabulario de los *Race Control Messages* está cerrado por el reglamento deportivo y sus plantillas léxicas no admiten variación sustantiva, condición en la que un conjunto de reglas explícitas supera en robustez, trazabilidad y mantenibilidad a cualquier modelo estadístico equivalente. Sobre 609 mensajes extraídos de seis grandes premios de las temporadas 2024 y 2025, el analizador alcanza cobertura del 100 % en las categorías críticas para la estrategia (banderas, despliegues de coche de seguridad y transiciones del DRS) y del 93 % en la categoría heterogénea *Other*, donde caen las investigaciones, sanciones y eventos atípicos; el residuo no clasificado representa el 3,1 % del corpus de validación, equivalente a 19 mensajes.

Dos limitaciones acompañan a esa cifra. La primera afecta a la métrica reportada: es cobertura, no precisión; el analizador asigna a cada mensaje uno de 23 tipos de evento (incluida la categoría residual *OTHER*), pero el cuaderno N23 no compara las asignaciones contra una anotación gold par a par, por lo que las decisiones individuales no se verifican una a una y el determinismo del propio analizador opera como garantía implícita de corrección. La segunda es que el residuo del 7 % de la categoría *Other* y el 3,1 % global de la categoría residual constituyen el conjunto adecuado para entrenar un clasificador supervisado en trabajos futuros que afine la granularidad de las investigaciones e infracciones sin sacrificar la robustez del enfoque determinista en las categorías críticas. La latencia del analizador, medida sobre ochenta repeticiones por dispositivo, queda en torno a los dos microsegundos en GPU y los cuatro en CPU, y resulta despreciable frente al resto de la cadena de inferencia descrita en el Sección 5.2.

5.1.13 Recuperación reglamentaria

La recuperación reglamentaria descansa sobre un índice vectorial construido a partir de las tres ediciones del reglamento deportivo de la FIA (temporadas 2023, 2024 y 2025). Los documentos se fragmentan en ventanas solapadas de 512 caracteres con 64 de solapamiento, se codifican con BGE-M3 a un espacio

Tabla 5.12: Variantes evaluadas para la recuperación reglamentaria (cuaderno N30B) sobre el conjunto de quince consultas con verdad de referencia. *Precision@5* exige acierto simultáneo de artículo y palabra clave, mientras que *Content P@5* relaja la restricción al solo contenido textual.

Configuración	P@1	P@5	Content P@5	MRR	P95 (ms)	Decisión
MiniLM-L6-v2 chunk 512	0,200	0,200	0,800	0,232	48,2	Alternativa ligera
BGE-M3 chunk 256	0,067	0,133	0,800	0,108	94,3	Descartada
BGE-M3 chunk 512	0,200	0,200	0,800	0,235	243,7	Modelo de producción

denso de 1024 dimensiones y se almacenan en una colección local de Qdrant. El paradigma se describe en el Sección 2.6 y su integración dentro del sistema en el Subsección 4.2.7. La evaluación cuantitativa se realiza en el cuaderno N30B sobre un conjunto de quince consultas redactadas manualmente en inglés, el idioma original del reglamento, y distribuidas en cinco categorías reglamentarias (DRS, banderas y sanciones, paradas en boxes, coche de seguridad y asignación de neumáticos). Cada consulta lleva asociada una verdad de referencia formada por el artículo correcto y una lista de palabras clave verificadas literalmente contra el PDF original. Se contrastan tres configuraciones. La primera, adoptada como modelo de producción, combina el codificador BGE-M3 con ventanas de 512 caracteres. La segunda sustituye BGE-M3 por *all-MiniLM-L6-v2*, codificador entrenado exclusivamente en inglés con 384 dimensiones y alrededor de seis veces menos parámetros, evaluado como alternativa ligera apta para despliegues sin GPU. La tercera conserva BGE-M3 pero reduce el tamaño de fragmento a 256 caracteres, para medir si una granularidad más fina mejora la recuperación al precio de partir artículos extensos entre dos fragmentos. Sobre cada configuración se reportan dos métricas de precisión. La estricta (*Precision@k*) exige acierto simultáneo de artículo y palabra clave. La de contenido (*Content P@5*) relaja la restricción al solo contenido textual y aísla así la calidad del recuperador del ruido introducido por el etiquetado de artículos durante la ingesta. El conjunto de evaluación, el cuaderno N30B y el fichero *queries_v1.json* con la verdad de referencia se mantienen versionados en el repositorio del sistema para que cualquier reejecución sea reproducible.

La Tabla 5.12 admite tres lecturas en paralelo. La primera es la brecha sistemática entre la columna *Precision@5* estricta y la columna *Content P@5*. Las tres configuraciones recuperan el fragmento correcto en el ochenta por ciento de las consultas, pero solo aciertan el campo *article* del payload en el veinte por ciento. El desfase no procede del codificador semántico sino de la expresión regular del indexador, que asigna a cada fragmento la primera referencia *Article X.Y* que encuentra dentro del texto. Cuando esa referencia es una cita cruzada al artículo de otro capítulo, el campo de metadato queda mal etiquetado aunque el contenido sea relevante. Mitigar este efecto exige post-procesar el payload con el encabezado de sección más cercano al fragmento, refinamiento que queda registrado como mejora futura. La segunda lectura es la comparación entre los dos codificadores. BGE-M3 y MiniLM-L6-v2 empatan en precisión y solo se distinguen por tres milésimas de MRR a favor de BGE-M3. En cambio, la diferencia de latencia es de cinco veces a favor de MiniLM (48,2 ms frente a 243,7 ms en el percentil 95). La decisión de mantener BGE-M3 como modelo de producción responde a dos argumentos que no son estrictamente métricos. Por un lado, la capacidad multilingüe del codificador deja la puerta abierta a indexar reglamentos traducidos o consultas en castellano sin reentrenar. Por otro, el coste de latencia queda subordinado a la llamada al modelo de lenguaje aguas abajo (varios cientos de milisegundos con un modelo local), por lo que la etapa de recuperación no se convierte en el cuello de botella del agente. La tercera lectura es el efecto del tamaño de fragmento. Reducir las ventanas de 512 a 256 caracteres con el mismo codificador hunde *Precision@1* de 0,200 a 0,067 y la MRR de 0,235 a 0,108 sin ganancia perceptible en *Content P@5*. Una granularidad más fina parte artículos largos (el 30.2 del reglamento de neumáticos se reparte entre dos fragmentos en este régimen) y deja la cláusula numérica fuera del top-k. Los casos de fallo típicos observados, documentados en el propio cuaderno, son consultas que mezclan DRS y coche de seguridad, donde compiten los fragmentos

Tabla 5.13: Latencia por sub-agente sobre la vuelta sintética NOR-McLaren Melbourne 2025 vuelta 20, en una RTX 5070 con CUDA 12.8 y los modelos previamente cargados en memoria. Cada fila resume diez repeticiones del sub-agente invocado desde su *entry point* `run_*_from_state`. La columna LLM indica si la salida del sub-agente cierra con una llamada de síntesis al modelo de lenguaje servido por LM Studio; los sub-agentes que delegan en el modelo de lenguaje contraen además la salida sobre el *schema* Pydantic correspondiente. La unidad es el segundo y las cifras se redondean a milisegundo.

Sub-agente	P50 (s)	P95 (s)	LLM	Modelos locales
Pace	0,481	0,588	No	XGBoost lap-time delta
Tire	2,736	3,237	No	TCN + MC Dropout
Race Situation	2,845	4,140	Opcional	LightGBM adelantamiento + LightGBM coche de seguridad
Pit Strategy	3,538	4,436	Sí	HistGBT cuantílico pit + LightGBM undercut
Radio	1,213	1,890	Sí	BERT sentimiento + SetFit intención + BERT NER
RAG	2,354	13,743	Sí	BGE-M3 + Qdrant

del artículo 55 y del artículo 56, y consultas sin filtro temporal explícito, donde el recuperador puede devolver un fragmento de la temporada equivocada cuando la cláusula numérica varía entre ediciones del reglamento. El comportamiento del componente integrado dentro del agente RAG completo, con el modelo de lenguaje que sintetiza la respuesta sobre los fragmentos recuperados, se aborda en el Sección 5.2.

5.2 Resultados por sub-agente

La medición de los sub-agentes desplaza el foco respecto a la Sección 5.1. En la sección anterior se reporta la métrica cruda del modelo sobre su conjunto de prueba; aquí se mide el sub-agente ya envuelto en su prompt de sistema, su *schema* de salida en Pydantic y, cuando procede, la llamada de síntesis al modelo de lenguaje servido por LM Studio. La arquitectura del orquestador y de los seis sub-agentes está descrita en el Subsección 4.2.8 y la implementación de cada uno se detalla en el Sección 4.2. El criterio cambia: no cuánta precisión alcanza el modelo, sino cuánto tarda el conjunto en devolver una salida que cumpla el contrato del orquestador y qué fracción de ese tiempo se invierte en inferencia local frente a la llamada al *endpoint* de síntesis. La medida se ejecuta sobre una vuelta sintética fija (NOR-McLaren en Melbourne 2025, vuelta 20), repetida diez veces por sub-agente en una RTX 5070 con CUDA 12.8 y los modelos cargados previamente en memoria, configuración que aproxima el régimen *warm* que verá el orquestador durante una carrera real. La canalización de procesamiento del lenguaje natural se reporta además sobre los dos dispositivos (CPU y GPU) en ochenta repeticiones por modalidad, requisito para verificar el RNF-01 de respuesta por vuelta declarado en el Capítulo 3.

La Tabla 5.13 admite tres lecturas en paralelo. La primera es la dispersión por orden de magnitud entre los seis sub-agentes. El sub-agente *Pace*, único que no delega ninguna fase al modelo de lenguaje y resuelve la predicción con una sola pasada de XGBoost, queda por debajo del segundo y marca la cota inferior del sistema. La banda intermedia, integrada por *Tire*, *Race Situation* y *Pit Strategy*, oscila entre 2,7 y 4,4 s en el percentil 95 por la combinación de modelos locales pesados (la TCN de degradación con cincuenta pasadas de MC Dropout, los dos LightGBM de adelantamiento y coche de seguridad, el HistGBT cuantílico y el LightGBM de undercut) y, en los sub-agentes que cierran con síntesis, la llamada al modelo de lenguaje. El sub-agente *Radio* encaja en la franja baja de esa banda porque las cadenas de sentimiento, intención y entidades operan sobre mensajes cortos cuya inferencia agregada no supera la decena de milisegundos en GPU, y el coste dominante proviene también de la síntesis. El sub-agente *RAG* presenta una mediana inferior a la del sub-agente *Pit Strategy* (2,4 s frente a 3,5 s) pero el percentil 95 más alto del sistema, y triplica al del siguiente sub-agente.

La segunda lectura es la cola pesada del sub-agente RAG. La mediana (2,4 s) y el percentil 95 (13,7 s) están separados por casi un orden de magnitud, asimetría que la media de 4,4 s, situada por encima de la mediana, confirma como cola por la derecha. La causa está en la fase de síntesis: la cláusula recuperada varía mucho en longitud entre artículos del reglamento y el modelo de lenguaje servido por LM Studio escala el tiempo de respuesta con el número de *tokens* de salida. Esta cola justifica el diseño del orquestador descrito en el Subsección 4.2.8: el sub-agente RAG no se invoca vuelta a vuelta sino bajo activación condicional (umbral 0,30 sobre `sc_prob_3lap` o referencia explícita al reglamento en la entrada), de modo que el coste se imputa solo cuando aporta valor estratégico y no penaliza el ciclo nominal del orquestador.

La tercera lectura es la canalización de procesamiento del lenguaje natural en sus dos dispositivos. Sobre las ochenta repeticiones reportadas, `run_pipeline` pasa de un percentil 95 de 44,2 ms en GPU a 364,1 ms en CPU, un factor de ocho que verifica el RNF-01 incluso en el régimen degradado: la vuelta media de Fórmula 1 dura entre setenta y ciento diez segundos en seco, y el ciclo de inferencia del orquestador completo, dominado por los sub-agentes pesados de la banda intermedia, queda más de un orden de magnitud por debajo de esa cota. El analizador determinista de *Race Control Messages* opera en microsegundos sobre ambos dispositivos y resulta despreciable en cualquier combinación. La compatibilidad con CPU permite desplegar el sistema en entornos sin GPU dedicada, ajustando los sub-agentes pesados a una cadencia más laxa o desactivando los condicionales del enrutador *rule-based* del orquestador, sin que el ciclo principal de recomendación deje de cumplir el requisito de respuesta por vuelta.

El cumplimiento del contrato Pydantic se verifica sobre los cuadernos de validación de extremo a extremo N32 y N34, discutidos en el Sección 5.3: sobre los 28 mensajes de radio y los 76 RCMs reales del GP de Baréin 2025, los catorce campos de la `StrategyRecommendation` se rellenan sin error en el 100 % de las invocaciones, y el orquestador ejecutado sobre Melbourne 2025 a tres vueltas en modo `-no-llm` produce las tres recomendaciones esperadas con el esquema íntegro.

5.3 Validación end-to-end del orquestador

La validación de extremo a extremo verifica que los seis sub-agentes encajen dentro del orquestador y produzcan una `StrategyRecommendation` consistente con su contrato de salida sobre datos reales de la temporada 2025. La pregunta es ortogonal a la de la sección anterior: aquí no se mide cuánto tarda cada sub-agente, sino si el conjunto, ensamblado tal como lo invoca la interfaz de simulación, llega hasta una recomendación válida sin desviarse del esquema Pydantic. La validación se articula en dos capas complementarias. La primera es la batería de pruebas automatizadas (`tests/test_smoke.py` y `tests/test_agents.py`) que verifica la estructura del repositorio, la importabilidad de los siete módulos del paquete `src/agents/` (los seis sub-agentes y el orquestador), los catorce campos del `schema StrategyRecommendation`, las cuatro `dataclasses` de salida y los tres `schemas` REST que enlazan los agentes con la interfaz Streamlit. La segunda es la batería de cuadernos de simulación (N32 y N34) que ejecuta el sistema completo sobre dos grandes premios reales de 2025, Melbourne y Baréin, y comprueba que el flujo desde `lap_state` hasta la recomendación se completa sin error de ejecución ni desviación contractual.

La Tabla 5.14 se lee en tres niveles. El primero es el contractual. El sistema impone tres capas de tipos articuladas en cascada: las `dataclasses` individuales (`PaceOutput`, `TireOutput`, `RaceSituationOutput` y `PitStrategyOutput`) que transportan la salida de cada sub-agente, el `schema StrategyRecommendation` de catorce campos con el que el orquestador sintetiza la decisión final, y los tres `schemas` REST que enlazan los sub-agentes con la interfaz Streamlit. Las tres capas se validan por separado en `test_agents.py`, lo que aísla las regresiones de tipos en su capa correspondiente y evita que un cambio de `schema` en un sub-agente se propague silenciosamente al orquestador.

Tabla 5.14: Verificación de extremo a extremo del orquestador estratégico sobre datos reales de la temporada 2025. Cada fila resume un *checkpoint* ejecutado por la prueba automatizada o el cuaderno indicado entre paréntesis en la columna Alcance. Las cifras de la columna Resultado son las contadas en la celda correspondiente; todos los *checkpoints* se ejecutan con los modelos previamente cargados en memoria.

Checkpoint	Alcance	Resultado
Estructura del repositorio	src/, notebooks/agents/ y pyproject.toml (test_smoke.py)	3 aserciones verdes
Schema	14 campos Pydantic productivos (test_agents.py)	Verde
StrategyRecommendation		
Dataclasses + schemas REST	PaceOutput, TireOutput, RaceSituationOutput, PitStrategyOutput + PaceRequest, TireRequest, RadioRequest (test_agents.py)	7 verdes
RaceStateManager	NOR vuelta 20: total_laps=57, posición 1, rivales no vacíos (test_smoke.py)	Verde
Melbourne 2025		
Sub-agentes individuales + CLI Melbourne	7 sub-agentes desde debug_agent.py en la vuelta 20 + run_simulation_cli.py a 3 vueltas en modo -no-llm (N32)	7 exit 0 + 3 recomendaciones
Corpus de radio Baréin 2025	28 archivos MP3 reales → Whisper turbo + caché atómica (N34)	28/28; caché 0,0 s
Corpus de RCMs Baréin 2025	76 mensajes FIA → analizador determinista (N34)	76/76
Round-trip N29 Baréin vuelta 4	RadioOutput con alert=PROBLEM sobre HAM y dos eventos de RCM contemporáneos (N34)	Esquema válido

El segundo nivel es el de la ejecución sobre datos reales. El cuaderno N32 ejecuta cada sub-agente y el CLI completo sobre Melbourne 2025 (vuelta 20 y vueltas 1-3 en modo -no-llm) y verifica que el flujo desde el *lap_state* hasta la *StrategyRecommendation* termina con código de salida cero y respeta el esquema Pydantic. El cuaderno N34 somete el subsistema de radio a un caso de carga superior: 28 mensajes MP3 reales del muro de boxes durante el GP de Baréin 2025 y 76 RCMs emitidos por dirección de carrera, procesados con caché atómica y *round-trip* completo al sub-agente Radio en la vuelta 4 (*RadioOutput* con alerta *PROBLEM* sobre el radio de HAM y dos eventos de RCM contemporáneos). La caché reduce la inicialización de la canalización desde unos treinta segundos en la primera lectura a 0,0 segundos en la segunda, condición indispensable para que un usuario interactivo no espere medio minuto al cargar otra carrera.

El tercer nivel es el de las cotas que esta validación no cubre. Los cuadernos N32 y N34 verifican corrección estructural pero no recorren la carrera completa con instrumentación de latencia *warm* vuelta a vuelta, ni miden la tasa de *fallback* de las salvaguardas del sub-agente Pit Strategy (que residen en el *system prompt* del modelo de lenguaje, no en código duro), ni reportan el coste de las llamadas al modelo de lenguaje servido por LM Studio frente a una alternativa comercial. Ese registro cuantitativo está fuera del alcance de las pruebas *smoke* y se traslada a los casos de uso del Sección 5.5, donde se ejecutan carreras reales completas de la temporada 2025 sobre las tres superficies del sistema (Streamlit, CLI y Arcade) y se documenta la salida del orquestador frente a la decisión observada del muro de boxes. La verificación reportada aquí sustenta, en cambio, una afirmación más modesta y suficiente: el sistema se ensambla, ejecuta y devuelve recomendaciones conformes al esquema sobre dos grandes premios reales de la temporada 2025.

5.4 Comparación con la literatura previa

La comparación con la literatura previa de aplicaciones de aprendizaje automático a la Fórmula 1 enfrenta un problema de inconmensurabilidad. A fecha de redacción de este trabajo no existe un *benchmark* público que defina conjuntos de prueba, métricas estándar y baselines aceptados por la comunidad para tareas estratégicas en este deporte, condición que obliga a cada grupo de investigación a definir sus propios

Tabla 5.15: Comparación del sistema actual con los siete trabajos del campo más cercanos al problema planteado. La columna Eje diferencial sintetiza la decisión metodológica que separa el sistema actual del trabajo previo correspondiente, sin afirmar superioridad cuantitativa: las publicaciones citadas evalúan tareas distintas sobre escenarios distintos y no existe un *benchmark* público común que permita comparación par a par.

Trabajo	Objetivo	Técnica	Eje diferencial
Heilmeier et al. (2020a)	Simulación probabilística de carrera completa	Monte Carlo sobre estrategias fijas	El sistema actual emplea Monte Carlo como capa intermedia (500 trayectorias sobre ventana de 5 vueltas) dentro de un orquestador multi-agente
Heilmeier et al. (2020b)	Decisión binaria pit / no pit	Red neuronal supervisada	Aquí cinco acciones discretas más catorce campos contractuales más <i>reasoning</i> en lenguaje natural
Sasikumar et al. (2025)	Decisión de parada en boxes	Aprendizaje profundo	El sub-agente Pit Strategy combina cuantiles HistGBT, clasificador undercut y salvaguardas en prompt; cobertura ampliada al resto de decisiones estratégicas
Machine Learning at Cambridge Laboratory (2019)	Modelado general de carreras de Fórmula 1	Método no especificado en la fuente pública (colaboración confidencial con equipo F1)	Aquí descomposición explícita en seis sub-agentes con contrato Pydantic y validación de extremo a extremo
Urdhwareshe (2025)	Predicción de posición final ($R^2 = 0,75$)	TabNet sobre temporadas 2010-2023	Aquí predicción operacional vuelta a vuelta sobre la temporada 2025 en lugar de <i>outcome</i> agregado de temporada
Bansal et al. (2025)	Predicción de puntos del constructor	XGBoost sobre 1950-2024	Aquí XGBoost de delta de tiempo de vuelta sobre 2023-2024 (entrenamiento) y 2025 (prueba), objetivo operacional no agregado
Gomathi L. (2025)	Agrupación no supervisada de circuitos	K-Means sobre telemetría pública	Aquí K-Means $K = 4$ con silueta 0,201 e integración como <i>feature</i> categórica en los sub-agentes Pace y Tire (Subsección 5.1.2)

escenarios de evaluación y a reportar cifras que no son directamente comparables entre trabajos. Las publicaciones revisadas resuelven además problemas distintos al planteado aquí: la mayoría predice el resultado agregado de la carrera o la temporada (posición final, puntos del constructor) y no la decisión estratégica vuelta a vuelta. La Tabla 5.15 sitúa el sistema actual frente a los siete trabajos del campo más cercanos al problema planteado y deja explícitos los ejes de diferenciación: el alcance del problema, la técnica empleada y el horizonte de operación.

Tres ejes de diferenciación se observan en la Tabla 5.15. El primero es el alcance: la mayoría de los trabajos revisados aborda una tarea aislada (decisión de pit stop, posición final, puntos del constructor o agrupación de circuitos), mientras que aquí se ensamblan seis sub-agentes especializados bajo un orquestador que devuelve una recomendación operacional con catorce campos contractuales. El segundo es el horizonte temporal: los trabajos que predicen el resultado agregado de temporada usan ventanas históricas largas (1950-2024 (Bansal et al., 2025), 2010-2023 (Urdhwareshe, 2025)) y no son operables en tiempo real, mientras que el sistema actual opera vuelta a vuelta sobre la temporada vigente y reconoce la deriva conceptual descrita en la Sección 2.2. El tercero es la incorporación de modalidades textuales: ninguno de los trabajos revisados combina decisión estratégica con análisis de la radio piloto-muro y recuperación reglamentaria, novedad del sistema actual que descansa en la canalización de procesamiento del lenguaje natural reportada de la Subsección 5.1.8 a la Subsección 5.1.12 y en el agente RAG de la Subsección 5.1.13.

5.5 Casos de uso

Las tres subsecciones que cierran este capítulo ejercitan el sistema sobre carreras reales del calendario 2025 desde las tres superficies descritas en el Subsección 4.2.9. El reparto de carreras entre superficies no es accidental. La Subsección 5.5.1 elige una carrera distinta a las usadas en los otros dos casos para ilustrar el *workflow* propio de Streamlit, que combina chat conversacional sobre el sistema con *tool calling* contra el servidor FastMCP y generación posterior de un informe estructurado de la sesión. El panel descrito en el Subsección 4.2.9 ofrece bastantes pestañas y vistas adicionales, aunque esta subsección se restringe a la parte directamente ligada al escenario de carrera. La Subsección 5.5.2 reproduce desde la CLI un tramo corto del GP de Hungría 2025 (las cinco vueltas en torno a la parada de Leclerc en V20, donde Ferrari cubrió el undercut de PIASTRI con compuesto duro en lugar del medio que la pista admitía). El rango breve permite mostrar el ciclo completo de recomendación sin saturar el capítulo. La Subsección 5.5.3 cierra la serie sobre el replay de Arcade con el error documentado de McLaren en el GP de Catar, reproducido sobre la decisión real registrada por la prensa especializada y catalogado por ella como el peor error de estrategia de la temporada.

5.5.1 Análisis post-carrera con Streamlit

La superficie Streamlit (`f1-streamlit`) se ejercita sobre el GP de Australia 2025 como muestra representativa del flujo conversacional, carrera deliberadamente distinta a las usadas en los otros dos casos de uso para evitar duplicar contexto estratégico. El usuario arranca el panel desde el terminal con `f1-streamlit`, abre la página de exploración vuelta a vuelta y lanza el chat conversacional sobre la sesión cargada. Una primera consulta del tipo “*compara los tiempos por vuelta de PIASTRI y NORRIS en Australia 2025 entre las vueltas 13 y 25*” dirige al modelo de lenguaje a la herramienta `get_lap_times` publicada por el servidor FastMCP descrito en el Subsección 4.2.9, que devuelve la serie de tiempos retornada por el *endpoint* de telemetría sobre la ventana solicitada. El modelo de lenguaje la resume en una respuesta sintética acompañada de los valores concretos, identificando una tendencia a la mejora del orden de un segundo y siete décimas en el tramo para NORRIS, recogida en la Figura 5.1.

El comportamiento más interesante del ciclo aparece cuando los datos retornados por la herramienta son parciales. En el caso ilustrado, la salida solo contiene los registros de NORRIS y el modelo de lenguaje lo verbaliza explícitamente como limitación de la respuesta antes de ofrecer una comparación reducida y dos rutas alternativas (intentar la comparación vuelta a vuelta sobre el tramo donde sí existe registro de PIASTRI, o ampliar el análisis con un compuesto distinto), comportamiento alineado con el *system prompt* del *chat_engine* que prohíbe fabricar valores cuando el dato no está presente. Una segunda consulta sobre la degradación de neumático del piloto seguido dirige al modelo a la herramienta `predict_tire` discutida en el Subsección 5.1.3, que devuelve las bandas P10/P50/P90 de `laps_to_cliff` y el etiquetado de aviso correspondiente sobre la vuelta consultada. El chat ejecuta una herramienta por turno y el modelo de lenguaje decide cuál a partir de los esquemas JSON tirados en vivo de FastMCP, lo que garantiza que el chat interno del frontend y los clientes MCP externos como Claude Desktop o Cursor consuman exactamente el mismo conjunto de herramientas sin lógica duplicada.

La interacción se sirve sobre eventos SSE (*Server-Sent Events*) emitidos por el *endpoint* `/chat/tool-message-stream`, que el chat consume en cuatro fases sucesivas, `stage` para indicar la etapa backend en curso, `tool_result` cuando llega la salida estructurada de la herramienta, `token` para el resumen incremental del modelo de lenguaje y `done` como marcador final con la metadata del proveedor. El frontend acompaña este flujo con un indicador de progreso vivo que refleja la etapa real del backend (consultando periódicamente al *endpoint* auxiliar `/chat/status`), renderizado Markdown del texto generado



Figura 5.1: Chat conversacional del panel Streamlit ejecutando la herramienta `get_lap_times` del servidor FastMCP sobre el GP de Australia 2025 (Piastri y Norris, vueltas 13 a 25). Panel superior con el turno usuario-asistente, banda inferior con la gráfica de tiempos por vuelta retornada por la herramienta y rango analizado marcado entre las dos discontinuas. Barra lateral izquierda con historial de conversaciones y acceso a *Reports*.

y un botón de *Stop* que interrumpe la generación, lo que mantiene perceptible la latencia de los modelos de mayor coste sin bloquear la interfaz.

El panel del chat dispone además de un botón *Download Report* en su cabecera que sintetiza la sesión completa en un informe estructurado, recogiendo las cifras retornadas por las herramientas consultadas durante la conversación junto a la valoración cualitativa del modelo de lenguaje. El resultado se persiste como archivo Markdown descargable y reutilizable fuera de la sesión interactiva. La Figura 5.2 reproduce un fragmento del informe generado tras la conversación demostrativa, donde el modelo estructura las conclusiones por piloto y reconoce explícitamente la limitación de la comparativa parcial, comportamiento operativamente útil para un *post-mortem* estratégico que necesita distinguir entre afirmaciones fundadas y especulaciones.

El panel admite dos extensiones operativas descritas en el Subsección 4.2.9. La primera permite sustituir el proveedor de modelo de lenguaje por una instancia local servida con LM Studio mediante la variable de entorno `F1_LLM_PROVIDER`, modo aprovechable sin conexión al exterior siempre que el modelo descargado soporte *tool calling* en el formato estándar de la API de OpenAI, que es el protocolo común esperado por el chat con independencia del proveedor. La segunda es el modo de voz, conmutable desde la barra lateral del chat, que sustituye la entrada por teclado por una transcripción con Whisper sobre el audio del micrófono y la salida en pantalla por una respuesta sintetizada con Edge-TTS, lo que permite formular las mismas consultas estratégicas en formato conversacional sin teclado y recibir la respuesta en audio.

El valor diferencial frente a las superficies CLI y Arcade reside en el ciclo completo de pregunta libre, llamada a una herramienta MCP, respuesta sintética y volcado a informe. Ninguna de las otras dos superficies expone ese ciclo, lo que justifica la coexistencia de Streamlit en el sistema pese a un solapamiento parcial de funcionalidad con la simulación CLI. La validación de la subsección es cualitativa, no cuantitativa, ya que la métrica de utilidad relevante (cobertura de consultas estratégicas reales por un usuario operacional) requiere un protocolo de evaluación con usuarios humanos que queda fuera del alcance de este trabajo.

5.5.2 Replay del GP de Hungría 2025 desde la CLI

El GP de Hungría 2025 se elige por su combinación poco habitual de cobertura de undercut y restricción reglamentaria. Charles Leclerc lidera desde la *pole position* hasta la vuelta 18, cuando Oscar Piastri entra a

```

• Lap 13: 91.318
• Lap 25: 89.601
• Net improvement over the stint segment: about 1.7 s
• Some laps were marked not valid, so those times should be treated cautiously

3) Strategic conclusions reached

a) PIAstri tyre condition
• At lap 25 in Australia, PIAstri's tyres appeared to be in manageable condition
• The tyre life estimate suggested no immediate cliff risk
• This implied the stint could likely continue for several more laps if race context allowed

b) Norris pace trend
• Norris showed a clear pace improvement across laps 13-25
• The trend suggested either:
  • tyre warm-up / track evolution benefits,
  • fuel burn-off,
  • or a cleaner stint phase
• However, because the comparison was incomplete, no direct conclusion versus PIAstri could be made

c) Overall limitation
• The main limitation in the conversation was missing comparative data for PIAstri in the lap-time overlay
• As a result, the only firm strategic takeaway was about PIAstri's tyre condition, not a full Norris-vs-PIAstri pace verdict

4) Final note
If you want, I can also turn this into a shorter executive summary or a race-engineering style report with bullet-point recommendations.

```

Figura 5.2: Fragmento del informe Markdown producido por el botón *Download Report* tras la conversación demostrativa sobre el GP de Australia 2025. Tres secciones (estado del neumático de PIAstri, tendencia de ritmo de Norris en V13 a V25, limitación por comparativa parcial) y cierre con propuesta de variantes de informe (resumen ejecutivo o estilo *race-engineering* por sub-agente).

boxes y monta neumático duro intentando un undercut sobre el coche de cabeza, con Leclerc por delante por unos dos segundos y medio en pista. Ferrari cubre el movimiento al cierre de la vuelta 19, también con duro, y Leclerc sale aún por delante de PIAstri por un segundo escaso. La elección del compuesto duro frente al medio fue criticada a posteriori por el propio Leclerc, que declaró desconocer la razón de salir en duro cuando con el medio el coche funcionaba bien, y la prensa especializada la encuadra dentro de las medidas conservadoras que Ferrari arrastraba desde el inicio de la temporada para mitigar el desgaste excesivo de la plancha tras la descalificación de Lewis Hamilton en el GP de China (Harris, 2025). El caso pone a prueba la capacidad del sistema para sopesar el ritmo contra la restricción regulatoria y emitir una recomendación coherente con la lectura agresiva, divergiendo de la decisión real del muro de Ferrari.

La superficie elegida es la línea de comandos (*f1-sim*) descrita en el Subsección 4.2.9, invocada con un rango breve de tres vueltas (V17 a V19) que cierra la ventana de simulación en la vuelta crítica de la decisión, de modo que la recomendación del orquestador en V19 aparece en el bloque grande de detalle del panel Rich y no comprimida en el histórico. El comando, ejecutado sobre la temporada 2025 con Leclerc como piloto seguido y PIAstri como rival, es *f1-sim Budapest LEC Ferrari -year 2025 -laps 17-19 -rival PIA -provider openai*. El asistente interactivo *f1-strat* ofrece la misma cadena de simulación con un menú guiado por teclado para usuarios que prefieran no recordar la sintaxis de los argumentos, alternativa documentada en el Subsección 4.2.9. La salida en terminal con paneles Rich actualizados por vuelta despliega tres bloques verticales descritos en el Subsección 4.2.9, un histórico de decisiones de vueltas anteriores con acción del orquestador y puntuaciones Monte Carlo de las cuatro estrategias candidatas, un bloque central con la vuelta en curso que detalla la salida tipada de los seis sub-agentes acompañada de su predicción cuantitativa y un glifo de alerta junto al razonamiento textual, y un bloque inferior con la planificación multi-vuelta (modo de ritmo, postura de riesgo, vuelta esperada de fin de relevo, contingencias condicionales y rival más cercano). El código cromático verde-amarillo-rojo permite identificar el estado de la cadena en la vuelta crítica sin necesidad de leer el detalle textual completo.

La trayectoria de recomendaciones del orquestador a lo largo de las tres vueltas evaluadas revela el momento óptimo de reacción según el sistema y la divergencia con la decisión real del muro de Ferrari. En

Lap	Tyre	Age	Pos	Lap (s)	Gap	Fwd	Rival	Decision	Plan	Conf	Stay	Pit	Ucut	Ocut	Reasoning
17	MED/C4	17	1	82.220	—	P2	MED/17 +2.5s	PIT_NOW NTRL-BAL	→ L17 HARD vs PIA	0.94	-1.787	-1.042	-1.031	0.176	PIT_NOW on lap 17 be P50 is at lap 1.7 fr delta is already +0. the previous lap, so stint is at the end
18	MED/C4	18	1	82.316	—	P2	MED/18 +4.9s	PIT_NOW L&C-BAL	→ L18 HARD vs PIA	0.97	-1.828	-1.010	-0.997	0.136	PIT_NOW on lap 18 be P50 is at 1.6 laps a already showing +0.0 versus the prior ref out risks crossing t immediately.
19	MED/C4	19	1	84.552	—	P5	HAR/2 +19.2s	OVERCUT MNGR-BAL	→ L22 HARD	0.73	-0.912	-1.259	-0.939	0.362	OVERCUT on lap 19: t P50 is only 3.4 laps has already lost +2. current stint is ent window but still has defe

Lap 19		
● Pace	Δnext +0.062s	pred 84.61s vs median +1.53s ±2.36s Lap 19: predicted 84.614s (+0.062s, slower than prev). CI [82.1-86.9s]. +1.535s vs median.
● Tire	cliff in ~3 laps	range 3-3 laps deg 1.146s/lap MONITOR Warning level: MONITOR, laps to cliff (P50): 3.4. Although the degradation rate is currently negative indicating improving pace, the P10 estimate shows only about 3 laps remain...
● Situation	threat LOW	overtake 0% safety car 1% Threat level: LOW. P(overtake) = 0.0 (gap > 2.5s), P(SC 3-lap) = 0.012 (very low SC risk). No overtaking opportunity and minimal Safety Car risk indicate a low strategic threat.
● Radio	no alerts	2 radios · 0 rcm The first radio message indicates the driver is confirming a strategic adjustment, likely a plan A minus five, and mentions that the tyres are worn out. This suggests the team i...
○ Pit	idle	triggers on cliff pressure, compound change, or problem radio
○ RAG	idle	triggers on compound change, SC >30%, or FIA warning/penalty

● Pit plan	Lap 22 → HARD	target 84.600s
● Pace mode	MANAGE	
○ Stint end	Lap 22	
● Branch	if SC deployed within 3... +2 more → PIT_NOW, UNDERCUT	
● Risks	Medium tyre cliff is ne... +4 more	
Rival	P5 HAR/2 +19.2s	

Figura 5.3: Panel Rich de f1-sim sobre el GP de Hungría 2025 con Leclerc como piloto seguido y Piastrri como rival, simulación cerrada en la vuelta 19 tras tres vueltas evaluadas. Histórico superior con las recomendaciones consecutivas del orquestador en V17, V18 y V19. Bloque inferior con la salida vuelta a vuelta de los seis sub-agentes en V19 y la planificación multivuelta del orquestador. Las cifras se discuten en el cuerpo del texto.

V17 y V18 el orquestador emite PIT_NOW con confianzas de 0,94 y 0,97 respectivamente sobre neumático medio en primera posición, identificando la ventana reactiva en la que Verstappen y Piastrri ejecutan sus respectivas paradas. Ferrari mantiene fuera a Leclerc durante esas dos vueltas y para finalmente al cierre de V19, una vuelta más tarde de la ventana óptima señalada por el sistema. En V19 el orquestador, observando que Piastrri ya ha pitado en V18 y se encuentra a +19,2 s en quinta posición sobre neumático duro fresco, cambia su recomendación a OVERCUT con confianza 0,73 y planifica la siguiente parada en la vuelta 22 manteniendo el neumático medio durante tres vueltas adicionales. La divergencia con la decisión real es de *timing*, el sistema ofrece dos ventanas de reacción óptimas (parada inmediata en V17 o V18 para cerrar el undercut, o extensión hasta la vuelta 22 para apurar el primer relevo) mientras que Ferrari elige una opción intermedia que pierde ambas ventajas.

El bloque grande de la Figura 5.3 muestra el detalle del orquestador en V19 con los seis sub-agentes desplegados. El sub-agente Pace predice un tiempo de vuelta de 86,61 s con un delta vuelta a vuelta de +0,062 s. El sub-agente Tire detecta el cliff de degradación en torno a tres vueltas adicionales sobre el neumático medio actual, con etiqueta MONITOR, lo que justifica la planificación de la parada en la vuelta 22. El sub-agente Race Situation reporta amenaza LOW con probabilidad de adelantamiento del 0 % y probabilidad de coche de seguridad del 0 %, sin presión competitiva inmediata. El sub-agente Radio no detecta alertas activas, el sub-agente Pit permanece en estado idle a la espera de los disparadores configurados (cliff de neumático, cambio de compuesto o radio de problema), y el sub-agente RAG ha cargado el contexto regulatorio. La rama condicional Branch del orquestador activaría PIT_NOW o UNDERCUT si el coche de seguridad apareciera en las tres vueltas siguientes, salvaguarda que mantiene robusta la decisión OVERCUT frente a eventos imprevistos.

La validación de este caso es cualitativa frente a la decisión real del muro de boxes y no frente a un *benchmark* cuantitativo público, limitación común a los casos de uso de esta sección y discutida en el Sección 5.4. El sistema propone una recomendación operacional anclada en sus modelos predictivos sin

asumir oráculo retrospectivo sobre el desenlace de la carrera, condición que separa una sugerencia de *outperform* reproducible de una afirmación retrospectiva no falsable.

5.5.3 Sesión en directo con Arcade sobre el GP de Qatar 2025

El GP de Qatar 2025 se elige por contener el que la prensa estratégica especializada cataloga como el peor error de muro de boxes de la temporada. En la vuelta 7, una colisión entre Nico Hülkenberg y Pierre Gasly en la primera curva activa el coche de seguridad y abre la ventana clásica de parada gratuita bajo bandera amarilla. Diecisiete coches pitan en esa misma vuelta, entre ellos Max Verstappen, aspirante al título y rival directo de los dos McLaren en la lucha por el campeonato de pilotos. Los dos coches de McLaren, en primera y tercera posición, deciden quedarse fuera bajo el argumento explícito de Andrea Stella (jefe de equipo) de evitar la salida en tráfico tras la parada. La realidad operativa es la opuesta. Los coches que paran bajo el SC ganan un delta-pit prácticamente gratuito y McLaren se ve obligada a parar más adelante bajo bandera verde por el mandato Pirelli de 25 vueltas máximas por compuesto en Lusail, perdiendo posición efectiva frente a Verstappen. La carrera termina con Verstappen ganando por ocho segundos sobre Piastri, Norris cuarto detrás de Carlos Sainz (Williams) y el campeonato reabierto con doce puntos de margen para la última cita de la temporada.

La superficie elegida es la réplica visual con Arcade (`f1-arcade`) descrita en el Subsección 4.2.9 por dos razones complementarias. La primera es de oportunidad demostrativa. El contraste visual entre el coche que sigue en pista en el panel replay y la recomendación divergente que el panel estratégico mantiene en pantalla durante esa vuelta sintetiza el argumento central del caso, dado que el sistema marca la decisión que el muro de McLaren no tomó. La segunda es de pertinencia técnica. El panel de telemetría comparada de la Figura 4.28 hace visible la pérdida de velocidad punta y los puntos de frenada del compuesto degradado frente al rival con neumático fresco, información operacional sobre la que la decisión estratégica se apoya. El comando ejecutado sobre la temporada 2025 con Piastri como piloto seguido y Verstappen como rival es `f1-arcade -viewer -year 2025 -round 23 -driver PIA -team McLaren -driver2 VER -strategy`, atajo de línea de comandos que salta el menú interactivo y carga directamente la réplica de carrera con las tres ventanas coordinadas por el servidor TCP local descrito en el Subsección 4.2.9.

La primera ejecución del sistema sobre la vuelta 7 reproduce con exactitud el sesgo del muro de boxes real, recomendando `STAY_OUT` con una confianza del 92 % y modo de ritmo `MANAGE`. El diagnóstico del comportamiento revela una cadena de tres frenos en la arquitectura del orquestador. El sub-agente `Radio` detecta correctamente el `RCM SAFETY_CAR_DEPLOYED` emitido por dirección de carrera y lo expone como evento estructurado en el *lap state*, pero el sub-agente `Race Situation` reporta una probabilidad de coche de seguridad del 10 % porque su modelo `LightGBM` está entrenado para predecir eventos futuros y no integra la señal de coche de seguridad ya activo. La salvaguarda de stint mínimo del sub-agente `Pit Strategy`, que exige un mínimo de doce vueltas sobre compuesto medio, descarta cualquier parada antes de ese umbral con independencia del contexto. El agente reglamentario `RAG` solo se activa cuando la probabilidad de coche de seguridad reportada por `Race Situation` supera el treinta por ciento, condición que la lectura ciega del sub-agente nunca cumple sobre esta vuelta. El sistema, en consecuencia, replica la decisión humana errónea por una razón distinta a la del muro real.

La identificación de esta cadena de frenos motiva la implementación de un módulo de integración cruzada que post-procesa la salida del sub-agente `Race Situation` a partir de los RCMs activos del *lap state*. El módulo, descrito con detalle en el Capítulo 6, fuerza la probabilidad de coche de seguridad a la unidad cuando el parser determinista detecta un evento `SAFETY_CAR_DEPLOYED` no acompañado de su correspondiente RCM de cierre, propaga un indicador booleano `sc_currently_active` a través del *lap state* y anula la

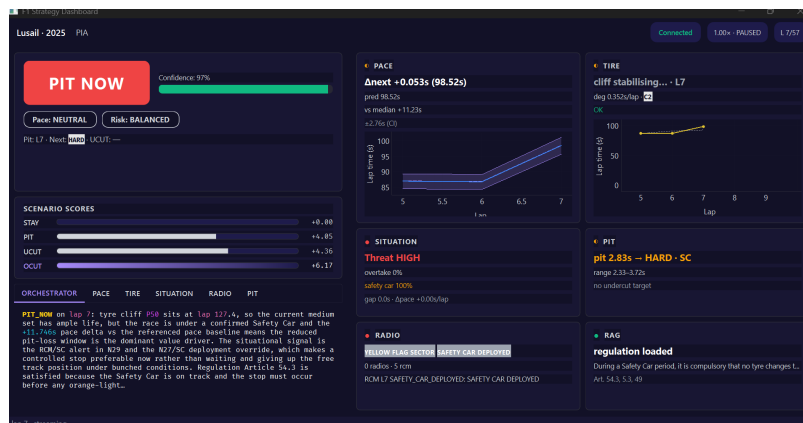


Figura 5.4: Réplica visual con Arcade del GP de Catar 2025 con Piastri como piloto seguido y Verstappen como rival, simulación pausada en la vuelta 7 con coche de seguridad activo. Panel izquierdo con la ventana de replay, tabla de clasificación y abandono de Hülkenberg tras la colisión que motivó la neutralización. Panel derecho con el detalle del orquestador tras activar el módulo RCMContextResolver descrito en el Capítulo 6. Las cifras y el razonamiento del orquestador se discuten en el cuerpo del texto.

salvaguarda de stint mínimo del sub-agente Pit Strategy bajo esa condición. La re-ejecución del sistema sobre la misma vuelta 7 con el módulo activo, recogida en la Figura 5.4, produce ahora una recomendación de PIT_NOW con confianza del 97 %, modo de ritmo NEUTRAL y postura BALANCED. El sub-agente Race Situation reporta amenaza HIGH con probabilidad de coche de seguridad del 100 %, el sub-agente Pit Strategy estima una parada de 2,83 s con compuesto siguiente duro bajo régimen de coche de seguridad y el agente reglamentario RAG cita el Artículo 36.3 sobre la obligación de los equipos de entrar a boxes durante una neutralización por coche de seguridad. El razonamiento del orquestador menciona explícitamente la activación del módulo (*"the N27/N28 deployment override"*) como detonante de la decisión, lo que sustituye el razonamiento opaco del muro humano por una secuencia auditable de señales y umbrales. La trazabilidad cubre desde la detección del RCM en el sub-agente Radio (YELLOW FLAG SECTOR y SAFETY CAR DEPLOYED visibles en su tarjeta) hasta la justificación reglamentaria del agente RAG, pasando por el override forzado de la probabilidad de coche de seguridad en Race Situation.

Conviene precisar el alcance de la corrección. La decisión real del muro de boxes de McLaren no es irracional en términos absolutos, sino una apuesta estratégica plausible bajo la asunción de que un coche de seguridad desplegado muy temprano sobre una carrera de cincuenta y siete vueltas puede ser corto y que un evento ulterior podría reabrir una ventana de parada más favorable. El sistema corregido prioriza la decisión cuantitativa fundada en el delta de pit gratuito bajo coche de seguridad activo, lo que en este caso concreto se valida frente al desenlace real, pero al hacerlo descarta el tipo de juicio probabilístico contextual que un estratega humano puede aportar en escenarios donde la apuesta por un coche de seguridad posterior tiene mérito empírico. El sistema debe entenderse, por tanto, como un copiloto que provee una señal estructurada y reproducible, no como un sustituto del juicio del muro de boxes. La aportación es complementaria a la decisión humana, no excluyente, y el caso de Catar 2025 ilustra el valor del contraste entre ambas lógicas.

La validación de este caso, como en los anteriores, es cualitativa frente a la decisión real del muro de boxes. La aportación demostrativa es doble. Por un lado, la primera ejecución del sistema identifica con precisión técnica la cadena de frenos que llevó al orquestador a replicar el sesgo humano. Por otro, el módulo de integración cruzada elimina ese sesgo y produce una recomendación divergente de la decisión real, validada a posteriori por el desenlace de la carrera y el consenso de la prensa especializada. La diferencia con el caso de Hungría es la magnitud de la consecuencia, ya que esta decisión comprometió la matemática del campeonato de pilotos y dejó la disputa abierta a doce puntos para la última cita de la temporada.

6 CONCLUSIONES y TRABAJOS FUTUROS

¿Cómo sería la vida si solo hiciéramos lo necesario?

— Niki Lauda, tricampeón de Fórmula 1¹

Este capítulo cierra la memoria con un balance honesto del trabajo realizado. La revisión se ordena en seis bloques que recorren el cumplimiento de los cuatro objetivos específicos enunciados en el Capítulo 1, las aportaciones principales del sistema descrito en el Capítulo 4 y validado en el Capítulo 5, el refinamiento arquitectónico realizado durante la fase experimental, las limitaciones detectadas, las líneas de trabajo futuro y una valoración personal del aprendizaje. Solo la sección dedicada al módulo `RCMContextResolver` introduce material técnico no presentado antes, motivada por el caso de Catar 2025 discutido en Subsección 5.5.3.

6.1 Cumplimiento de objetivos

Conviene abrir el balance por la revisión más directa de todas, la de los compromisos asumidos al inicio del Capítulo 1. Para ordenarla, la Tabla 6.1 confronta cada uno de ellos con la evidencia cuantitativa que respalda su cierre y remite a las secciones del Capítulo 5 donde esa evidencia se discute en detalle. La sección actúa así como mapa de retorno hacia los resultados ya presentados, sin reabrir el análisis que allí se desarrolla.

El balance que arroja la tabla es positivo en su lectura operativa, todos los compromisos cuentan con implementación efectiva y respaldo cuantitativo. Conviene matizar, no obstante, que el contraste frente a decisiones reales del muro de boxes se ha realizado por casos representativos en Sección 5.5, y no mediante un protocolo agregado sobre las veinticuatro carreras de la temporada. Esa comparativa sistemática se difiere a Sección 6.5 como extensión natural del trabajo presentado.

6.2 Aportaciones principales

Las aportaciones que sostienen este balance se enumeran a continuación por capa arquitectónica, desde la integración de datos hasta las decisiones metodológicas, y cada una se apoya en un dato o en una decisión ya documentados en el Capítulo 4 o en el Capítulo 5.

1. **Integración abierta y de extremo a extremo de telemetría, aprendizaje automático, procesamiento de lenguaje natural, recuperación reglamentaria y modelo de lenguaje** sobre datos públicos (FastF1, OpenF1 y los PDFs reglamentarios de la FIA), frente a las plataformas cerradas de los equipos profesionales descritas en el Capítulo 3. El sistema se distribuye como repositorio bajo licencia Apache 2.0.
2. **Flujo de canalización de procesamiento de lenguaje natural específico del dominio F1** que encadena ASR Whisper turbo, sentimiento RoBERTa con ajuste fino (exactitud 0,84), intención SetFit sobre ModernBERT (macro F1 0,5338), NER BIO con BERT-large CoNLL-03 (F1 0,4151) y parser

¹Respuesta de Lauda a quienes criticaban la Fórmula 1 como un riesgo innecesario, recogida en una entrevista con *Top Gear* y reproducida en publicaciones posteriores como *Speedhunters* (2019). Traducción propia del original en inglés: *A lot of people criticise Formula 1 as an unnecessary risk. But what would life be like if we only did what is necessary?*

Tabla 6.1: Cumplimiento de los cuatro objetivos específicos enunciados en el Capítulo 1. La columna de evidencia recoge una muestra representativa de las métricas obtenidas en el Capítulo 5.

#	Objetivo específico	Estado	Evidencia cuantitativa y referencia
1	Conjunto de datos 2023–2024 para entrenamiento y 2025 para test, familia de siete modelos predictivos y agrupación de circuitos en cuatro arquetipos	Cumplido	Lap time MAE 0,4104 s, undercut AUC-ROC 0,7708, overtake AUC-PR 0,5491, safety car con lift $1,67\times$ sobre tasa base, pit duration P50 MAE 0,487 s, tire degradation TCN MAE 0,7078 s global y 0,5501 s para C2 fine-tuned, clustering K=4 con silueta 0,201 (Sección 5.1)
2	Flujo de procesamiento de lenguaje natural sobre radio de equipo, sentimiento, intención, NER y procesamiento de RCM	Cumplido	RoBERTa de sentimiento con accuracy 0,84, SetFit de intención con macro F1 0,5338, BERT-large CoNLL-03 para NER con F1 0,4151, parser determinista de RCM con cobertura del 100 % sobre tipos críticos y pipeline completo con latencia P95 GPU de 59,4 ms (Subsección 5.1.8, Subsección 5.1.9, Subsección 5.1.10, Subsección 5.1.11, Subsección 5.1.12)
3	Subsistema de recuperación reglamentaria y arquitectura multi-agente con seis sub-agentes coordinados por un orquestador supervisor compatible con LLM local y en la nube	Cumplido	Recuperación reglamentaria sobre 2.279 chunks indexados con BGE-M3 con MRR 0,235 y latencia P95 de 243,7 ms, orquestador N31 con 14 campos contractuales en StrategyRecommendation y provider conmutable entre LM Studio y OpenAI (Subsección 5.1.13, Sección 5.2, Subsección 4.2.8)
4	Motor de simulación con tres superficies de uso (CLI, Arcade, Streamlit) y validación de extremo a extremo sobre la temporada 2025 con comparación frente a decisiones reales del muro de boxes	Cumplido	Smoke completo sobre Melbourne 2025 y Baréin 2025 (Sección 5.3) y tres casos de uso reproducidos con datos reales para Australia, Hungría y Catar 2025, dos de ellos con divergencia validada respecto a la decisión real del muro de boxes (Sección 5.5)

determinista de RCM, unificada con latencia P95 por debajo de 60 ms en GPU (Subsección 5.1.8, Subsección 5.1.9, Subsección 5.1.10, Subsección 5.1.11, Subsección 5.1.12).

- Familia de modelos predictivos competitivos para estrategia de carrera** que comprende tiempo de vuelta con XGBoost y bootstrap CI (MAE 0,4104 s), undercut con LightGBM (AUC-PR 0,6739 y AUC-ROC 0,7708), duración de parada con HistGBT cuantílico (MAE P50 0,487 s y cobertura del 70,5 %), adelantamiento con LightGBM (AUC-PR 0,5491) y safety car (lift $1,67\times$ sobre tasa base), todos validados con partición temporal estricta sobre 2023–2024 y prueba sobre 2025 (Sección 5.1).
- Validación del clustering de circuitos en cuatro arquetipos** (*strategic chaos*, *long high-speed*, *warm conservative* y *high-stress outliers*) ajustado sobre 2023–2024 y aplicado sin reajuste a 2025, utilizado de forma transversal como rasgo por los modelos predictivos que lo consumen.
- Arquitectura multi-agente con LangGraph en patrón supervisor**, compuesta por seis sub-agentes especializados (Pace, Tire, Race Situation, Pit Strategy, Radio y RAG reglamentario) y un orquestador N31 con tres capas (enrutador MoE basado en reglas, simulación Monte Carlo de quinientas trayectorias por estrategia y síntesis con LLM y structured output sobre catorce campos Pydantic), distribuida

por tres superficies de uso con copia local del orquestador en Arcade para evitar dependencia HTTP (Sección 5.2, Subsección 4.2.8).

6. **Decisión metodológica documentada sobre el resultado negativo del TCN de adelantamiento (N12B)**, donde el modelo TCN causal con aproximadamente dieciocho mil secuencias rinde AUC-PR cercana a 0,10 mientras que el LightGBM con *features* `pace_delta_rolling3` y `gap_trend` alcanza AUC-PR 0,5491. La aportación es metodológica, el *feature engineering* explícito supera al *sequence modeling* bruto cuando el conjunto es pequeño y los patrones temporales relevantes se conocen de antemano.

Estas seis aportaciones materializan los objetivos enunciados en la propuesta inicial del proyecto y los resultados de aprendizaje de la titulación vinculados a ella. La arquitectura final difiere del alcance previsto en tres puntos concretos, la sustitución del motor de reglas Experta por el orquestador supervisor de tres capas descrito en el ítem anterior, la composición del banco de agentes con seis sub-agentes especializados frente a los tres planificados, y la consolidación de Arcade como superficie de uso completa con copia local del orquestador en lugar de visualización opcional.

6.3 Refinamiento durante la validación: módulo `RCMContextResolver`

El recorrido entre el cierre del modelado y la entrega de la memoria ha dejado varios refinamientos puntuales además del módulo que da nombre a esta sección. El descarte del TCN causal de adelantamiento N12B en favor del LightGBM con *features* explícitas `pace_delta_rolling3` y `gap_trend` (Sección 5.1), la sustitución de NuNER *zero-shot* y con ajuste fino por BERT-large CoNLL-03 BIO para el reconocimiento de entidades (Subsección 5.1.11), el descarte de DeBERTa-v3-large frente a SetFit sobre ModernBERT para la clasificación de intención (Subsección 5.1.10) y la elección del modelo de embedding BGE-M3 sobre MiniLM y de *chunking* de doscientos cincuenta y seis *tokens* para la recuperación reglamentaria (Subsección 5.1.13) son ajustes razonados durante la validación que no llegaron a alterar la topología del sistema pero sí cambiaron su contenido. El módulo `RCMContextResolver` es el único refinamiento que tocó la propia composición del grafo de agentes y por eso recibe descripción técnica autónoma a continuación.

La primera ejecución del caso de Catar 2025 vuelta 7 con coche de seguridad activo, descrita en Subsección 5.5.3, recomendaba `STAY_OUT` al 92 % replicando el sesgo del muro de boxes real. La cadena de tres barreras identificada fue Race Situation ciego al SC ya activo, salvaguarda de stint mínimo de Pit Strategy bloqueando la parada y umbral del 30 % del agente reglamentario silenciando la cita pertinente. El parser determinista de RCM detectaba el evento `SAFETY_CAR_DEPLOYED` pero esa señal no se propagaba al enrutamiento del orquestador ni al estado de los sub-agentes.

El módulo `RCMContextResolver` cubre cinco tipos de RCM asociados a SC y VSC, post-procesa la salida del sub-agente Race Situation a partir de los RCMs activos del *lap state*, propaga un indicador booleano `sc_currently_active` a través de un campo nuevo en `RaceSituationOutput` y desactiva la salvaguarda de stint mínimo del sub-agente Pit Strategy cuando esa señal está activada. La implementación es estrictamente aditiva sobre los *schemas* Pydantic y no requiere reentrenar modelos. La re-ejecución del caso de Catar 2025 vuelta 7 con el módulo activo produce `PIT_NOW` con confianza alta, integra al agente reglamentario que cita el mandato Pirelli de veinticinco vueltas en Lusail y diverge así de la decisión del muro de McLaren.

Conviene precisar que la decisión real del muro de McLaren no es irracional en términos absolutos. El sistema corregido prioriza la lectura cuantitativa del delta de pit gratuito bajo SC activo y descarta el juicio probabilístico contextual de que un SC corto seguido de otro evento ulterior pueda reabrir una ventana

más favorable. La conclusión operativa es que el sistema debe entenderse como copiloto que aporta señal estructurada y reproducible al muro de boxes, no como sustituto del juicio humano del estratega. Cada nuevo caso de carrera al que se exponga al orquestador revelará condiciones de borde donde la cadena de inferencia falla o donde conviene ajustar las salvaguardas, y el módulo aquí presentado es el primer ejemplo documentado de ese proceso de descubrimiento iterativo más que un cierre definitivo.

6.4 Limitaciones

Las limitaciones identificadas a lo largo del proyecto se enumeran aquí como parte del rigor académico, no como defensa anticipada. Cada una recibe respaldo con la cifra concreta o la decisión arquitectónica que la motiva.

1. **Datos públicos frente a telemetría propietaria.** FastF1 y OpenF1 exponen una fracción del canal telemétrico real de los equipos profesionales, sin sensores internos de neumático, combustible ni temperatura exacta de freno. Esta carencia tiene efecto cuantificable sobre el modelo de degradación, donde solo el compuesto C2 dispuso de muestras suficientes para un ajuste fino específico (MAE 0,5501 s) mientras que C1, C3, C4, C5 y C6 caen al TCN global (MAE 0,7078 s) por falta de vueltas etiquetadas por compuesto.
2. **Hardware de entrenamiento limitado.** El trabajo se desarrolló sobre una única RTX 5070 de 16 GB de VRAM, lejos de las granjas de A100 o H100 que utilizan los equipos profesionales. Esta restricción techa las decisiones arquitectónicas, desde la preferencia por TCN compacto frente a un transformador completo para series temporales hasta el uso de Whisper turbo en lugar de *large-v3* o de modelos locales de tamaño moderado frente a opciones de setenta mil millones de parámetros.
3. **Eventos raros con precisión limitada.** El modelo de coche de seguridad opera con una tasa base del 4-5 % y una AUC-PR de 0,0723. Pese al *lift* de $1,67\times$ sobre tasa base, se reporta como estimador de riesgo relativo y no como predictor determinista. Predecir SC con alta precisión requiere señales fuera del alcance del conjunto actual, como cámaras de pista o comunicaciones FIA internas.
4. **Validación de extremo a extremo cualitativa.** Las recomendaciones del orquestador se han contrastado con tres casos reales de la temporada 2025 (Australia para el chat MCP, Hungría para la CLI y Catar para Arcade), no con una comparación sistemática vuelta a vuelta sobre las veinticuatro carreras del calendario. La aportación demostrativa es por casos representativos, no por una métrica agregada de coincidencia con el muro de boxes.
5. **NER con F1 0,4151 sobre corpus auto-etiquetado.** El corpus de radio F1 (399 ejemplos, nueve entidades semánticas clausales) está etiquetado por el propio autor sin panel de revisión ni consenso inter-anotador. El BERT-large CoNLL-03 BIO supera a GLiNER *zero-shot* y a NuNER con ajuste fino, lo que indica que el techo está marcado tanto por el tamaño del corpus como por la ausencia de validación cruzada de las anotaciones.
6. **Cola de latencia del orquestador.** El flujo de canalización PLN cumple holgadamente el RNF-01 de 60 ms P95 en GPU, pero el orquestador completo arrastra una cola de 13,7 s en el sub-agente RAG. Esto significa que el sistema cumple la ventana operativa de decisión en la mediana, pero no en el percentil 95, especialmente cuando el agente reglamentario interviene en la cadena.
7. **Sin verificación de calibración probabilística.** Los clasificadores de undercut, adelantamiento y coche de seguridad emiten probabilidades que no se han confrontado con *reliability diagrams* ni

análisis de *Brier score*. No existe garantía de que las predicciones del 80 % acierten 80 % de las veces sobre el conjunto de prueba, lo que limita su uso como umbrales de decisión sin calibración posterior.

8. **Ausencia de evaluación con usuarios y visualización Arcade limitada.** El sistema no se ha sometido a una prueba A/B con estrategias profesionales, su utilidad operacional se valora por consistencia interna y trazabilidad de la cadena de inferencia. Adicionalmente, la superficie Arcade está construida sobre una biblioteca 2D Python que carece de las animaciones avanzadas y los efectos visuales propios de las pantallas de muro de boxes profesionales, lo que limita su adopción más allá del uso didáctico e interactivo.

6.5 Trabajos futuros

Las líneas de trabajo futuro se proponen como extensiones naturales del sistema actual, cada una con un objetivo medible o una capacidad concreta a alcanzar.

1. **Comparativa retrospectiva sistemática sobre la temporada 2025 completa.** Diseñar un protocolo que, para cada vuelta con decisión estratégica relevante del muro de boxes (parada, cambio de compuesto, no parada bajo SC), registre la recomendación del orquestador junto al resultado real de la carrera y produzca una métrica agregada de coincidencia sobre las cuatro acciones discretas (STAY_OUT, PIT_NOW, UNDERCUT, OVERCUT) extendida a las veinticuatro carreras del calendario.
2. **Ampliación del corpus etiquetado de radio F1.** Crecer el etiquetado hasta unos dos mil ejemplos (cinco veces el tamaño actual) con un panel de etiquetadores y consenso inter-anotador, con el objetivo de superar F1 0,60 a nivel de *token*. Considerar *self-training* sobre los 530 audios ya transcritos con Whisper.
3. **Adaptación al reglamento técnico y deportivo de 2026.** El cambio de unidad de potencia con reparto del 50 % eléctrico y eliminación de la MGU-H, el rediseño aerodinámico con alerones activos, la introducción de coches más ligeros y la previsible reformulación de los compuestos Pirelli reintroducirán deriva conceptual significativa. El trabajo inmediato consiste en reajustar los *clusters* de circuitos con datos de la primera mitad de la temporada 2026, reetiquetar los compuestos con los nuevos mapeos, regenerar las *features* de ritmo y degradación y monitorizar la degradación de rendimiento de los modelos hasta que los *splits* 2023-2024 dejen de ser representativos.
4. **Despliegue de un año completo con telemetría en vivo y reentrenamiento progresivo por Gran Premio.** Para cada GP de la temporada $N+1$, entrenar los modelos predictivos con los datos históricos de ese mismo circuito en la temporada N y afinarlos en caliente con los libres y la clasificación del propio fin de semana. Como ejemplo, para Baréin 2027 los modelos partirían de Baréin 2026 como base y se afinarían con los libres y la clasificación del viernes y sábado del propio fin de semana. El objetivo es pasar del régimen actual de partición temporal estricta a un régimen de aprendizaje incremental por carrera que aproveche la dinámica específica de cada pista en su año vigente.
5. **Extensión del módulo `RCMContextResolver` a más tipos de RCM.** El módulo cubre actualmente cinco tipos asociados a SC y VSC. Las líneas naturales de continuación son añadir *override* específico para bandera roja (forzar pit window según artículo del reglamento), penalización aplicada (forzar parada en N vueltas siguientes) y pit lane cerrado (bloqueo de recomendaciones de parada). El esqueleto del módulo lo soporta sin cambios estructurales.

6. **Ajuste fino de un modelo de lenguaje abierto sobre corpus específico de F1.** Línea anticipada como extensión natural en el Capítulo 3, donde la dependencia del modelo de lenguaje se identificó como restricción crítica del sistema. La propuesta consiste en curar un corpus específico del dominio (terminología técnica, jerga del paddock, articulado del reglamento deportivo, comentario estratégico de carrera y casos históricos relevantes) y realizar un ajuste fino mediante LoRA sobre Gemma 4 o sucesor de la familia Gemma (Gemma Team, 2025), con el objetivo de dotar al razonador final de un conocimiento profundo y especializado en Fórmula 1 que ningún modelo generalista alcanza por defecto. La métrica de éxito esperable es una mejora simultánea en la tasa de validación del *schema* Pydantic de catorce campos en *StrategyRecommendation* y en la exactitud de la cita reglamentaria respecto a la baseline actual basada en *gpt-4.1-mini* para los sub-agentes y *gpt-5.4-mini* para la capa de síntesis del orquestador.
7. **Orquestador por mezcla de agentes (MoA) ajustado finamente.** Sustituir el supervisor basado en *prompting* por un modelo MoA ajustado finamente sobre trazas de razonamiento del sistema actual, siguiendo el patrón (Wang et al., 2024). El objetivo es reducir la cola de latencia del orquestador identificada en Sección 6.4 y mejorar la consistencia de la acción seleccionada.
8. **Integración con telemetría WebSocket en tiempo real.** Sustituir el motor de replay desde *parquets* por un consumidor WebSocket sobre OpenF1 durante un GP en directo, verificando que los siete agentes mantienen la latencia admisible bajo carga real y permitiendo el cierre del ciclo previsto en la línea anterior.

6.6 Valoración personal

El cierre del capítulo invita a recoger antes los aprendizajes acumulados a lo largo del proyecto. El primero, de orden técnico, es que integrar componentes heterogéneos como series temporales, procesamiento de lenguaje natural, recuperación reglamentaria y modelo de lenguaje en un sistema coherente exige más ingeniería de la que aporta el conocimiento teórico aislado. El segundo, de orden metodológico, ha sido constatar que la validación temporal estricta sobre las temporadas 2023-2024 y prueba sobre 2025 y el registro honesto de los resultados negativos del TCN de adelantamiento N12B, de DeBERTa-v3-large para la clasificación de intención y de NuNER para el reconocimiento de entidades han resultado más formativos que los éxitos aislados. El tercero, sobre la propia naturaleza de la validación, lo aportó el caso de Catar 2025, que forzó a identificar y corregir un fallo arquitectónico en un sistema que parecía funcional durante la propia fase de redacción de la memoria y dejó claro que la validación retrospectiva con casos reales es herramienta de descubrimiento, no solo de demostración.

Visto en retrospectiva, dos decisiones se tomarían de otro modo, abordar antes el etiquetado del corpus de radio, donde la NER quedó techada por los datos y no por la arquitectura, y considerar desde el inicio un protocolo de evaluación de extremo a extremo frente a decisiones reales del muro de boxes. El sistema entregado se entiende como punto de partida operativo y no como cierre definitivo, con el código publicado bajo licencia Apache 2.0 y las líneas de continuación detalladas en Sección 6.5. La intersección entre aprendizaje automático y dominio Fórmula 1 explorada aquí queda como terreno fértil en el que seguir trabajando más allá del cierre formal de esta memoria.

7 BIBLIOGRAFÍA

Las citas deben ser realizadas bajo la Norma APA.

- Aston Martin F1 Team. (2026, marzo). *Cohere joins Aston Martin Aramco as Official Generative AI Partner to help accelerate AI innovation* [Clave bib heredada cohere_2024_astonmartin; el anuncio real es del 4 de marzo de 2026, no de 2024]. Consultado el 20 de abril de 2026, desde <https://www.astonmartinf1.com/en-GB/news/announcement/cohere-joins-aston-martin-aramco-as-official-generative-ai-partner>
- Bai, S., Kolter, J. Z., & Koltun, V. (2018). *An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling*. arXiv: 1803.01271. <https://arxiv.org/abs/1803.01271>
- Bansal, A., Arora, A., Bhati, L., Sethia, K., Verma, I., & Kapoor, N. (2025). Advanced Machine Learning Approaches for Formula 1 Race Performance Prediction: A Comprehensive Analysis of Championship Point Forecasting [*Preprint* no revisado por pares sobre la predicción de resultados de Fórmula 1 mediante XGBoost y gradient boosting sobre datos 1950–2024; atribuye un 23,8 % de importancia agregada a la variable *Year*, evidencia que justifica mantenerla en el conjunto de predictores del sistema pese a la deriva conceptual entre temporadas]. Consultado el 1 de mayo de 2026, desde <https://www.researchgate.net/publication/394015807>
- Bengio, Y., Léonard, N., & Courville, A. (2013). *Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation*. arXiv: 1308.3432. <https://arxiv.org/abs/1308.3432>
- Bergmeir, C., & Benítez, J. M. (2012). On the Use of Cross-Validation for Time Series Predictor Evaluation [Referencia canónica de la validación temporal por bloques en series temporales; justifica la sustitución de la validación cruzada aleatoria por un esquema de *hold-out* respetando el orden cronológico (train en el periodo más antiguo, validación en el siguiente, test sobre el más reciente)]. *Information Sciences*, 191, 192-213. <https://doi.org/10.1016/j.ins.2011.12.028>
- Bogdanov, S., Constantin, A., Bernard, T., Crabbé, B., & Bernard, É. (2024). NuNER: Entity Recognition Encoder Pre-training via LLM-Annotated Data [NuNER_Zero-span baseline zero-shot evaluado en N22]. *Proceedings of EMNLP*. <https://aclanthology.org/2024.emnlp-main.660/>
- Brawn, R., & Parr, A. (2016). *Total Competition: Lessons in Strategy from Formula One* [Libro de referencia sobre estrategia y toma de decisiones en F1. Fuente del epígrafe del cap. 1 y respaldo conceptual del cap. 2.1.4 (undercut, overcut, *pit window*)]. Simon & Schuster. <https://www.worldcat.org/isbn/9781471162404>
- Carter, D. (2025, agosto). *How Ferrari, Mercedes, Red Bull and McLaren are using AI and cloud computing to gain an edge in F1* [Clave bib heredada allegretti_2025_aiteams; autor real verificado en el sitio: David Carter (no Diego Allegretti). Fuente de prensa motor empleada únicamente para contextualizar el conjunto de partenariados AI en F1 descritos en el cap. 1 (Ferrari–AWS, Red Bull–Oracle, McLaren–Dell, Aston Martin–Cognizant), cada uno respaldado a su vez por sus correspondientes comunicados oficiales]. ScuderiaFans. Consultado el 25 de enero de 2025, desde <https://scuderiafans.com/how-ferrari-mercedes-red-bull-and-mclaren-are-using-ai-and-cloud-computing-to-gain-an-edge-in-f1/>
- Casella, G., & Berger, R. L. (2002). *Statistical Inference* (2.^a ed.) [Referencia clásica de inferencia estadística; cubre las distribuciones normal y de Bernoulli empleadas en el muestreo de la simulación Monte Carlo del orquestador]. Duxbury. <https://www.worldcat.org/isbn/9780534243128>
- Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., & Liu, Z. (2024). *BGE M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation* [Modelo

- BAAI/bge-m3; embeddings densos (1024-d) usados en el RAG sobre PDFs FIA 2023–2025]. arXiv: 2402.03216. <https://arxiv.org/abs/2402.03216>
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System [XGBoost; modelo de *delta lap time* en N06 (MAE 0,392 s sobre test 2025)]. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794. <https://doi.org/10.1145/2939672.2939785>
- Cheng, K., et al. (2025, septiembre). *Computational Fluid Dynamics Optimization of F1 Front Wing using Physics Informed Neural Networks* [Clave bib heredada cheng_2024_pinnf1; el preprint en realidad se subió en septiembre de 2025 según el identificador arXiv]. arXiv: 2509.01963. <https://arxiv.org/abs/2509.01963>
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation [Paper canónico de la unidad GRU; introducida originalmente en traducción automática neural pero generalizada a múltiples dominios. En el proyecto legacy se evaluaron LSTM y GRU sobre la serie temporal de carrera y se descartaron en favor de la TCN causal]. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1724-1734. <https://doi.org/10.3115/v1/D14-1179>
- Conventional Commits. (2019). *Conventional Commits 1.0.0* [Especificación abierta de mensajes de *commit* estructurados con prefijos (*feat*, *fix*, *chore*, *ci*, *docs*, etc.) que habilita la inferencia automática del versionado semántico y la generación de notas de versión; adoptada en el flujo de publicación del proyecto]. Consultado el 11 de mayo de 2026, desde <https://www.conventionalcommits.org/en/v1.0.0/>
- Craven, P., & Arcade contributors. (2025). *The Python Arcade Library* [Biblioteca Python de código abierto para el desarrollo de juegos 2D y aplicaciones gráficas interactivas, construida sobre Pyglet y OpenGL]. Consultado el 13 de mayo de 2026, desde <https://api.arcade.academy/>
- Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C., & Bontempi, G. (2018). Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy [Referencia canónica en detección de fraude con tarjeta bajo deriva conceptual incremental: modela explícitamente la evolución de las tácticas de los atacantes y propone una estrategia de aprendizaje en línea con ventana deslizante]. *IEEE Transactions on Neural Networks and Learning Systems*, 29(8), 3784-3797. <https://doi.org/10.1109/TNNLS.2017.2736643>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding [BERT-large como backbone del reconocimiento de entidades F1 (NER token-level) en N22]. *Proceedings of NAACL-HLT*, 4171-4186. <https://arxiv.org/abs/1810.04805>
- Digital Engineering 24/7. (2024). *Formula 1 By Design*. Consultado el 25 de enero de 2025, desde <https://www.digitalengineering247.com/article/formula-1-by-design>
- Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise [Referencia canónica del algoritmo DBSCAN; alternativa basada en densidad considerada y descartada para el agrupamiento de circuitos por la inestabilidad de sus hiperparámetros ε y *minPts* con un corpus de 24 muestras]. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, 226-231. <https://aaai.org/papers/kdd96-037-a-density-based-algorithm-for-discovering-clusters-in-large-spatial-databases-with-noise/>
- Evidently AI. (2025, enero). *What is concept drift in ML, and how to detect and address it* [Clave bib heredada evidently_2024_mlmonitoring; la página actual usa el título arriba (rebrandeada en enero 2025). El título original "Machine Learning Monitoring, Part 5: Data and Concept Drift" pertenecía a la serie antigua de Evidently AI]. Consultado el 25 de enero de 2025, desde <https://www.evidentlyai.com/ml-in-production/concept-drift>

- Fédération Internationale de l'Automobile. (2024, febrero). *2024 Formula 1 Sporting Regulations — Issue 4* [Aprobado por el WMSC el 13 de febrero de 2024 (110 páginas). Índice de versiones del reglamento deportivo FIA: <https://www.fia.com/regulation/category/110>. Fuente documental del RAG Agent (N30) junto con las ediciones 2023 y 2025]. FIA. Consultado el 24 de abril de 2026, desde https://www.fia.com/sites/default/files/fia_2024_formula_1_sporting_regulations_-_issue_4_-_2024-02-13.pdf
- Fédération Internationale de l'Automobile. (2025). *2025 Formula 1 Technical Regulations* [Reglamento técnico oficial FIA 2025 para Fórmula 1; fija la masa máxima de combustible por carrera en 110 kg y el caudal máximo de combustible en 100 kg/h. NOTA: la URL aquí enlazada es el índice de la categoría 110 (regulación deportiva y técnica) de la FIA; para la versión PDF concreta del reglamento técnico 2025 hay que navegar al enlace específico desde ese índice]. Consultado el 22 de abril de 2026, desde <https://www.fia.com/regulation/category/110>
- Formula 1. (2024, diciembre). *Facts and Stats: McLaren's record 26-year title wait ends and Leclerc scores podium milestone* [Resumen oficial publicado por F1.com tras la confirmación matemática del Campeonato de Constructores 2024 en el Gran Premio de Abu Dabi (8 de diciembre de 2024); fija en 26 años el lapso desde el anterior título de constructores de McLaren (1998)]. Consultado el 25 de abril de 2026, desde <https://www.formula1.com/en/latest/article/facts-and-stats-mclarens-record-26-year-title-wait-ends-and-leclerc-scores.2eFJvtofZMatNNagF8f2NT>
- Formula 1. (2025a, octubre). *2025 Singapore Grand Prix race report and highlights: George Russell storms to victory in Singapore as McLaren seal Teams' Championship* [Crónica oficial F1.com del Gran Premio de Singapur del 5 de octubre de 2025, donde McLaren selló matemáticamente el Campeonato de Constructores 2025 a falta de seis carreras; primer doblete de títulos consecutivos del equipo desde 1990-1991]. Consultado el 25 de abril de 2026, desde <https://www.formula1.com/en/latest/article/russell-storms-to-victory-in-singapore-as-mclaren-seal-teams-championship.7cMITGtnc9fdxEaukmGVvI>
- Formula 1. (2025b, mayo). *Explained: What is the new two-stop rule for the Monaco Grand Prix and how does it work?* Consultado el 25 de enero de 2025, desde <https://www.formula1.com/en/latest/article/explained-what-is-the-new-two-stop-rule-for-the-monaco-grand-prix-and-how.1qropbdmR05S3BqgYsHtpE>
- Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine [Fundamento teórico del *gradient boosting*; sustenta la familia XGBoost/LightGBM/HistGBT empleada en N06/N12/N14/N15/N16]. *Annals of Statistics*, 29(5), 1189-1232. <https://doi.org/10.1214/aos/1013203451>
- Gal, Y., & Ghahramani, Z. (2016). Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning [Fundamento teórico del MC Dropout; base de la cuantificación de incertidumbre P10/P50/P90 en N09/N10 (degradación por compuesto)]. *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 1050-1059. <https://arxiv.org/abs/1506.02142>
- Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A Survey on Concept Drift Adaptation [Survey de referencia que cataloga la taxonomía clásica de deriva (súbita, gradual, incremental, recurrente) y emplea la estacionalidad como ejemplo arquetípico del régimen recurrente]. *ACM Computing Surveys*, 46(4), 1-37. <https://doi.org/10.1145/2523813>
- Gemma Team. (2025, marzo). *Gemma 3 Technical Report* [Familia de modelos abiertos multimodales de 1 a 27 mil millones de parámetros con contexto de hasta 128K tokens y entrenamiento en más de 140 idiomas; mencionada en Capítulo 3 como candidata para un eventual ajuste fino del razonador del orquestador sobre dominio F1, línea de extensión recogida en Capítulo 6]. arXiv: 2503.19786. Google DeepMind. <https://doi.org/10.48550/arXiv.2503.19786>
- GitHub. (2024a). *Dependabot Documentation* [Servicio integrado en GitHub que detecta vulnerabilidades en las dependencias declaradas y abre *pull requests* con su actualización; el servicio original fue

- adquirido por GitHub en mayo de 2019 e integrado de forma nativa en la plataforma a partir de 2020]. Consultado el 11 de mayo de 2026, desde <https://docs.github.com/en/code-security/dependabot>
- GitHub. (2024b). *GitHub Actions Documentation* [Plataforma nativa de integración y entrega continua de GitHub, basada en flujos de trabajo declarados en YAML bajo el directorio `.github/workflows/`; disponible de forma general desde noviembre de 2019. Empleada para los tres trabajos paralelos de *test*, *lint* y *typecheck* sobre cada *pull request*]. Consultado el 11 de mayo de 2026, desde <https://docs.github.com/en/actions>
- Gomathi L. (2025, julio). *What Makes F1 Circuits Truly Different? A Data Dive Into Unexpected Similarities* [Artículo divulgativo en Medium sobre la agrupación no supervisada de circuitos de la Fórmula 1 a partir de telemetría pública, publicado el 18 de julio de 2025; antecedente directo del clustering K-Means desarrollado en este trabajo]. Consultado el 30 de abril de 2026, desde <https://medium.com/@gomathi.l/what-makes-f1-circuits-truly-different-a-data-dive-into-unexpected-similarities-c61b7ee968b9>
- Google APIs. (2024). *release-please: automated releases based on conventional commits* [Herramienta de Google que infiere el incremento semántico (*patch*, *minor*, *major*) a partir de los mensajes de *commit* acumulados desde la última etiqueta, abre el *pull request* con las notas de versión y, al fusionarse, publica los artefactos del nuevo *release*; la versión 4 de su acción de GitHub se publicó en 2024]. Consultado el 11 de mayo de 2026, desde <https://github.com/googleapis/release-please>
- GrandPrix247. (2024). *How artificial intelligence helps Formula 1 teams with race analytics*. Consultado el 25 de enero de 2025, desde <https://www.grandprix247.com/2024/09/18/the-role-of-mathematical-computing-in-formula-1-how-ai-artificial-intelligence-helps-with-race-analytics/>
- Hamilton, W. L., Leskovec, J., & Jurafsky, D. (2016). Diachronic Word Embeddings Reveal Statistical Laws of Semantic Change [Paper canónico sobre desplazamiento semántico diacrónico medido con *word embeddings* sobre corpus históricos; empleado en cap. 2.2 como ejemplo arquetípico de deriva gradual]. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1489-1501. <https://doi.org/10.18653/v1/P16-1141>
- Harris, D. (2025, agosto). *How the Hungarian GP exposed fundamental design flaw with 2025 Ferrari F1 car* [Autor real verificado: Daniel Harris, publicado el 7 de agosto de 2025. Análisis técnico sobre el problema estructural de la SF-25 con el desgaste de plancha, arrastrado desde el inicio de la temporada 2025 y detonante de las medidas conservadoras (presiones, modos de motor) que Ferrari adoptó en Hungría tras la descalificación de Hamilton en China]. Consultado el 14 de mayo de 2026, desde <https://www.motorsportweek.com/2025/08/07/how-the-hungarian-gp-exposed-fundamental-design-flaw-with-2025-ferrari-f1-car/>
- He, P., Gao, J., & Chen, W. (2023). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing [DeBERTa-v3 con preentrenamiento estilo ELECTRA y *gradient-disentangled embedding sharing*; modelo descartado en N21 tras experimentos preliminares (304 M parámetros incapaces de converger sobre el corpus de 370 ejemplos etiquetados; resultado negativo documentado)]. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2111.09543>
- Heilmeier, A., Graf, M., Betz, J., & Lienkamp, M. (2020a). Application of Monte Carlo Methods to Consider Probabilistic Effects in a Race Simulation for Circuit Motorsport [Existe un Erratum publicado en <https://www.mdpi.com/2076-3417/10/17/5745> sobre el artículo original]. *Applied Sciences*, 10(12), 4229. <https://doi.org/10.3390/app10124229>
- Heilmeier, A., Thomaser, A., Graf, M., & Betz, J. (2020b). Virtual Strategy Engineer: Using Artificial Neural Networks for Making Race Strategy Decisions in Circuit Motorsport [Referencia para stint-length averages y fallback compound logic en N28 Pit Strategy Agent]. *SAE Technical Paper 2020-01-1413*. <https://doi.org/10.3390/app10217805>

- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory [Paper canónico de la arquitectura LSTM; referencia obligada al introducir las redes recurrentes con memoria a largo plazo en el marco teórico]. *Neural Computation*, 9(8), 1735-1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Holding, J. (2023, septiembre). *High-downforce vs power tracks in F1 – differences explained* [Categorización de prensa motorsport en *high-downforce* (Hungria, Imola, Singapur, España, Mónaco, Spa, Silverstone) frente a *power* (Monza, Bakú, Canadá, México, Austria); el autor reconoce expresamente que «no son términos oficiales»]. Motorsport.com. Consultado el 25 de enero de 2025, desde <https://www.motorsport.com/f1/news/high-downforce-power-tracks-f1-differences-explained/6130973/>
- Hovakimyan, G., & Bravo, J. M. (2024). Evolving Strategies in Machine Learning: A Systematic Review of Concept Drift Detection [Clave bib heredada *hinder_2024_conceptdrift*; autores reales segun Crossref: Gurgen Hovakimyan y Jorge Miguel Bravo (no Hinder et al.)]. *Information*, 15(12), 786. <https://doi.org/10.3390/info15120786>
- Hutto, C. J., & Gilbert, E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text [Modelo de sentimiento basado en léxico ponderado; baseline previo al fine-tuning de RoBERTa en N20]. *Proceedings of the Eighth International AAAI Conference on Weblogs and Social Media (ICWSM-14)*, 216-225. <https://ojs.aaai.org/index.php/ICWSM/article/view/14550>
- Jacobs, R. A., Jordan, M. I., Nowlan, S. J., & Hinton, G. E. (1991). Adaptive Mixtures of Local Experts. *Neural Computation*, 3(1), 79-87. <https://doi.org/10.1162/neco.1991.3.1.79>
- Johnson, N. L., Kotz, S., & Balakrishnan, N. (1995). *Continuous Univariate Distributions* (2.^a ed., Vol. 2) [Tratado de referencia sobre familias de distribuciones continuas univariadas; capítulo 26 dedicado a la distribución triangular empleada para muestrear las vueltas restantes hasta el *cliff* y la duración de la parada en boxes]. Wiley. <https://www.worldcat.org/isbn/9780471584940>
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree [LightGBM; empleado en N12 adelantamiento, N14 coche de seguridad y N16 undercut]. *Advances in Neural Information Processing Systems 30 (NIPS)*, 3146-3154. <https://papers.nips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>
- Kim, H., Kim, S., Mok, J., & Yoon, S. (2025). TAFAS: Battling the Non-stationarity in Time Series Forecasting via Test-time Adaptation [Aceptado en AAAI 2025; método candidato considerado en el plan de N09 sin llegar a implementarse]. *Proceedings of the AAAI Conference on Artificial Intelligence*. <https://doi.org/10.48550/arXiv.2501.04970>
- Kim, T., Kim, J., Tae, Y., Park, C., Choi, J.-H., & Choo, J. (2022). Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift [Partial RevIN applied in N06 lap-time prediction v5 experiment]. *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=cGDAkQo1C0p>
- Kraus, A., & van der Aa, H. (2025). Machine learning-based detection of concept drift in business processes [Clave bib heredada *adams_2025_driftbusiness*; autores reales: Kraus y van der Aa (no Adams). El paper propone CV4CDD-4D (computer vision para concept drift en process mining), no es referencia foundational de ADWIN ni DDM]. *Process Science*, 2, 5. <https://doi.org/10.1007/s44311-025-00012-w>
- LangChain Inc. (2024). *LangGraph: Stateful, orchestration framework for building agents* [Framework de orquestación de agentes empleado para construir los sub-agentes ReAct N25–N30 y el supervisor del orquestador N31]. <https://www.langchain.com/langgraph>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [Paper canónico de RAG; fundamento del subsistema N30 de recuperación sobre el

- reglamento deportivo FIA]. *Advances in Neural Information Processing Systems 33 (NeurIPS)*. <https://arxiv.org/abs/2005.11401>
- Liu, O., Fu, D., Yogatama, D., & Neiswanger, W. (2024). *DeLLMa: Decision Making Under Uncertainty with Large Language Models* [Framework de decisión bajo incertidumbre referenciado en N31]. arXiv: 2402.02392. <https://arxiv.org/abs/2402.02392>
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach* [RoBERTa-base, backbone del modelo de sentimiento de radio (*fine-tune* en N20, accuracy real 0,84 sobre test 2025; el 87,5 % del header del notebook esta desactualizado)]. arXiv: 1907.11692. <https://arxiv.org/abs/1907.11692>
- Liu, Y., et al. (2023). *Handling Concept Drift in Global Time Series Forecasting* [arXiv preprint; propone Error Contribution Weighting y Gradient Descent Weighting sobre dos modelos entrenados con datos recientes vs. totales. Clave bib heredada `liu_2023_gfnormalization` pese a que el título real no menciona normalización]. arXiv: 2304.01512. <https://arxiv.org/abs/2304.01512>
- Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic Gradient Descent with Warm Restarts. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1608.03983>
- Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions [SHAP (*SHapley Additive exPlanations*); herramienta de interpretabilidad por descomposición aditiva basada en valores de Shapley empleada sobre los modelos de *gradient boosting* del sistema]. *Advances in Neural Information Processing Systems 30 (NIPS)*, 4765-4774. <https://arxiv.org/abs/1705.07874>
- Machine Learning at Cambridge Laboratory. (2019). *Machine Learning for Modelling Formula One Races* [Proyecto del grupo de aprendizaje automático de la Universidad de Cambridge (Carl Henrik Ek, Ferenc Huszár y Neil Lawrence). Año estimado a partir de la actividad del grupo, ya que la página no fecha la publicación. Citado en este trabajo únicamente para acreditar la existencia de la colaboración Cambridge-equipo de F1 mencionada en el estado del arte]. Consultado el 25 de enero de 2025, desde <https://mlatcl.github.io/internal/projects/machine-learning-for-modelling-formula-one-races.html>
- Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs [Algoritmo HNSW sobre el que Qdrant construye el índice aproximado usado por el RAG del proyecto]. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824-836. <https://doi.org/10.1109/TPAMI.2018.2889473>
- Marr, B. (2023). *How Artificial Intelligence, Data And Analytics Are Transforming Formula One in 2023*. Consultado el 25 de enero de 2025, desde <https://bernardmarr.com/how-artificial-intelligence-data-and-analytics-are-transforming-formula-one-in-2023/>
- McLaren Racing. (2024). *How AI is revolutionising F1* [Contenido presentado por Dell Technologies, publicado el 15 de agosto de 2024]. Consultado el 25 de enero de 2025, desde <https://www.mclaren.com/racing/partners/dell-technologies/how-ai-is-revolutionising-f1-presented-by-dell-technologies/>
- Meyes, R., Lu, M., de Puiseau, C. W., & Meisen, T. (2019). Ablation Studies in Artificial Neural Networks [Formalización metodológica del estudio de ablación aplicado a redes neuronales artificiales: se modifica o retira un único componente del modelo manteniendo el resto constante, para aislar su contribución al rendimiento]. <https://doi.org/10.48550/arXiv.1901.08644>
- Noble, J., & Somerfield, M. (2023, septiembre). *The four key concept changes that transformed McLaren's MCL60 F1 car* [Análisis técnico publicado el 28 de septiembre de 2023 sobre las cuatro modificaciones aerodinámicas y conceptuales que reformularon el MCL60 a partir del paquete de actualizaciones del Gran Premio de Austria; empleado en cap. 2.2 como evidencia de la transformación de rendimiento de McLaren a lo largo de la temporada 2023]. Motorsport.com. Consultado el 25 de abril de 2026, desde <https://www.motorsport.com/f1/news/the-four-key-concept-changes-that-transformed-mclarens-mcl60-f1-car/10525546/>

- Oehrly, P. (2024). *FastF1: A Python package for accessing and analyzing Formula 1 results, schedules, timing data and telemetry* [Biblioteca open source de Python utilizada como fuente principal de telemetría histórica, tiempos por vuelta y mensajes de dirección de carrera para las temporadas 2023–2025 del corpus]. Consultado el 29 de abril de 2026, desde <https://github.com/theOehrly/Fast-F1>
- OpenAI. (2024). *Whisper: Robust Speech Recognition via Large-Scale Weak Supervision* [Repositorio oficial; tabla comparativa de modelos disponibles, requisitos de VRAM y velocidad relativa en el README de la rama main. La variante turbo se publicó en octubre de 2024.]. Consultado el 27 de abril de 2026, desde <https://github.com/openai/whisper>
- OpenF1 Project. (2024). *OpenF1: Real-time and historical Formula 1 data API* [API REST y WebSocket abierta sobre datos de la Fórmula 1; complementa a FastF1 con cobertura específica de comunicaciones de radio piloto-muro y eventos de sesión, base del flujo NLP del sistema]. Consultado el 29 de abril de 2026, desde <https://openf1.org>
- Partridge, J. (2025). *How Does An F1 Wind Tunnel Work?* [Autor real verificado: Jarrod Partridge en F1 Chronicle]. Consultado el 25 de enero de 2025, desde <https://f1chronicle.com/how-does-an-f1-wind-tunnel-work-f1-technology/>
- Pirelli Motorsport. (2025a). *2025 compounds: something new for Jeddah* [Nominación oficial de compuestos C1–C5 para la temporada 2025; clasifica implícitamente Suzuka y Sakhir como dos de los circuitos más exigentes con el neumático y reservó los compuestos más blandos para Australia]. Consultado el 26 de abril de 2026, desde <https://press.pirelli.com/2025-compounds-something-new-for-jeddah/>
- Pirelli Motorsport. (2025b). *F1 Technical Insights and Race Previews* [Portal oficial de prensa; utilizado para mapeo de compuestos C1–C5 por GP]. Consultado el 25 de enero de 2025, desde <https://press.pirelli.com/?h=1&t=formula+1>
- Platt, J. C. (1999). Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods [Calibración de Platt; ajuste sigmoide sobre los *scores* de un clasificador no calibrado mediante una regresión logística de un parámetro. Aplicada en N12, N14 y N16 sobre el conjunto de validación 2024 para convertir las salidas de los modelos LightGBM en probabilidades calibradas]. En A. J. Smola, P. Bartlett, B. Schölkopf & D. Schuurmans (Eds.), *Advances in Large Margin Classifiers* (pp. 61-74). MIT Press. <https://mitpress.mit.edu/9780262194488/advances-in-large-margin-classifiers/>
- Qureshi, N. (2025, marzo). *Ranking Formula 1 Tracks/Circuits by Similarity* [Autor real Noor Qureshi (no Nabeel); publicado el 28 de marzo de 2025]. INST414 Data Science Techniques, Medium. Consultado el 25 de enero de 2025, desde <https://medium.com/inst414-data-science-tech/ranking-formula-1-tracks-circuits-by-similarity-43fc0bcb9c34>
- Raceteq. (2024). *CFD in F1: How does Computational Fluid Dynamics drive performance?* Consultado el 25 de enero de 2025, desde <https://www.raceteq.com/articles/2024/10/cfd-performance-in-f1>
- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision* [Whisper; transcripción automática de radio de equipo en N18]. arXiv: 2212.04356. OpenAI. <https://arxiv.org/abs/2212.04356>
- Regan, H. (2024). *How F1 teams are turning to AI to improve performance on the track*. CNBC Technology. Consultado el 25 de enero de 2025, desde <https://www.cnbc.com/2024/12/24/how-f1-teams-are-turning-to-ai-to-improve-performance-on-the-track.html>
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks [Fundamento conceptual de los *embeddings* de oración; antecedente directo de BGE-M3 y de SetFit]. *Proceedings of EMNLP-IJCNLP*, 3982-3992. <https://arxiv.org/abs/1908.10084>
- Reynolds, D. A. (2009). Gaussian Mixture Models [Revisión de modelos de mezcla gaussiana; alternativa probabilística considerada y descartada para el agrupamiento de circuitos por el coste de estimar matrices de covarianza con apenas 24 muestras en un espacio de 14 dimensiones]. En S. Z. Li &

- A. Jain (Eds.), *Encyclopedia of Biometrics* (pp. 659-663). Springer US. https://doi.org/10.1007/978-0-387-73003-5_196
- Sasikumar, A., Anny Leema, A., & Balakrishnan, P. (2025). Data-driven pit stop decision support for Formula 1 using deep learning models [Vellore Institute of Technology (India). Indexado en PMC: PMC12626961]. *Frontiers in Artificial Intelligence*, 8. <https://doi.org/10.3389/frai.2025.1673148>
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1701.06538>
- Siemens Digital Industries. (2021). *Racing team uses Simcenter STAR-CCM+ and Fibersim to support continuous innovation – Renault F1 Team*. Consultado el 25 de enero de 2025, desde <https://resources.sw.siemens.com/en-US/case-study-renault-formula-one/>
- Snowflake Inc. & Streamlit contributors. (2025). *Streamlit: A faster way to build and share data apps* [Marco de trabajo de código abierto en Python para la construcción de aplicaciones interactivas de datos y aprendizaje automático sin necesidad de desarrollo *front-end* dedicado]. Consultado el 13 de mayo de 2026, desde <https://streamlit.io/>
- Tjong Kim Sang, E. F., & De Meulder, F. (2003). Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition [Esquema de etiquetado BIO y *benchmark* original de NER que antecede al fine-tuning F1 custom en N22]. *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, 142-147. <https://doi.org/10.3115/1119176.1119195>
- Tunstall, L., Reimers, N., Jo, U. E. S., Bates, L., Korat, D., Wasserblat, M., & Pereg, O. (2022). Efficient Few-Shot Learning Without Prompts [Framework SetFit utilizado en N21 intent classification]. *Findings of EMNLP*. <https://arxiv.org/abs/2209.11055>
- Urdhwaresh, A. (2025, abril). *The Use of Machine Learning in Predicting Formula 1 Race Outcomes* [Preprint enviado el 18 de abril de 2025; aplica TabNet a datos de las temporadas 2010–2023 para predecir la posición final del piloto ($R^2 = 0,75$, RMSE=2,87) y los puntos del constructor ($R^2 = 0,71$); referenciado en el panorama académico de Capítulo 3]. <https://doi.org/10.20944/preprints202504.1471.v1>
- Vega Sobral, V. (2025, mayo). *F1 Strategy Manager AI: Integrating AI and Rule-Based Systems for Strategic Decision-Making in Formula 1* (Informe final de asignaturas, Grado en Ingeniería en Sistemas Inteligentes) (Informe en formato artículo, no publicado. Versión preliminar en solitario desarrollada como proyecto conjunto de las asignaturas *Procesamiento del Habla y del Lenguaje Natural*, *Aprendizaje Automático Avanzado*, *Visión por Computador* y *Sistemas Inteligentes* (2.º cuatrimestre de 3.º, curso 2024–2025). Cierre: 9 de mayo de 2025). Universidad Intercontinental de la Empresa (UIE), Facultad de Ingeniería y Tecnología Empresarial. A Coruña, España. https://github.com/VforVitorio/F1_Strat_Manager/blob/main/documents/docs_legacy_strat_manager/F1_Strategy_Manager_AI.pdf
- Vega Sobral, V., & Souto Ortega, S. (2025, noviembre). F1 Telemetry Manager: análisis multimodal de telemetría de Fórmula 1 con interfaz conversacional [Coautoría: V. Vega Sobral (análisis de telemetría, chat multimodal, integración de modelos) y S. Souto Ortega (almacenamiento en la nube y gestión de usuarios, componentes no heredados en el TFG). Incorporado al TFG como submódulo Git en `src/telemetry/`. Cierre: 29 de noviembre de 2025]. https://github.com/VforVitorio/F1_Strat_Manager
- Vela, D., Sharp, A., Zhang, R., Nguyen, T., Hoang, A., & Pinykh, O. S. (2022). Temporal Quality Degradation in AI Models [Estudio empírico revisado por pares en *Nature Scientific Reports* que documenta la degradación temporal de modelos de IA en producción en cuatro sectores y discute la disrupción de la pandemia de COVID-19 como fuente de deriva súbita]. *Scientific Reports*, 12(1), 11654. <https://doi.org/10.1038/s41598-022-15245-z>

- Wang, J., Wang, J., Athiwaratkun, B., Zhang, C., & Zou, J. (2024). *Mixture-of-Agents Enhances Large Language Model Capabilities* [Patrón de agregación de razonamiento referenciado en N31 Strategy Orchestrator]. arXiv: 2406.04692. <https://arxiv.org/abs/2406.04692>
- Warner, B., Chaffin, A., Clavié, B., Weller, O., Hallström, O., Taghadouini, S., Gallagher, A., Biswas, R., Ladhak, F., Aarsen, T., Cooper, N., Adams, G., Howard, J., & Poli, I. (2024). *Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference* [ModernBERT-base backbone de SetFit en N21]. arXiv: 2412.13663. <https://arxiv.org/abs/2412.13663>
- Williams Racing. (2026). *Anthropic and Atlassian Williams F1 Team announce multi-year partnership, naming Claude the team's «Official Thinking Partner»*. Consultado el 20 de abril de 2026, desde <https://www.williamsf1.com/articles/0f1eece4-e632-49c9-a767-148159bbbf87/anthropic-atlassian-williams-f1-team-multi-year-partnership-claude-official-thinking-partner>
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation* [Selective tool invocation referenciado como inspiración del router condicional de N31]. arXiv: 2308.08155. <https://arxiv.org/abs/2308.08155>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models* [Patrón ReAct usado en los sub-agentes N25–N30 vía `create_react_agent` de LangGraph]. arXiv: 2210.03629. <https://arxiv.org/abs/2210.03629>
- Zaratiana, U., Tomeh, N., Holat, P., & Charnois, T. (2024). GLiNER: Generalist Model for Named Entity Recognition using Bidirectional Transformer. *Proceedings of NAACL*. <https://arxiv.org/abs/2311.08526>
- Zhao, Z., Liu, H., & Prakash, B. A. (2025). *ShiftS: Tackling Time-Series Forecasting Generalization via Mitigating Concept Drift* [arXiv preprint, enviado 16 oct. 2025; método candidato considerado en el plan de N09 sin llegar a implementarse]. arXiv: 2510.14814. <https://doi.org/10.48550/arXiv.2510.14814>

8 ANEXOS

8.1 Anexo 1 Declaración de uso de IAG

Declaración de Uso de IA Generativa en la Elaboración del TFG/TFM

Apellidos	Vega Sobral	Nombres	Víctor
Titulación	Grado en Ingeniería en Sistemas Inteligentes	GISI	

Título de trabajo

Sistema Multi-Agente Inteligente para Análisis de Telemetría y Optimización de Estrategias en Carreras

1. Objetivo de la Declaración.

El presente formulario tiene como finalidad dejar constancia del uso de herramientas de Inteligencia Artificial Generativa (IAG) durante el proceso de elaboración del Trabajo Fin de Grado (TFG) o Trabajo Fin de Máster (TFM), garantizando la transparencia, el uso ético y la integridad académica.

2. He utilizado herramientas de IA Generativa en este trabajo.

SÍ

☒

NO

☐

Si marcas **SÍ**, debes completar las siguientes partes de esa declaración

3. Uso Ético y Responsable.

3.1 Protección de Datos.

☒

Mi utilización de la herramienta de IA Generativa ha respetado sus términos de uso.

☒

Declaro que no he introducido datos personales sensibles de terceros en herramientas de IAG.

☒

He utilizado datos personales únicamente de forma anonimizada o sintética.

☒

En caso de haber tratado datos reales, explico las medidas implementadas para protegerlos:

Pese a tratarse de datos reales, todos ellos están accesibles a través de fuentes de datos públicas, y en ningún caso se tratan de datos confidenciales de los equipos de Fórmula 1, sino de datos extraídos de la propia retransmisión de la carrera. Por tanto, no se han implementado medidas para protegerlos por su naturaleza no confidencial. Pese a ello, marco esta opción para que quede presente la transparencia a la hora de tratar estos datos durante la elaboración del proyecto.

3.2 Responsabilidad Académica.

☒

Declaro que el contenido final del TFG/TFM es de mi autoría y que la IAG ha sido utilizada como herramienta de apoyo.

☒

Asumo la responsabilidad de verificar, corregir y validar toda la información generada mediante IAG.

☒

Declaro haber citado y reconocido explícitamente el uso de herramientas de IAG en el trabajo.

Declaración de Uso de IA Generativa en la Elaboración del TFG/TFM

4. Uso Técnico de Herramientas de IA Generativa.

4.1 Apoyo en Escritura y Redacción.

- ☐ Revisión ortográfica y gramatical de textos.
- ☒ Revisión o mejora de redacción de párrafos.
- ☒ Reformulación de ideas manteniendo el contenido original.
- ☒ Generación de resúmenes o esquemas preliminares.

4.2 Documentación y Consulta.

- ☒ Solicitud de explicaciones conceptuales.
- ☒ Búsqueda de referencias preliminares.
- ☒ Respuestas a preguntas concretas técnicas o teóricas.

4.3 Diseño y Metodología.

- ☒ Propuesta de Metodologías de Diseño y/o Desarrollo.
- ☐ Tratamiento de datos: generación, recogida, análisis, cruce de datos...
- ☒ Análisis de Resultados.
- ☒ Procesos de optimización.
- ☒ Generación de imágenes, esquemas, diagramas o visualizaciones.

4.4 Generación de Código.

- ☒ Generación de fragmentos de código.
- ☒ Depuración o explicación de errores.
- ☒ Propuestas de algoritmos o estructuras.
- ☒ Generación de módulos, bibliotecas y/o APIs.

Declaración de Uso de IA Generativa en la Elaboración del TFG/TFM

5. Herramientas Utilizadas.

☒ ChatGPT	☐ Perplexity	☐ Meta Llama
☐ Gemini	☒ Claude	☐ Semantic Scholar
☐ Copilot	☐ DeepSeek	☐ Scite.ai

El sistema desarrollado en este Trabajo integra modelos de IA generativa como componentes propios del objeto de estudio, por lo que conviene distinguir dos modalidades de uso no recogidas explícitamente en los apartados anteriores. Como apoyo a la elaboración de la memoria y del código, además de las herramientas marcadas (ChatGPT y Claude, esta última también vía la interfaz CU

6. Reflexiones y aclaraciones.

El sistema desarrollado en este Trabajo integra modelos de IA generativa como componentes propios del objeto de estudio, por lo que conviene distinguir dos modalidades de uso no recogidas explícitamente en los apartados anteriores. Como apoyo a la elaboración de la memoria y del código se han empleado las herramientas marcadas (ChatGPT y Claude), esta última también a

6. Declaración Final.

Declaro que la información aquí consignada es veraz y que el uso de IA generativa ha sido realizado conforme a los principios éticos, legales y académicos exigidos por la UIE.

Firma del estudiante: _____ Fecha: _____

Firmado por VEGA SOBRAL VICTOR - ***5591** el día 16/05/2026 con un certificado emitido por AC FNMT Usuarios

8.2 Anexo 2 Manual de instalación, despliegue y uso

Este anexo recoge el procedimiento operativo para que un lector técnico pueda clonar el repositorio, instalar dependencias, descargar artefactos y ejecutar el sistema sin contactar al autor. Las rutas relativas que aparecen toman como raíz el repositorio de código `F1_Strat_Manager`.

8.2.1 II.1 Requisitos del sistema

El entorno de desarrollo del autor combina un equipo portátil con una NVIDIA GeForce RTX 5070 Laptop de 8 GB de *VRAM* GDDR7, un procesador AMD Ryzen AI 9 365 y 32 GB de memoria RAM DDR5, configuración que permite servir el modelo *Whisper turbo* en GPU con margen suficiente para los modelos NLP en memoria. Como mínimo realista para ejecutar el sistema en modo CPU se recomiendan 16 GB de RAM y 30 GB de almacenamiento, considerando que los pesos y datos descargados ocupan en torno a 15 GB. La aceleración por GPU es opcional pero acorta tiempos de inferencia en *Whisper* y en los componentes transformer del agente de radio.

A nivel de *software* se exige Python 3.10 o 3.11, dado que la rueda CUDA del proyecto y varias dependencias del *stack* ML aún no soportan Python 3.12 o superior. Se necesitan Git y Git LFS para clonar el conjunto de datos desde Hugging Face. Docker con Compose es opcional y se reserva al despliegue contenedorizado del *backend* FastAPI y el *frontend* Streamlit. Node.js 18 o superior solo es necesario si se desea recompilar el componente React de visualización de audio del *frontend* Streamlit, no para el flujo normal de uso.

En cuanto a credenciales externas, el sistema admite dos proveedores de modelo grande de lenguaje. La opción de referencia es OpenAI a través de la variable `OPENAI_API_KEY`, con `gpt-4.1-mini` para los sub-agentes especializados N25–N30 y `gpt-5.4-mini` para el orquestador N31 y el chat conversacional del *backend*. Como alternativa local se acepta el proveedor `lmstudio` apuntando a un servidor LM Studio en `http://localhost:1234/v1`. Cabe matizar que, aunque el sistema soporta el proveedor `lmstudio`, las familias de modelos que se ensayaron en su día (Llama, Qwen y Gemma) no se incorporaron como capa de razonamiento en la entrega final, de modo que la baseline de producción combina `gpt-4.1-mini` para los sub-agentes y `gpt-5.4-mini` para la capa de síntesis del orquestador.

8.2.2 II.2 Instalación con uv, flujo recomendado

El gestor `uv` es el camino recomendado por dos motivos. Resuelve la rueda CUDA `cu128` de PyTorch automáticamente a través de la sección `[tool.uv.sources]` del `pyproject.toml` y reproduce el *lockfile* de forma determinista. El flujo completo desde una máquina limpia es el siguiente.

```
# 1. Instalar uv (Windows PowerShell)
irm https://astral.sh/uv/install.ps1 | iex

# 2. Clonar el repositorio
git clone https://github.com/VforVitorio/F1-StratLab.git
cd F1-StratLab

# 3. Sincronizar el entorno (resuelve cu128 automaticamente)
uv sync --all-extras

# 4. Ejecutar el asistente interactivo
uv run f1-strat
```

El flag `-all-extras` instala los grupos opcionales dev (*pytest, ruff, mypy y jupyter*) y voice (*edge-tts, ffmpeg-python*) declarados en `[project.optional-dependencies]` del `pyproject.toml`. Para un despliegue mínimo orientado solo al uso del sistema basta con `uv sync -extra voice`, que omite la cadena de herramientas de desarrollo. Tras `uv sync` quedan disponibles cuatro puntos de entrada de consola, descritos en la sección II.8.

8.2.3 II.3 Instalación como herramienta global

Si el lector solo desea ejecutar el sistema sin trabajar sobre el código, `uv tool install` instala la última versión publicada en su propio entorno aislado y deja los ejecutables en el `PATH`.

```
uv tool install "git+https://github.com/VforVitorio/F1-StratLab.git"
```

```
f1-strat      # asistente interactivo
f1-sim        # simulacion CLI no interactiva
f1-arcade     # tres ventanas pygame + PySide6
f1-streamlit  # cuadro de mando Streamlit
```

En esta modalidad, el primer arranque dispara la descarga única de modelos y datos cacheados bajo `~/f1-strat/`. Los arranques siguientes operan sin red.

8.2.4 II.4 Alternativa con pip y entorno virtual

Para entornos donde `uv` no esté disponible se ofrece un flujo basado en `pip` y `venv`. Conviene avisar que `pip` no lee `[tool.uv.sources]` y, por tanto, conviene instalar PyTorch a mano contra el índice CUDA antes del proyecto.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# Instalar PyTorch con CUDA primero (ajustar a la version local)
pip install torch --index-url https://download.pytorch.org/whl/cu128

# Luego el proyecto en modo editable
pip install -e .
```

8.2.5 II.5 Despliegue con Docker Compose

El repositorio incluye dos archivos `docker-compose.yml`. El de la raíz arranca un par mínimo de servicios, backend en el puerto 8000 y frontend en el 8501, y es suficiente para una demostración. El segundo, ubicado en `src/telemetry/`, añade montajes de volumen para recarga de código en vivo y acceso a los datos. Este segundo *compose* monta `../src` y `../data` en modo solo lectura y deja `data/rag` en modo escritura para que el agente N30 pueda actualizar el índice Qdrant.

```
# Despliegue minimo, desde la raiz
docker-compose up --build
```

Tabla 8.1: Variables de entorno reconocidas por el sistema

Variable	Obligatoria	Valor por defecto	Descripción
BACKEND_URL	No	http://localhost:8000	URL del <i>backend</i> usada por el <i>frontend</i> Streamlit. El módulo Arcade lee la variable equivalente F1_BACKEND_URL.
FRONTEND_URL	No	http://localhost:8501	URL del <i>frontend</i> usada para CORS.
F1_LLM_PROVIDER	No	lmstudio	Proveedor del LLM, valores lmstudio u openai.
OPENAI_API_KEY	Si provider=openai	sin valor	Clave de API de OpenAI.
F1_STRAT_DATA_ROOT	No	<repo>/data	Sobrescribe el directorio de datos.
F1_STRAT_OFFLINE	No	0	Si vale 1, suprime cualquier descarga remota.
F1_STRAT_NO_FIRST_RUN	No	0	Si vale 1, suprime el banner y las barras de progreso del primer arranque.

```
# Despliegue completo con volúmenes
cd src/telemetry
docker-compose up --build
```

El *frontend* llama al *backend* mediante el nombre de servicio Docker `http://backend:8000`. Si se desea integrar LM Studio dentro del despliegue, se accede al *host* con `http://host.docker.internal:1234/v1`.

8.2.6 II.6 Variables de entorno

Las variables que controlan rutas, proveedores y comportamiento de primer arranque se definen en un archivo `.env` situado en la raíz del repositorio. La Tabla 8.1 resume las que reconoce el sistema, consolidando lo documentado en `docs/pages/setup.md` y en el README del paquete `src/f1_strat_manager`.

8.2.7 II.7 Descarga del conjunto de datos y de los modelos

Los pesos entrenados, el corpus de radio y los *parquets* de carrera se publican en el conjunto `VforVitorio/f1-strategy-dataset` de Hugging Face, con un tamaño total cercano a 15 GB. Existen dos formas de poblar el árbol local.

En instalación clonada, el módulo `data_cache.py` ubicado en `src/f1_strat_manager/` descarga los artefactos vía `huggingface_hub` la primera vez que se ejecuta `f1-strat` o `f1-sim`. Los modelos se descargan de forma temprana porque el orquestador los necesita en arranque, mientras que el audio de radios y los volcados de FastF1 se traen perezosamente por carrera cuando el usuario simula ese Gran Premio. En instalación global vía `uv tool install`, el árbol equivalente queda bajo `~/.f1-strat/`.

Si se prefiere poblar la caché manualmente, basta con clonar el repositorio del *dataset* con Git LFS habilitado.

```
git lfs install
```

```
git clone https://huggingface.co/datasets/VforVitorio/f1-strategy-dataset data
```

El árbol esperado bajo data/ se muestra a continuación.

```
data/
  raw/2025/<GP>/laps.parquet
  processed/laps_featured_2025.parquet
  models/lap_time/                # N06 XGBoost
  models/tire_degradation/        # N09/N10 TireDegTCN
  models/overtake_probability/    # N12 LightGBM
  models/safety_car_probability/  # N14 LightGBM
  models/pit_prediction/          # N15 HistGBT + N16 undercut
  models/nlp/                     # pipeline_config_v1.json
  models/agents/                  # configuracion JSON por agente
  rag/                            # indice Qdrant FIA
  tire_compounds_by_race.json
```

8.2.8 II.8 Primera ejecución, tres caminos

El sistema expone tres superficies de uso sobre la misma base de código. La elección depende del caso de uso: inspección rápida desde terminal, visualización inmersiva o cuadro de mando posterior a la carrera.

8.2.8.1 Camino 1, CLI con Rich

La interfaz de consola ofrece dos modos. El asistente interactivo `f1-strat` despliega un menú con flechas para elegir Gran Premio, piloto, equipo y proveedor de LLM. La forma no interactiva `f1-sim` acepta los mismos parámetros como argumentos posicionales.

```
# Asistente interactivo
uv run f1-strat

# Forma no interactiva, todas las vueltas
uv run f1-sim Melbourne NOR McLaren --no-llm

# Rango de vueltas y proveedor explícito
uv run f1-sim Melbourne NOR McLaren --laps 15-35 --provider openai
```

La opción `-no-llm` omite la capa de síntesis del orquestador y muestra solo las salidas ML y de Monte Carlo, útil para iteraciones rápidas. La opción `-provider` acepta los valores `openai` y `lmstudio`. La salida en vivo renderiza una tabla Rich con columnas para vuelta, compuesto, vida del neumático, acción recomendada, confianza, escenarios STAY/PIT/UDCT/OVCT y razonamiento del orquestador.

8.2.8.2 Camino 2, modo Arcade de tres ventanas

El modo *arcade* reproduce la carrera en 2D y abre simultáneamente un cuadro de mando estratégico y una ventana de telemetría. El proceso *arcade* es dueño del bucle de simulación y publica el estado en el puerto TCP 127.0.0.1:9998. La ventana de *dashboard* y la de telemetría se suscriben a ese flujo.

```
python -m src.arcade.main --viewer --year 2025 --round 3 \
    --driver VER --team "Red Bull Racing" \
    --driver2 LEC --strategy
```

Los controles de teclado están gestionados por `F1ArcadeView.on_key_press`. Space pausa o reanuda la reproducción, las flechas izquierda y derecha avanzan o retroceden una vuelta, las teclas + y - modifican la velocidad y Escape devuelve al menú o cierra la aplicación si se arrancó con `-viewer`. Omitiendo `-driver2` se renderiza un único piloto y la traza Delta queda vacía. La bandera `-strategy` solo está soportada en 2025, dado que el *pipeline* multiagente se entrena con las *features* de esa temporada.

8.2.8.3 Camino 3, Streamlit más FastAPI

El cuadro de mando posterior a la carrera se sirve en dos procesos. El *backend* FastAPI expone los *endpoints* REST de los agentes y el *frontend* Streamlit consume esa API. Conviene levantarlos en dos terminales.

```
# Terminal 1, backend FastAPI
cd src/telemetry
uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload

# Terminal 2, frontend Streamlit
cd src/telemetry
streamlit run frontend/app/main.py --server.port 8501
```

El *backend* ofrece Swagger UI en `http://localhost:8000/docs` con los *endpoints* `/api/v1/strategy/pace`, `/tire`, `/situation`, `/pit`, `/radio`, `/rag` y `/recommend`, complementados por *routers* adicionales para telemetría, chat conversacional, modo voz, comparación entre pilotos y análisis de dominio de circuito. El *frontend* se abre en `http://localhost:8501` y reúne las páginas de análisis posterior a la carrera, chat estratégico, laboratorio de modelos y modo voz.

8.2.9 II.9 Inventario de scripts en scripts

El directorio `scripts/` contiene los puntos de entrada de línea de comandos y herramientas de mantenimiento. La Tabla 8.2 resume los más relevantes para el uso operativo del sistema. Existen además scripts con prefijo `bench_` (`bench_nlp_pipeline_cpu.py`, `bench_pace_baselines.py`, `bench_subagent_latency.py`, `bench_whisper.py`) que reproducen las métricas del Capítulo 5 y no son necesarios para el uso normal del sistema.

8.2.10 II.10 Resolución de incidencias frecuentes

La Tabla 8.3 recoge los problemas más habituales detectados durante el desarrollo y su corrección. Conviene revisar primero esta tabla antes de profundizar en *logs*.

Tabla 8.2: Scripts disponibles en scripts/

Script	Propósito
run_simulation_cli.py	Simulación completa con tabla Rich y RaceReplayEngine. Acepta -laps, -no-llm, -provider y -whisper-model.
debug_agent.py	Ejecuta un agente aislado contra datos reales de Melbourne 2025 con <i>fallback</i> sintético. Soporta -override sobre el <i>lap state</i> .
download_fia_pdfs.py	Descarga los PDF del reglamento deportivo FIA desde el sitio oficial. Prerrequisito de build_rag_index.py.
build_rag_index.py	Construye el índice Qdrant a partir de los PDF de reglamento deportivo FIA. Necesario antes del primer uso del agente N30.
build_radio_dataset.py	Genera el corpus estático de radios por Gran Premio que consume ensure_radio_corpus.
download_data.py	Descarga manual del árbol data/ desde Hugging Face.
f1_cli.py	Implementación del asistente interactivo f1-strat.
run_streamlit.py	Lanzador del <i>frontend</i> Streamlit equivalente al ejecutable f1-streamlit.

Tabla 8.3: Incidencias frecuentes y su resolución

Síntoma	Causa probable	Resolución
FileNotFoundError sobre laps.parquet	Caché FastF1 sin construir	Ejecutar una vez sin -strategy o forzar python -m src.simulation.builder 2025 3.
OpenAI api_key missing	Falta .env o variable	Añadir OPENAI_API_KEY al .env o exportar F1_LLM_PROVIDER=lmstudio.
No module named pyqtgraph	uv sync incompleto	Repetir uv sync desde la raíz del repositorio.
"Backend offline. ^{en} el dashboard	El arcade no levantó el servidor TCP	Reiniciar con la bandera -strategy.
Puerto 9998 en uso	Otro proceso retiene el puerto	Cerrar el proceso colgado o reiniciar la máquina.
-strategy ignorado fuera de 2025	El pipeline multiagente solo soporta 2025	Limitar las ejecuciones con -strategy a la temporada 2025.
Carga DLL de scikit-learn en Windows	Conflicto con instalaciones antiguas del entorno base	Reinstalar dentro del .venv aislado o usar uv en lugar de pip global.
AttributeError: collections.Mapping	frozendict antiguo	Forzar frozendict>=2.4.0 en el entorno activo.

8.3 Anexo 3 Material complementario

Este anexo recoge material complementario al cuerpo de la memoria y se organiza en dos bloques. El primero documenta el inventario íntegro de cuadernos Jupyter del repositorio del sistema, que constituyen la documentación operativa detrás de cada modelo y experimento referenciado en los capítulos anteriores. El segundo recoge los diagramas completos de arquitectura cuyas versiones simplificadas aparecen en el Capítulo 4.

8.3.1 III.1 Mapa de notebooks del proyecto

Las afirmaciones del Capítulo 4 y del Capítulo 5 se respaldan en los cuadernos Jupyter del repositorio del sistema, donde cada modelo y experimento se documenta paso a paso con análisis exploratorio completo, registro de hiperparámetros, gráficas intermedias y discusión más extensa que la admitida por el formato de la memoria. Las Tabla 8.4, Tabla 8.5 y Tabla 8.6 reúnen el inventario completo agrupado por capa funcional. Cada bloque indica su ruta raíz dentro del repositorio y la columna *Ruta* contiene únicamente el nombre del fichero `.ipynb`. Las cifras clave remiten al Capítulo 5 y se cruzan literalmente con los valores reportados en las tablas y prosa de ese capítulo.

8.3.2 III.2 Diagramas completos del sistema

Las versiones simplificadas de los diagramas que aparecen en el cuerpo de la memoria omiten parte de los detalles operativos para no recargar la exposición. Esta subsección recoge las versiones completas, exportadas desde los ficheros `.drawio` versionados en el repositorio del sistema, para que el lector pueda consultar la arquitectura con la granularidad que cada diagrama soporta.

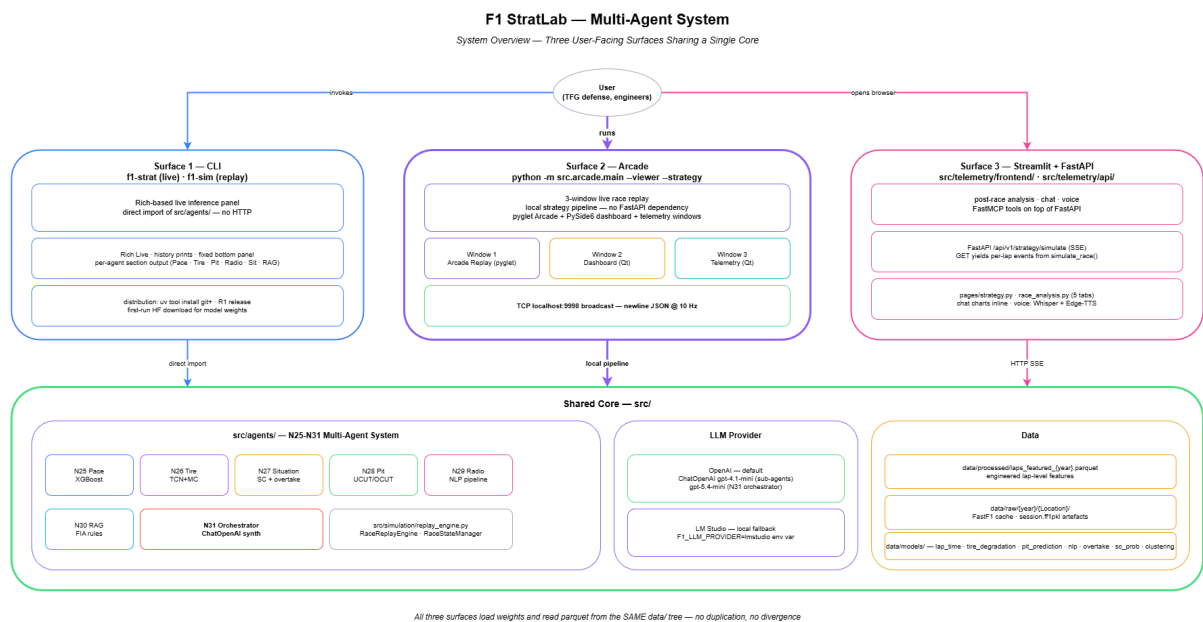


Figura 8.1: Vista global de F1 StratLab. Las tres superficies de uso (CLI, Arcade y Streamlit con FastAPI) comparten el núcleo de seis sub-agentes y el orquestador supervisor N31, el proveedor de modelos de lenguaje y el árbol de datos. El bloque inferior representa los modelos en producción y los artefactos descargados durante el primer arranque desde el conjunto abierto publicado en Hugging Face.

Tabla 8.4: Inventario de cuadernos Jupyter del proyecto F1 Strat Manager (parte 1 de 3: ingeniería de datos y aprendizaje supervisado clásico).

Sigla	Ruta	Propósito	Estado	Cifra clave
Bloque 1. Ingeniería de datos Ruta raíz: <code>notebooks/data_engineering/</code>				
N01	<code>N01_data_download.ipynb</code>	Descarga del corpus de 46 Grandes Premios 2023-2024 desde FastF1 y OpenF1 y consolidación en <i>parquet</i> maestro	Completado	46 GP
N02	<code>N02_eda_master.ipynb</code>	Análisis exploratorio del corpus maestro y caracterización de calidad, distribución y patrones agregados de vuelta	Completado	—
N03	<code>N03_circuit_clustering.ipynb</code>	<i>Clustering</i> KMeans de 24 circuitos sobre 14 variables y selección por codo de inercia	Completado	K=4, silueta 0,201
N04	<code>N04_feature_engineering.ipynb</code>	Construcción del conjunto de variables aguas abajo a partir del maestro y de los <i>clusters</i> de N03	Completado	—
Bloque 2. Aprendizaje supervisado clásico Subdirectorios bajo <code>notebooks/strategy/</code>				
<i>Subdirectorio:</i> <code>lap_time_prediction/</code>				
N05	<code>N05_laptime_eda.ipynb</code>	Análisis exploratorio del <i>dataset</i> <i>laps_featured</i> previo al regresor de tiempo por vuelta	Completado	—
N06	<code>N06_laptime_model.ipynb</code>	Regresor XGBoost de delta de tiempo por vuelta con <i>splits</i> temporales 2023-2025	Completado	MAE 0,4104 s, R ² 0,9947
<i>Subdirectorio:</i> <code>tire_degradation/</code>				
N07	<code>N07_tireddeg_eda.ipynb</code>	Análisis exploratorio de la degradación de neumático sobre el corpus de <i>stints</i> en seco	Completado	—
N08	<code>N08_tireddeg_sequence_config.ipynb</code>	Configuración de la canalización causal con <i>padding</i> y ventana global de 28 vueltas para la TCN	Completado	ventana 28
N09	<code>N09_tireddeg_tcn.ipynb</code>	TCN global de degradación con cuatro bloques residuales, <i>kernel</i> 3 y dilatación exponencial	Completado	MAE 0,7078, R ² 0,6048
N10	<code>N10_tireddeg_compound_finetuning.ipynb</code>	<i>Finetuning</i> por compuesto con MC <i>Dropout</i> de 50 pasadas y enrutado selectivo a C2	Completado	C2 MAE 0,5501
<i>Subdirectorio:</i> <code>overtake_probability/</code>				
N11	<code>N11_overtake_eda.ipynb</code>	Análisis exploratorio de pares atacante-defensor con 28.494 pares y tasa positiva 8,44 %	Completado	8,44 % positivos
N12	<code>N12_overtake_model.ipynb</code>	Clasificador LightGBM de adelantamiento sobre <i>snapshots</i> con umbral operativo 0,80	Completado	AUC-PR 0,5491, AUC-ROC 0,8758
N12B	<code>N12B_overtake_tcn.ipynb</code>	Alternativa TCN causal a N12 archivada como resultado negativo válido frente a LightGBM	Archivado	AUC-PR 0,0994
<i>Subdirectorio:</i> <code>sc_probability/</code>				
N13	<code>N13_sc_eda.ipynb</code>	Análisis exploratorio del coche de seguridad con 3.275 vueltas y horizontes 3, 5 y 7	Completado	3.275 vueltas
N14	<code>N14_sc_model.ipynb</code>	Clasificador LightGBM de probabilidad de coche de seguridad sobre tres horizontes	Completado	AUC-PR 0,0723, <i>lift</i> 1,67×
<i>Subdirectorio:</i> <code>pit_prediction/</code>				
N15	<code>N15_pit_duration.ipynb</code>	Regresor HistGBT cuantílico de duración de <i>pit stop</i> sobre P05, P50 y P95	Completado	P50 MAE 0,487 s
N16	<code>N16_undercut.ipynb</code>	Clasificador LightGBM binario de éxito del <i>undercut</i> con umbral 0,522	Completado	AUC-PR 0,6739, AUC-ROC 0,7708

Tabla 8.5: Inventario de cuadernos Jupyter del proyecto F1 Strat Manager (parte 2 de 3: procesamiento de lenguaje natural de radio y sub-agentes). Continuación de la Tabla 8.4.

Sigla	Ruta	Propósito	Estado	Cifra clave
Bloque 3. Procesamiento de lenguaje natural de radio Ruta raíz: <code>notebooks/nlp/</code>				
N17	<code>N17_radio_labeling.ipynb</code>	Construcción y etiquetado del corpus de mensajes de radio para los modelos aguas abajo	Completado	—
N18	<code>N18_radio_transcription.ipynb</code>	Transcripción automática de audio mediante Whisper turbo con caché atómica en disco	Completado	—
N19	<code>N19_sentiment_vader.ipynb</code>	Línea base de sentimiento con VADER previa al modelo neuronal de producción	Completado	—
N20	<code>N20_bert_sentiment.ipynb</code>	<i>Fine-tuning</i> de RoBERTa-base para sentimiento de radio sobre 80 mensajes de prueba	Completado	<i>accuracy</i> 0,84
N21	<code>N21_radio_intent.ipynb</code>	Clasificador de intención mediante SetFit sobre ModernBERT con régimen <i>few-shot</i>	Completado	macro F1 0,5338
N22	<code>N22_ner_models.ipynb</code>	Reconocimiento de entidades F1 con BERT-large CoNLL-03 BIO sobre 40 mensajes	Completado	F1 0,4151
N23	<code>N23_rcm_parser.ipynb</code>	Analizador determinista de <i>Race Control Messages</i> con cobertura íntegra de etiquetas FIA	Completado	cobertura 100 %/93 %
N24	<code>N24_nlp_pipeline.ipynb</code>	Canalización unificada que integra N20, N21, N22 y N23 con dos puntos de entrada	Completado	P95 GPU 59,4 ms
N33*	<code>N33_radio_dataset_builder.ipynb</code>	Constructor del corpus de radio etiquetado a partir de OpenF1 con filtros estructurales por vuelta	Completado	28 radios Baréin
Bloque 4. Sub-agentes y orquestador Ruta raíz: <code>notebooks/agents/</code>				
N25	<code>N25_pace_agent.ipynb</code>	Agente ReAct de ritmo con @tool sobre el modelo N06 y enrutado por <i>cluster</i> de circuito	Completado	—
N26	<code>N26_tire_agent.ipynb</code>	Agente ReAct de neumático con @tool sobre N09 y N10 e incertidumbre MC <i>Dropout</i>	Completado	—
N27	<code>N27_race_situation_agent.ipynb</code>	Agente ReAct de situación de carrera con @tool sobre N12 y N14 y calibradores asociados	Completado	—
N28	<code>N28_pit_strategy_agent.ipynb</code>	Agente ReAct de estrategia de <i>pit</i> con @tool sobre N15 y N16 y salvaguardas en <i>prompt</i>	Completado	—
N29	<code>N29_radio_agent.ipynb</code>	Agente ReAct de radio con @tool sobre la canalización N24 y emisión booleana de alertas	Completado	—
N30	<code>N30_rag_agent.ipynb</code>	Agente ReAct de recuperación reglamentaria con BGE-M3 en producción sobre el corpus FIA	Completado	—
N30B	<code>N30B_rag_benchmark.ipynb</code>	Banco de pruebas RAG con 15 consultas y verdad de referencia sobre tres configuraciones	Completado	Precision@5 y MRR
N31	<code>N31_strategy_orchestrator.ipynb</code>	Orquestador supervisor LangGraph con tres capas y <i>schemas</i> Pydantic de 14 campos	Completado	14 campos, 500 MC
N31v	<code>N31_mc_visualization.ipynb</code>	Cuaderno de visualización estática de la simulación Monte Carlo de la capa 2 del orquestador	Completado	—

Tabla 8.6: Inventario de cuadernos Jupyter del proyecto F1 Strat Manager (parte 3 de 3: integración y validación). Continuación de la Tabla 8.5.

Sigla	Ruta	Propósito	Estado	Cifra clave
Bloque 5. Integración y validación Ruta raíz: notebooks/agents/				
N32	N32_smoke_test.ipynb	Prueba de humo de los siete sub-agentes sobre Melbourne 2025 vuelta 20 y CLI a tres vueltas	Completado	7 exit 0, 3 recomendaciones
N33	N33_thresholds_and_calibration.ipynb	Guaderno hermano de visualización de umbrales y calibración a partir de artefactos serializados	Completado	—
N34	N34_radio_runner_smoke.ipynb	Prueba de humo del corredor de radio sobre Barén 2025 con 28 audios y 76 RCMs reales	Completado	28/28 radios, 76/76 RCMs

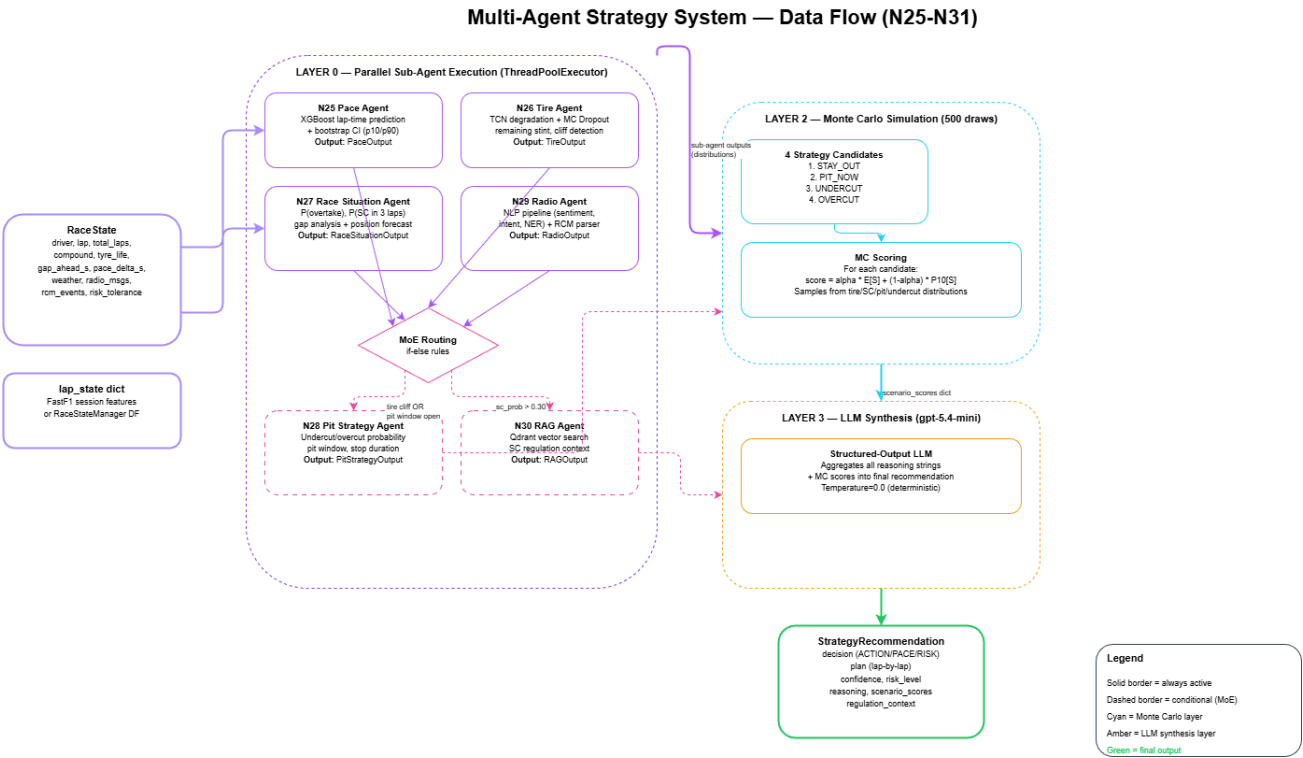


Figura 8.2: Flujo del orquestador supervisor N31. La capa de invocación dispara los sub-agentes especializados, la capa de enrutamiento por mezcla de expertos basada en reglas decide qué salidas pasan a la siguiente etapa, la capa de simulación Monte Carlo evalúa las cuatro acciones discretas STAY_OUT, PIT_NOW, UNDERCUT y OVERCUT sobre quinientas trayectorias por candidata y la capa de síntesis con modelo de lenguaje produce la recomendación final con salida estructurada de catorce campos Pydantic.

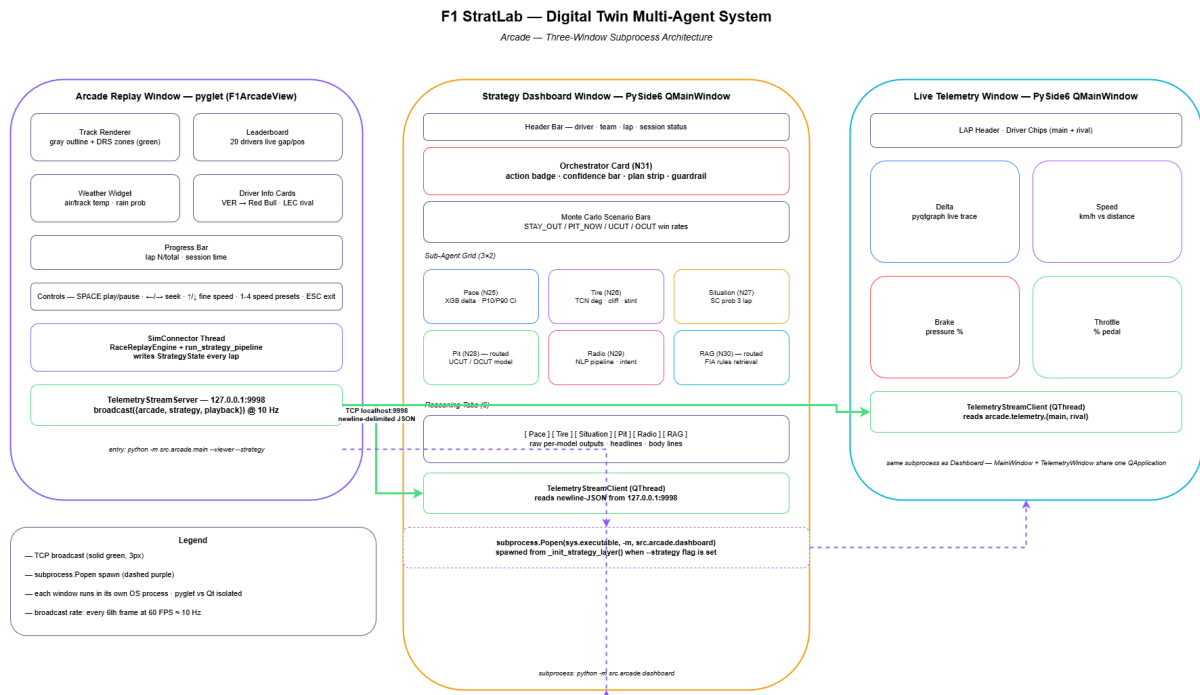


Figura 8.3: Arquitectura de tres ventanas del modo Arcade. El proceso *pyglet* del replay 2D actúa como dueño del bucle de simulación y publica el estado por TCP al puerto 127.0.0.1:9998 a diez hercios. Un único subprocesso lanzado con `subprocess.Popen` sobre `src.arcade.dashboard` abre las dos ventanas Qt (cuadro de mando estratégico y telemetría) dentro de la misma `QApplication`, suscritas al mismo flujo TCP.

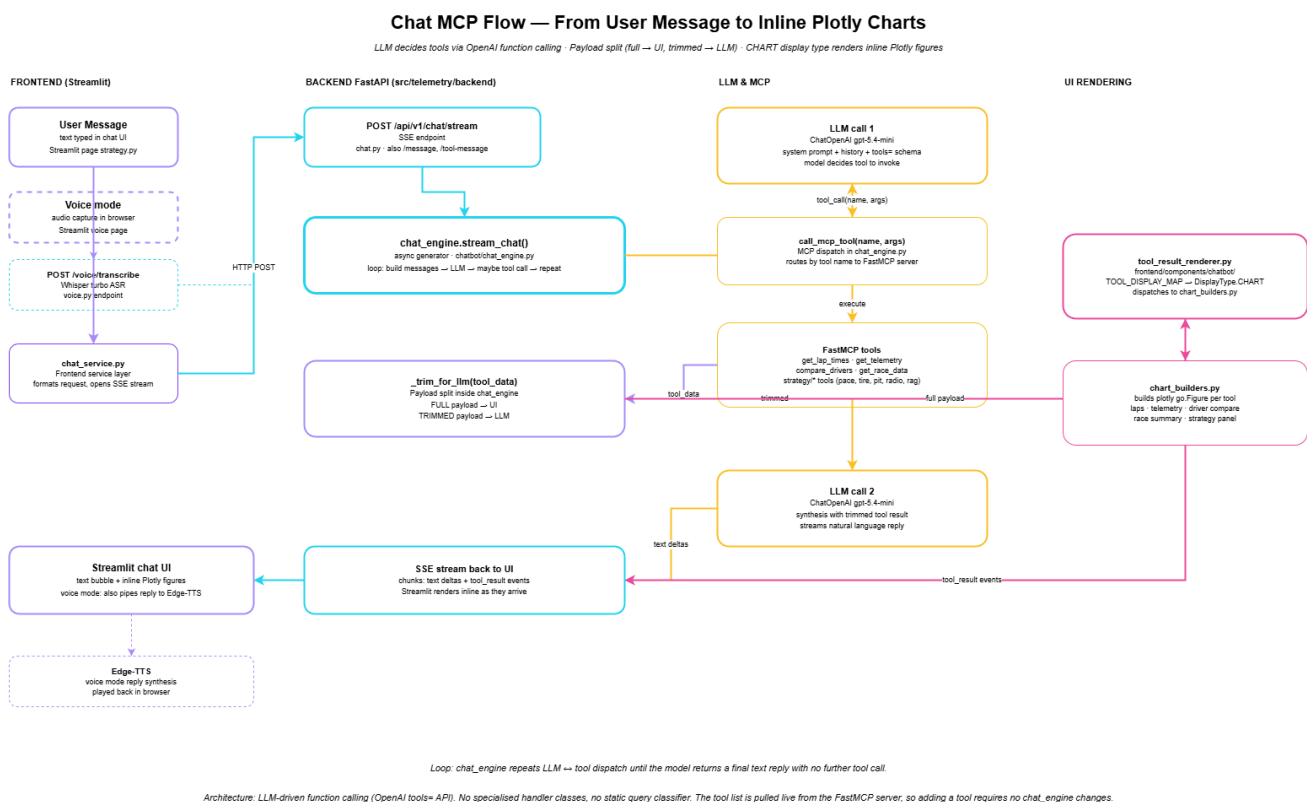


Figura 8.4: Flujo del chat conversacional con tool calling MCP. La consulta del usuario en la interfaz de Streamlit alcanza el endpoint `/api/v1/chat/stream` del *backend* FastAPI, que invoca `chat_engine.stream_chat`. El modelo `gpt-5.4-mini` decide la herramienta MCP a llamar mediante el mecanismo de *function calling* de OpenAI, el resultado de la herramienta se parte en el *payload* completo destinado a la interfaz y el *payload* reducido destinado a la segunda pasada del modelo, y el cliente renderiza las figuras Plotly de forma *inline* junto a la respuesta sintetizada.